



AKTU

B.Tech II-Year



CS IT & CS Allied

DATA STRUCTURE

Unit-4 : TREES

in ONE SHOT



By Dr. Nidhi Parashar Ma'am

- Ph.D (CSE)
- M.Tech Gold Medalist
- Net Qualified



Pdf Notes



AKTU PYQs



Imp. Questions



AKTU

B.Tech III-Sem



GATEWAY CLASSES

CS IT & CS Allied

DATA STRUCTURE

Unit-4 Lecture-1

Today's Target

- Introduction to Tree Data structure
- Binary tree
 - Types of binary trees
- AKTU PYQs
- DPP



By Dr. Nidhi Parashar Ma'am

Ph.D (CSE)

- M.Tech Gold Medalist
- Net Qualified



CS IT & CS Allied III-Sem

Subject : DATA STRUCTURE



Unit-4 : Syllabus

Trees: Basic terminology used with Tree, Binary Trees, Binary Tree Representation: Array Representation and Pointer(Linked List) Representation, Binary Search Tree, Strictly Binary Tree ,Complete Binary Tree . A Extended Binary Trees, Tree Traversal algorithms: Inorder, Preorder and Postorder, Constructing Binary Tree from given Tree Traversal, Operation of Insertation , Deletion, Searching & Modification of data in Binary Search . Threaded Binary trees, Traversing Threaded Binary trees. Huffman coding using Binary Tree. Concept & Basic Operations for AVL Tree , B Tree & Binary Heaps



AKTU



Gateway Classes



CS IT & CS Allied

Gateway Udaan Batch



Sem-IV

Validity Till
Sem Exam

COMBO of 5 Subjects



Video Lectures



Pdf Notes



AKTU PYQs



Most Imp. Questions

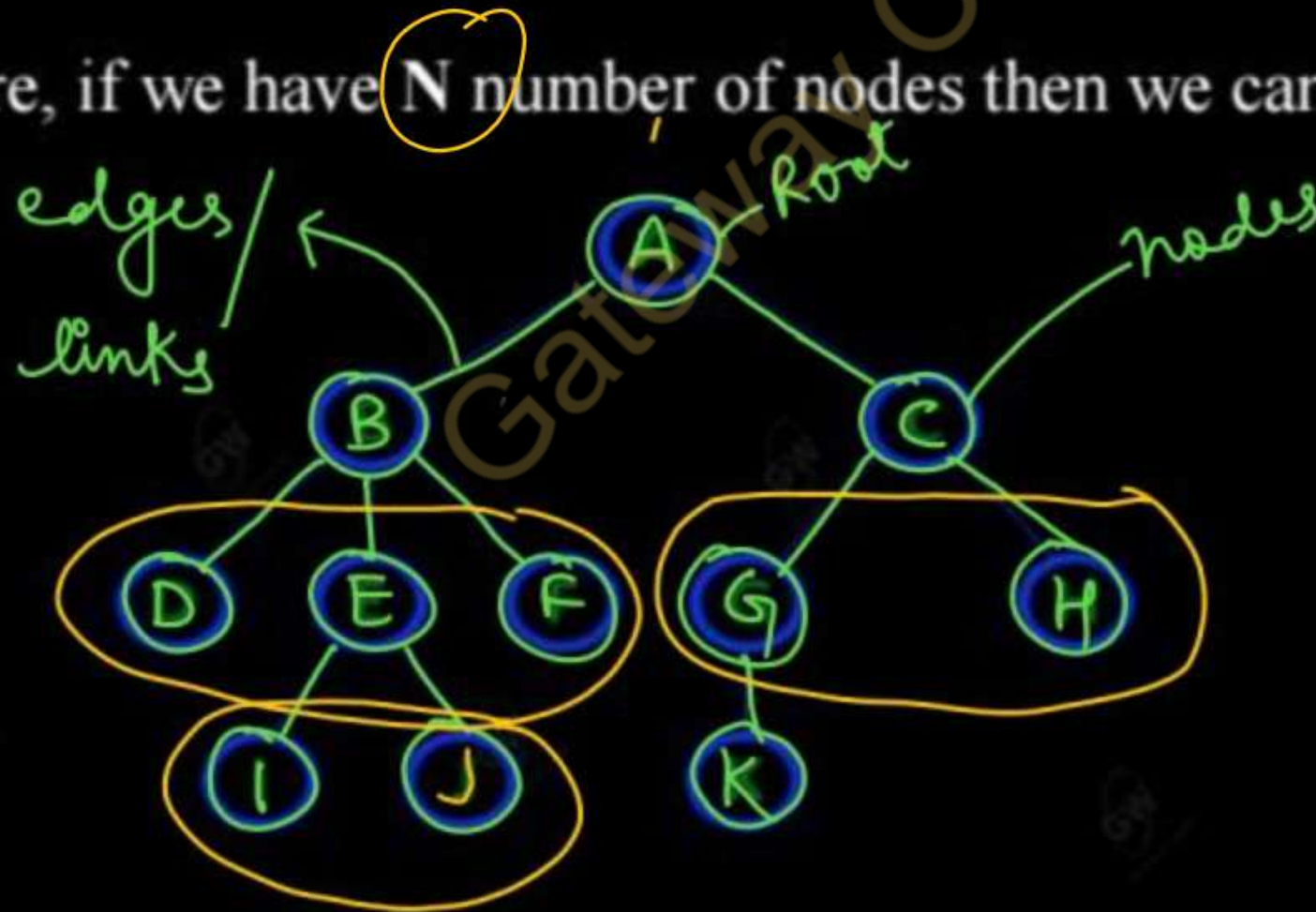
Helpline No-7819 0058 53



Tree - Terminology

Tree is a non-linear data structure which organizes data in hierarchical structure and this is a recursive definition.

- In tree data structure, every individual element is called as **Node**.
- Node in a tree data structure stores the actual data of that particular element and link to next element in hierarchical structure.
- In a tree data structure, if we have **N** number of nodes then we can have a maximum of **N-1** number of links.

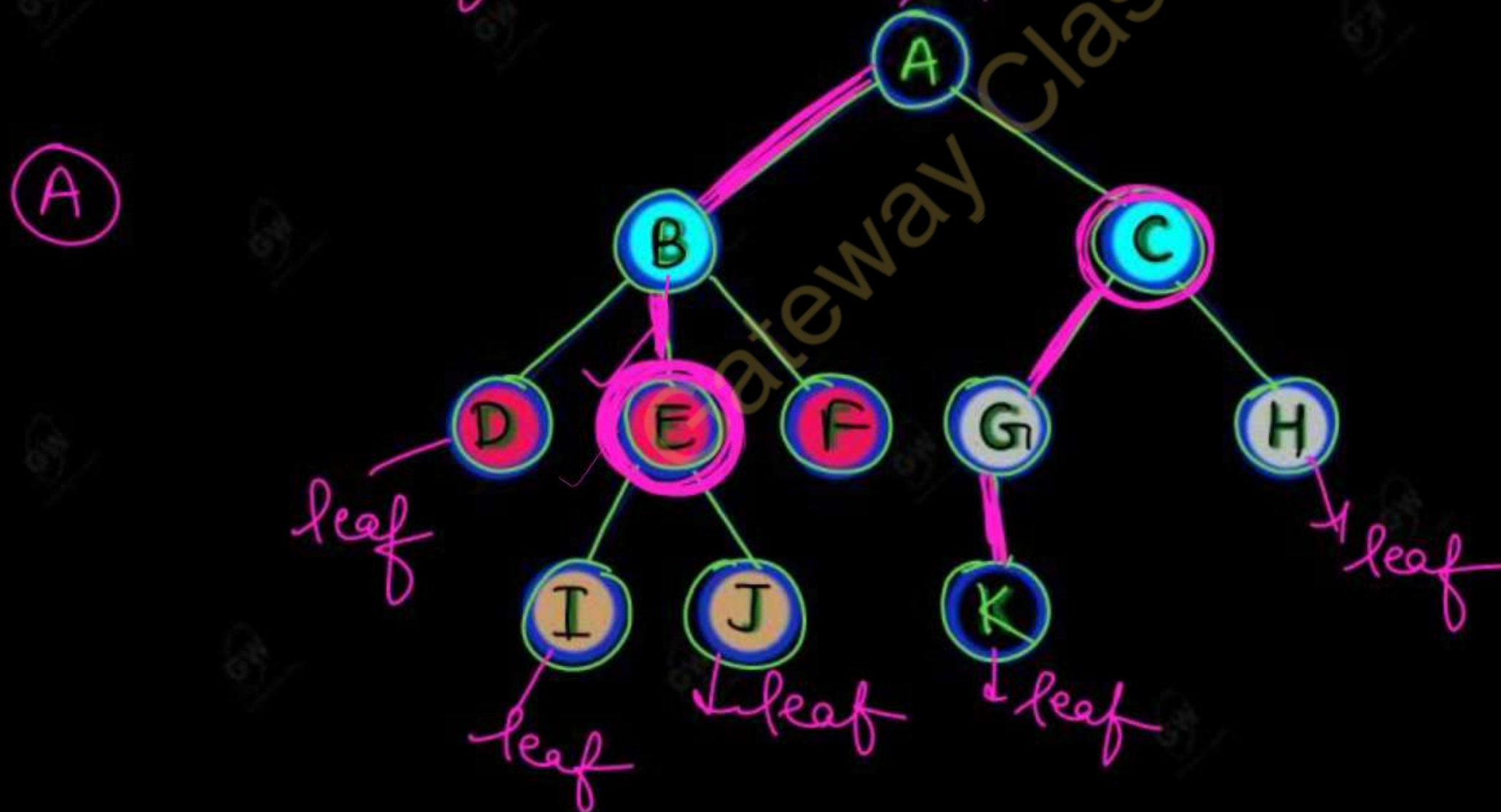


1. **Root:** The first node is called as **Root Node**. Every tree must have a root node. Root node is the origin of the tree data structure. In any tree, there must be only one root node. We never have multiple root nodes in a tree.
2. **Edge:** In a tree data structure, connecting link between any two nodes is called as EDGE. In a tree with 'N' number of nodes there will be a maximum of 'N-1' number of edges.
3. **Parent:** Node which is a predecessor of any node is called as **PARENT NODE**. Parent node can also be defined as "**The node which has child / children**".
4. **Child:** Node which is descendant of any node is called as **CHILD Node**. In a tree, any parent node can have any number of child nodes. In a tree, all the nodes except root are child nodes.
5. **Siblings:** Nodes which belong to same Parent are called as **SIBLINGS**.

Ancestors of $E \rightarrow B, A$

Descendant: Any successor node on the path from that node to leaf node.

Descendant of $C \rightarrow G, K, H$

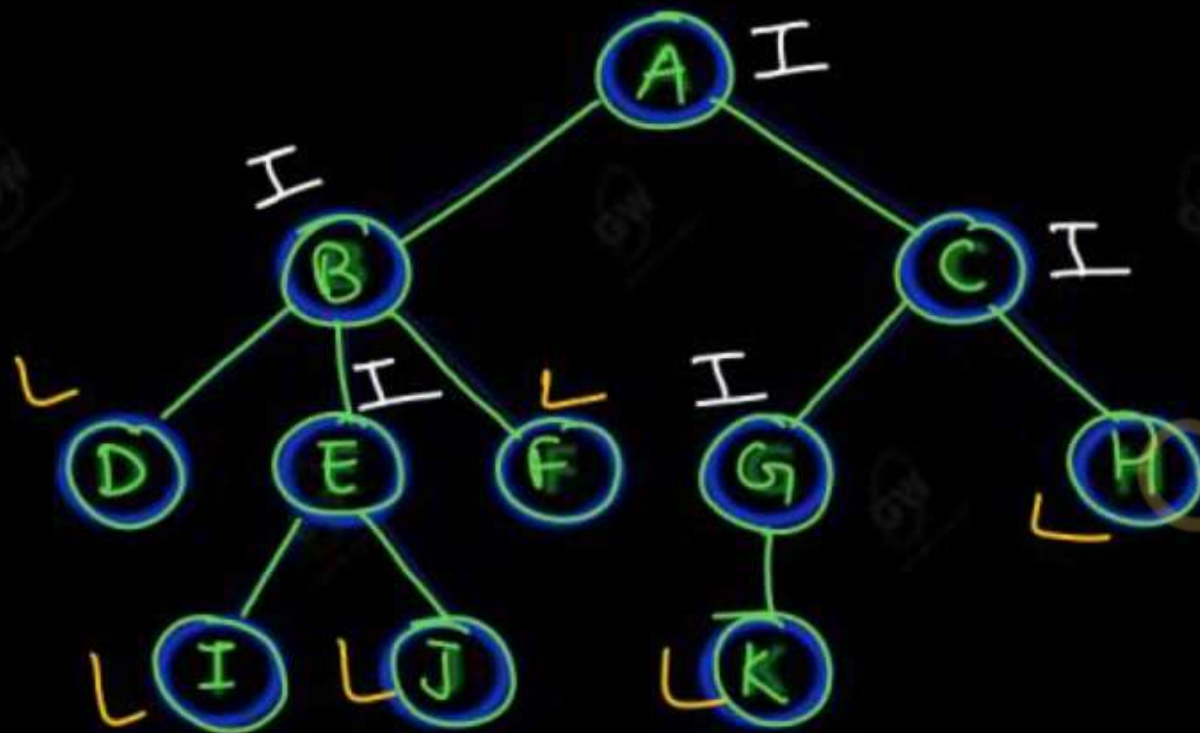


6. Leaf

Node which does not have a child is called as **LEAF Node**.

leaf nodes are also called as **External Nodes/ 'Terminal' node**.

leaf nodes → D, I, J, F, K, H



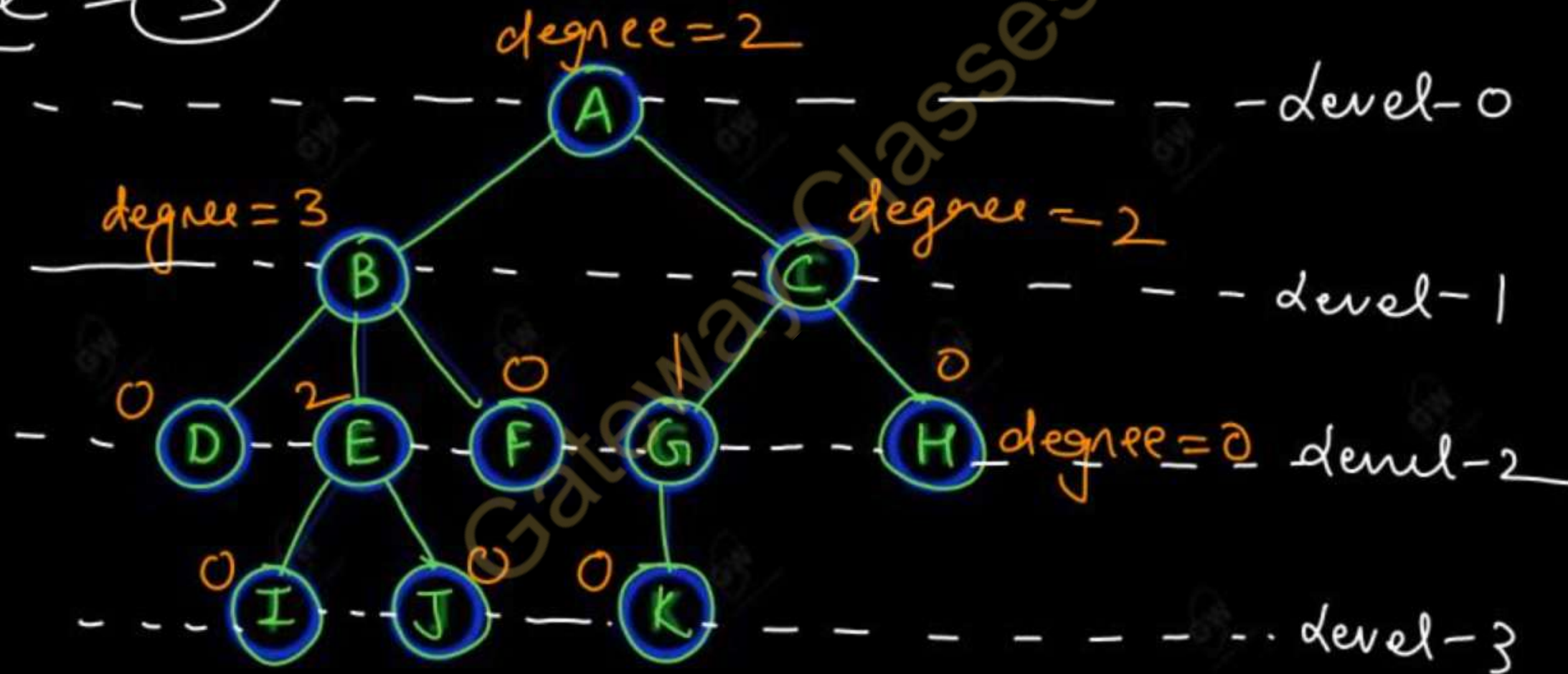
7. Internal Nodes

- Node having at least one child is called as **INTERNAL Node**.
- Nodes other than leaf nodes are called as **Internal Nodes**.
- Root node is also said to be Internal Node if the tree has more than one node.
- Internal nodes are also called as 'Non-Terminal' nodes.

8. Degree: Total number of children of a node is called as **DEGREE** of that Node.

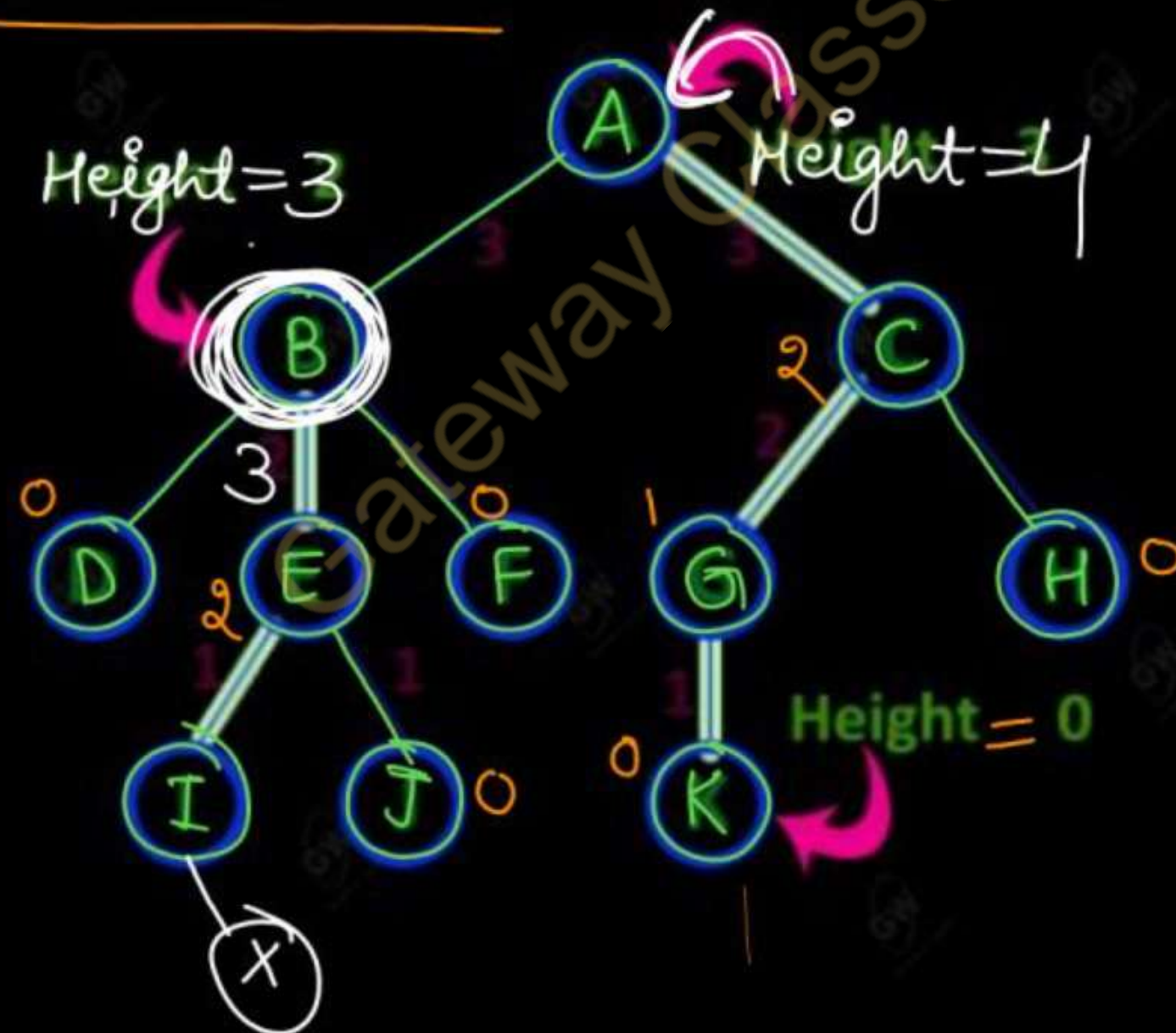
9. Level: Root node is said to be at Level 0 and the children of root node are at Level 1 and the children of the nodes which are at Level 1 will be at Level 2 and so on...

degree of Tree = 3



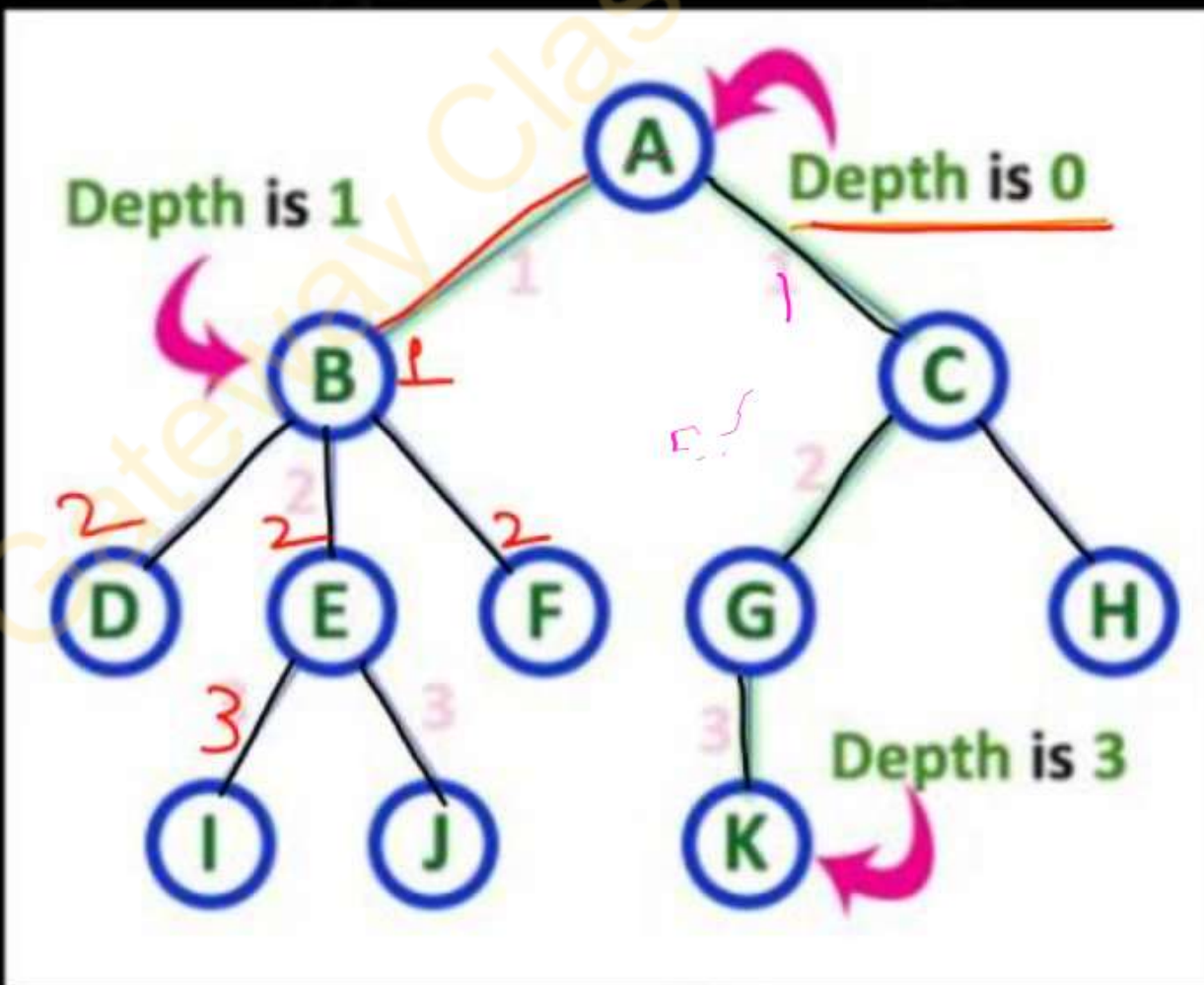
10. Height:

- Total number of edges from leaf node to a particular node in the longest path is called as **HEIGHT** of that Node.
- In a tree, height of the root node is said to be height of the tree.
- In a tree, height of all leaf nodes is '0'.



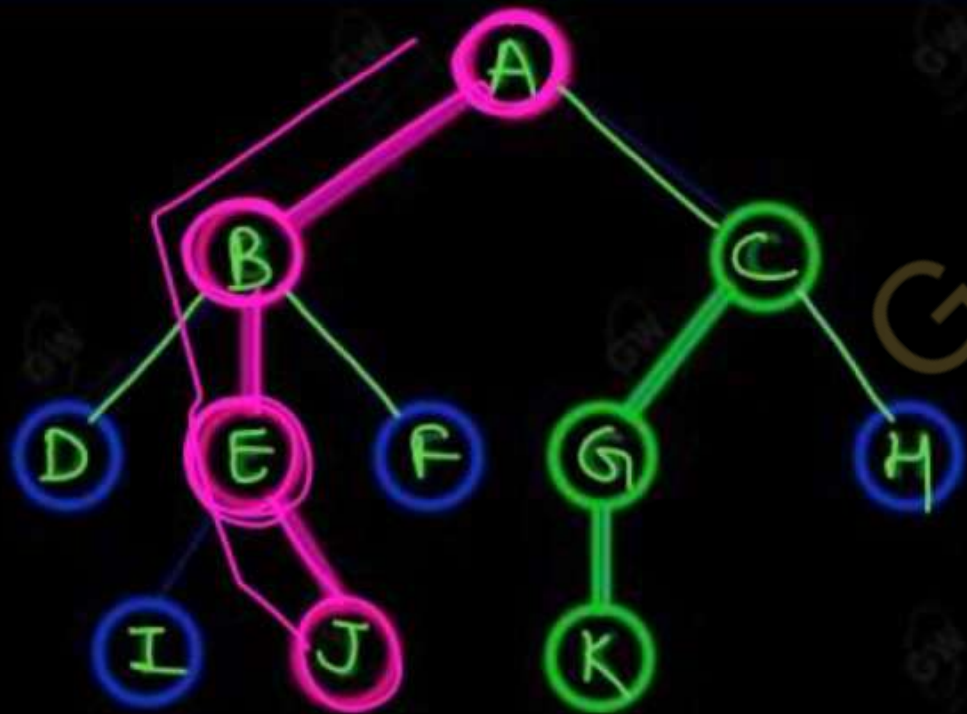
11. Depth:

- Total number of edges from root node to a particular node is called as DEPTH of that Node.
- In a tree, the total number of edges from root node to a leaf node in the longest path is said to be Depth of the tree.
- The highest depth of any leaf node in a tree is said to be depth of that tree.
- Depth of the root node is '0'.



12. Path:

- Sequence of Nodes and Edges from one node to another node is called as **PATH** between that two Nodes.
- **Length of a Path** is total number of nodes in that path. In below example **the path A - B - E - J** has **length 4**.



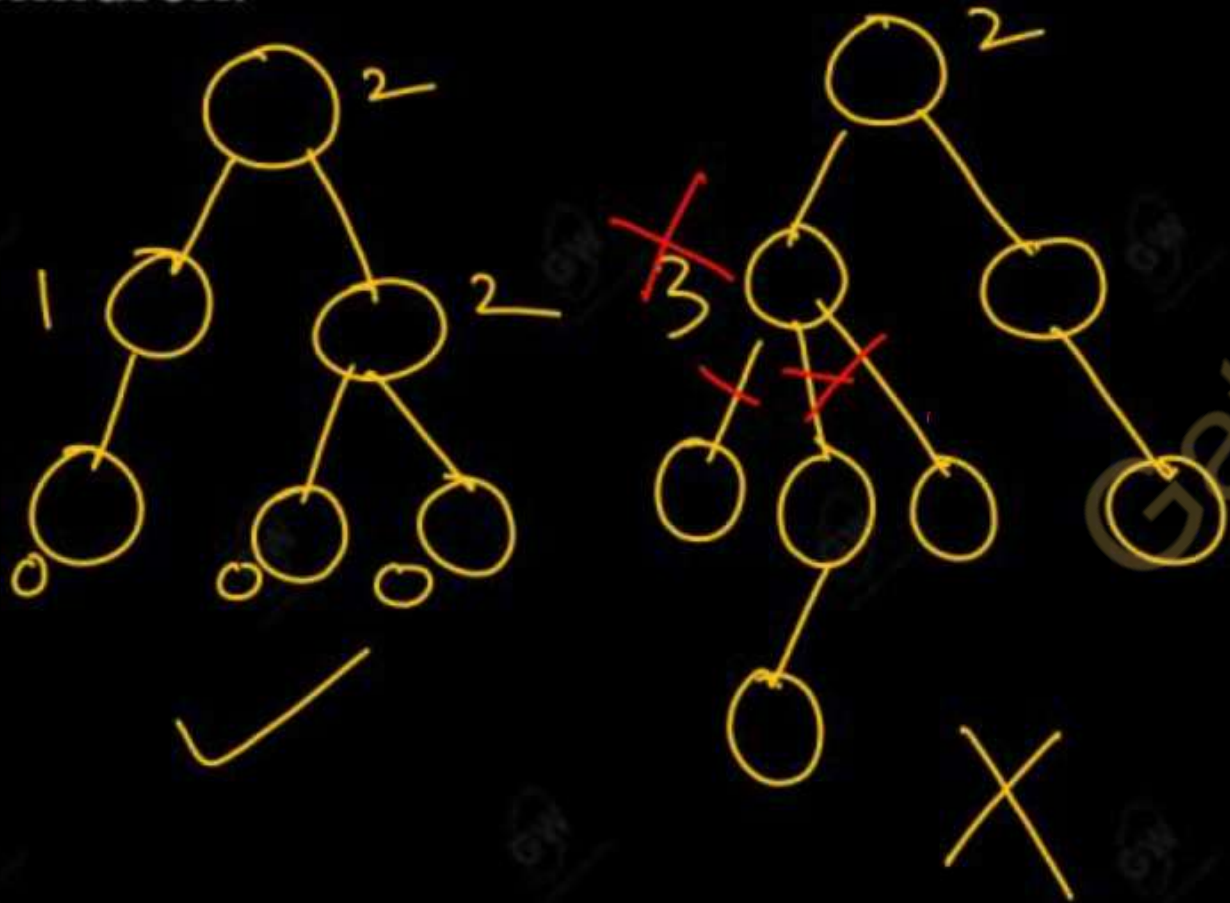
13. Sub Tree

- Each child from a node forms a subtree recursively.
- Every child node will form a subtree on its parent node.



Binary Tree

- In a normal tree, every node can have any number of children. A binary tree is a special type of tree in which every node can have a **maximum of 2 children**. One is known as a left child and the other is known as right child.
- In other words, every node can have either 0 children or 1 child or 2 children but not more than 2 children.

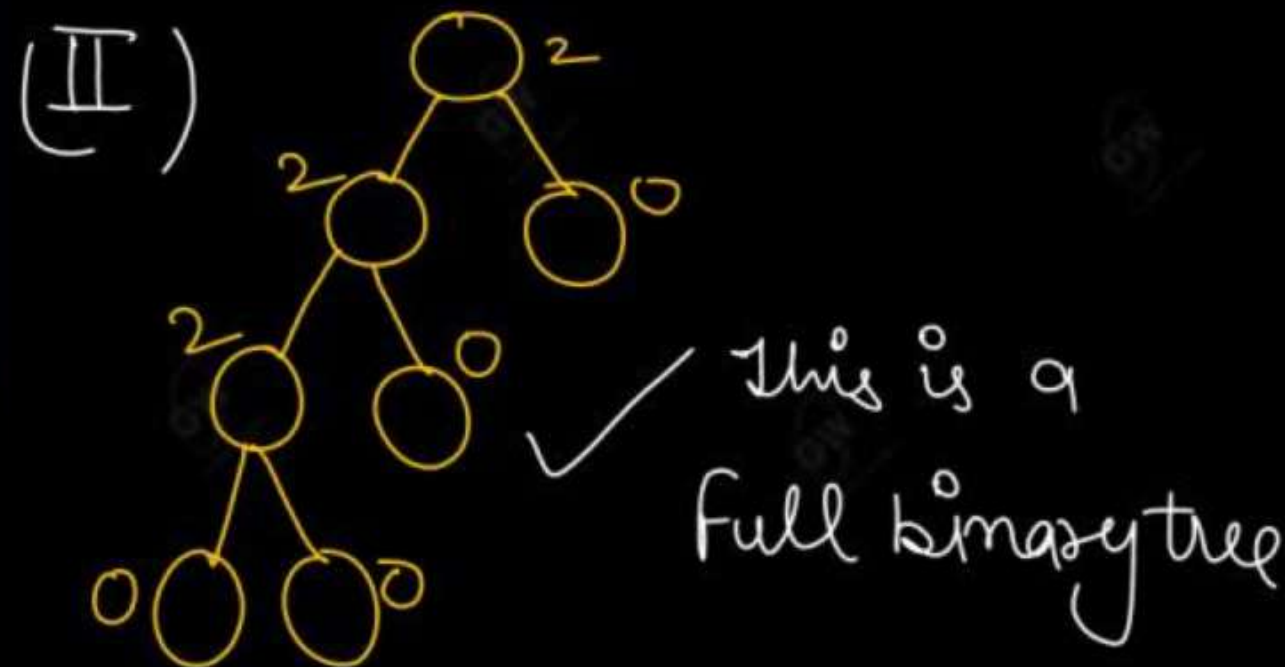
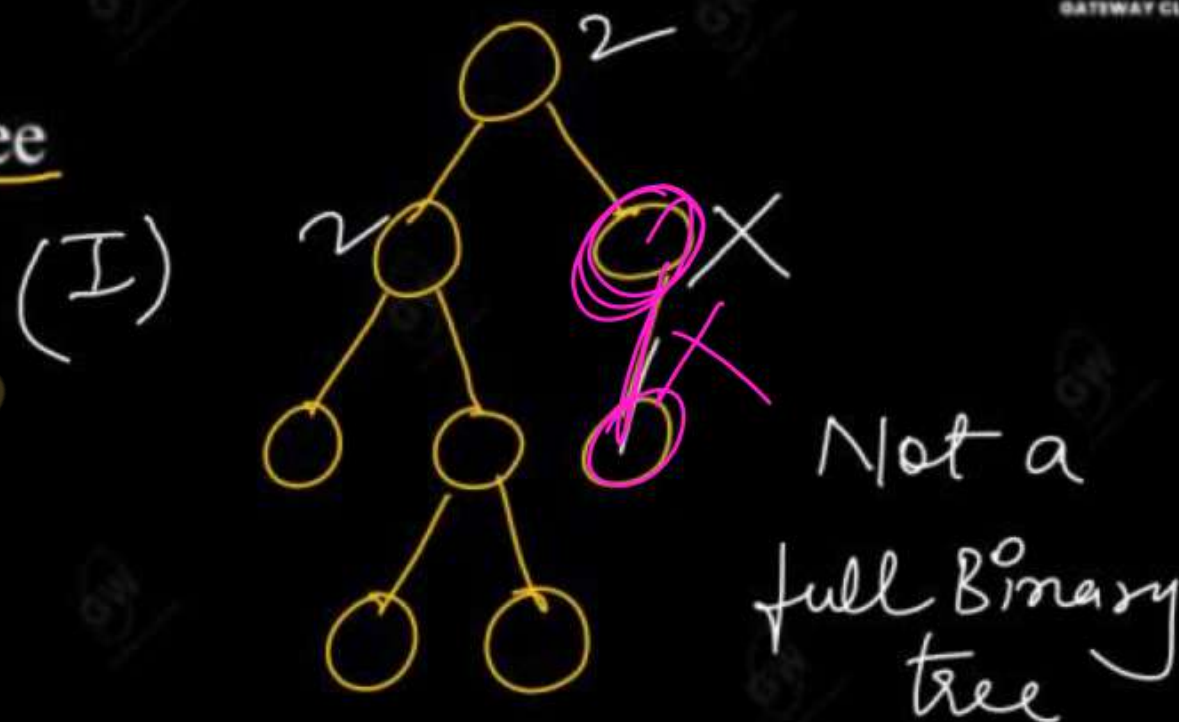


Types of Binary Tree (On the basis of number of children)

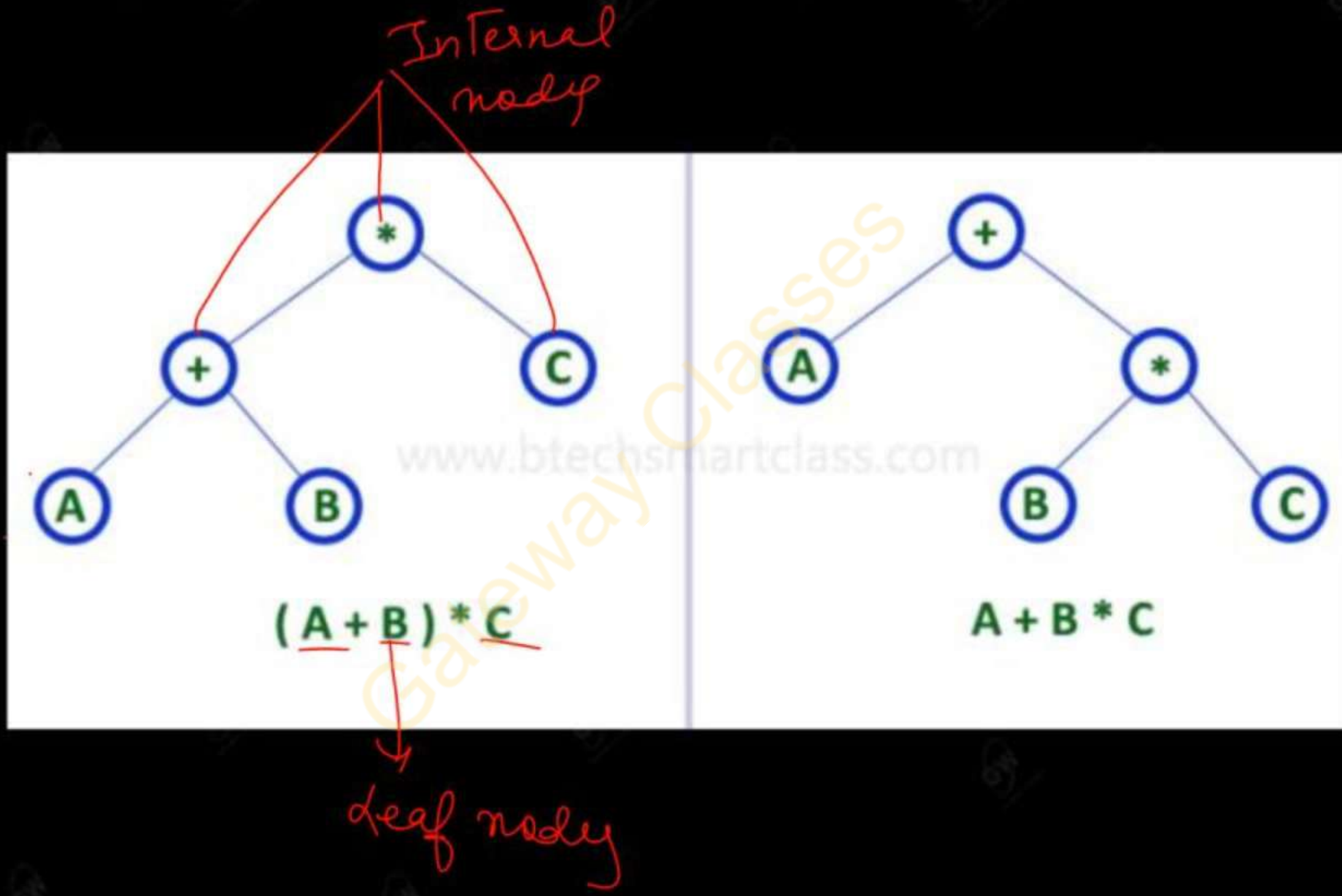
1. Full Binary Tree/Strict Binary Tree/Proper Binary Tree/2-Tree
2. Degenerate Binary Tree
3. Skewed Binary Trees

1. Full Binary Tree

- In full binary tree every node has either 0 or 2 children.
- All nodes except leaf nodes have two children.
- A special type of binary tree in which every parent node/internal node has either two or no children.



➤ Strict binary tree data structure is used to represent mathematical expressions.



2. Degenerate (or pathological) tree:

- Every internal node has one child either left or right.
- Such trees are performance-wise same as linked list.



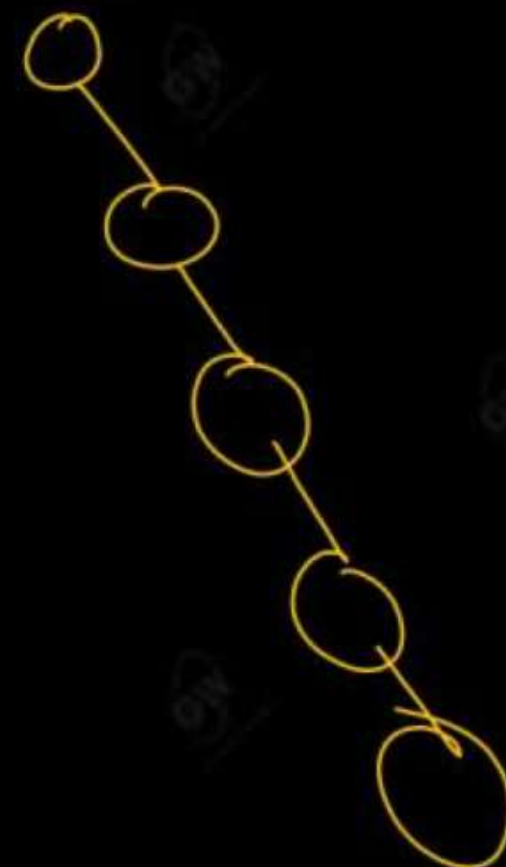
3. Skewed Binary Tree

A pathological/degenerate tree in which the tree is either dominated by the left nodes or the right nodes.

left-skewed



right-skewed

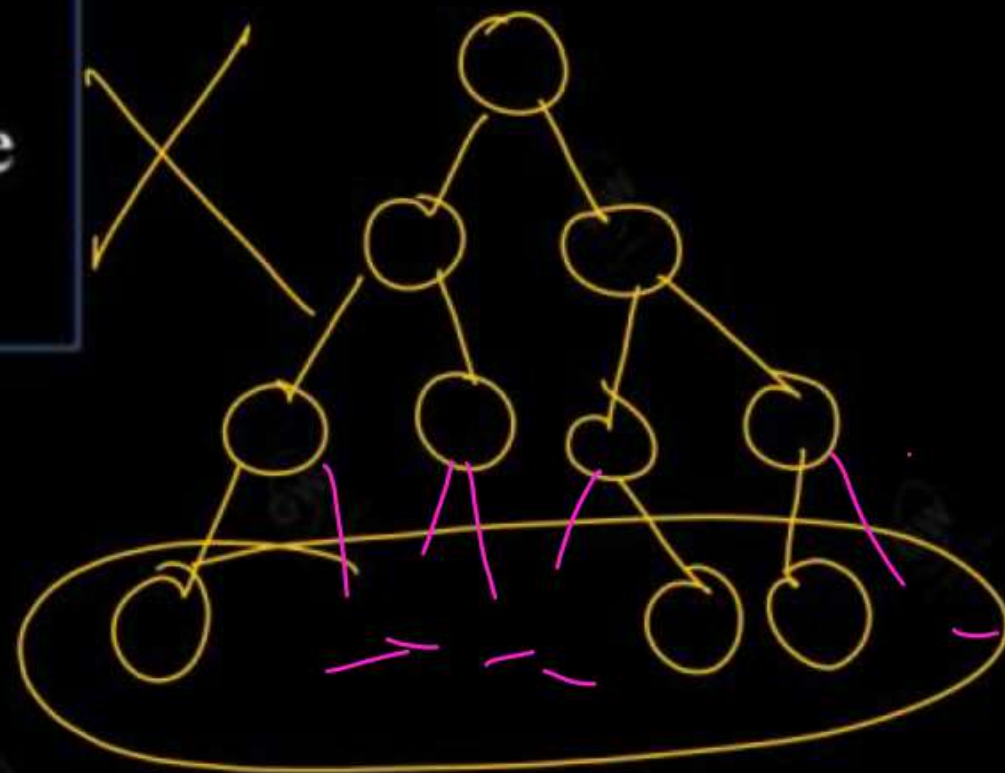
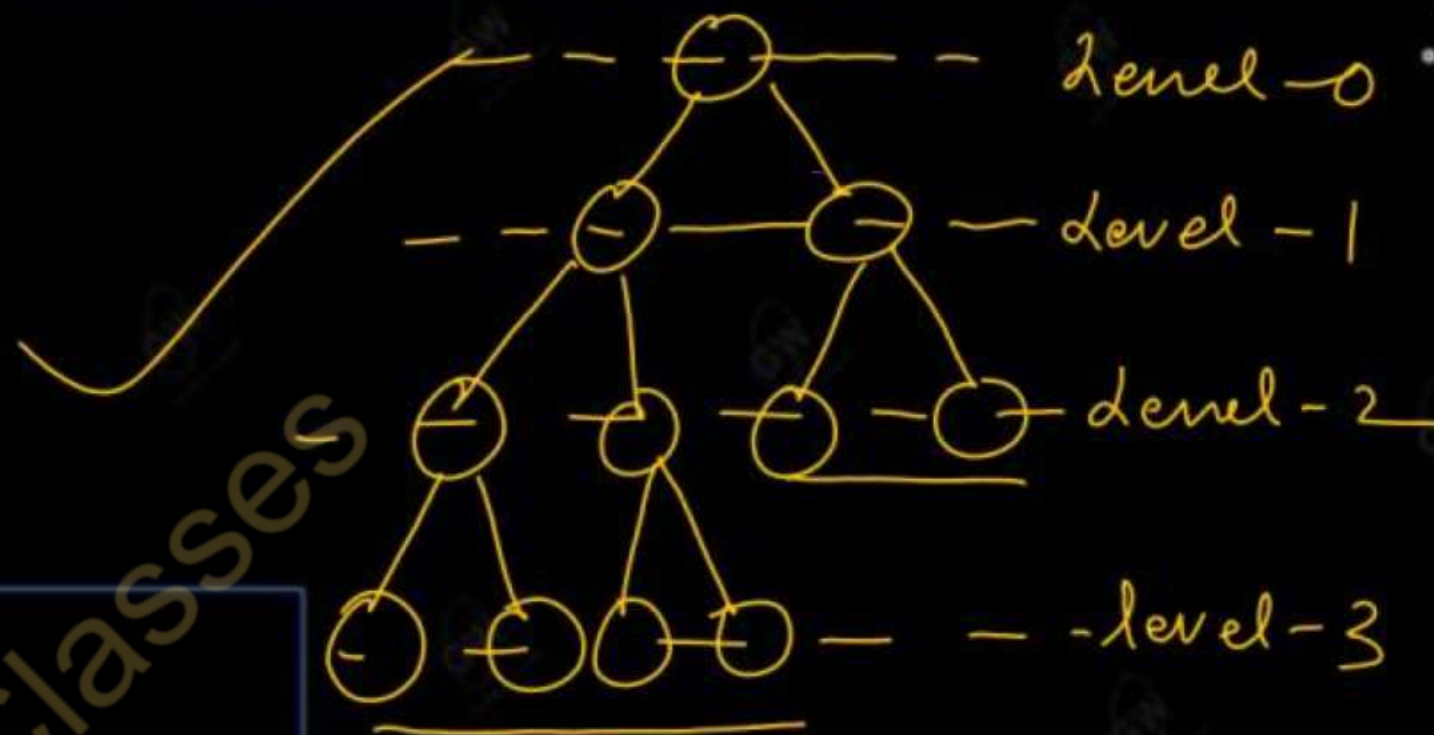


Types of Binary Tree (On the basis of Level completion)

1. Complete Binary Tree
2. Perfect Binary Tree
3. Balanced Binary Tree

1. Complete Binary Tree

- A Binary Tree is a Complete Binary Tree if all the levels are completely filled except possibly the last level and the last level is filled from left side.



2. Perfect Binary Tree:

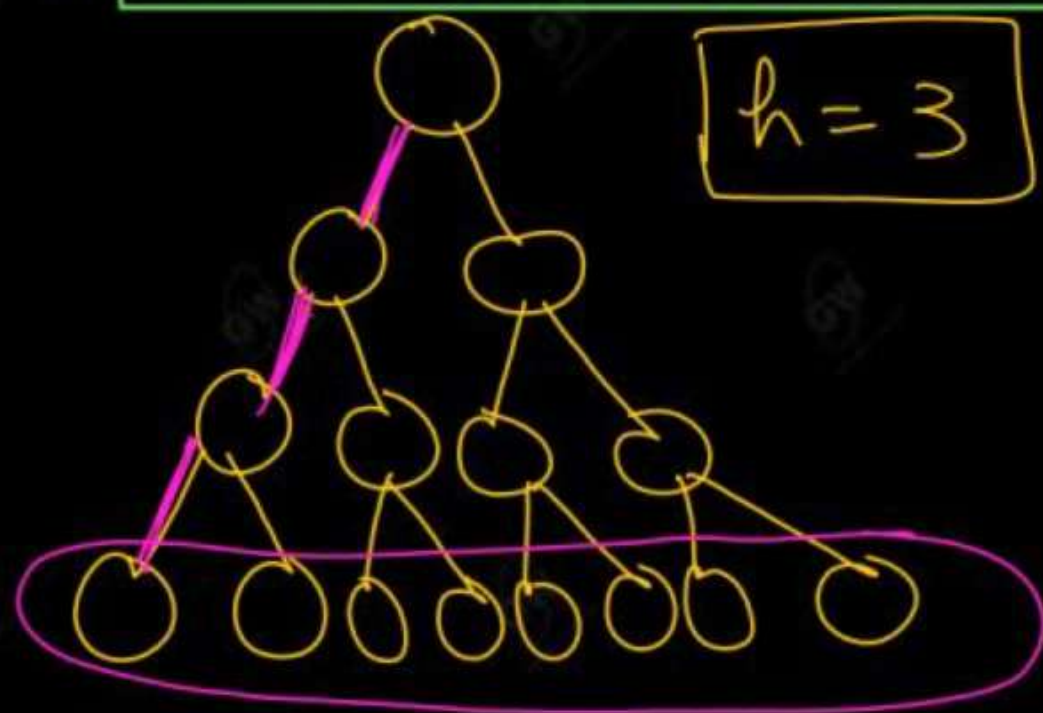
➤ In Perfect Binary Tree, all the internal nodes have two children and all leaf nodes are at the same level.

➤ A binary tree of height 'h' having the maximum number of nodes is a **perfect** binary tree.

➤ A perfect binary tree is a type of binary tree in which every internal node has exactly two child nodes and all the leaf nodes are at the same level.

leaf nodes $\swarrow$ $L = I + 1$ $\searrow$ Internal nodes

➤ In a Perfect Binary Tree, the number of leaf nodes is the number of internal nodes plus 1.



Max no. of nodes in h-height binary tree

$$\Rightarrow 2^{h+1} - 1 \Rightarrow 2^{3+1} - 1 = 2^4 - 1 = 15$$

No. of leaf nodes = 8

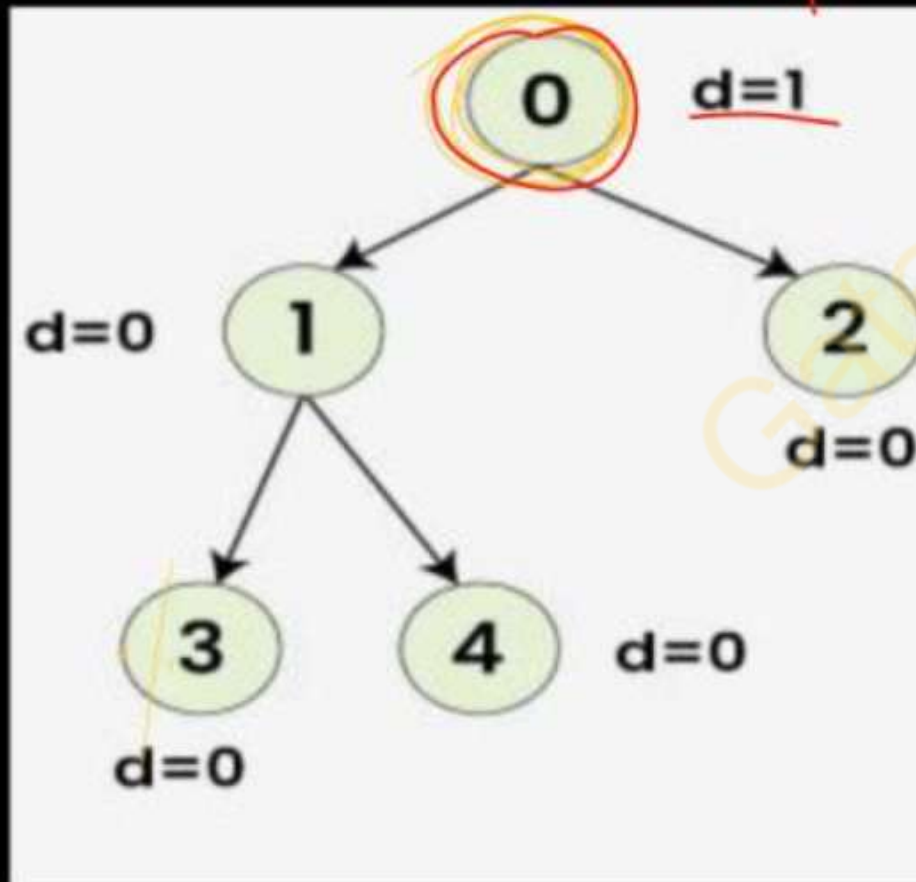
No. of internal nodes = 7

3. Balanced Binary Tree:

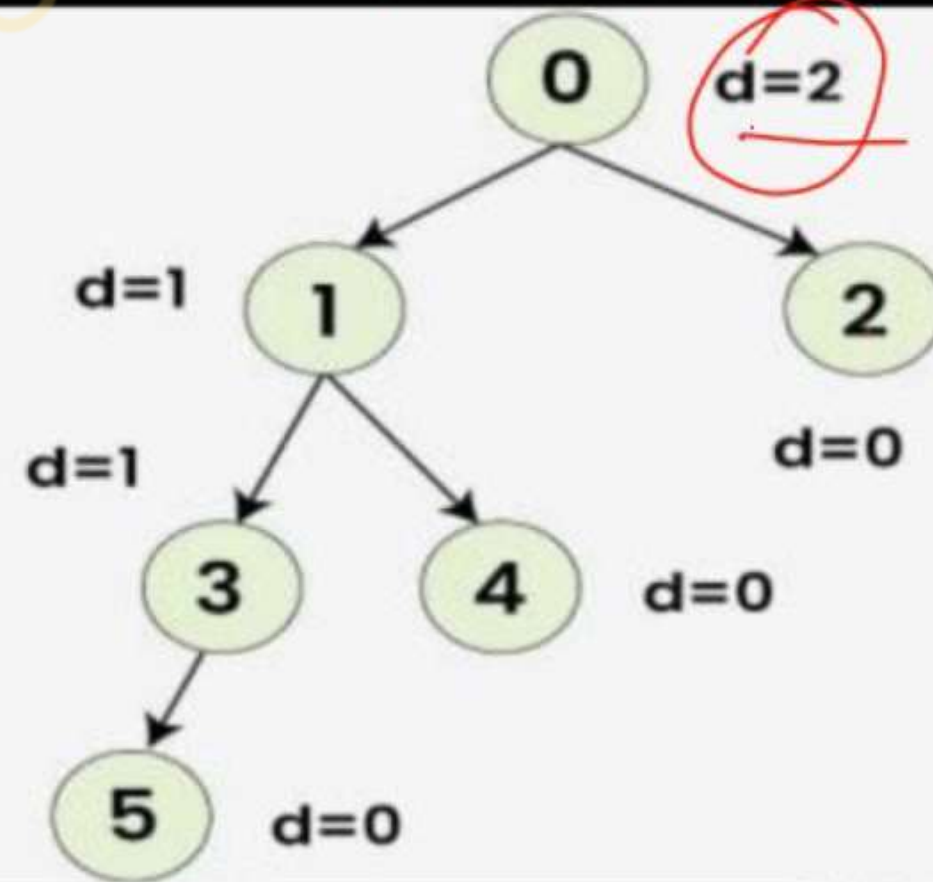
- A binary tree is balanced if the height of the tree is $O(\log n)$ where n is the number of nodes.
- For example, the AVL tree maintains $O(\log n)$ height by making sure that the difference between the heights of the left and right subtrees of every node is at most 1.
- Here, $d = |\text{Height of Left child} - \text{Height of Right child}|$

$+1 / -1, 0$

Balanced



unbalanced

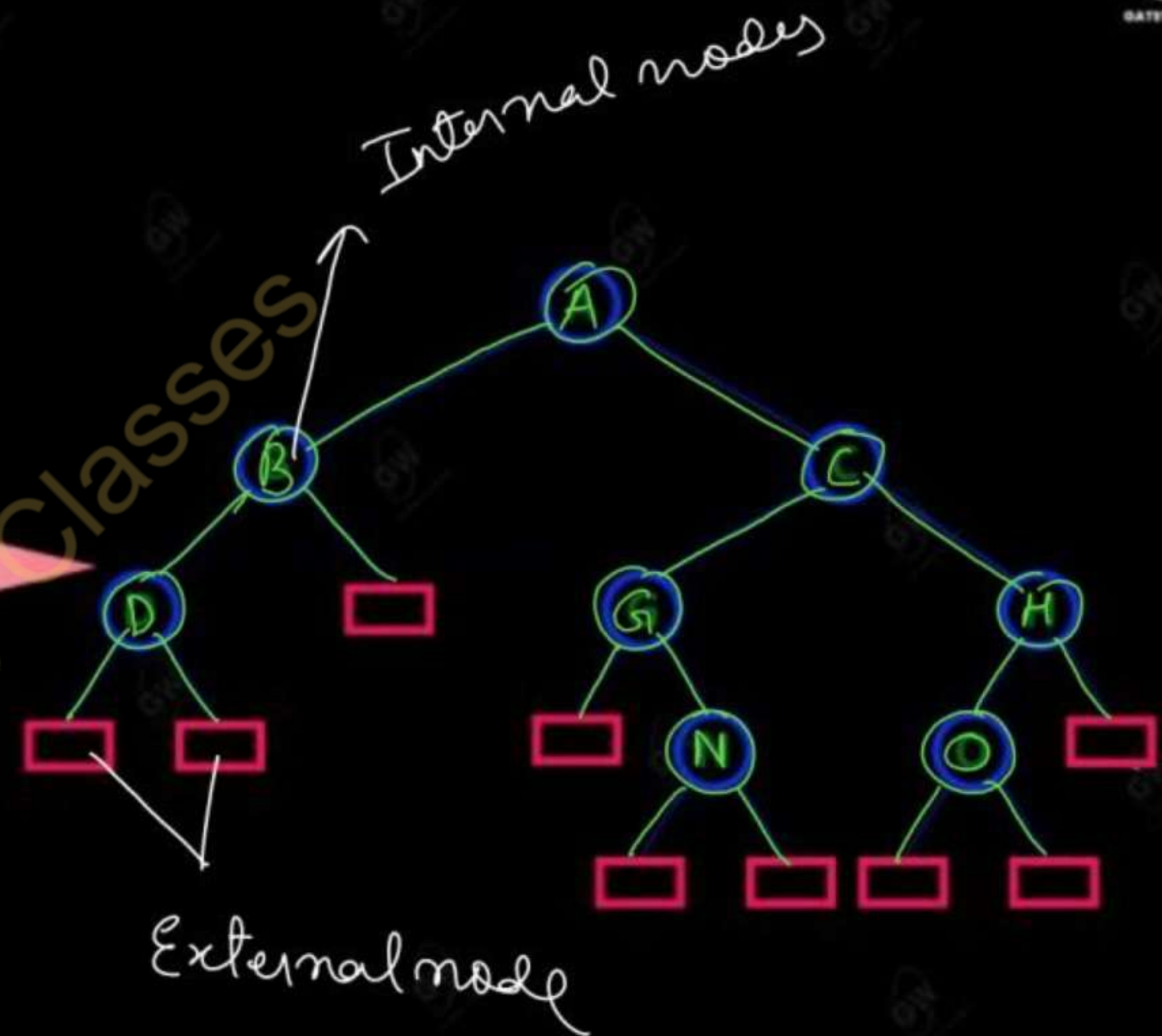
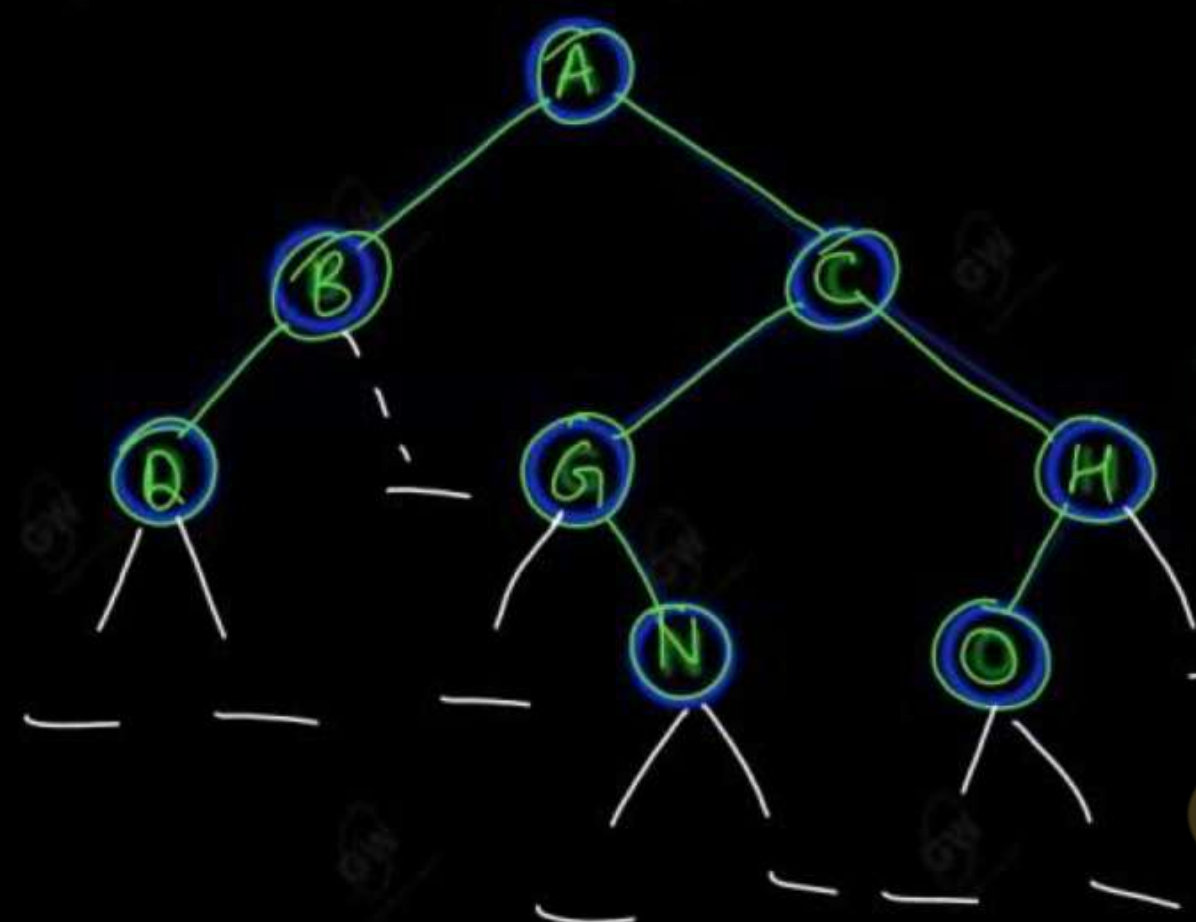


Extended Binary Tree:

- A binary tree can be converted into Full Binary tree by adding dummy nodes to existing nodes wherever required.
- A type of binary tree in which all the null sub tree of the original tree are replaced with special nodes called external nodes whereas other nodes are called internal nodes.
- Here the circles represent the internal nodes and the boxes represent the external nodes.

Properties of External binary tree:

- The nodes from the original tree are internal nodes and the special nodes are external nodes.
- All external nodes are leaf nodes and the internal nodes are non-leaf nodes.
- Every internal node has exactly two children and every external node is a leaf.
- It is useful for representation in algebraic expressions.



Internal nodes

External node

1. Maximum Nodes at Level 'l':

- A binary tree can have at most 2^l nodes at level l. Here, root is considered at level 0.

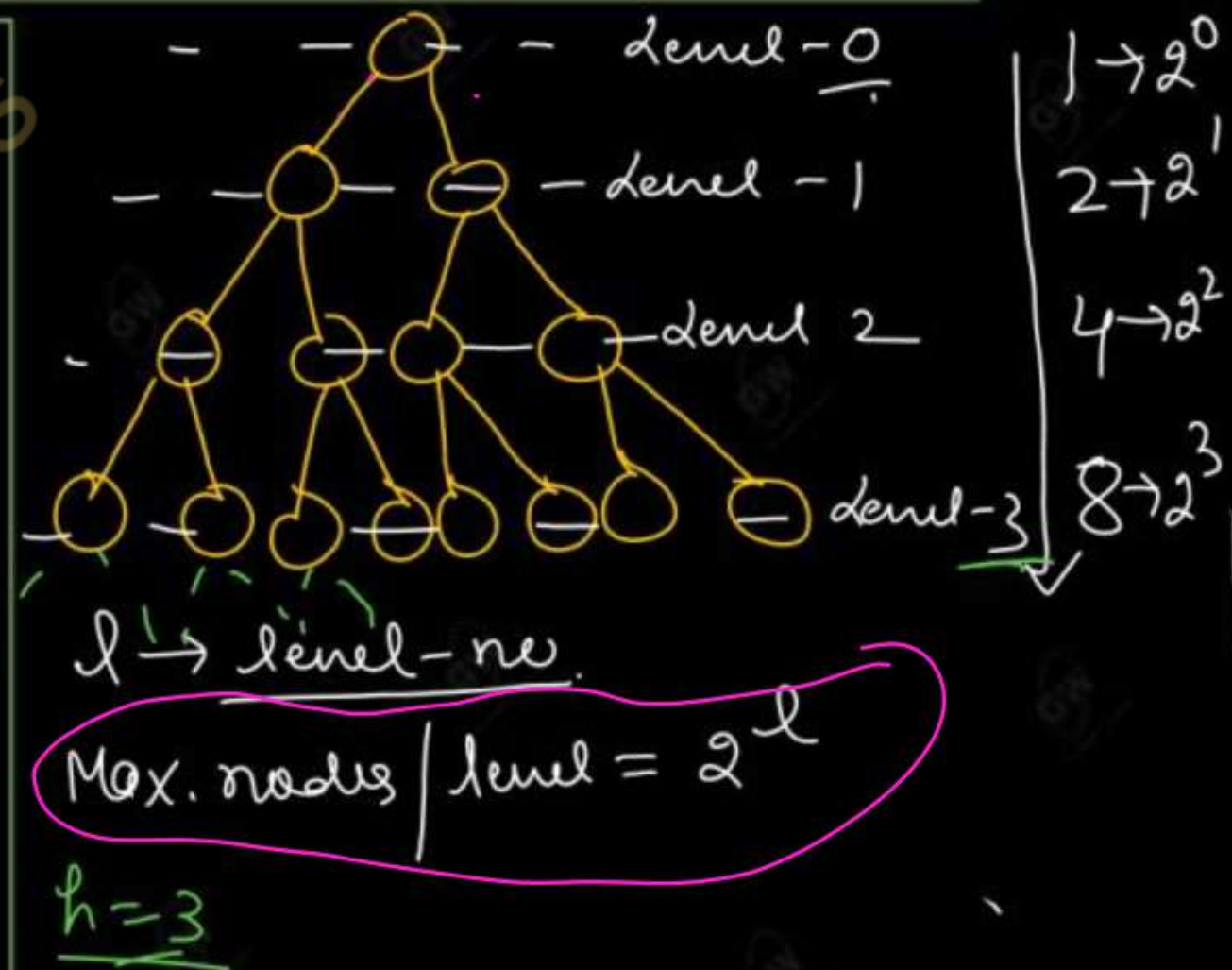
Max Nodes

2. Maximum Nodes in a Binary Tree of Height 'h':

A binary tree of height 'h' can have at most $2^{h+1} - 1$ nodes.

- Height of a tree is the longest path from root to a leaf node.
- A tree with only one root node has height 0 and an empty tree (or root is NULL) has height "-1".
- A tree has the maximum nodes when all levels are completely filled. Summing nodes at each level:

$$\underbrace{1}_{2^0} + \underbrace{2}_{2^1} + \underbrace{4}_{2^2} + \dots + 2^h = 2^{h+1} - 1$$



1. Maximum Nodes at Level 'l':

- A binary tree can have at most 2^l nodes at level l. Here, root is considered at level 0.

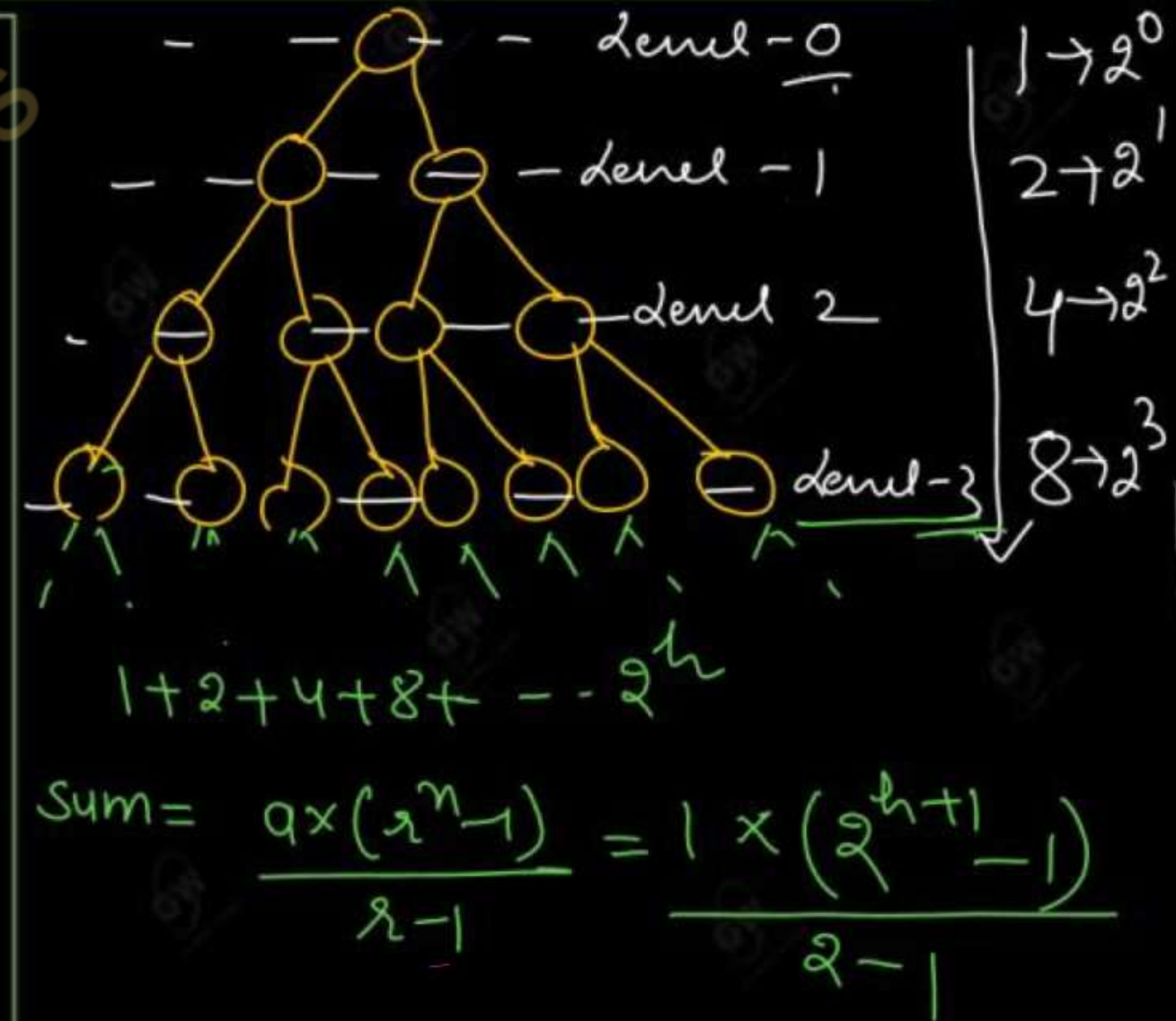
Max Nodes

2. Maximum Nodes in a Binary Tree of Height 'h':

A binary tree of height 'h' can have at most $2^{h+1} - 1$ nodes.

- Height of a tree is the longest path from root to a leaf node.
- A tree with only one root node has height 0 and an empty tree (or root is NULL) has height "-1".
- A tree has the maximum nodes when all levels are completely filled. Summing nodes at each level:

$$1 + 2 + 4 + \dots + 2^h = 2^{h+1} - 1$$



3. Minimum Nodes in a Binary Tree of Height 'h':

$h+1$

4. Maximum height of N Nodes in a Binary Tree:

$N-1$

5. Minimum Height for ' N ' Nodes: $\lceil \log_2 N \rceil$

➤ Explanation:

➤ A binary tree with height h can have at most $2^{h+1} - 1$ nodes.

Rearranging:

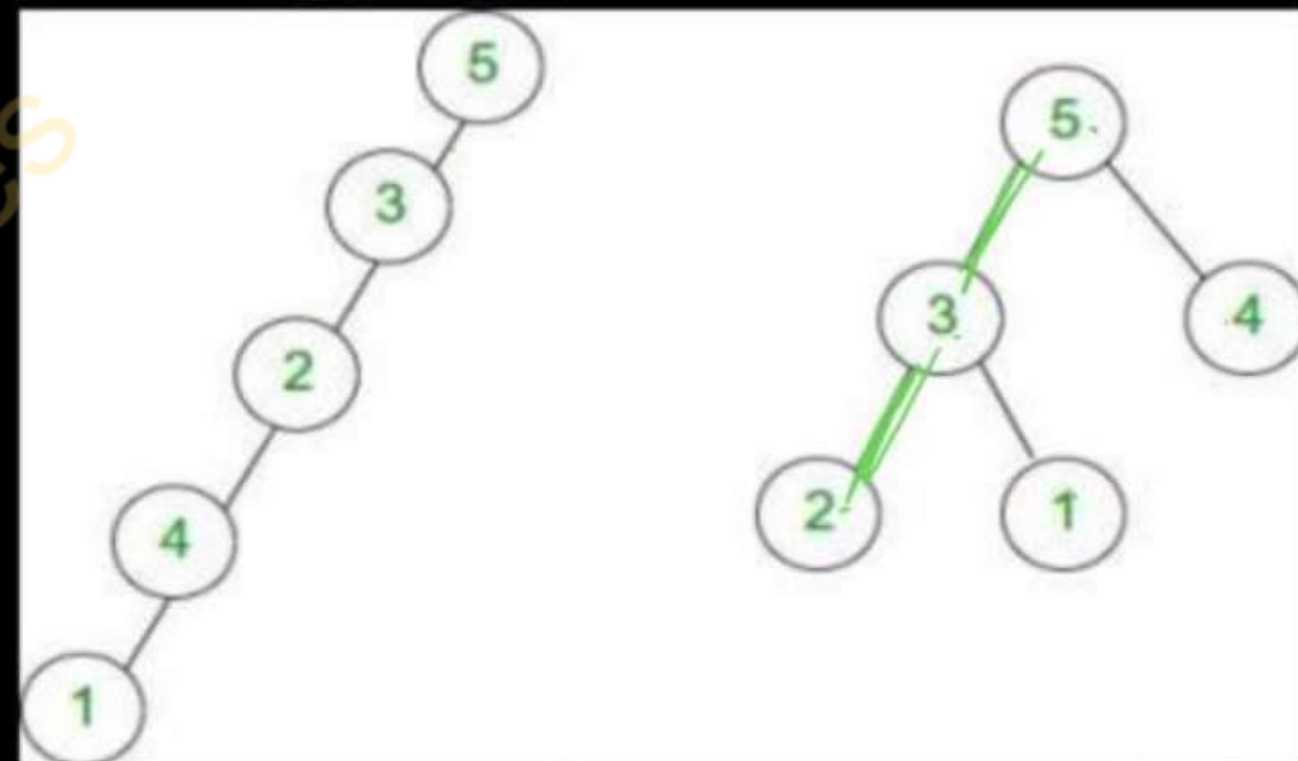
$$N \leq 2^{h+1} - 1$$

$$2^{h+1} \geq N+1$$

$$(h+1) \log_2 2 \geq \log_2 (N+1)$$

$$h \geq \log_2 (N+1) - 1 \text{ (Taking } \log_2 \text{ both sides)}$$

$$h = \lceil \log_2 (N+1) \rceil - 1$$



$$\begin{aligned} & \lceil \log_2 (N+1) \rceil - 1 \\ & \lceil \log_2 (6) \rceil - 1 \\ & = \lceil 2.585 \rceil - 1 \\ & = 3 - 1 = 2 \end{aligned}$$

6. Nodes with Two Children vs. Leaf Nodes:

In a full binary tree (where every node has either 0 or 2 children), the number of leaf nodes (L) is always one more than the internal nodes (T) with two children:

$$L = T + 1$$

7. Total Edges in a Binary Tree: In any non-empty binary tree with n nodes, total number of edges is $n - 1$.

Q: In a complete binary tree if number of nodes is 1000000. What will be height of complete binary tree. (AKTU 2022-23)

$$\begin{aligned} \text{Height} &= \lfloor \log_2 N \rfloor = \lfloor \log_2 1000000 \rfloor = \frac{\log_{10} 1000000}{\log_{10} 2} = \frac{\log_{10} 10^6}{\log_{10} 2} = \frac{6}{0.3010} \\ &\Rightarrow \lfloor 19.63 \rfloor \\ &\Rightarrow \underline{19} \end{aligned}$$

Representing binary trees

1. Array Representation
2. Linked Representation

Array Representation of Binay Tree:

Given a perfect binary tree, we store all nodes in an array according to the **order of level-order traversal, where each node corresponds to a unique array index.**

Mapping formula between the index of a parent node and its children:

Case-1: Array Index Starts with 1

If a node's index is : i

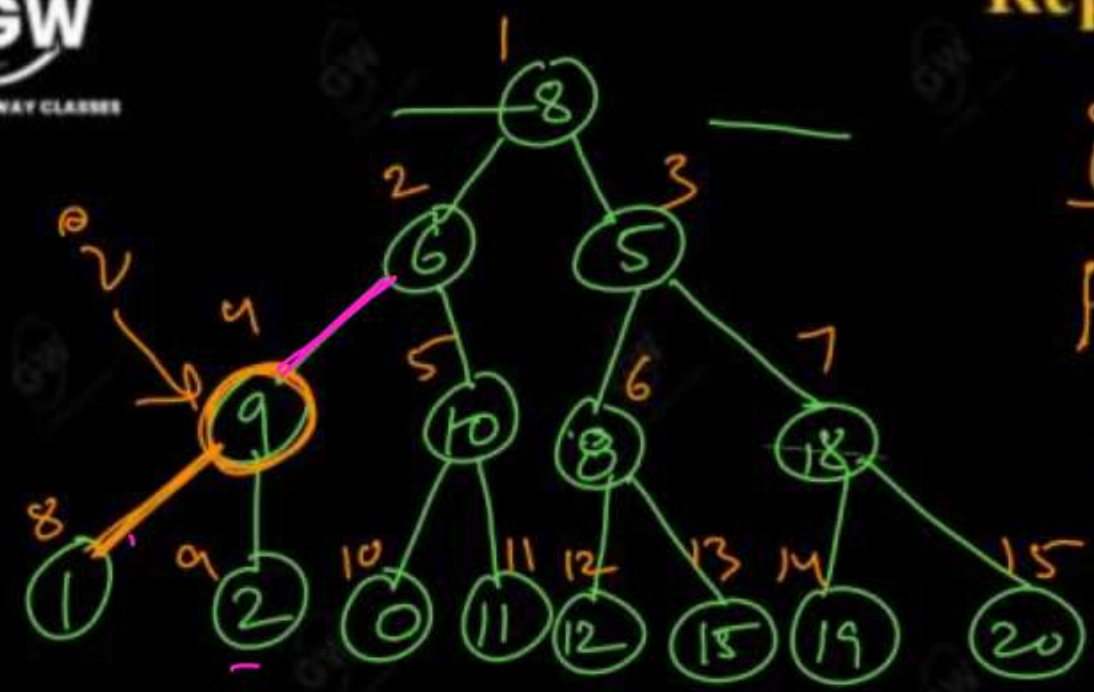
- Parent's index: $\text{floor}(i/2)$
- Index of its left child is: $2i$
- and the right child is: $2i+1$

Case-1: Array Index Starts with 0

If a node's index is : i

- Parent's index: $\text{floor}(i-1)/2$
- Index of its left child is: $2i+1$
- and the right child is: $2i+2$

Representing perfect binary trees



$i = 4$

$parent[i] = \lfloor \frac{i}{2} \rfloor = \lfloor \frac{4}{2} \rfloor = 2$ ✓

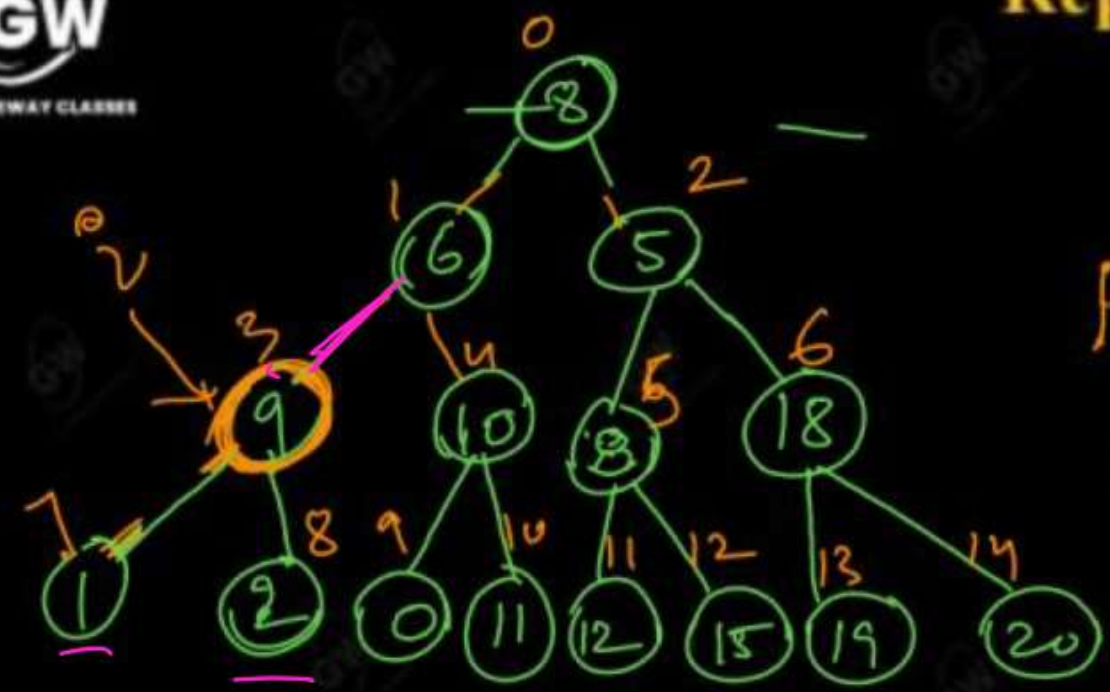
$left[i] = 2 * i = 2 * 4 = 8$

$Right[i] = 2 * i + 1 = 2 * 4 + 1 = 9$

here
Index starts
with 1

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
8	6	5	9	10	8	18	1	2	0	11	12	15	19	20

Representing perfect binary trees



$$i = 3$$

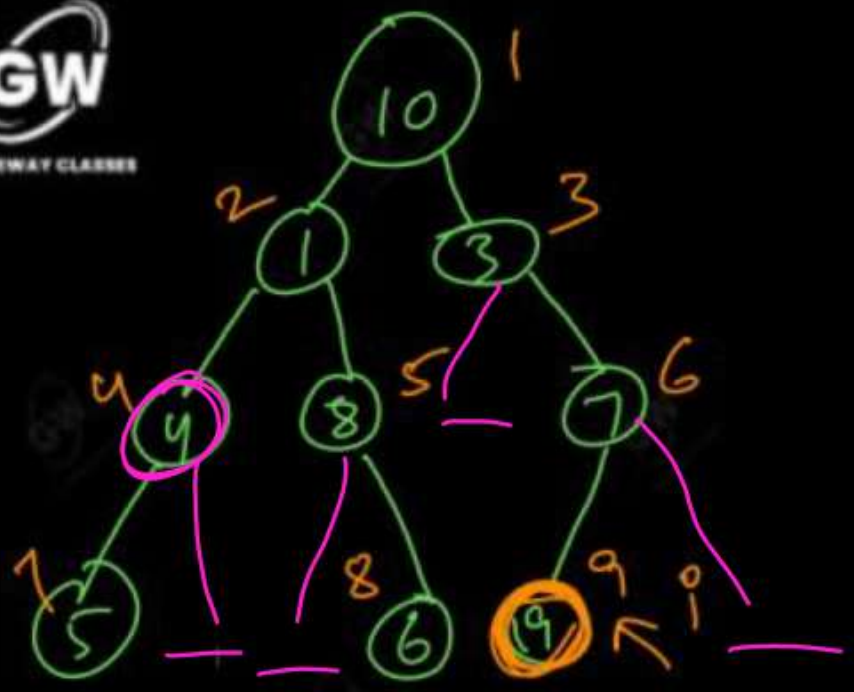
$$\text{parent}[i] = \left\lfloor \frac{(i-1)}{2} \right\rfloor = \left\lfloor \frac{3-1}{2} \right\rfloor = 1 \quad \checkmark$$

$$\text{left}[i] = 2 \times i + 1 \Rightarrow 2 \times 3 + 1 = 7$$

$$\text{Right}[i] = 2 \times i + 2 \Rightarrow 2 \times 3 + 2 = 8$$

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
8	6	5	9	10	8	18	1	2	0	11	12	15	19	20

Index starting from 0



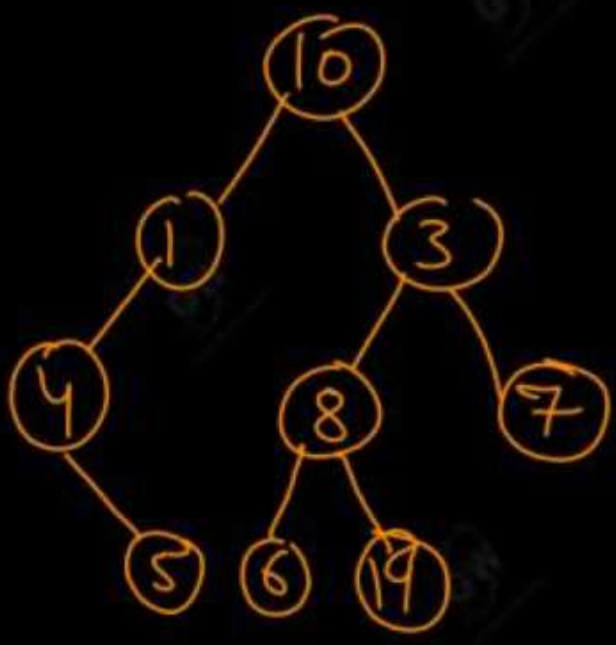
Representing any binary tree

$$i = 9$$

$$\text{parent}[i] = \left\lfloor \frac{i}{2} \right\rfloor$$

$$= \left\lfloor \frac{9}{2} \right\rfloor = 4$$

1	2	3	4	5	6	7	8	9
10	1	3	4	8	7	5	6	19



- Perfect binary trees are a special case;
- There are often many **None** values in middle levels of a binary tree.
- Sequence of level-order traversal does not include these None values, **means multiple binary tree structures can match the same level-order traversal sequence.**

➤ To solve this problem of Level-order traversal sequence corresponding to multiple binary tree possibilities, we explicitly write out all None values in the level-order traversal sequence.



$i = 7$

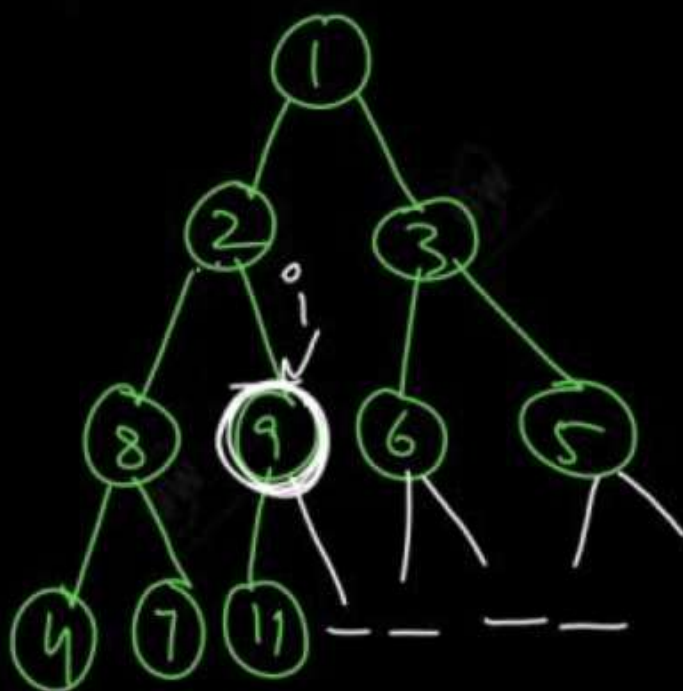
$parent[i] = \lfloor \frac{i}{2} \rfloor = \lfloor \frac{7}{2} \rfloor = 3 \checkmark$

$left[i] = 2 * i = 2 * 7 = 14 \checkmark$

$Right[i] = 2 * i + 1 = 15$

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
10	1	3	4	5	-	7	5			6			9	

- Representing complete binary tree are suitable for array representation.
- In a complete binary tree, None appears only at the bottom level and towards the right, meaning all None values definitely appear at the end of the level-order traversal sequence.
- So, using array to represent a complete binary tree, it's possible to omit storing all None values, which is very convenient.



1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	2	3	8	9	6	5	4	7	11	-	-	-	-	-

$$i = 5$$

$$\text{parent}[i] = \left\lfloor \frac{i}{2} \right\rfloor = \left\lfloor \frac{5}{2} \right\rfloor = 2$$

$$\text{left}[i] = 2 * i = 10$$

$$\text{Right}[i] = 2 * i + 1 = 11$$

Advantages and limitations of Array Representation of Binary tree

Advantages:

- Arrays are stored in contiguous memory spaces, which is cache-friendly and allows for faster access and traversal.
- It does not require storing pointers, which saves space.
- It allows random access to nodes.

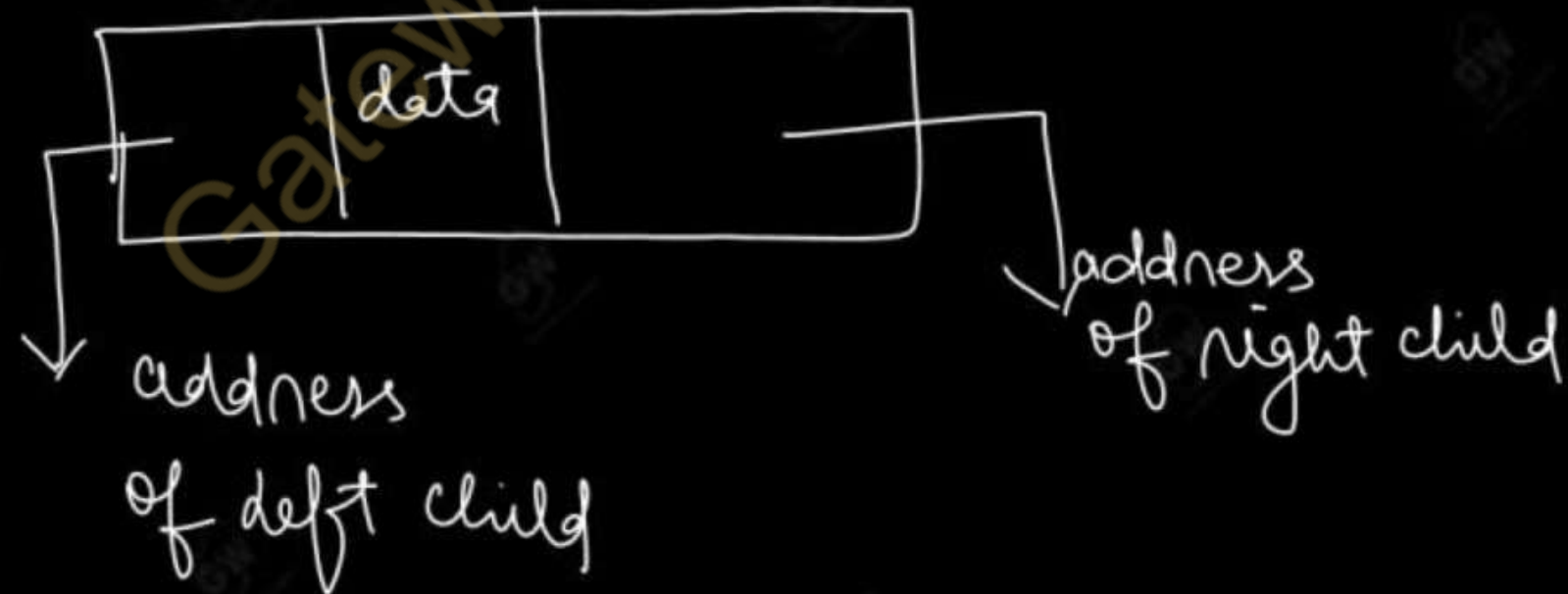
Limitations:

- Array storage requires contiguous memory space, so it is not suitable for storing trees with a large amount of data.
- Adding or deleting nodes requires array insertion and deletion operations, which are less efficient.
- When there are many, **None values** in the binary tree, the proportion of node data contained in the array is low, leading to lower space utilization.

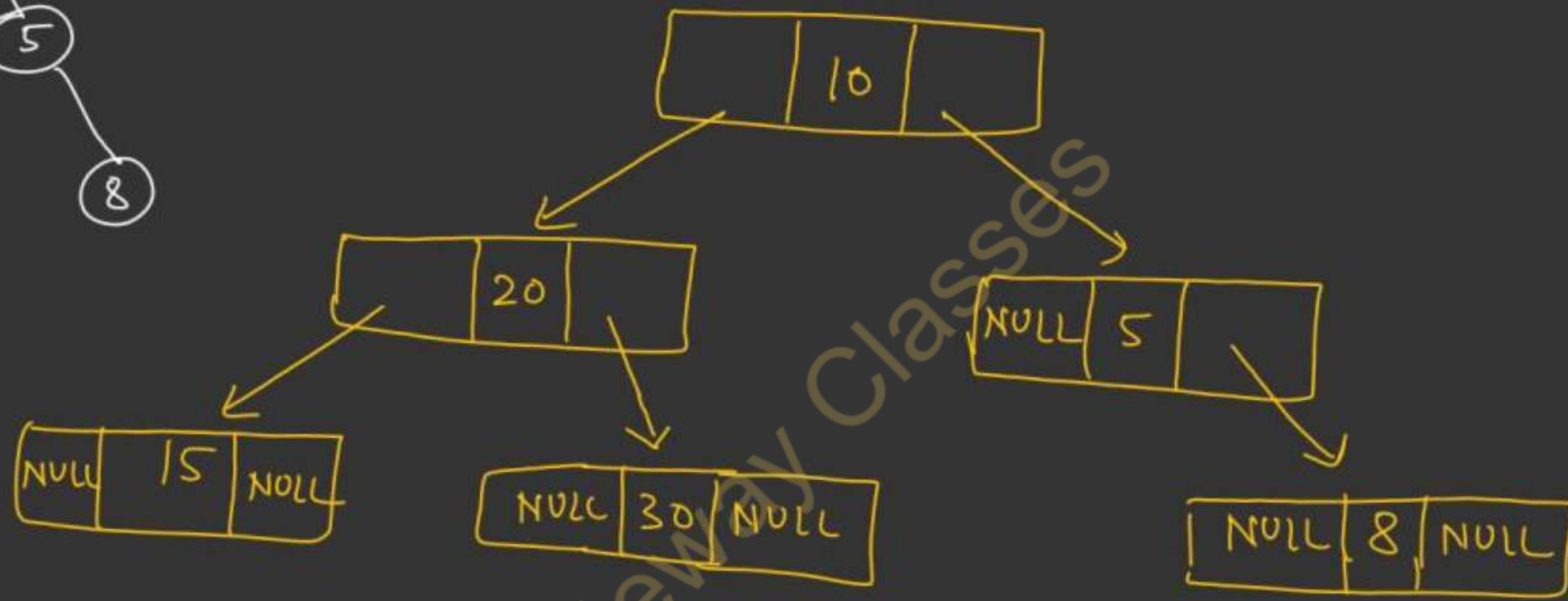
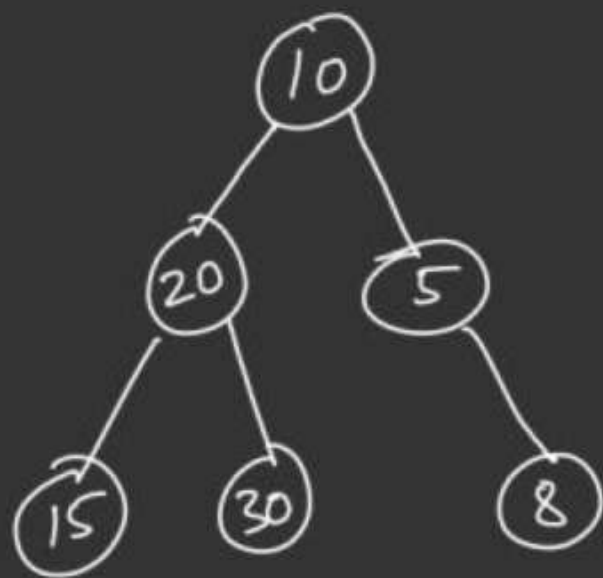
Linked Representation of Binary Tree:

- In linked and dynamic representation, the linked list data structure is used.
- Each node constitutes of a data part and two link parts. The two link parts store address of left and right child nodes.
- Data part is used to store the information about the binary tree element.
- This is a better representation as nodes can be added or deleted at any location.
- Memory utilization is better in this binary tree representation.

Node Structure



linked Representation



Advantages and limitations of Linked Representation of Binary tree

Advantages:

- It can easily grow or shrink as needed, so it uses only the memory it needs.
- Adding or removing nodes is straightforward and requires only pointer adjustments.
- Only uses memory for the nodes that exist, making it efficient for sparse trees.

Disadvantages:

- Needs extra memory for pointers.
- Finding a node can take longer because you have to start from the root and follow pointers.

1. Define extended binary tree, full binary tree, strictly binary tree and complete binary tree. (AKTU 2019-20)
2. What is binary tree? Explain. (AKTU 2017-18)
3. Define complete binary tree. Give example. (AKTU 2016-17)
4. In a complete binary tree if the number of nodes is 1000000. What will be the height of complete binary tree. (AKTU 2022-23)



AKTU

B.Tech III-Sem



GATEWAY CLASSES

CS IT & CS Allied

DATA STRUCTURE

Unit-4 Lecture-2

Today's Target

- Tree Traversals
 - Pre-order/In-order/Post-order
- Constructing Binary tree from given traversals
- AKTU PYQs
- DPP



By Dr. Nidhi Parashar Ma'am

Ph.D (CSE)

- M.Tech Gold Medalist
- Net Qualified

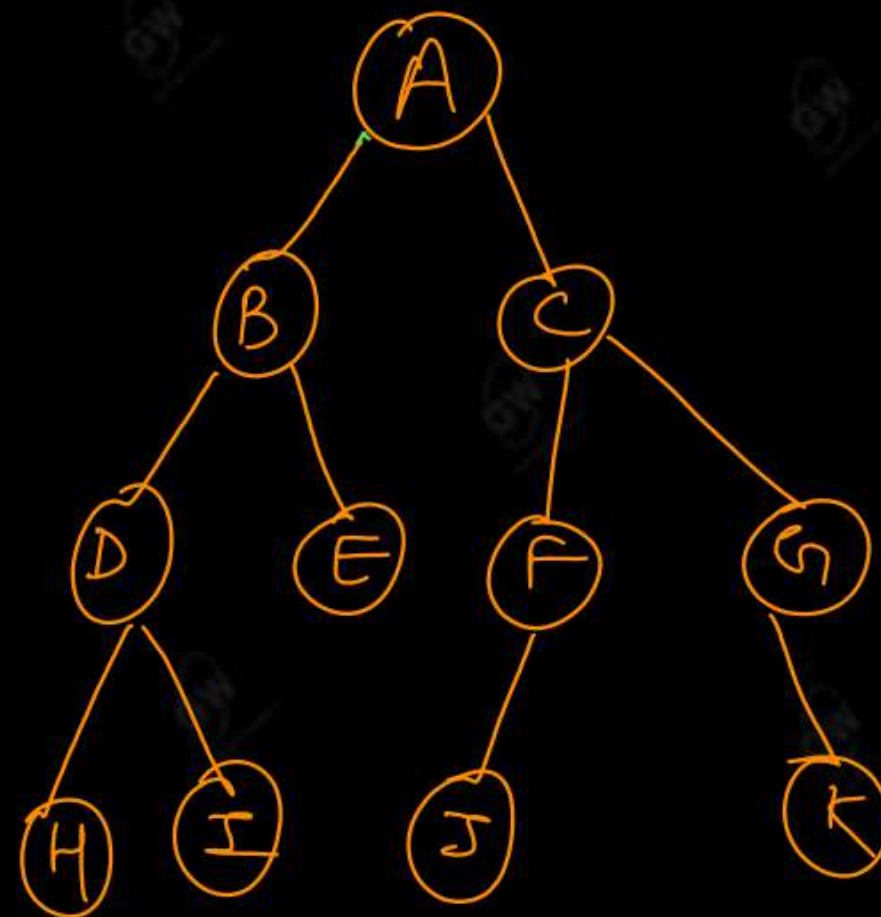
Binary Tree Traversals

- When we wanted to display a binary tree, we need to follow some order in which all the nodes of that binary tree must be displayed. In any binary tree, displaying order of nodes depends on the traversal method.

Displaying (or) visiting order of nodes in a binary tree is called as Binary Tree Traversal.

There are three types of binary tree traversals.

1. In - Order Traversal
2. Pre - Order Traversal
3. Post - Order Traversal



1. In - Order Traversal (leftChild - root - rightChild)

- In this traversal, the left child node is visited first, then the root node is visited and later we go for visiting the right child node. This is applicable for every root node of all subtrees in the tree.
- This is performed recursively for all nodes in the tree.

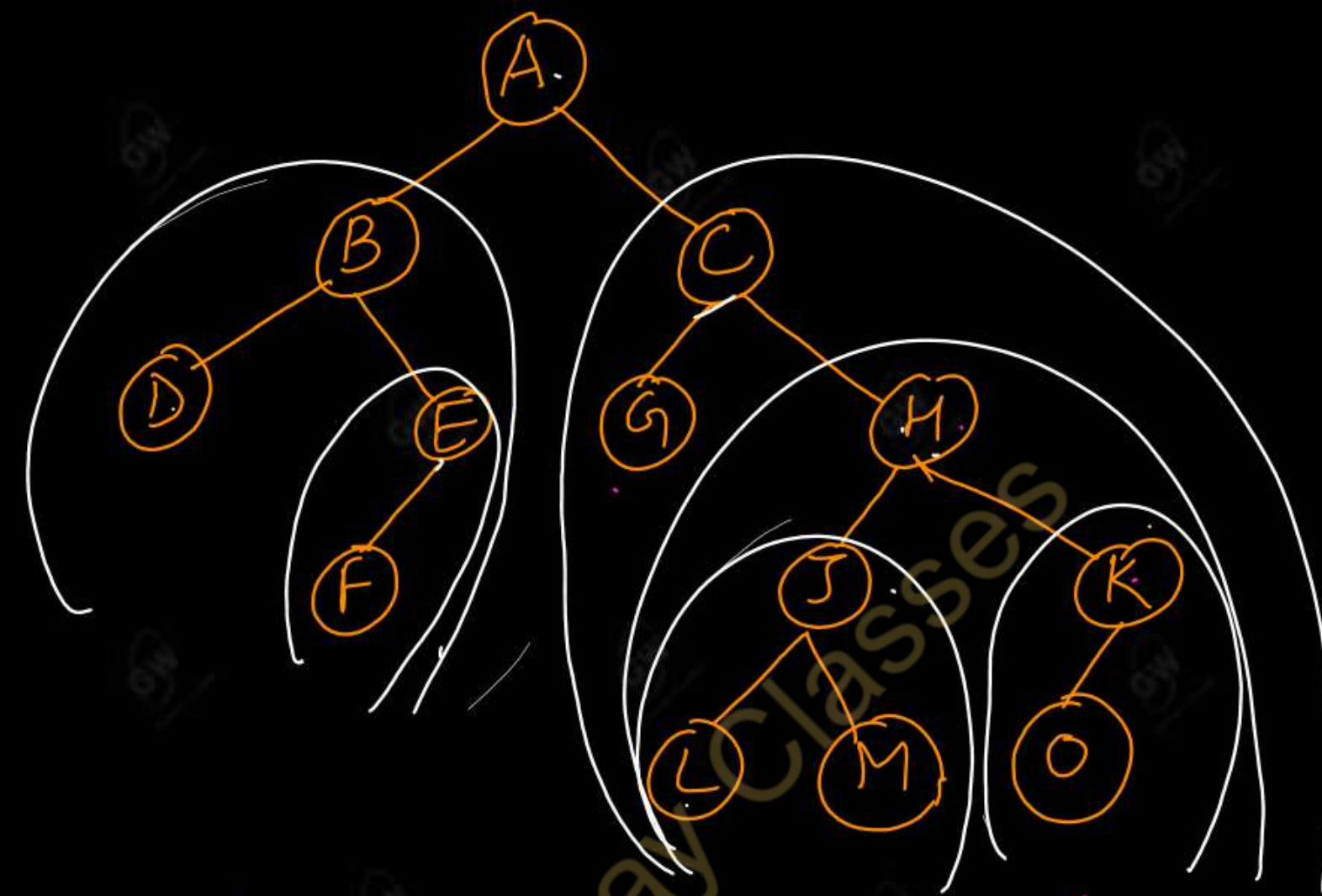


Left → Root → Right

I D J B F A G K C H ✓

In-Order Traversal for above example of binary tree is: I - D - J - B - F - A - G - K - C - H

Example

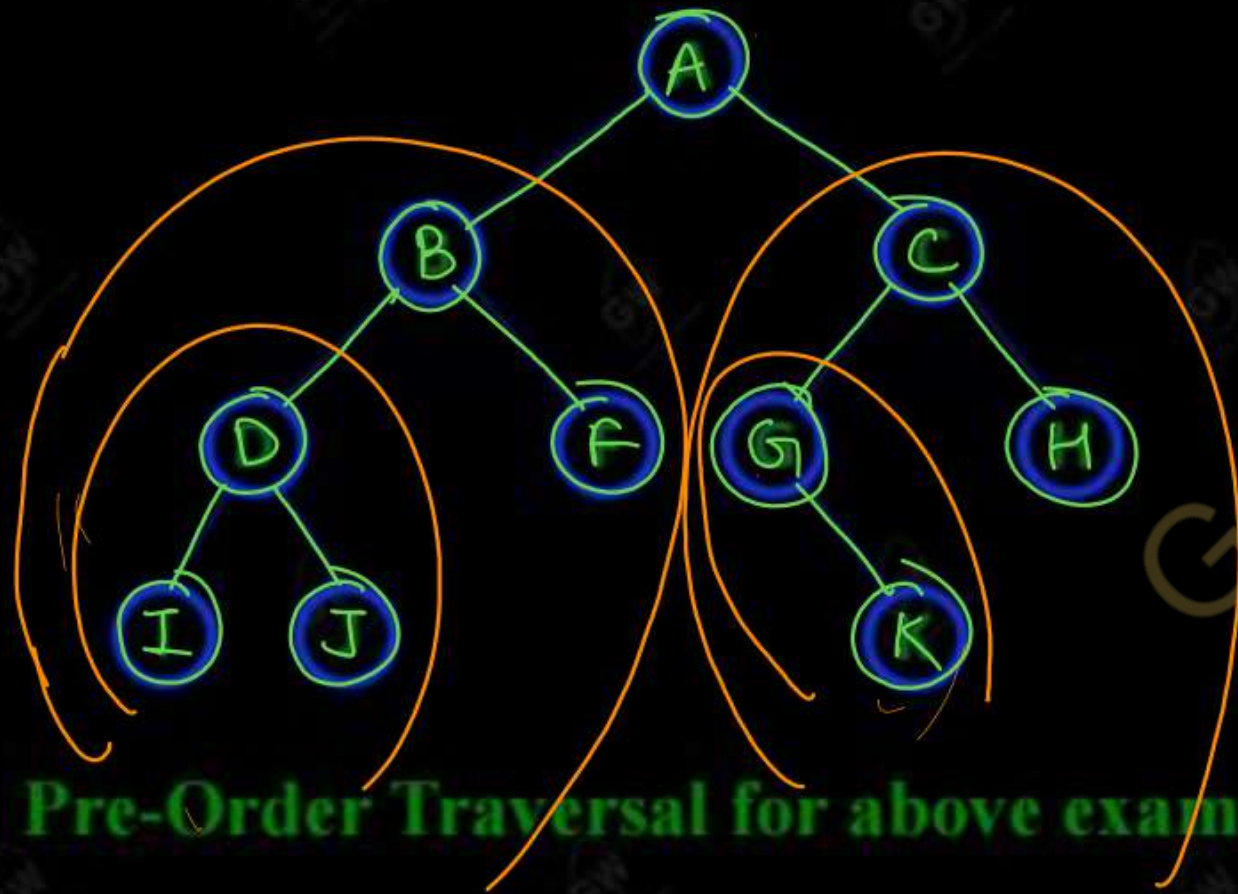


In-order Traversal -

D B F E A G C L I M H O K

2. Pre - Order Traversal (root - leftChild - rightChild)

- In Pre-Order traversal, the root node is visited before the left child and right child nodes.
- In this traversal, the root node is visited first, then its left child and later its right child. This pre-order traversal is applicable for every root node of all subtrees in the tree.
- This is performed recursively for all nodes in the tree.



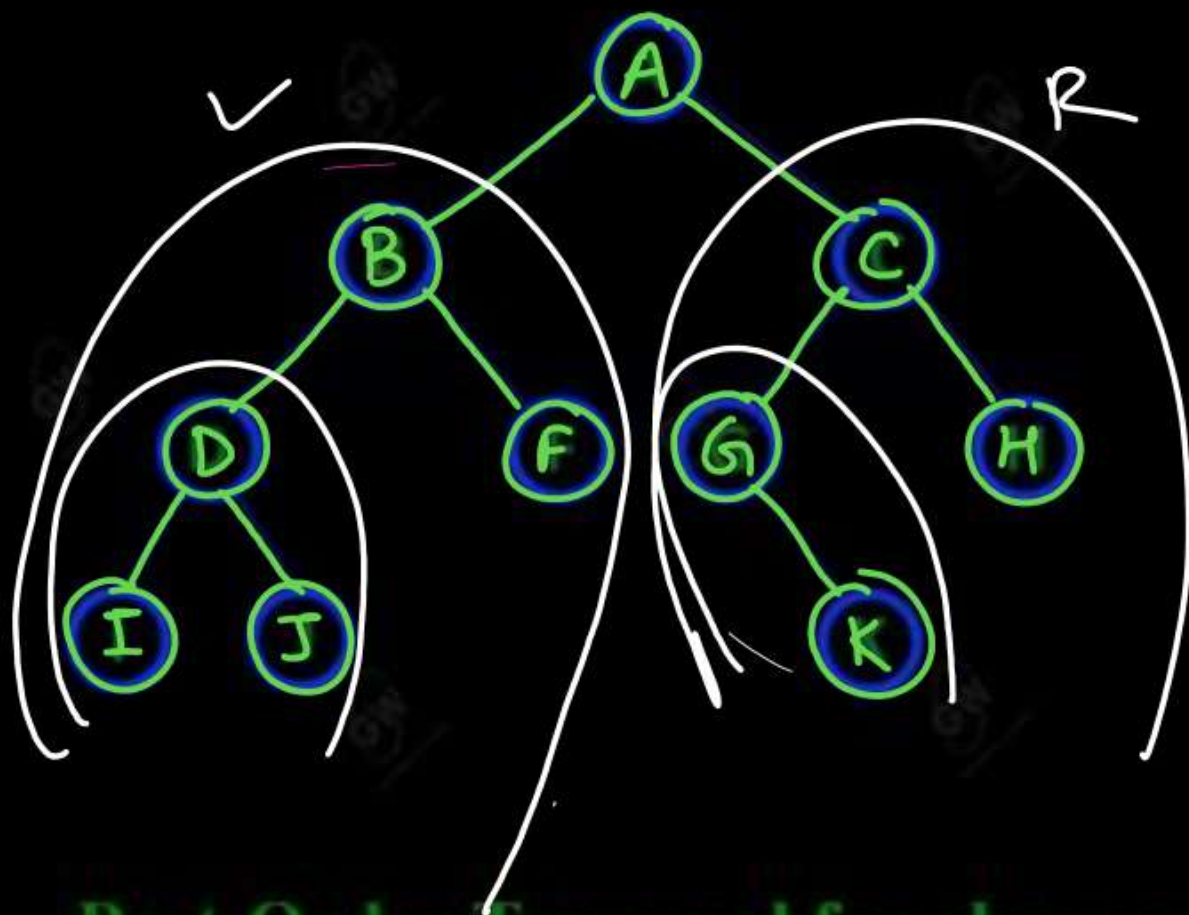
Root → Left → Right

A B D I J F C G K H

Pre-Order Traversal for above example binary tree is: A - B - D - I - J - F - C - G - K - H

3. Post- Order Traversal (leftChild - rightChild - root)

- In Post-Order traversal, the root node is visited after left child and right child.
- In this traversal, left child node is visited first, then its right child and then its root node.
- This is recursively performed until the right most node is visited.



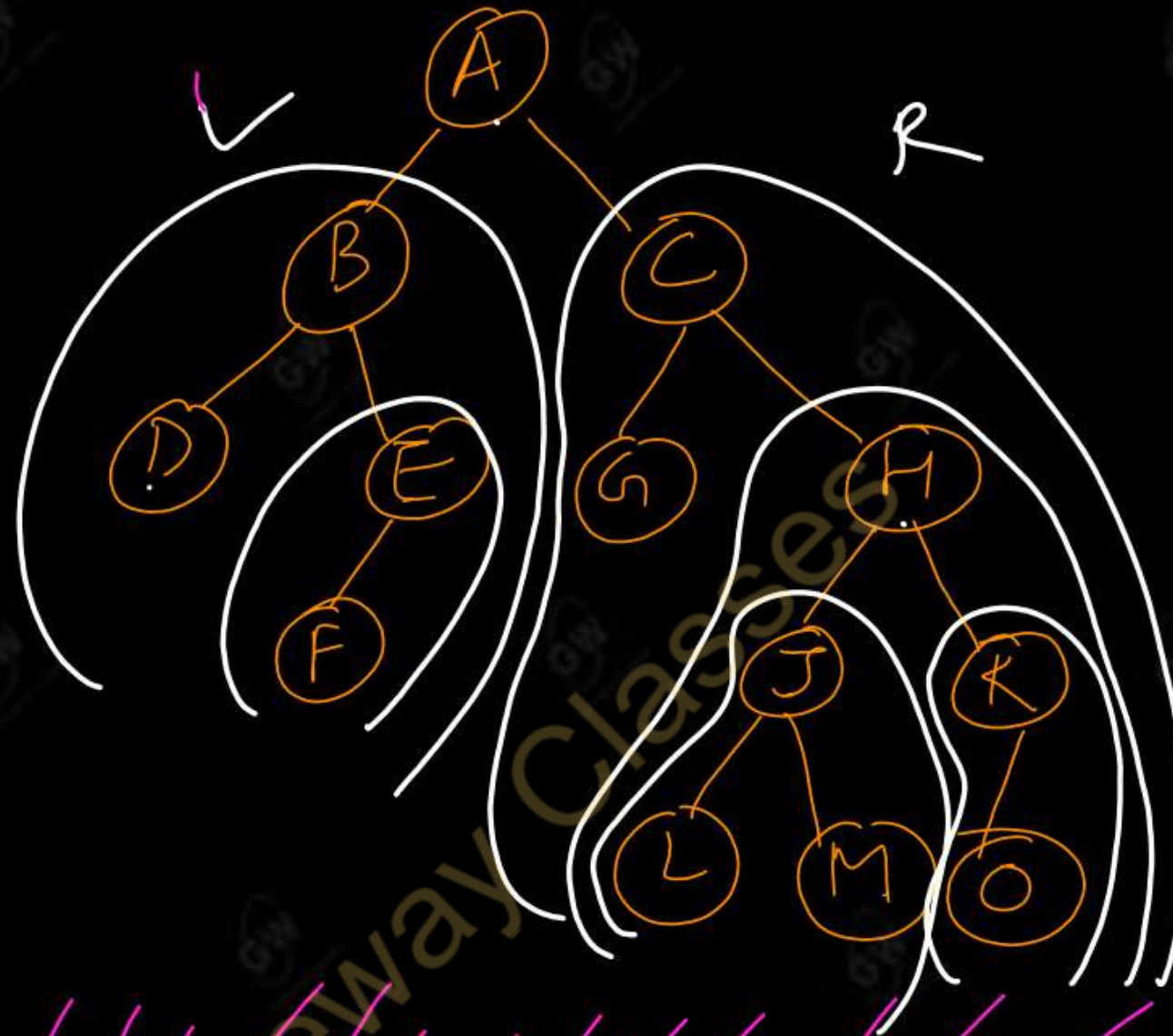
Left → Right → Root

I J D F B K G H C A

Post-Order Traversal for above example binary tree is: I - J - D - F - B - K - G - H - C - A

Example

Left → Right → Root

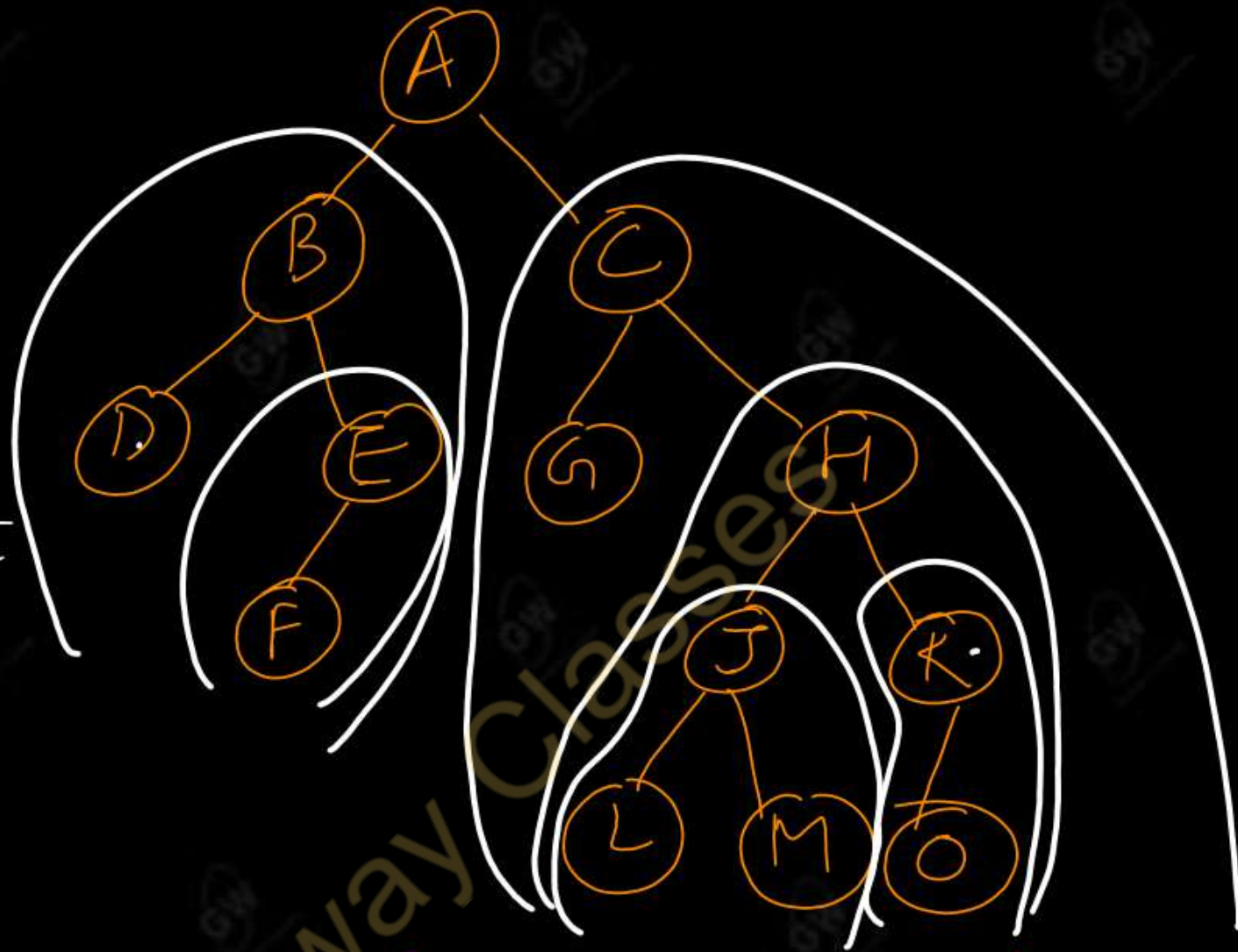


Post-order Traversal:-

✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓ ✓
D F E B G L M J O K H C A

Example

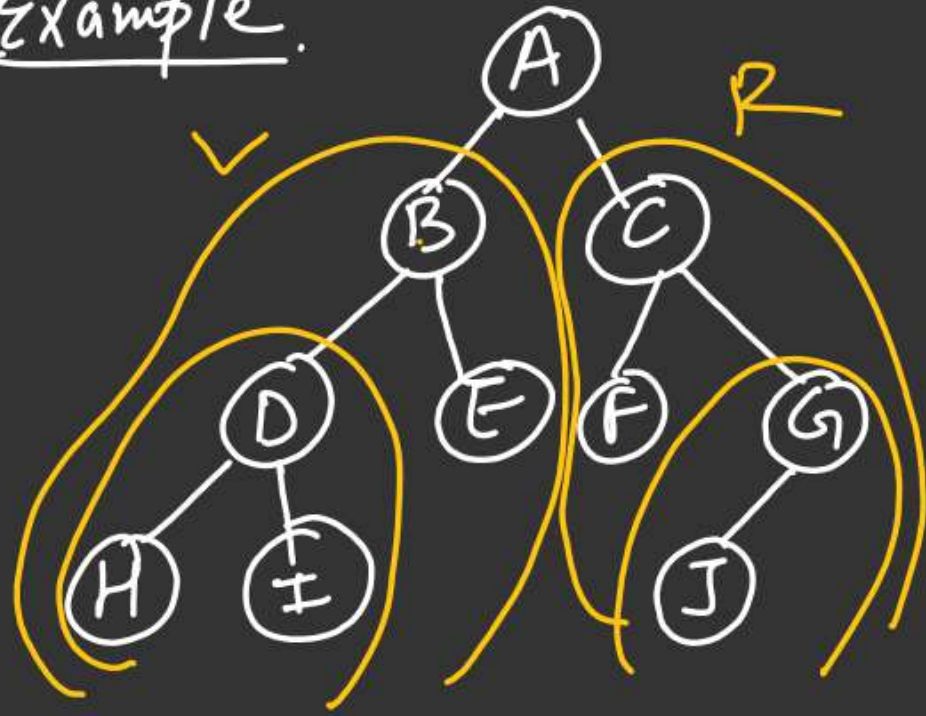
Root $\rightarrow$ Left $\rightarrow$ Right



Pre-order Traversal

A B D E F C G H J L M K O

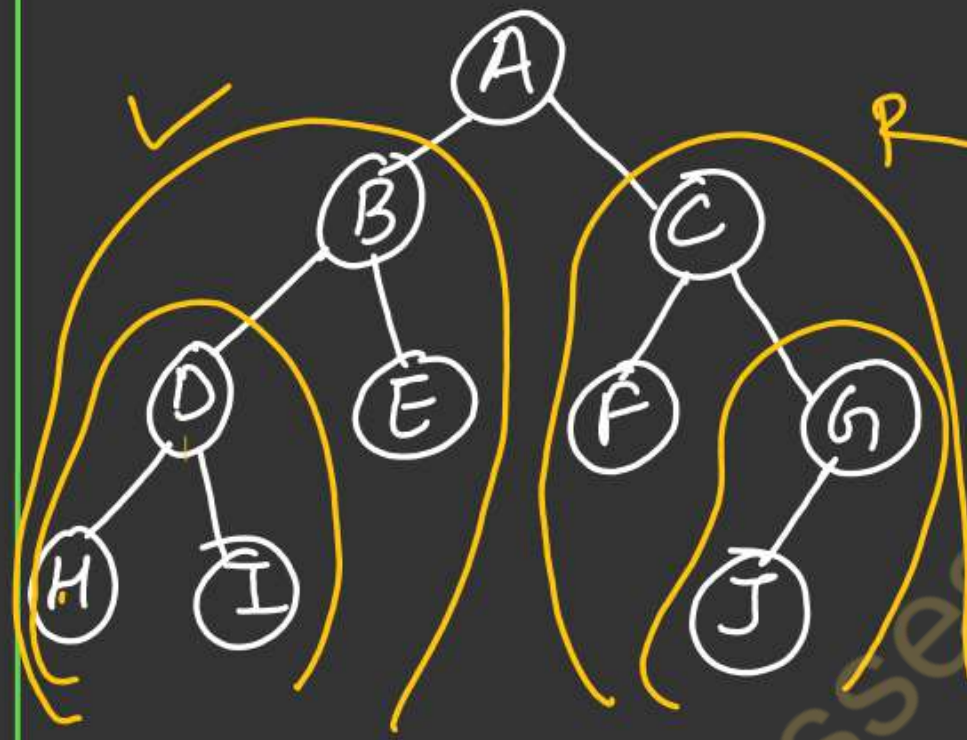
Example.



PRE-ORDER

Root → left → Right

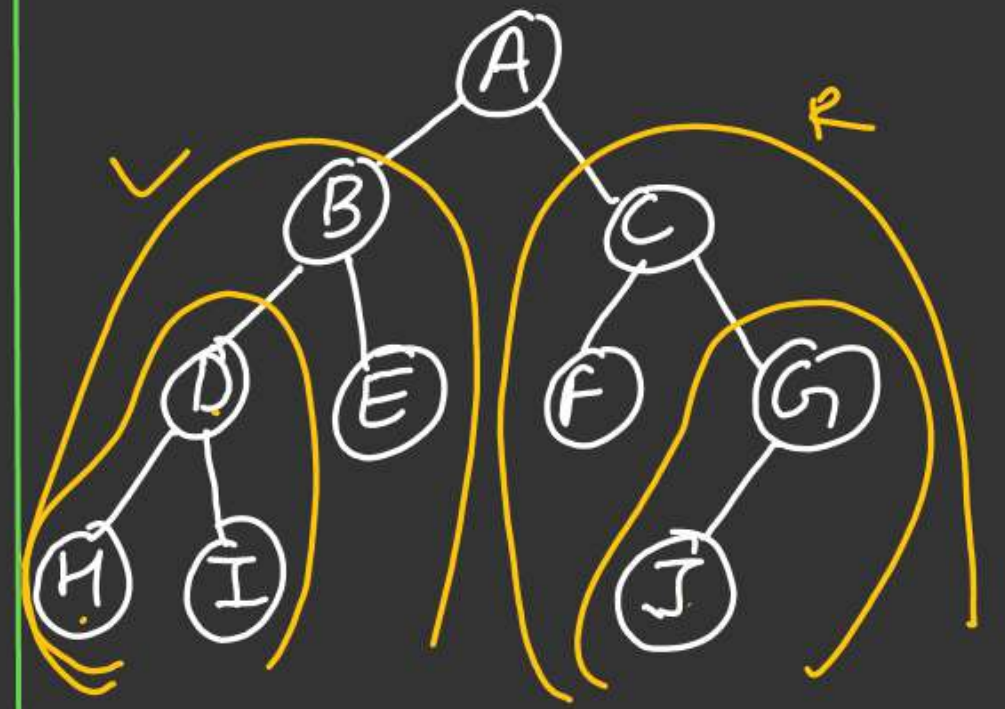
↑ Root
✓ A B D H I E C F G J



IN-ORDER

left → Root → Right

H I D B E A F C J G



POST-ORDER

left → Right → Root

H I D E B F J G C A

Q:Construct a binary tree with the following traversals: (AKTU 2024-25)

In-order: B C A E G D H F I J

Pre-order: A B C D E G F H I J

- The idea is to start with the root node, which would be the first item in the preorder sequence, and find the boundary of its left and right subtree in the inorder sequence.
- To find the boundary, search for the index of the root node in the inorder sequence.
- All keys before the root node in the inorder sequence become part of the left subtree, and all keys after the root node become part of the right subtree.
- Repeat this recursively for all nodes in the tree and construct the tree in the process.

Q: Construct a binary tree with the following traversals: (AKTU 2024-25)

In-order: B C A E G D H F I J

Pre-order: A B C D E G F H I J

(I)

Pre-order

A B C D E G F H I J

Root of the binary tree

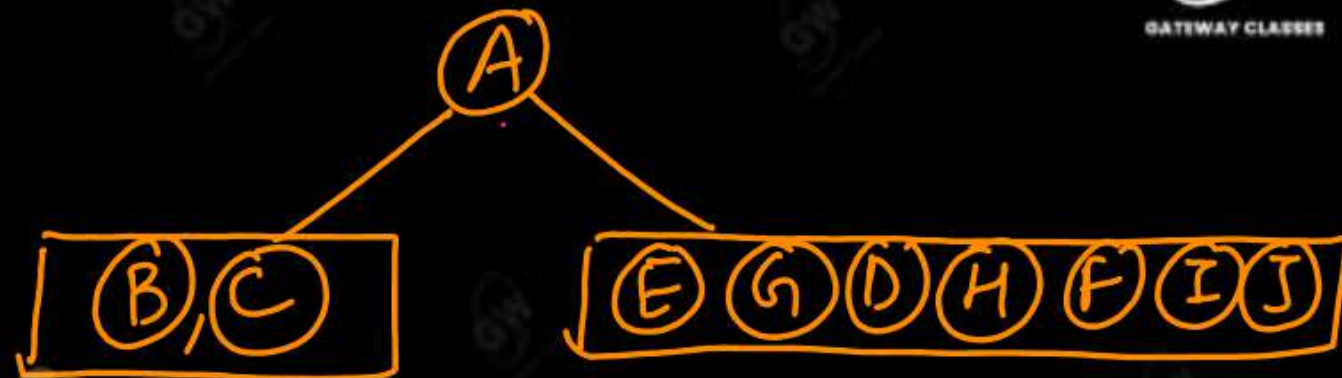
Inorder

B C A E G D H F I J

Elements
in the left subtree
rooted at A

Elements in the right
subtree rooted at A

locate A in the In-order Traversal



Q: Construct a binary tree with the following traversals: (AKTU 2024-25)

In-order: B C A E G D H F I J

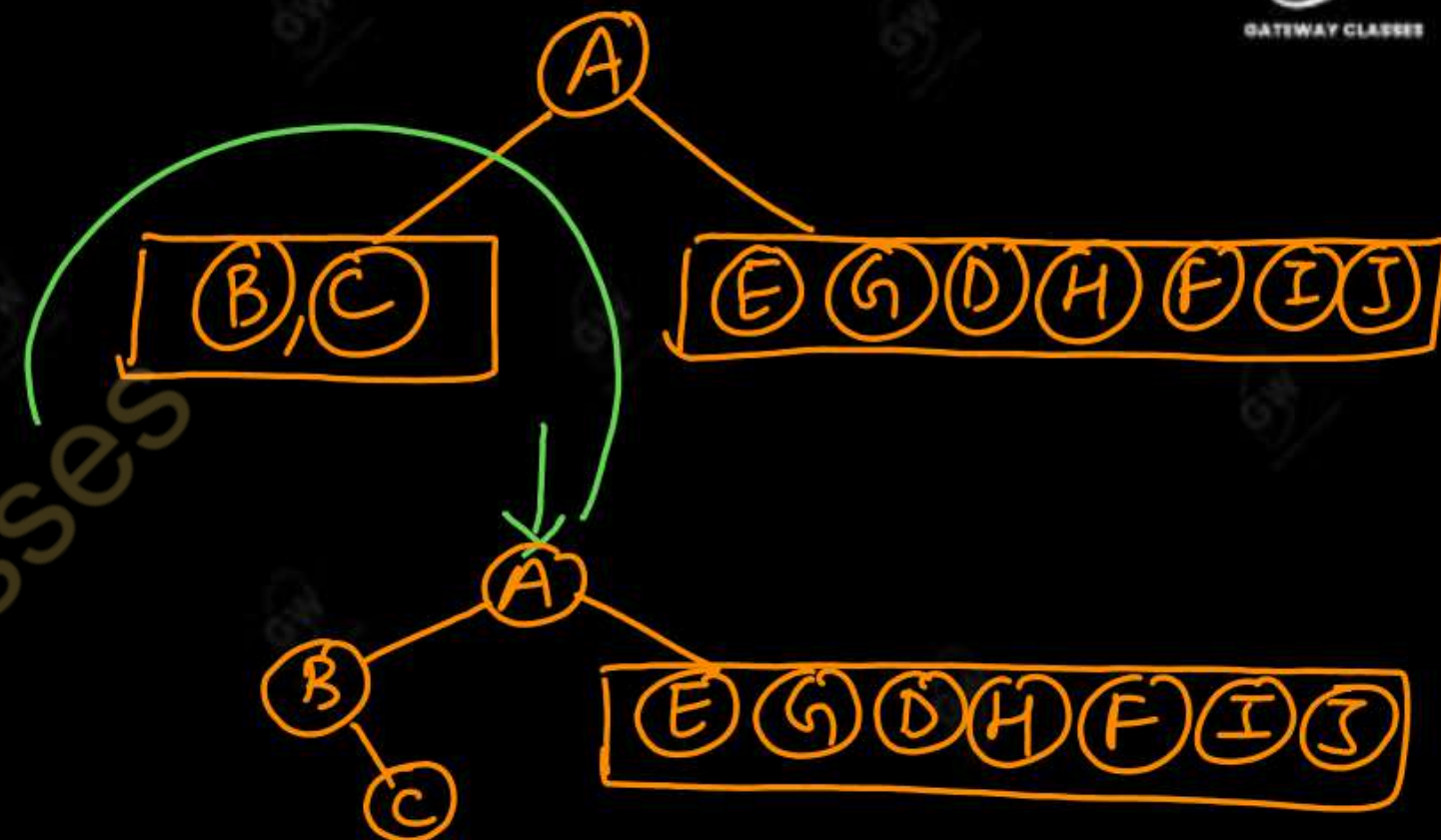
Pre-order: A B C D E G F H I J

(II)

Pre-order: A B C D E G F H I J

R-L

Inorder - B C A E G D H F I J



- Between (B) and (C), which will become root of the subtree?
- check out the pre-order sequence and find the nodes that comes first from L → R ⇒ (B) comes first.
- Now locate (B) in in-order seq.

Q: Construct a binary tree with the following traversals: (AKTU 2024-25)

In-order: B C A E G D H F I J

Pre-order: A B C D E G F H I J

(III)

Pre-order:

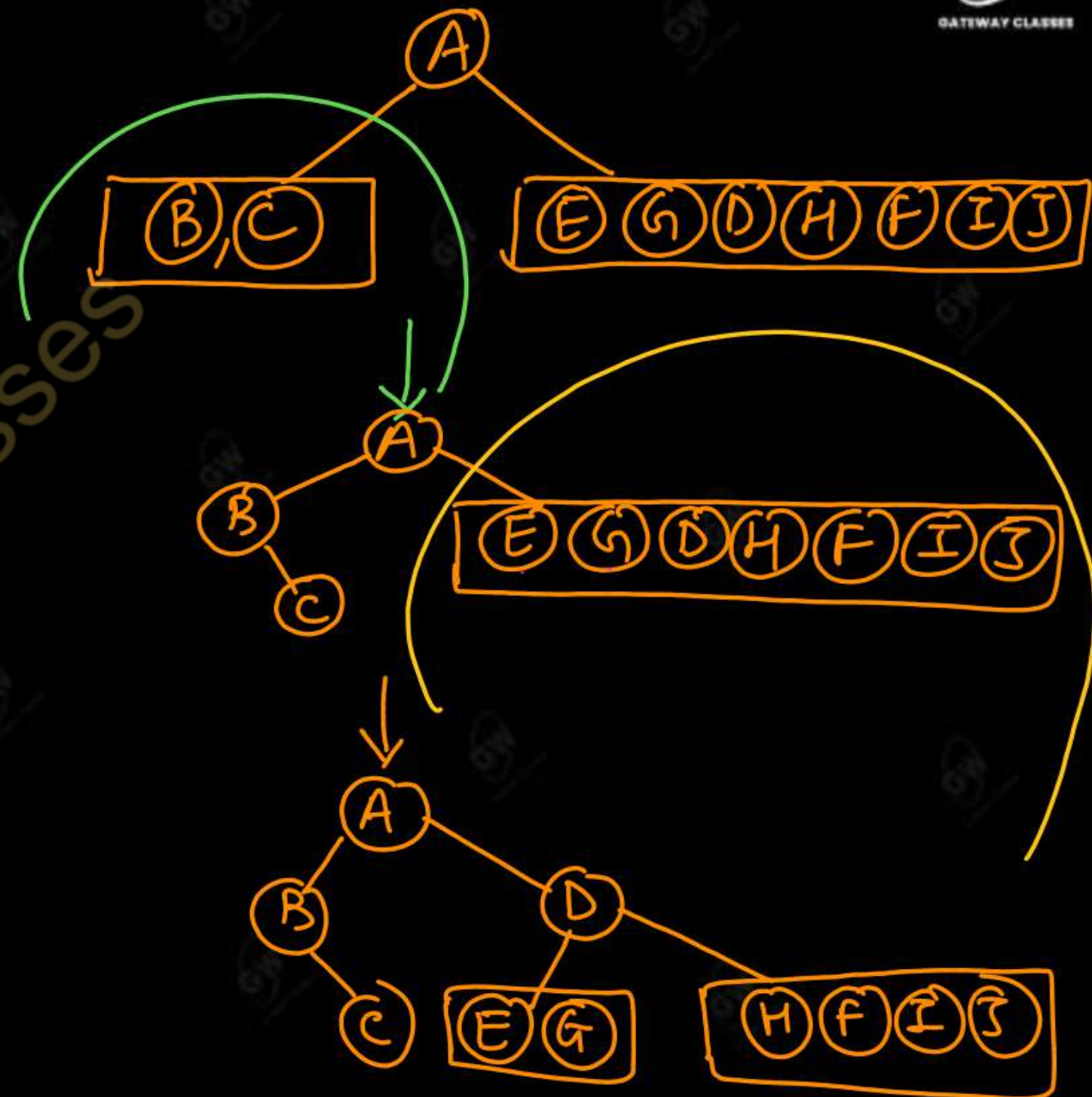
A	B	C	D	E	G	F	H	I	J
---	---	---	---	---	---	---	---	---	---

L → R

Inorder -

B	C	A	E	G	D	H	F	I	J
---	---	---	---	---	---	---	---	---	---

- Now in Pre-order sequence (D) comes first so it will be the root of the sub-tree
- Now locate (D) in In-order sequence to find elements in left and right subtrees rooted at (D).



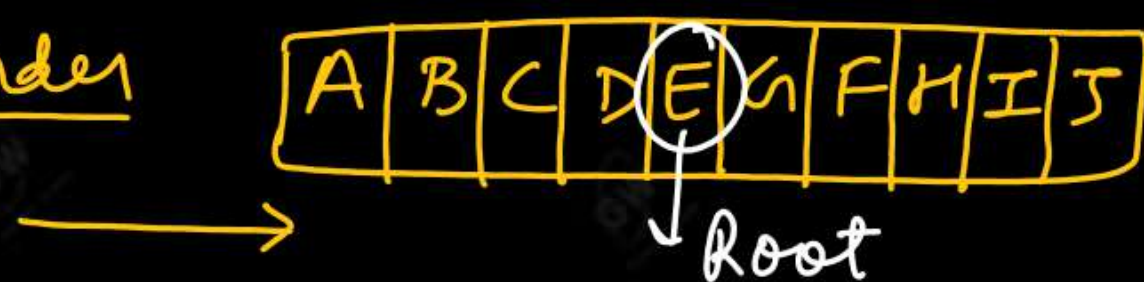
Q: Construct a binary tree with the following traversals: (AKTU 2024-25)

In-order: B C A E G D H F I J

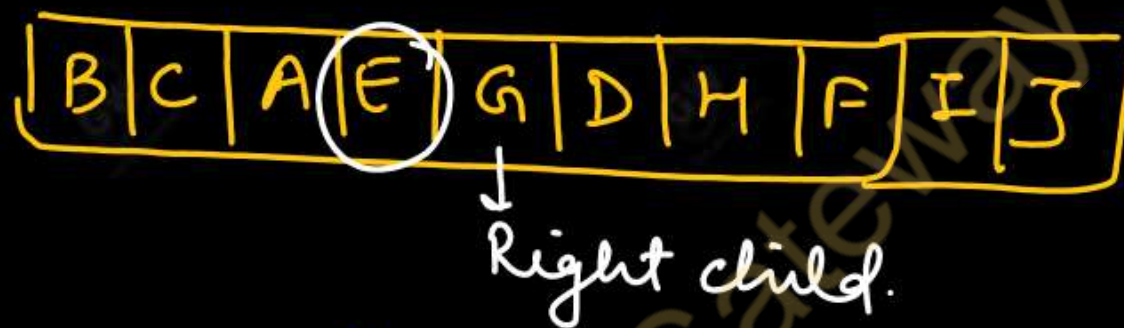
Pre-order: A B C D E G F H I J

(IV)

Pre-order



Inorder

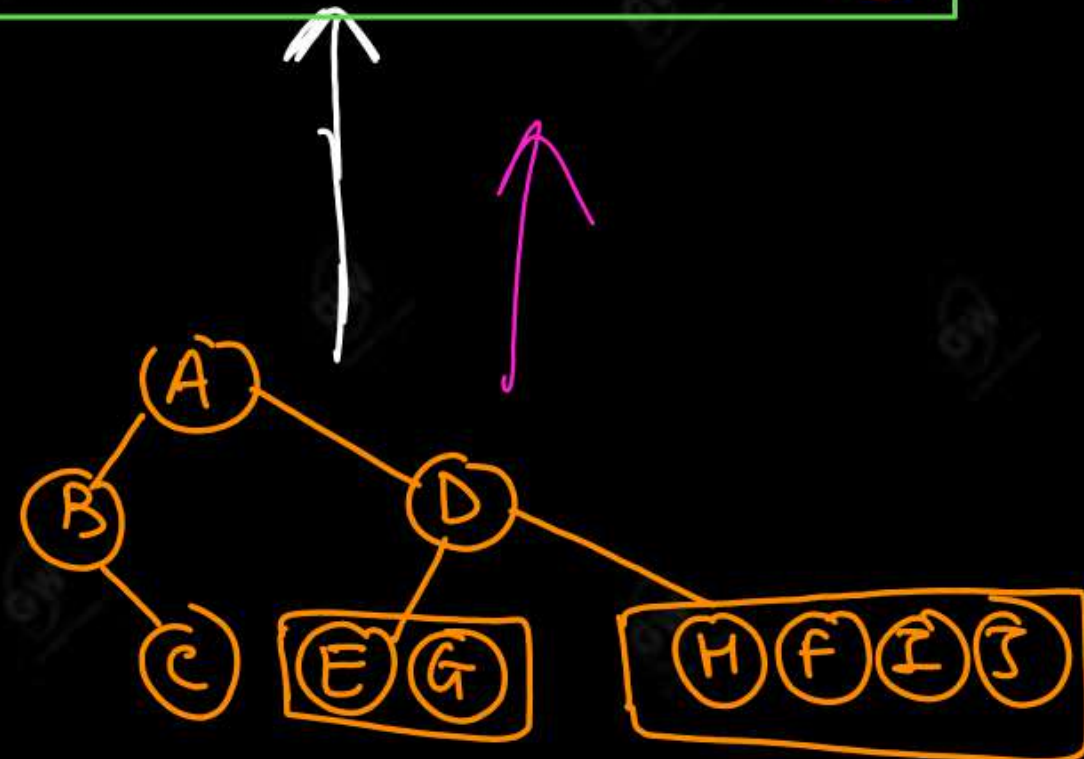
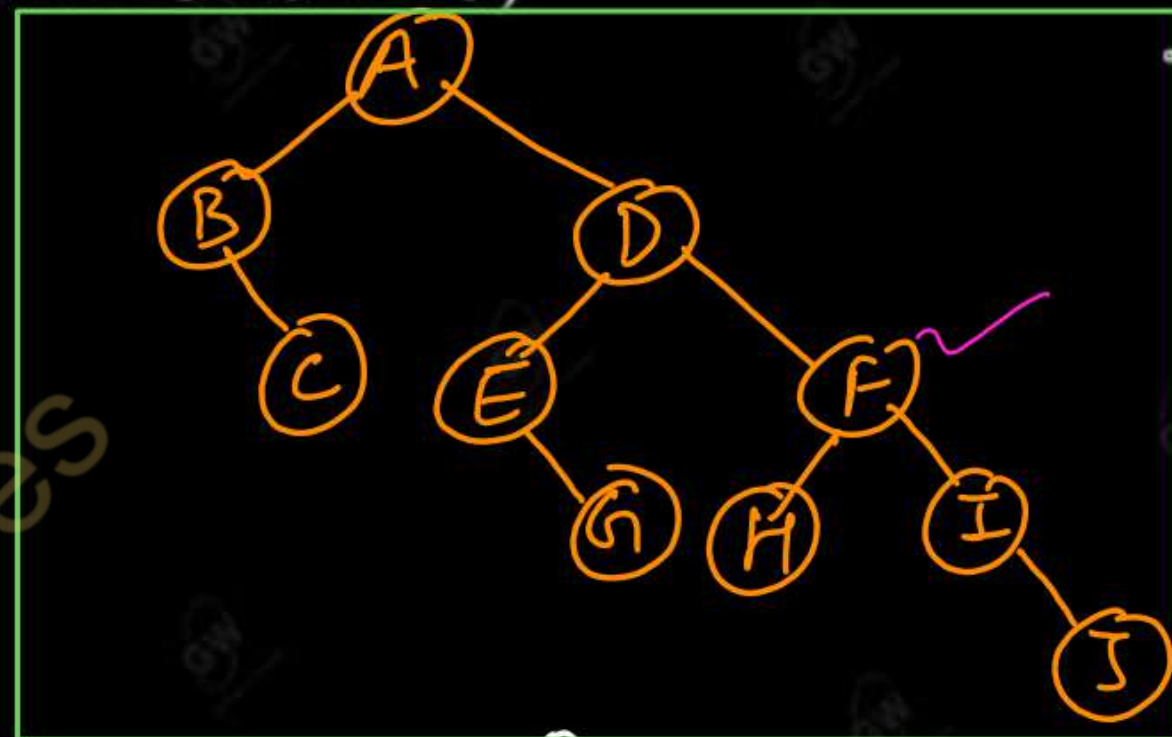
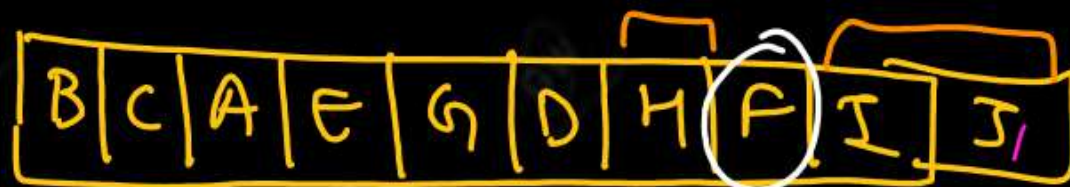


(V)

Pre-order



Inorder



Q: The order of nodes of a binary tree in inorder and postorder traversal are as follows:

In order : B, I, D, A, C, G, E, H, F.

Post order: I, D, B, G, C, H, F, E, A.

(i) Draw the corresponding binary tree.

(ii) Write pre order traversal of same tree. (AKTU 2022-23)

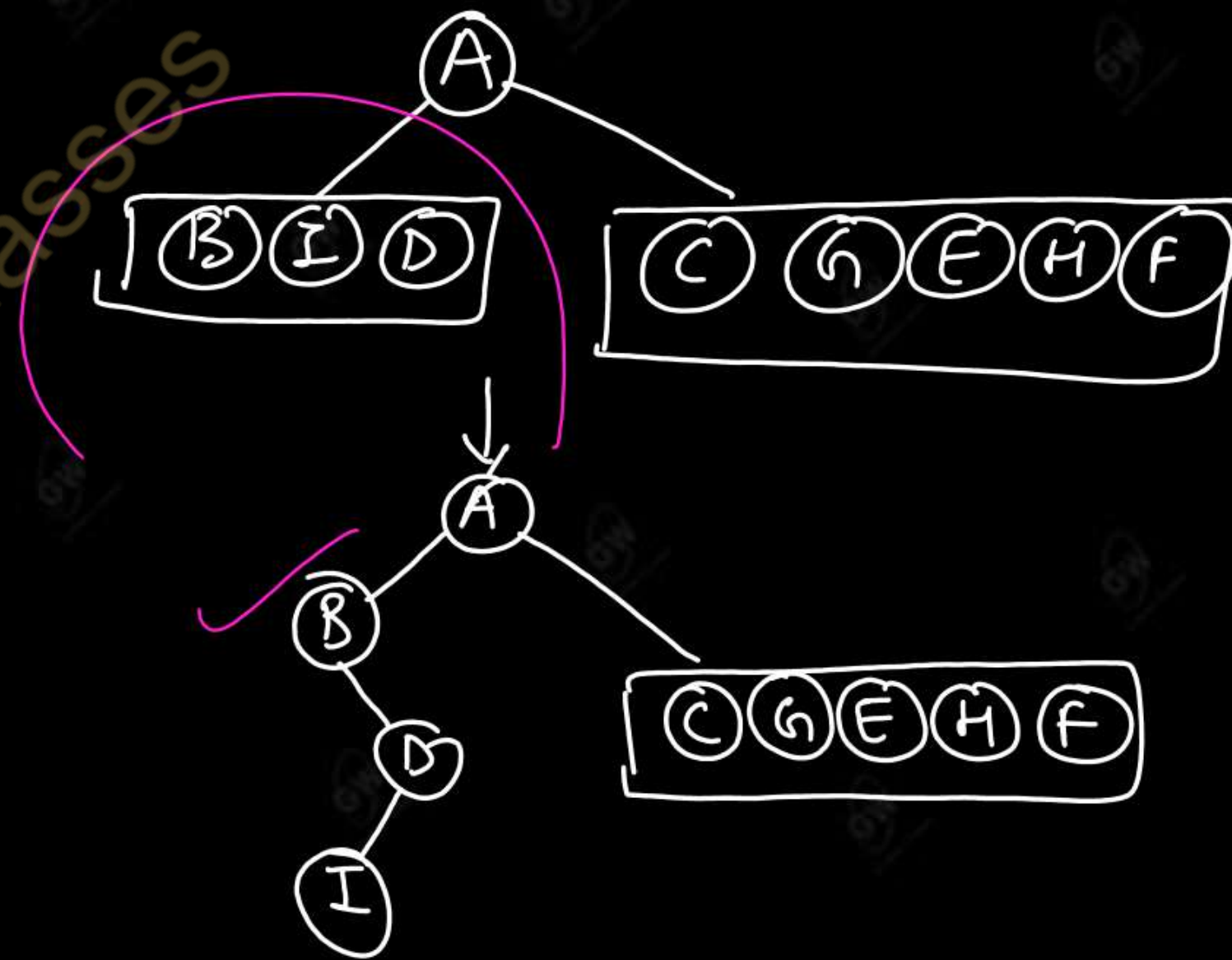
- The idea is to start with the root node, which would be the last item in the postorder sequence, and find the boundary of its left and right subtree in the inorder sequence.
- To find the boundary, search for the index of the root node in the inorder sequence.
- All keys before the root node in the inorder sequence become part of the left subtree, and all keys after the root node become part of the right subtree.
- Repeat this recursively for all nodes in the tree and construct the tree in the process.

(i) Draw the corresponding binary tree.

(ii) Write pre order traversal of same tree.

Diagram illustrating a hash table with 10 slots. The slots contain the letters B, I, D, A, C, G, E, H, and F. A blue arrow points to slot 0 (B), and a white arrow points to slot 3 (A). A blue bracket spans slots 0 to 3, and a white bracket spans slots 4 to 9.

A diagram of a linked list with nodes I, D, B, G, C, H, F, E, A. Node A is the root, and node B is the L-R node.



Q: The order of nodes of a binary tree in inorder and postorder traversal: (AKTU 2022-23, 2020-21)

In order : B, I, D, A, C, G, E, H, F.

Post order: I, D, B, G, C, H, F, E, A.

In-order -

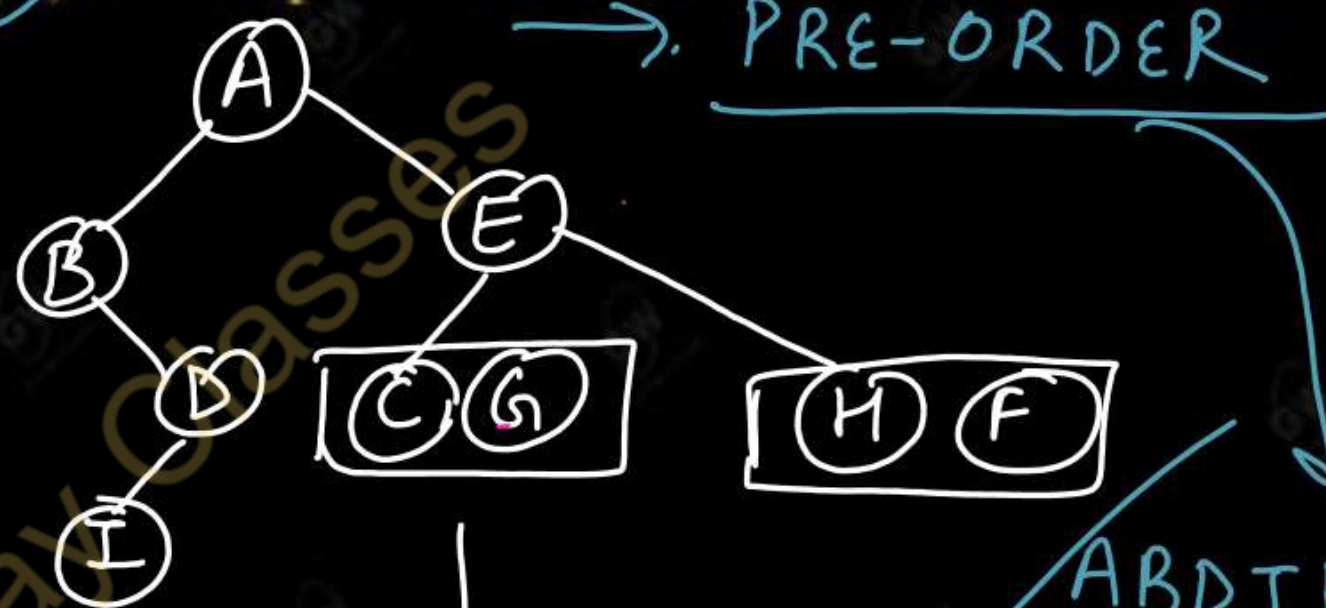
B	I	D	A	C	G	E	H	F
---	---	---	---	---	---	---	---	---

Post-order -

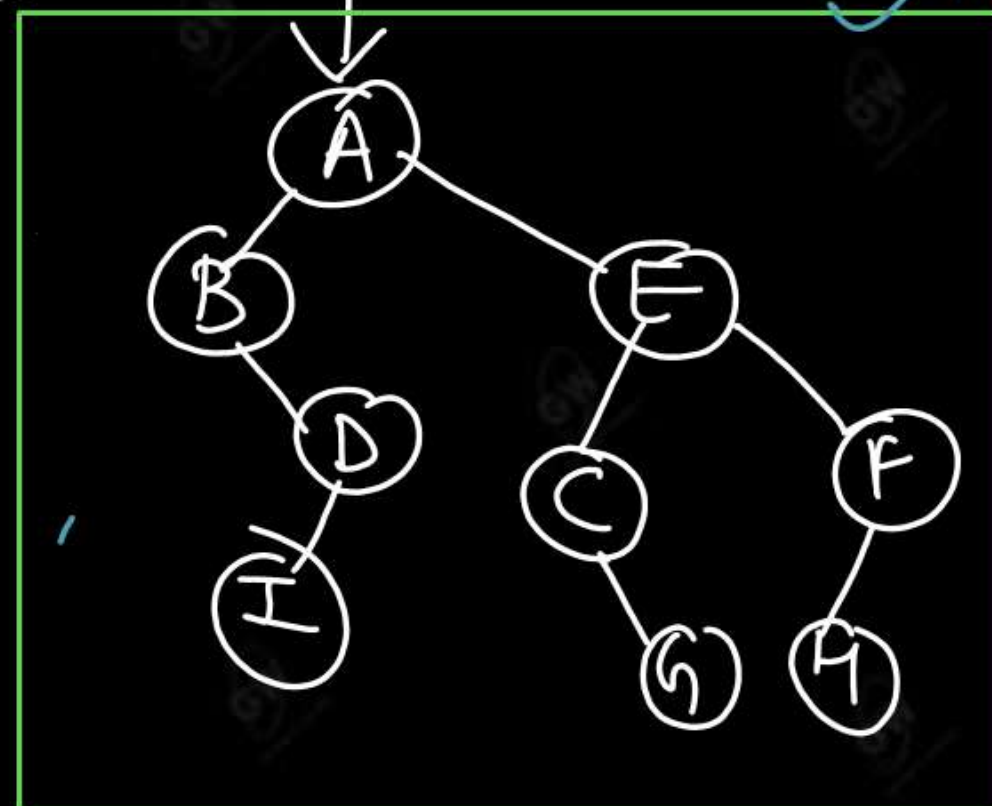
I	D	B	G	C	H	F	E	A
---	---	---	---	---	---	---	---	---

(i) Draw the corresponding binary tree.

(ii) Write pre order traversal of same tree.



ABDIECGFH

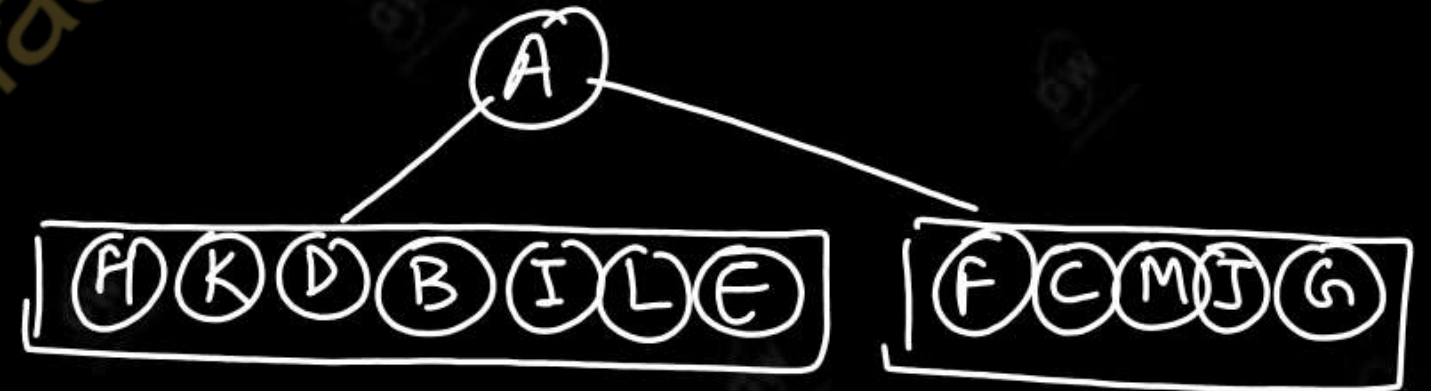
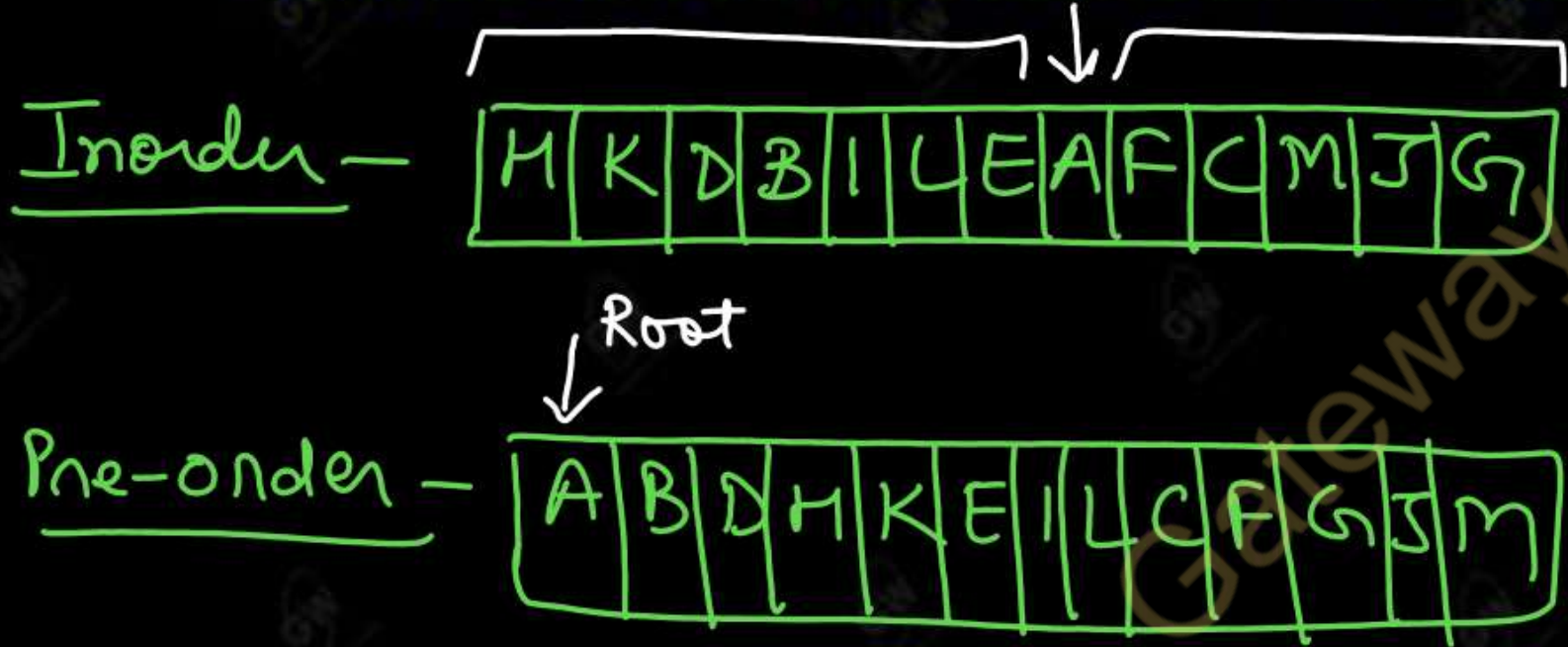


Q: Can you find a unique tree when any two traversals are given? Using the following traversals construct the corresponding binary tree:

INORDER: HKDBILEAF CMJG

PREORDER: ABDHKEILCFGJM

Also find the Post Order traversal of obtained tree. (AKTU 2019-20)



Q: Can you find a unique tree when any two traversals are given? Using the following traversals construct the corresponding binary tree:

INORDER: HKDBILEAF CMJG

PREORDER: ABDHKEILCFGJM

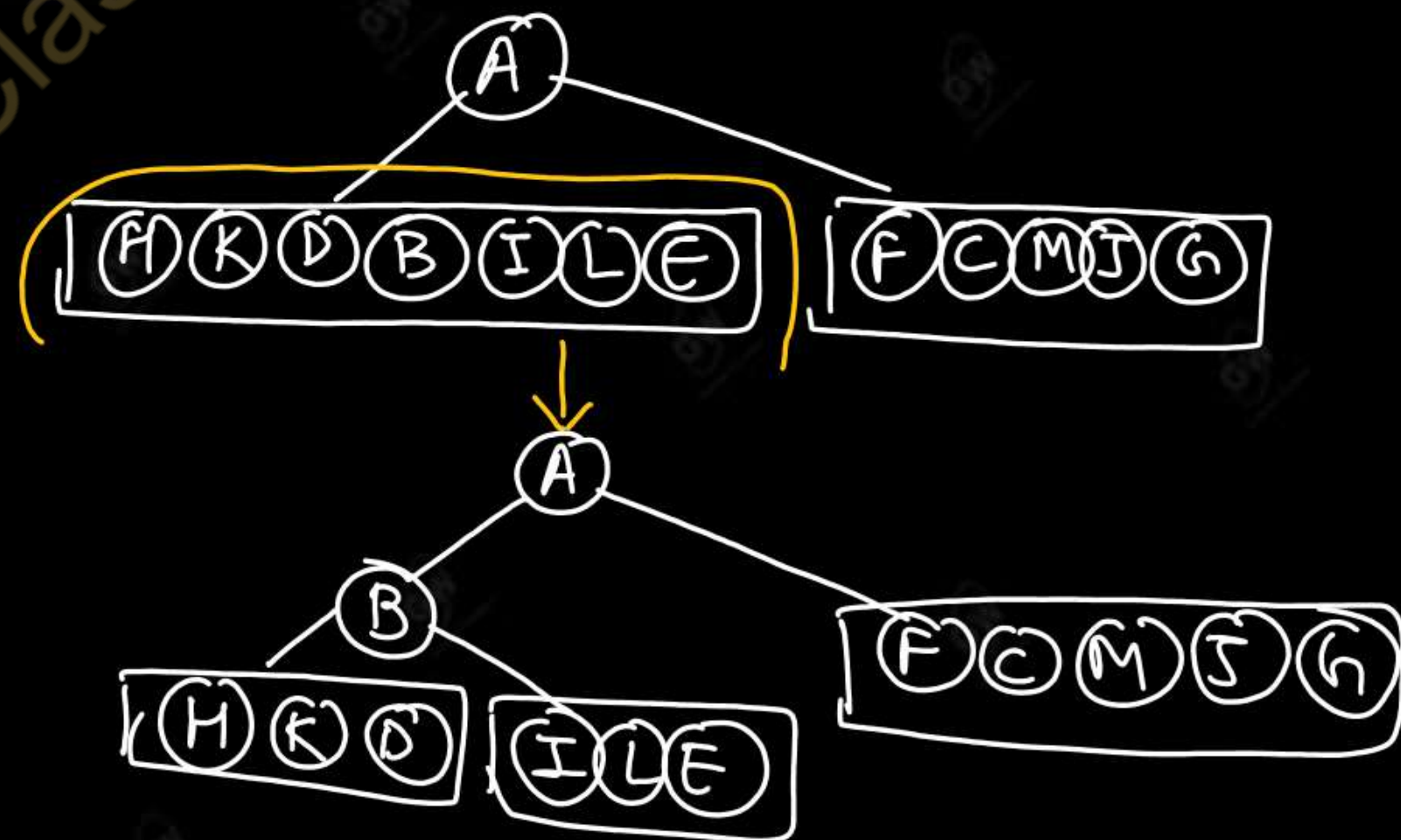
Also find the Post Order traversal of obtained tree. (AKTU 2019-20)

Inorder —

H	K	D	B	I	L	E	A	F	C	M	J	G
---	---	---	---	---	---	---	---	---	---	---	---	---

Pre-order —

A	B	D	H	K	E	I	L	C	F	G	J	M
---	---	---	---	---	---	---	---	---	---	---	---	---



Q: Can you find a unique tree when any two traversals are given? Using the following traversals construct the corresponding binary tree:

INORDER: HKDBILEAF CMJG

PREORDER: ABDHKEILCFGJM

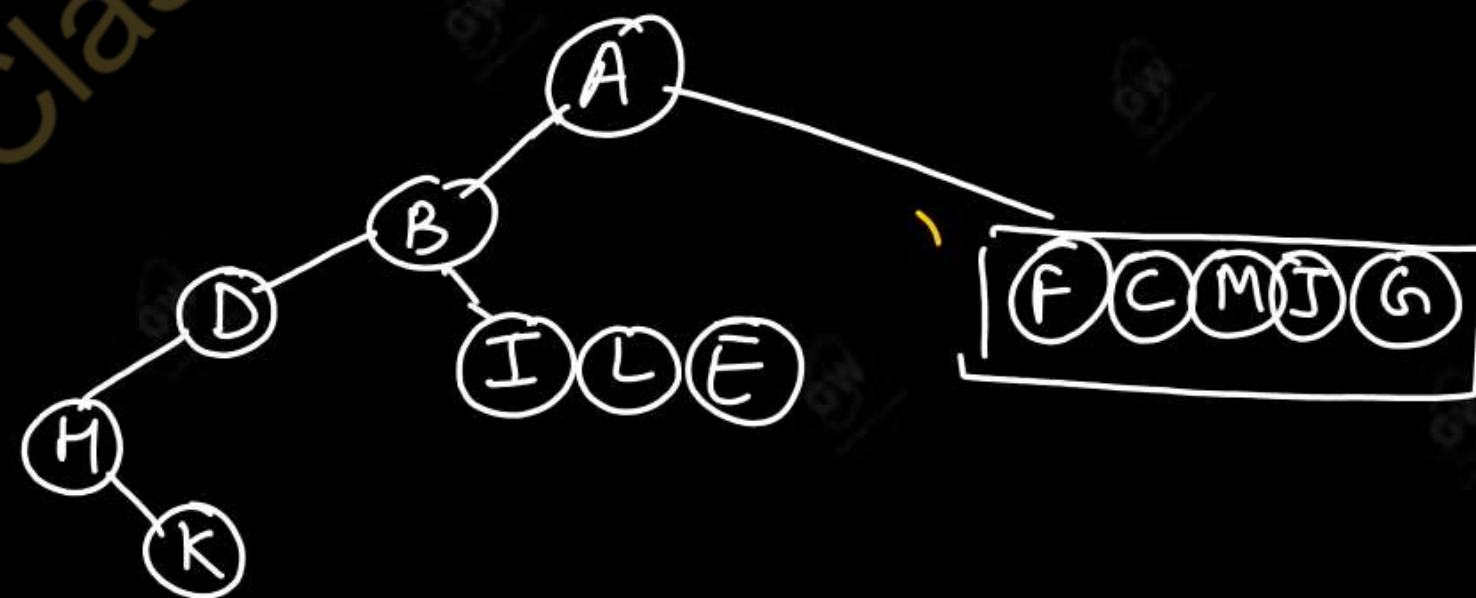
Also find the Post Order traversal of obtained tree. (AKTU 2019-20)

Inorder -

H	K	D	B	I	L	E	A	F	C	M	J	G
---	---	---	---	---	---	---	---	---	---	---	---	---

Pre-order -

A	B	D	H	K	E	I	L	C	F	G	J	M
---	---	---	---	---	---	---	---	---	---	---	---	---

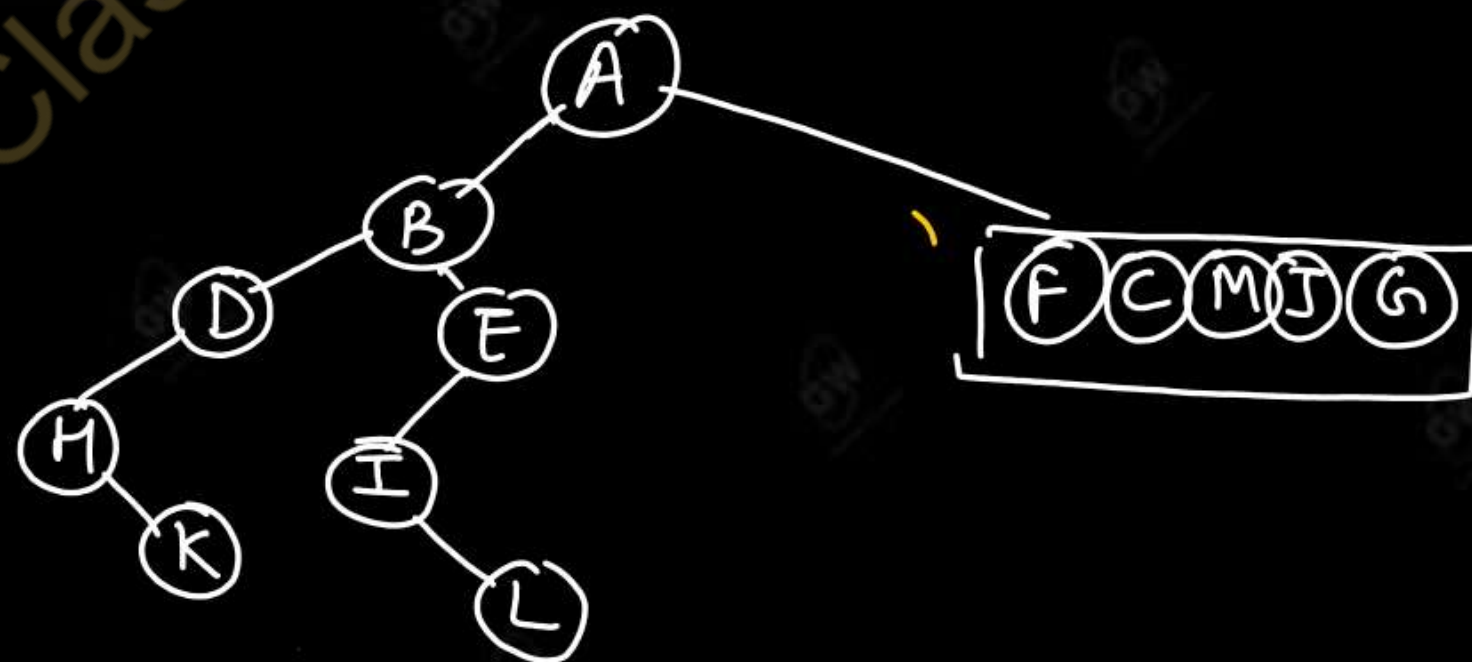
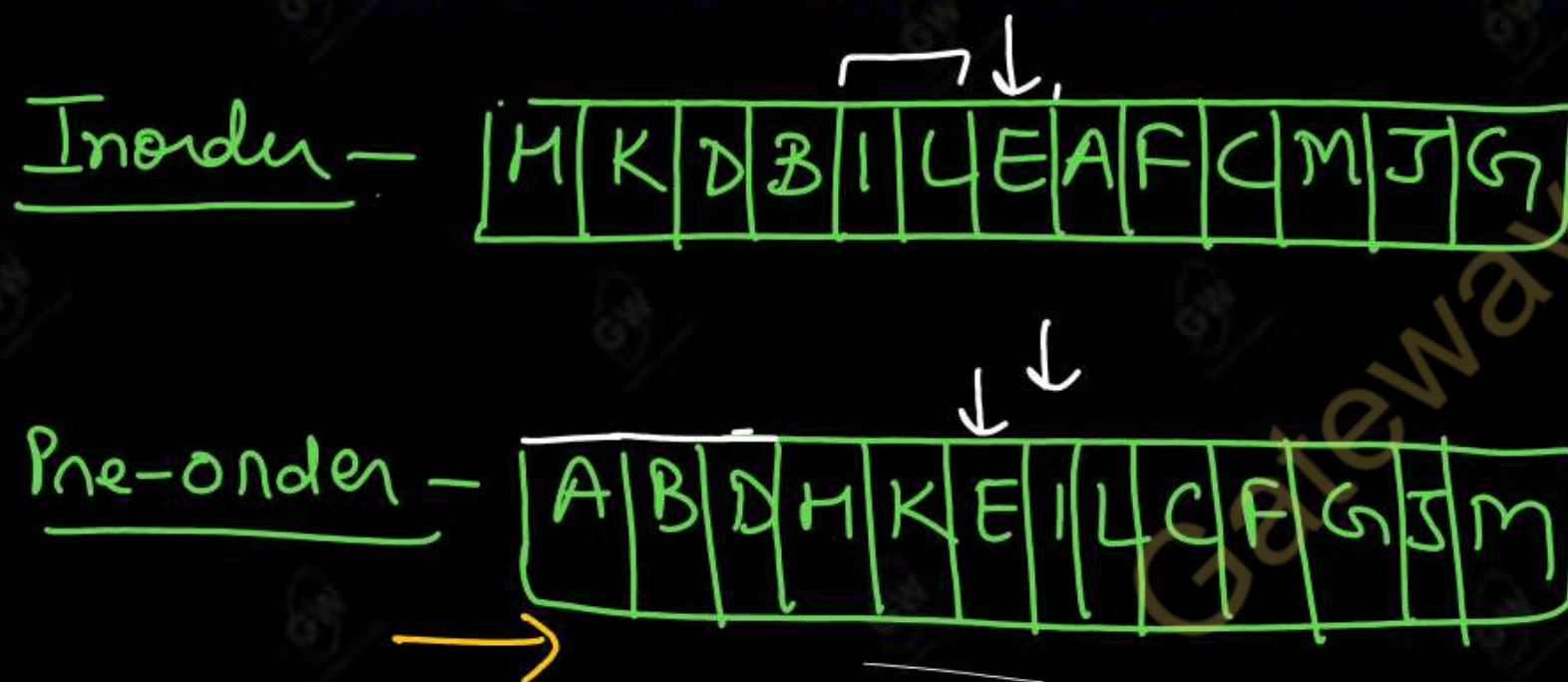


Q: Can you find a unique tree when any two traversals are given? Using the following traversals construct the corresponding binary tree:

INORDER: HKDBILEAF CMJG

PREORDER: ABDHKEILCFGJM

Also find the Post Order traversal of obtained tree. (AKTU 2019-20)



Q: Can you find a unique tree when any two traversals are given? Using the following traversals construct the corresponding binary tree:

INORDER: HKDBILEAF CMJG

PREORDER: ABDHKEILCFGJM

Post-order Traversal

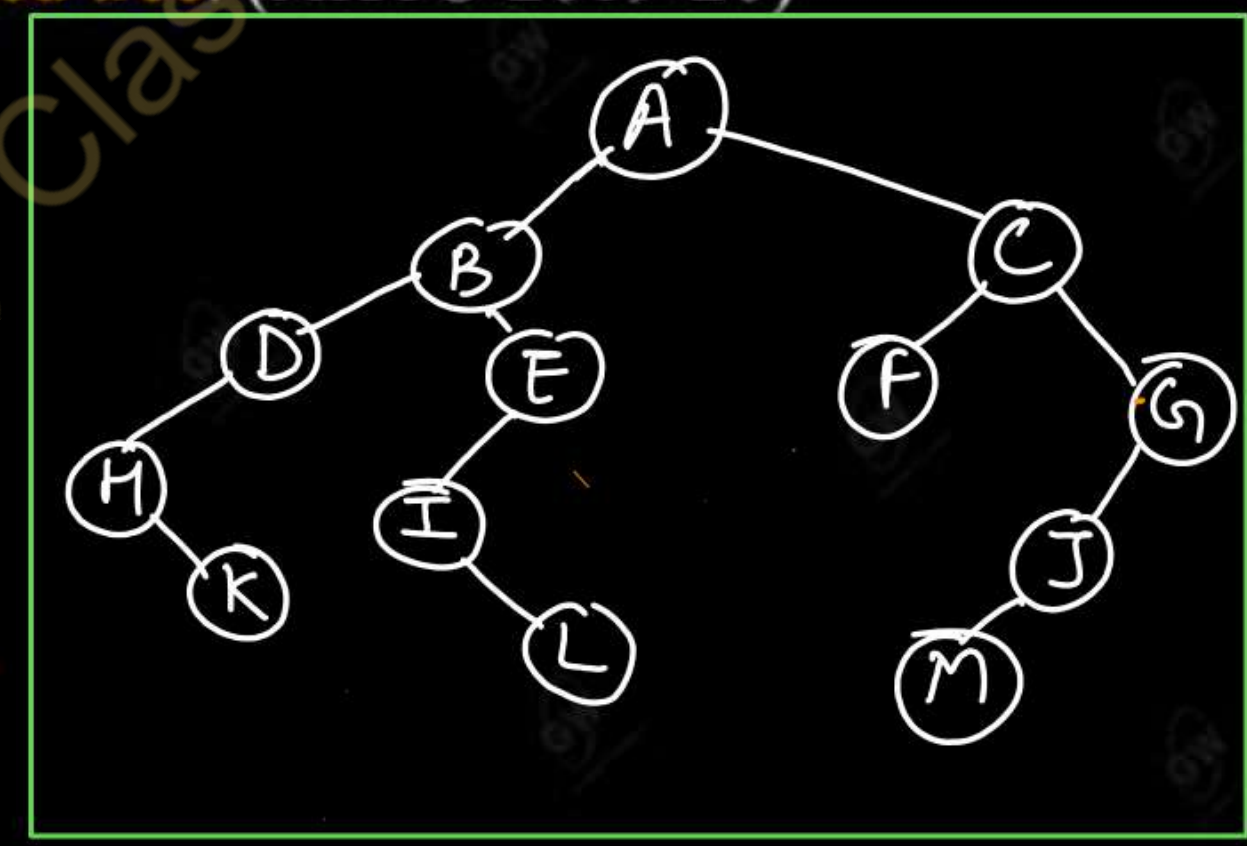
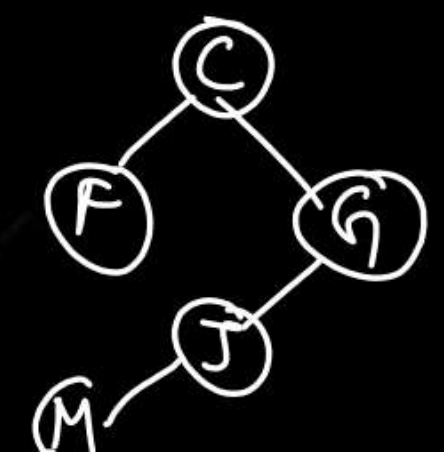
KHDLIEBFMTJGCA

Also find the Post Order traversal of obtained tree. (AKTU 2019-20)

Inorder -



Pre-order -





AKTU

B.Tech III-Sem



GATEWAY CLASSES

CS IT & CS Allied

DATA STRUCTURE

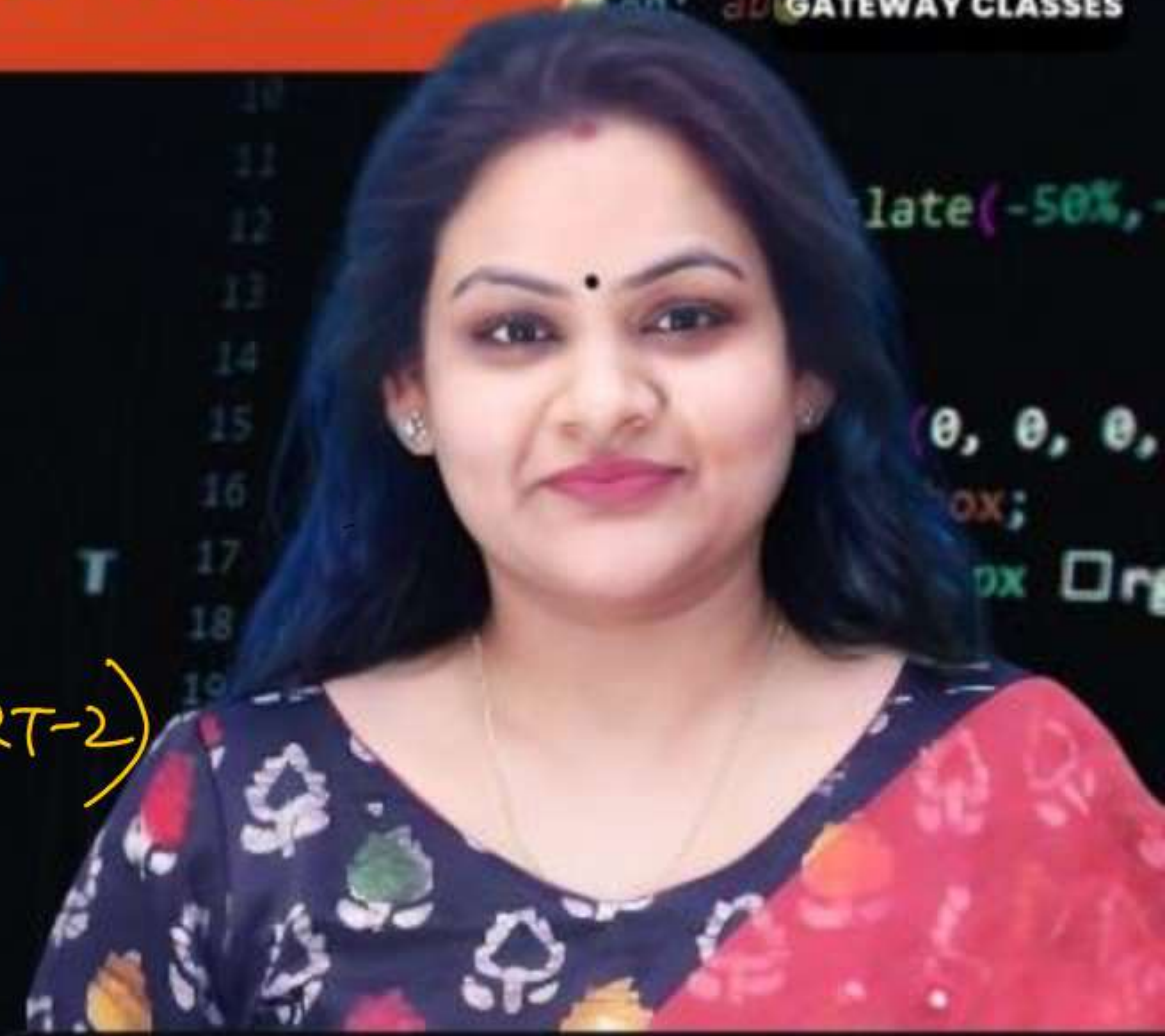
Unit-4 Lecture-3 (PART-1 + PART-2)

Today's Target

- Binary Search Tree
- BST Operations
 - BST Search
 - BST Insertion
 - BST Deletion
- AKTU PYQs
- DPP

PART-1

PART-2



By Dr. Nidhi Parashar Ma'am

Ph.D (CSE)

- **M.Tech Gold Medalist**
- **Net Qualified**

➤ A binary search tree (BST) is used to organize and manage data efficiently. BST is a special type of binary tree that maintains its elements in a sorted order.

➤ Used for fast searching, insertion, and deletion operations.

➤ Each node can have up to two children.

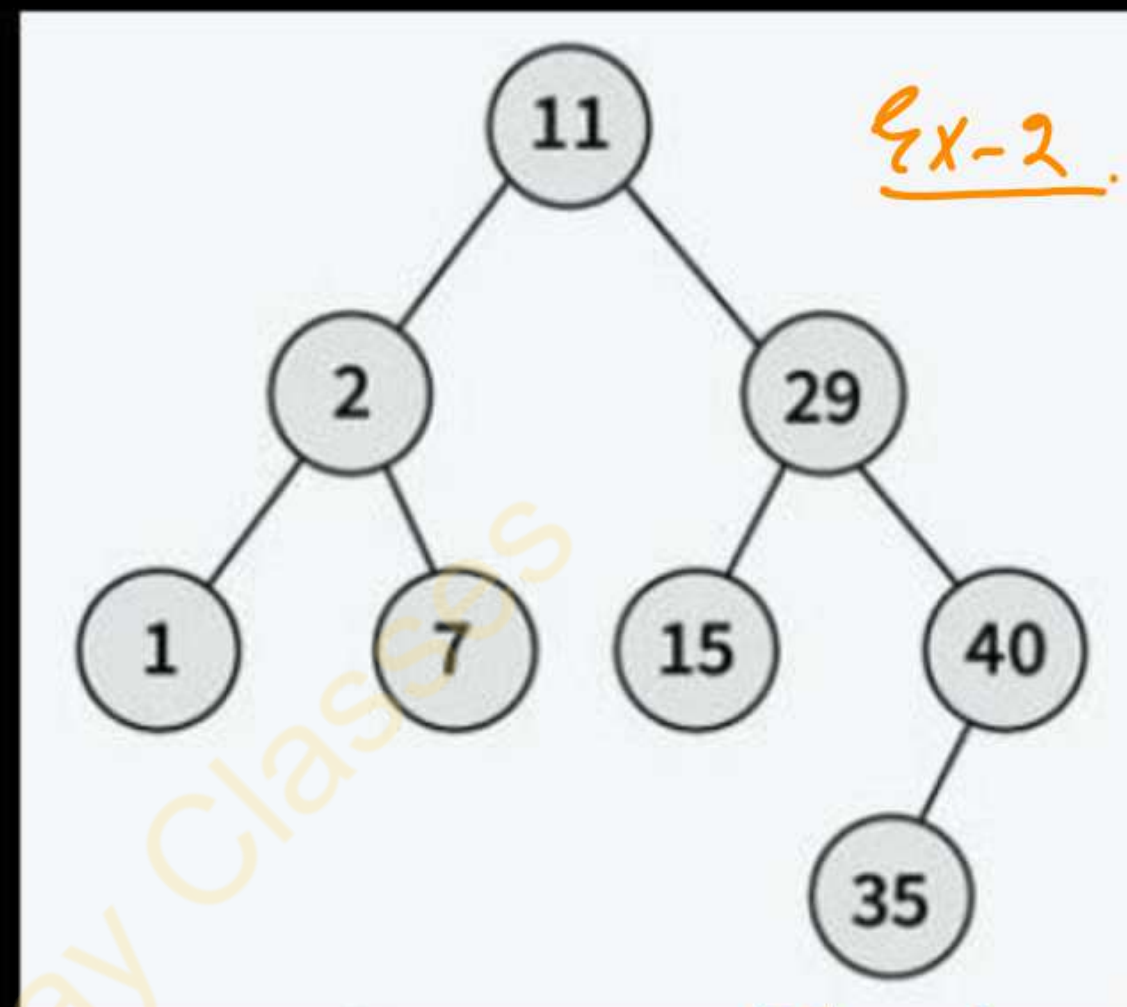
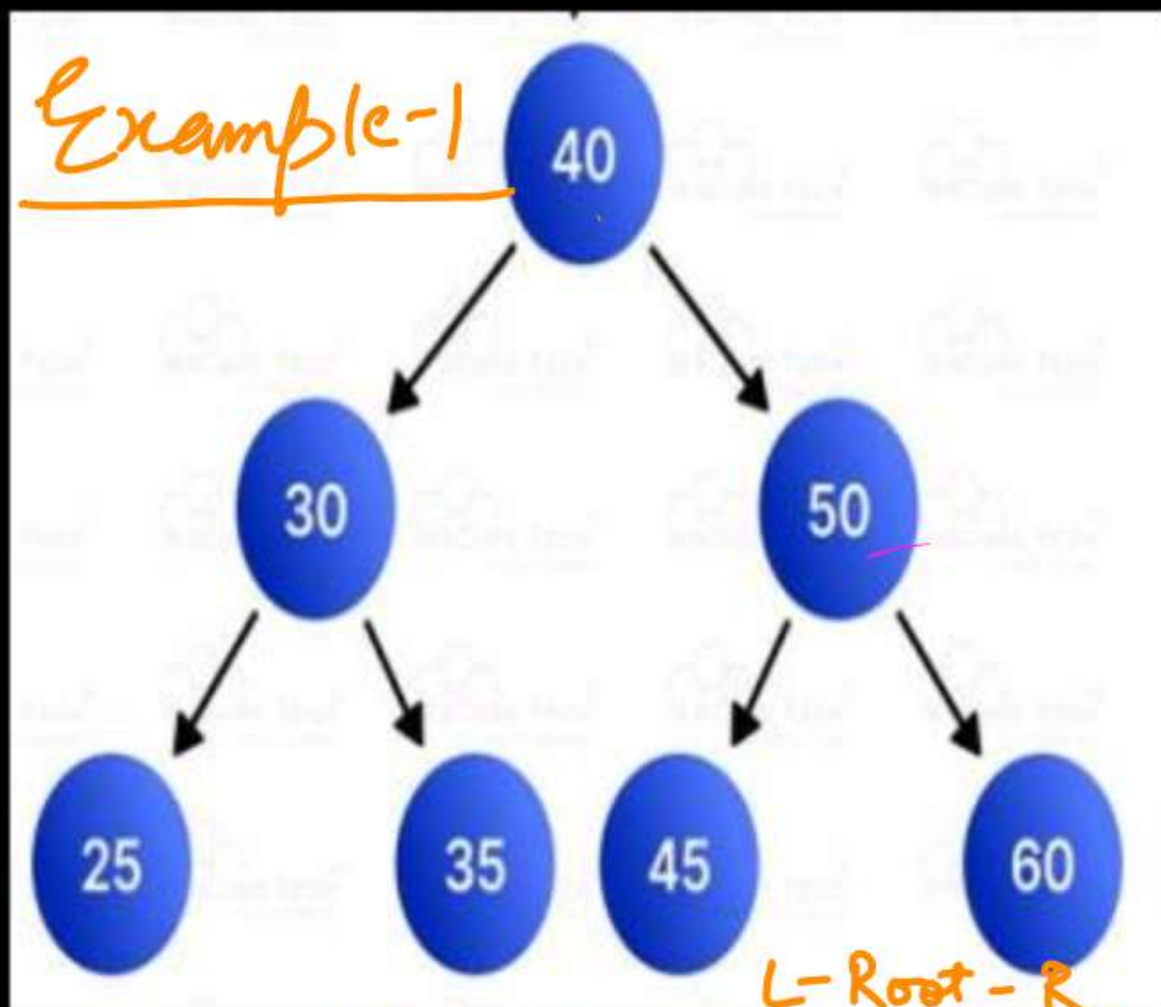
$O(\log n)$ ✓

The main rule of a BST is that for any node:

➤ All nodes in its left subtree have values less than the node's value.

➤ All nodes in its right subtree have values greater than the node's value.

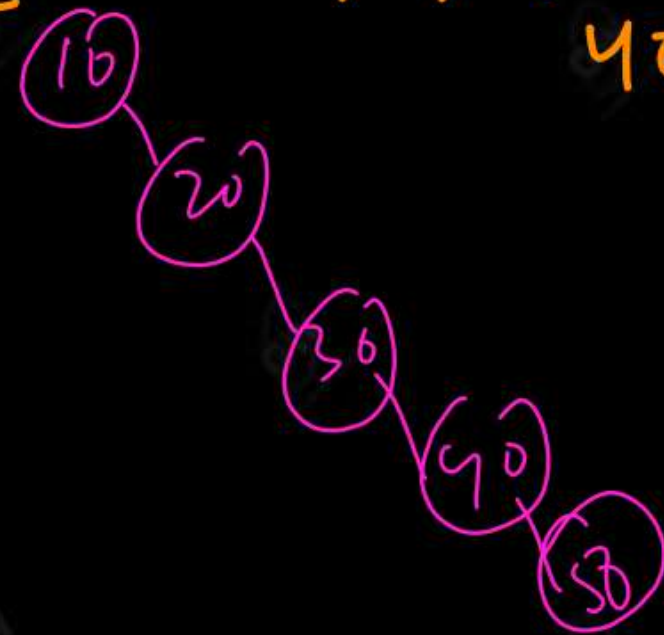
This structure helps to quickly find, add, or remove values from the tree. This property ensures that each comparison allows the operation to skip about half of the remaining tree, making BST operations much faster than linear structures like arrays or linked lists.



Key Properties: In-order: — 25, 30, 35, 40, 45, 50, 60

Inorder — 1, 2, 7, 11, 15, 29, 35, 40

- Unique ordering of elements means duplicates are usually not allowed.
- In-order traversal of a BST gives sorted order of elements.
- Average height: $O(\log n)$ (for balanced BST).
- Worst case height: $O(n)$ (when tree becomes skewed).



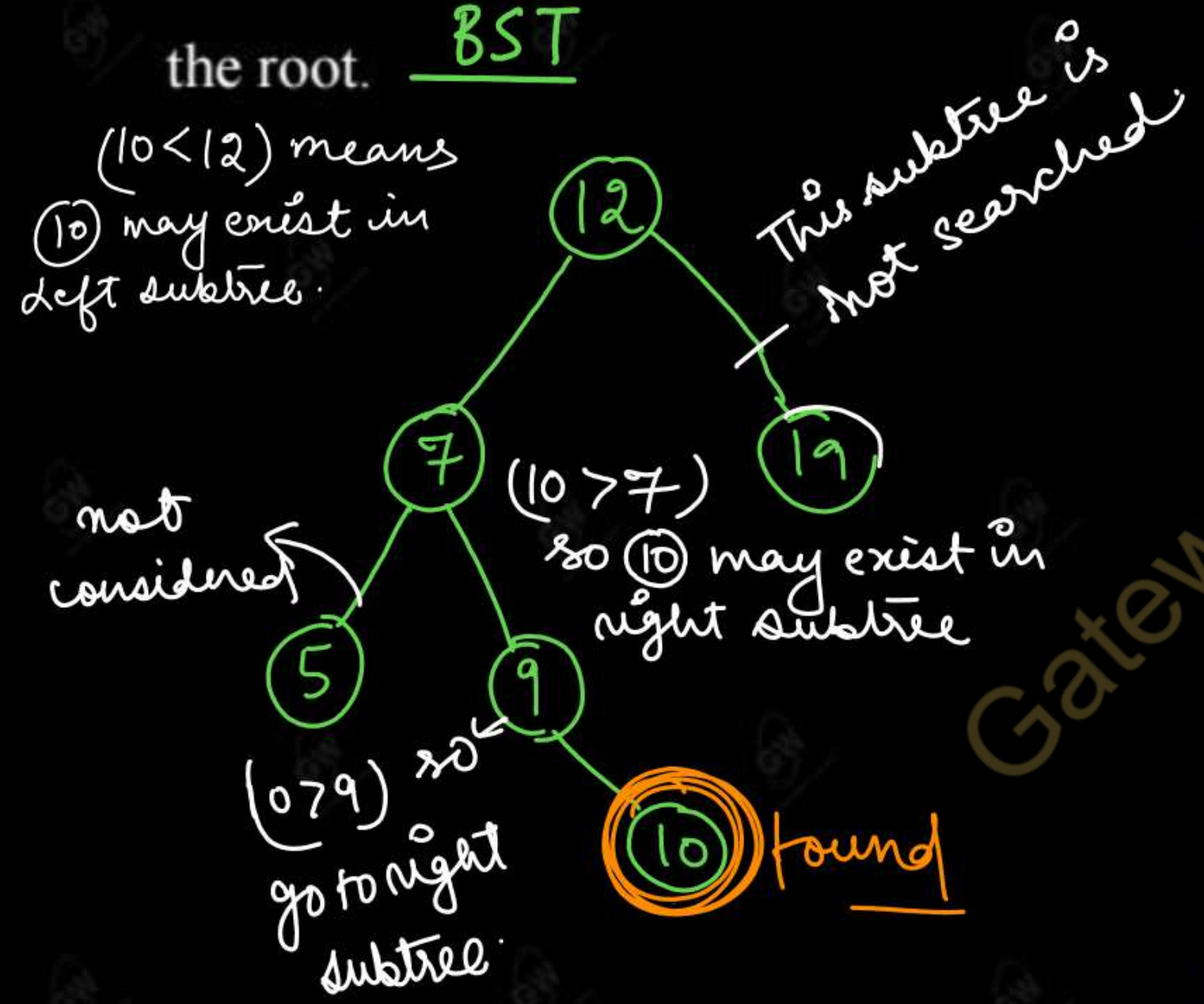
How Binary Search Tree Works?

- Depends upon element to be inserted, search, or deleted, the algorithm performs all operations by comparing values with the root and its further child nodes in the left or right sub-tree accordingly.
- So, the algorithm can easily drop the left or right subtree of the root node.

BST operations:

1. **Search:** searches element from the binary tree
2. **Insert:** adds an element to binary tree
3. **Delete:** delete element from a binary tree

Always initiate analyzing tree at the root node and then move further to either the right or left subtree of the root node depending upon the element to be located is either less or greater than the root. BST



1. The element to be searched is 10
2. Compare the element with the root node 12, $10 < 12$, hence you move to the left subtree. No need to analyze the right-subtree
3. Now compare 10 with node 7, $10 > 7$, so move to the right-subtree
4. Then compare 10 with the next node, which is 9, $10 > 9$, look in the right subtree child
5. 10 matches with the value in the node, $10 = 10$, return the value to the user.

BST Search (Recursive)

starting with root $\rightarrow$ Key being searched

Search (x, K)

1. if $x == \text{Nil}$ or $K == x.\text{key}$
2. return x
3. if $K < x.\text{key}$
4. return Search ($x.\text{left}, K$)
5. else return Search ($x.\text{right}, K$)

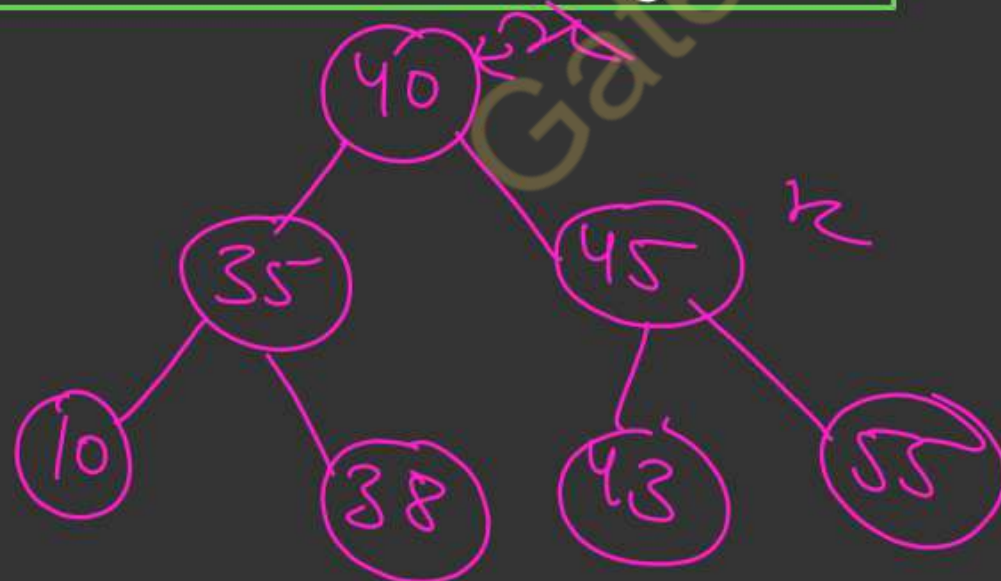
AKTU (2022-23)

(BST-Search (Iterative))

Search (x, K)

1. while $x \neq \text{Nil}$ and $K \neq x.\text{key}$
2. if $K < x.\text{key}$
3. $x = x.\text{left}$
4. else $x = x.\text{right}$
5. return x

K=43



Insert Operation

- First, the root node is inserted, then the next value is compared with the root node.
- If the value is greater than root, it is added to the right subtree, and if it is lesser than the root, it is added to the left subtree.

Example :- Insert 12, 7, 9, 19, 5, 10

Initially BST is empty

1) Insert 12

(12)

2) Insert 7

(12)
(7)

(7 < 12)
so it will be
inserted in the left
subtree of (12)

3) Insert 9

(12)
↓
(7)
↓
(9)

(9 < 12) ⇒ go to the left subtree
(9 > 7) go to right subtree

4) Insert 19

(12)
↙ ↘
(7) (19)
↓
(9)

(19 > 12) so go to right subtree

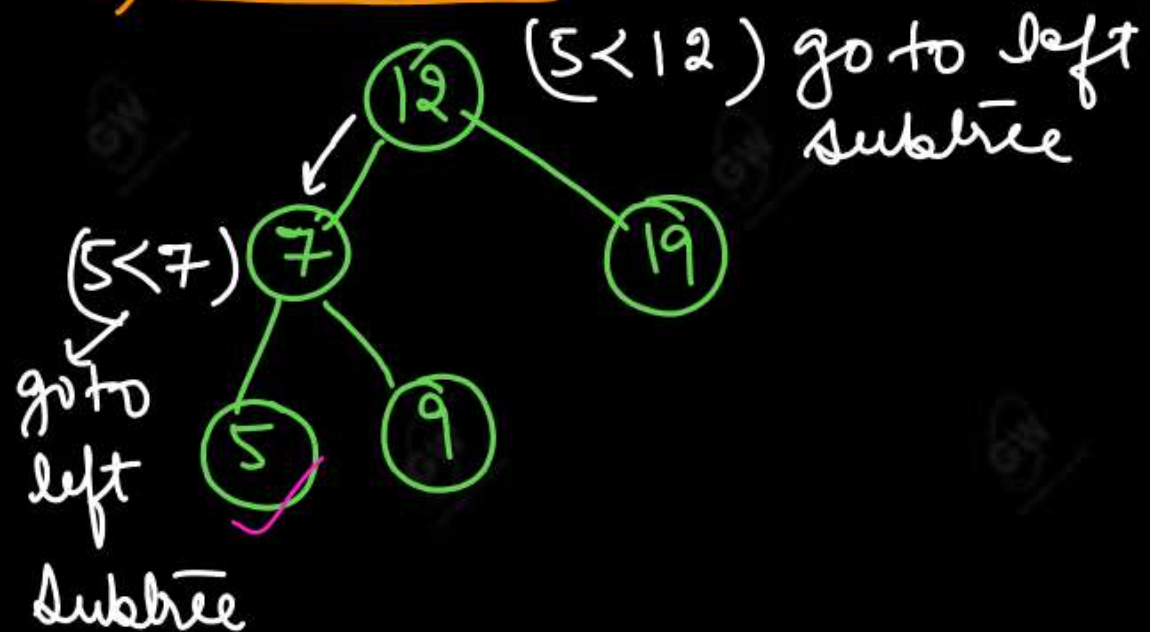
Insert Operation

- First, the root node is inserted, then the next value is compared with the root node.
- If the value is greater than root, it is added to the right subtree, and if it is lesser than the root, it is added to the left subtree.

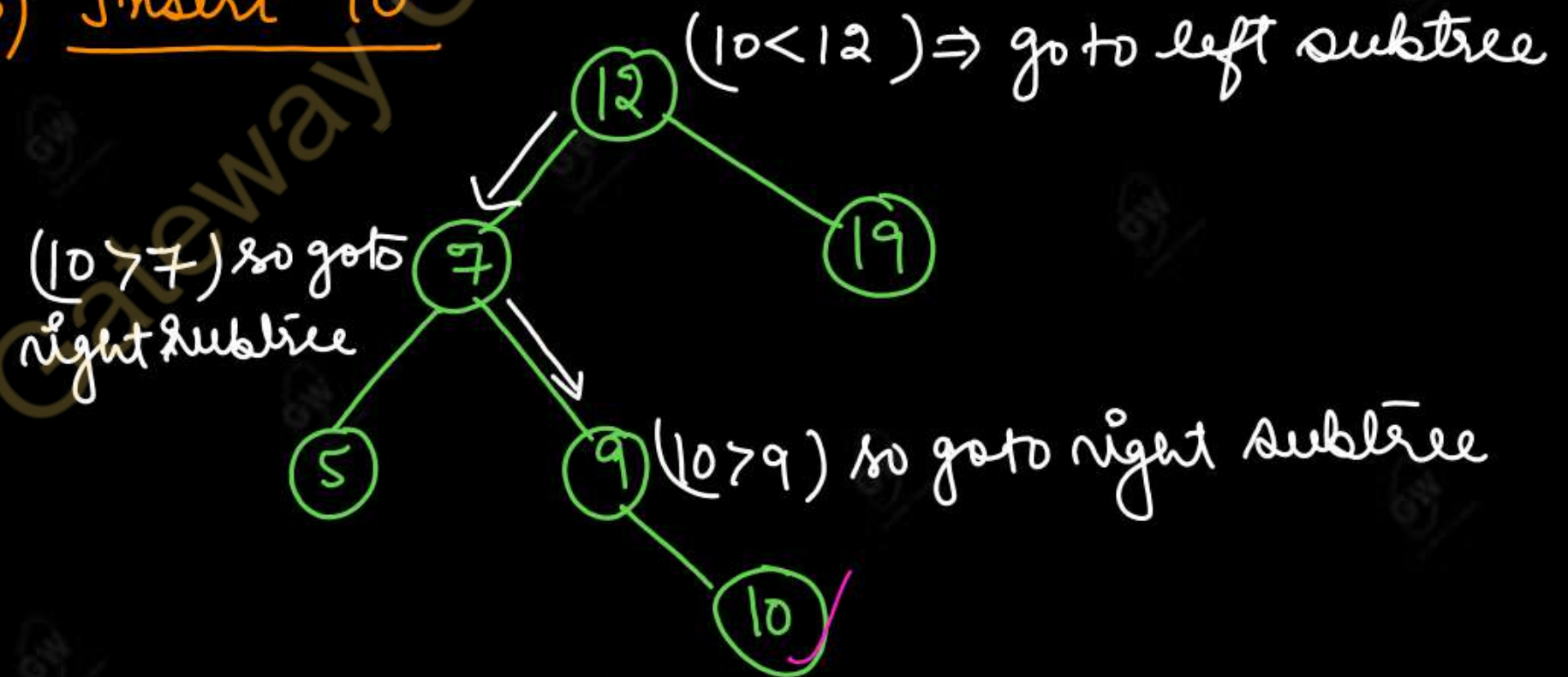
Example :- Insert 12, 7, 9, 19, 5, 10

Initially BST is empty

5) Insert 5



6) Insert 10



Pseudocode for Inserting a Node in BST

TreeInsert(T, z) $\rightarrow$ node to be inserted

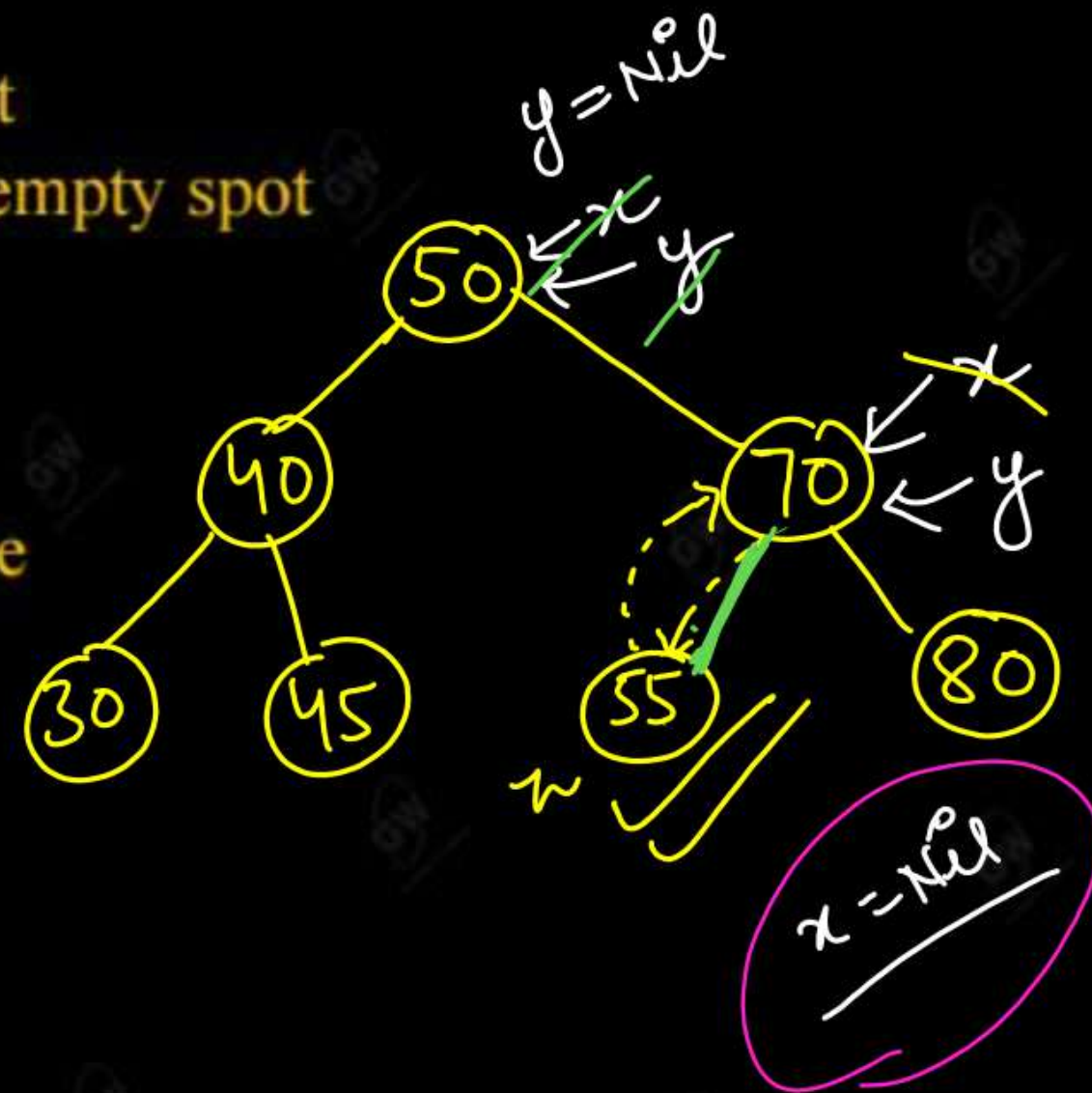
```
01  $y \leftarrow \text{NIL}$ 
02  $x \leftarrow \text{root}[T]$ 
03 while  $x \neq \text{NIL}$ 
04    $y \leftarrow x$ 
05   if  $\text{key}[z] < \text{key}[x]$ 
06     then  $x \leftarrow \text{left}[x]$ 
07     else  $x \leftarrow \text{right}[x]$ 
08  $p[z] \leftarrow y$ 
09 if  $y = \text{NIL}$ 
10   then  $\text{root}[T] \leftarrow z$ 
11   else if  $\text{key}[z] < \text{key}[y]$ 
12     then  $\text{left}[y] \leftarrow z$ 
13     else  $\text{right}[y] \leftarrow z$ 
```

y will be the parent of x
Start searching from the root
Continue until we reach an empty spot
Track the parent of x
Compare values
Go to left subtree
Otherwise go to right subtree
Set parent of z
If tree was empty

Insert as left child
Insert as right child

Key [z] = 55

Insert 55

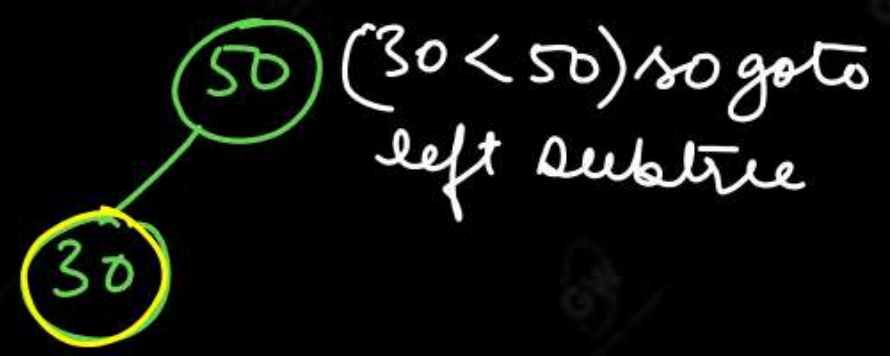


Q: Construct a Binary Search Tree (BST) using the following sequence of numbers: 50, 30, 70, 20, 40, 60, 80, 35, 45. Perform In-order traversal of Tree. (AKTU 2024-25)

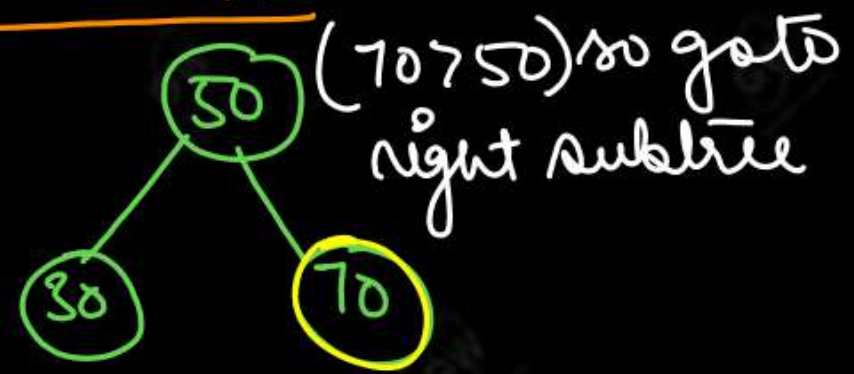
1) Insert 50



2) Insert 30



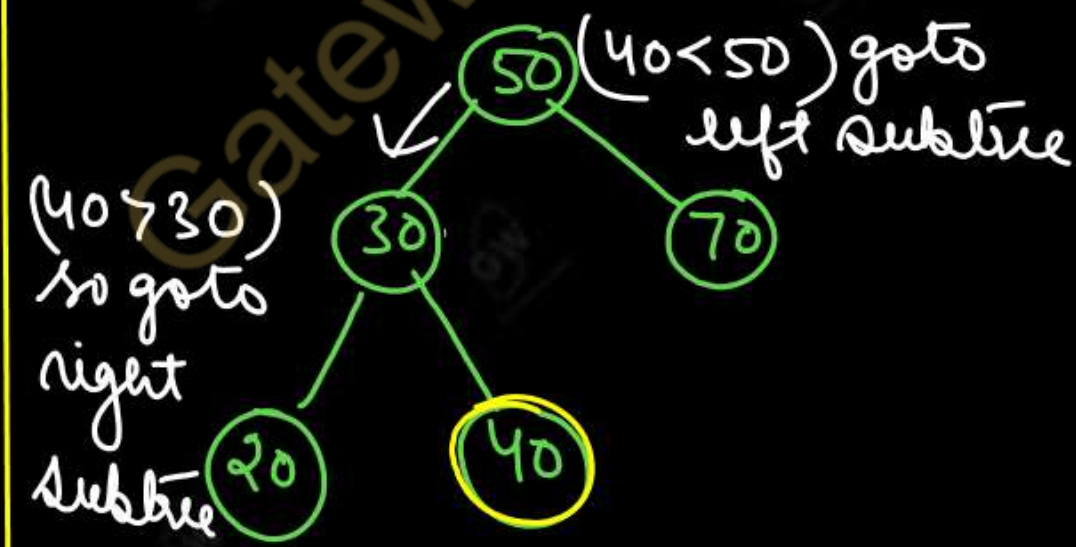
3) Insert 70



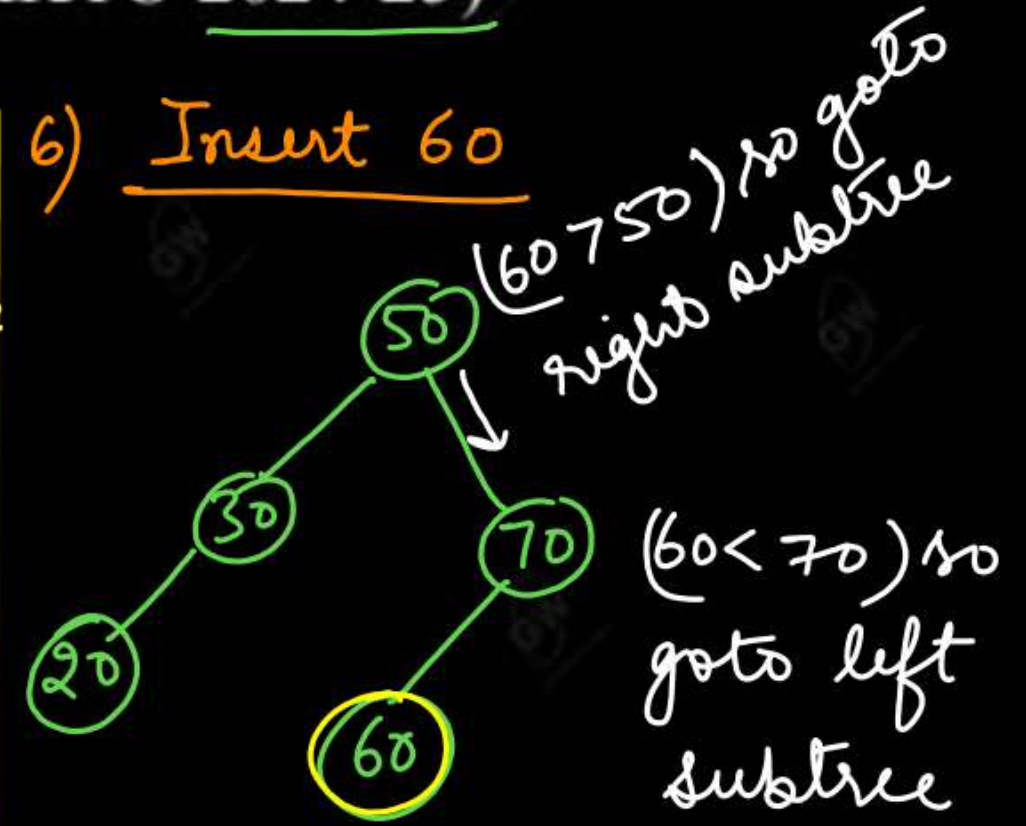
4) Insert 20



5) Insert 40

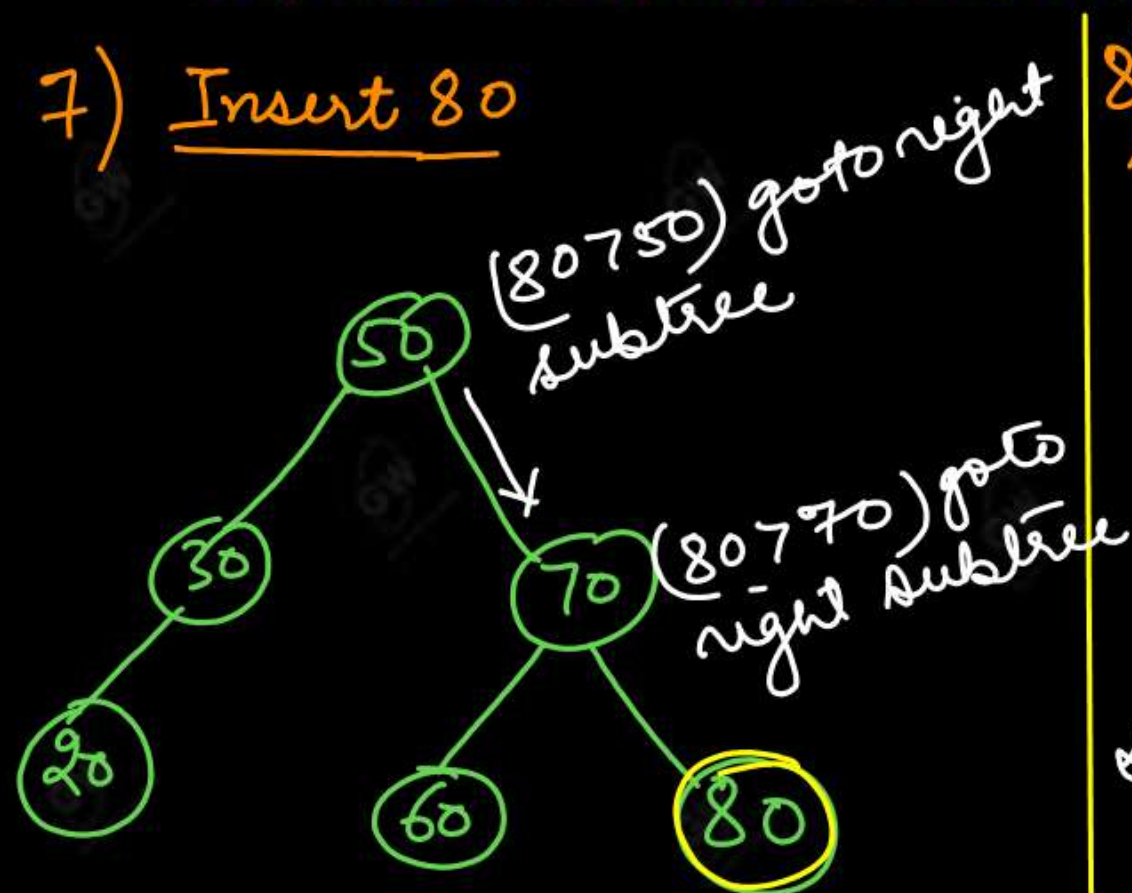


6) Insert 60

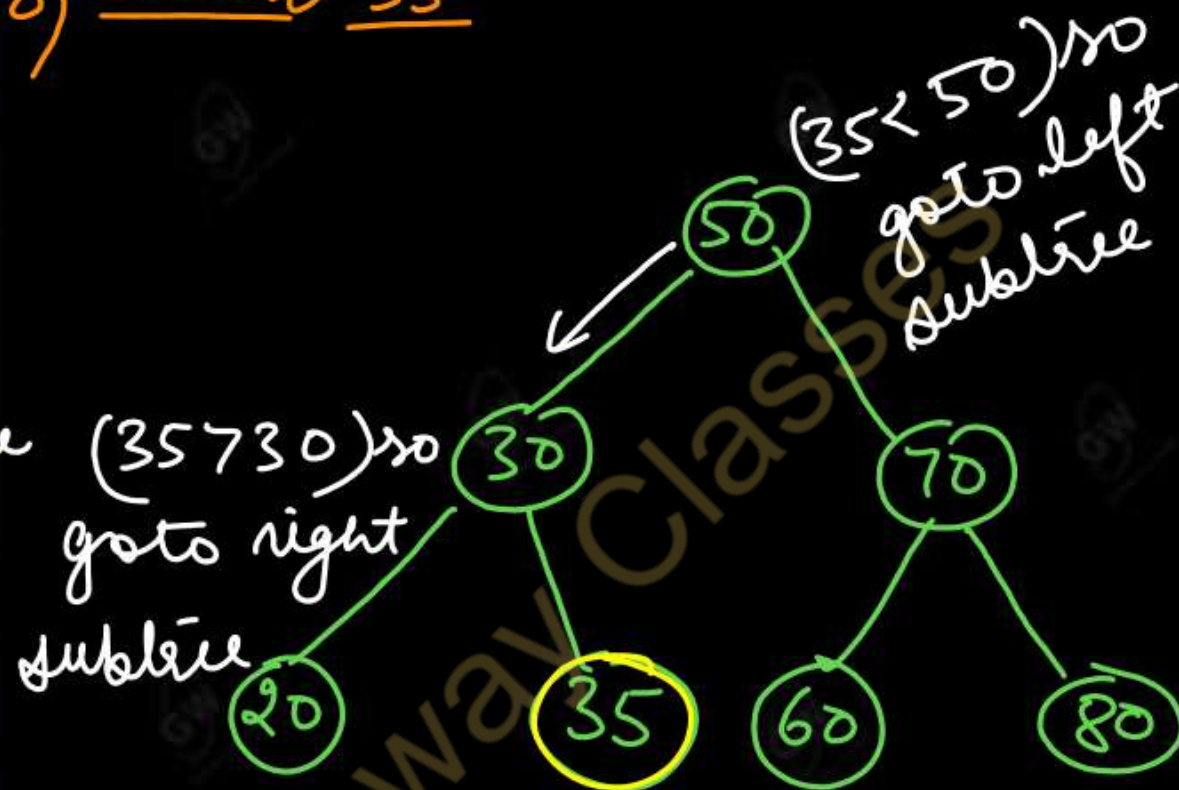


Q: Construct a Binary Search Tree (BST) using the following sequence of numbers: 50, 30, 70, 20, 40, 60, 80, 35, 45. Perform In-order traversal of Tree. (AKTU 2024-25)

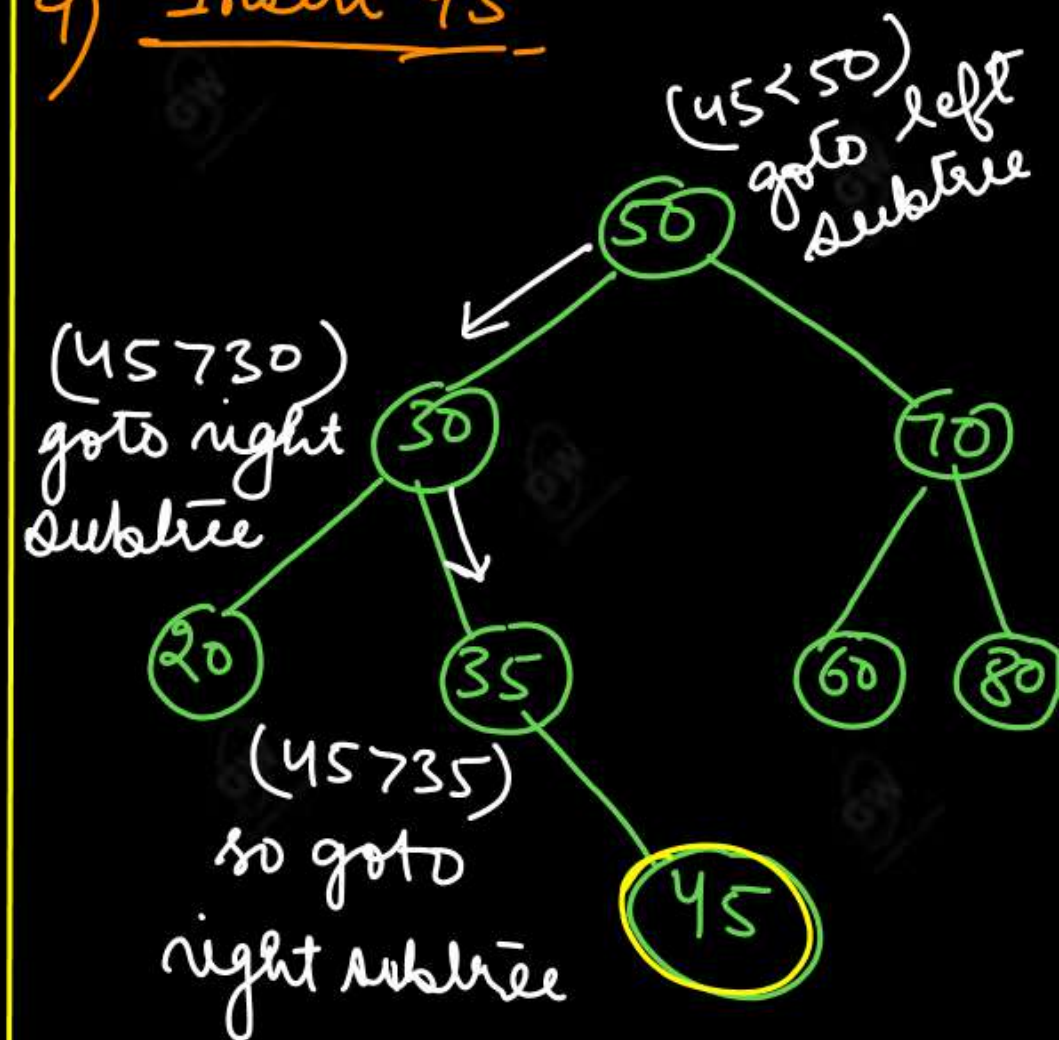
7) Insert 80



8) Insert 35



9) Insert 45



Inorder →

20 30 35 45 50 60 70 80

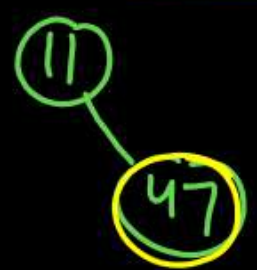
Final Tree

Q: Draw the binary search tree that results from inserting the following numbers in sequence starting with 11: 11, 47, 81, 9, 61, 10, 12 (AKTU 2021-22)

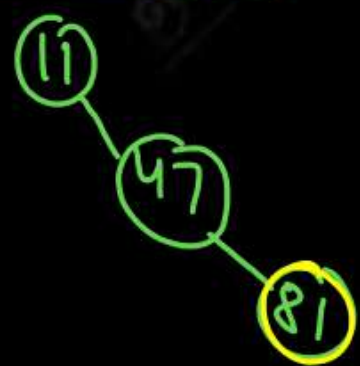
1) Insert 11



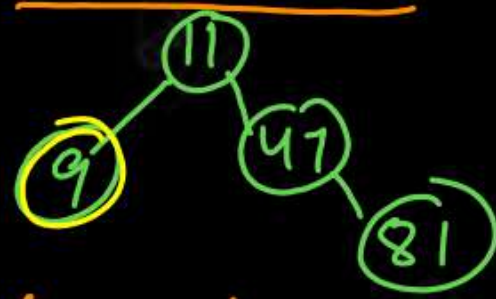
2) Insert 47



3) Insert 81



4) Insert 9



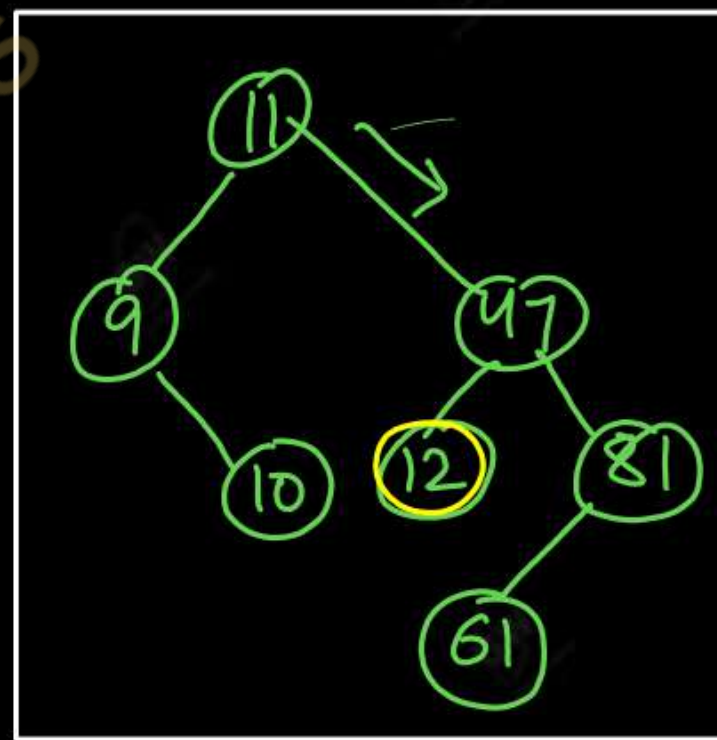
5) Insert 61



6) Insert 10



7) Insert 12

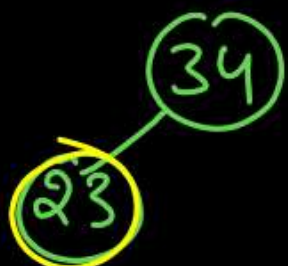


Q: For a given sequence of numbers, construct a BST. 34, 23, 67, 45, 12, 54, 87, 43, 98, 75, 84, 93, 31. (AKTU 2019-20)

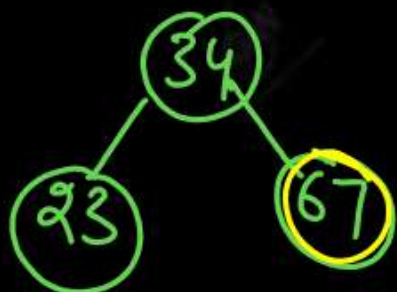
1) Insert 34



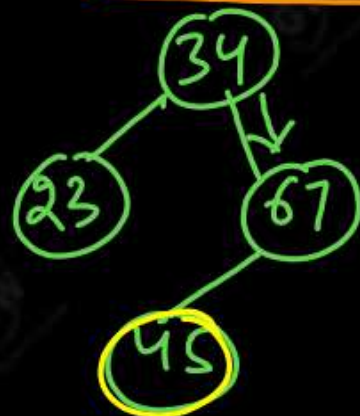
2) Insert 23



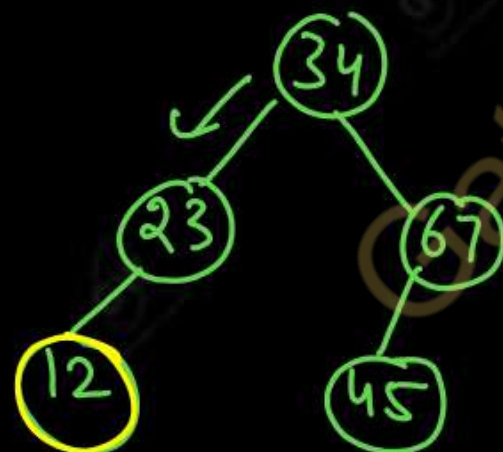
3) Insert 67



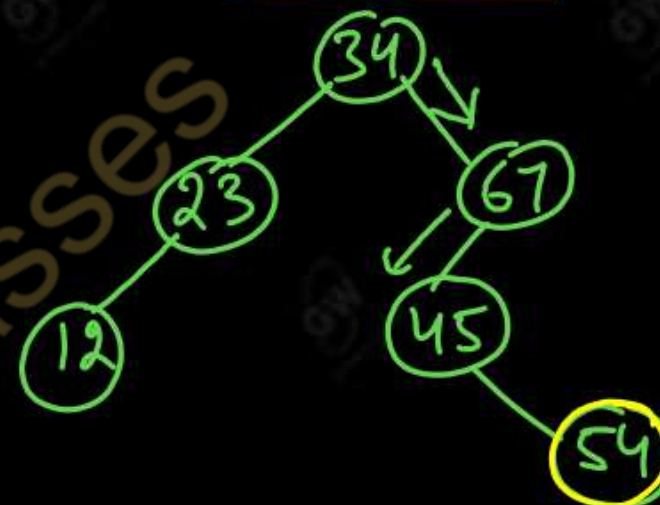
4) Insert 45



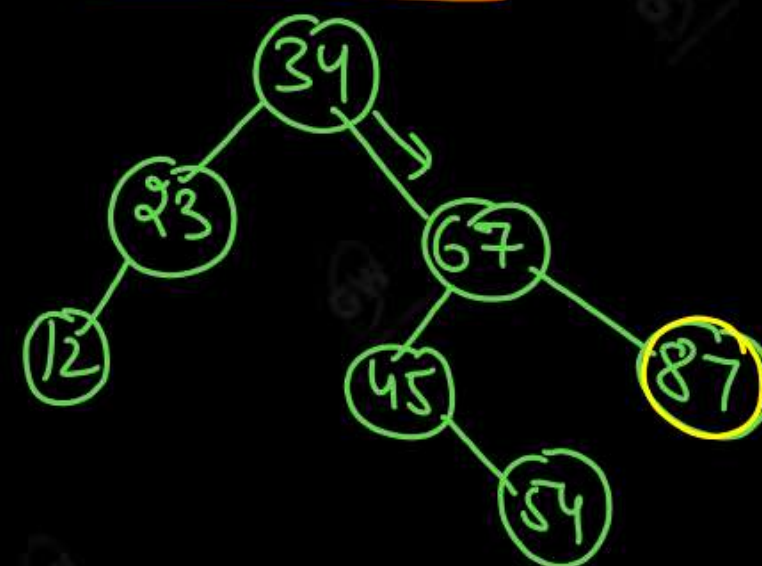
5) Insert 12



6) Insert 54

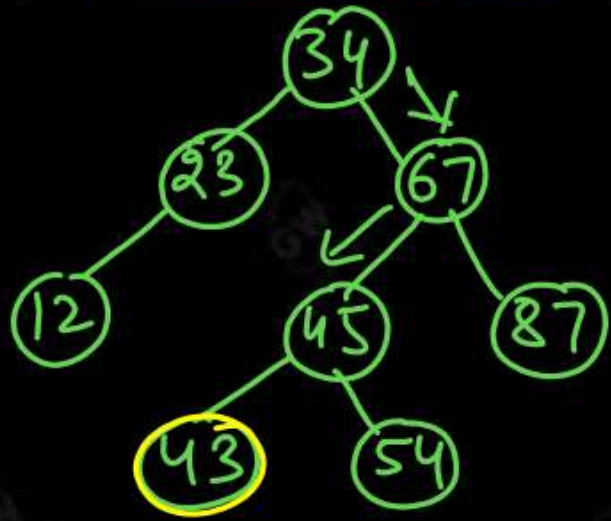


7) Insert 87

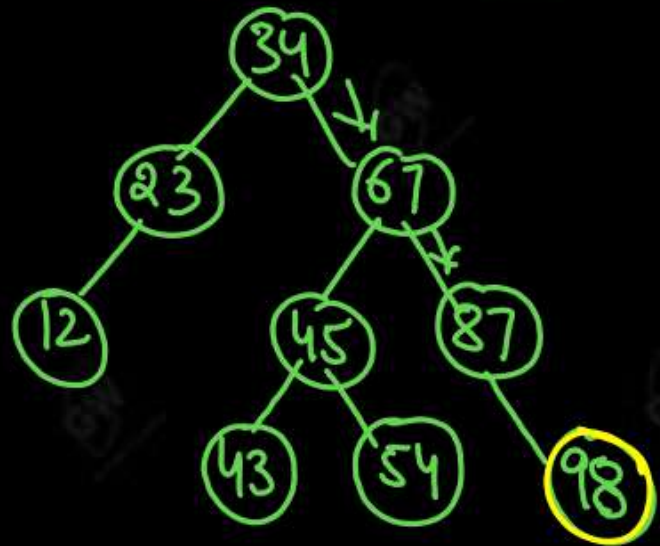


Q: For a given sequence of numbers, construct a BST. 34, 23, 67, 45, 12, 54, 87, 43, 98, 75, 84, 93, 31. (AKTU 2019-20)

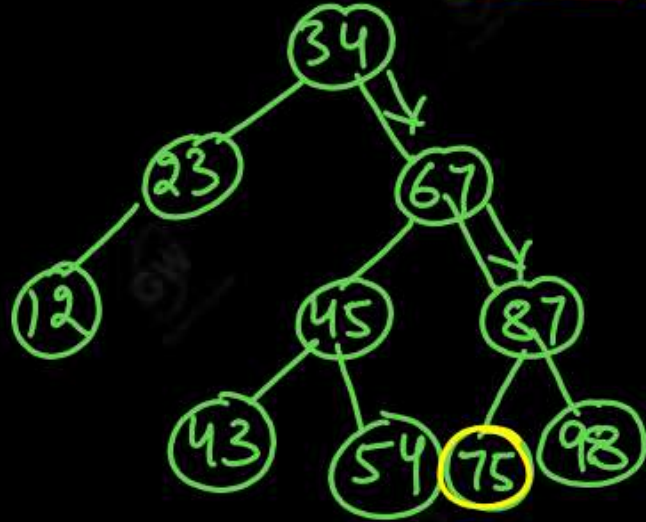
8) Insert 43



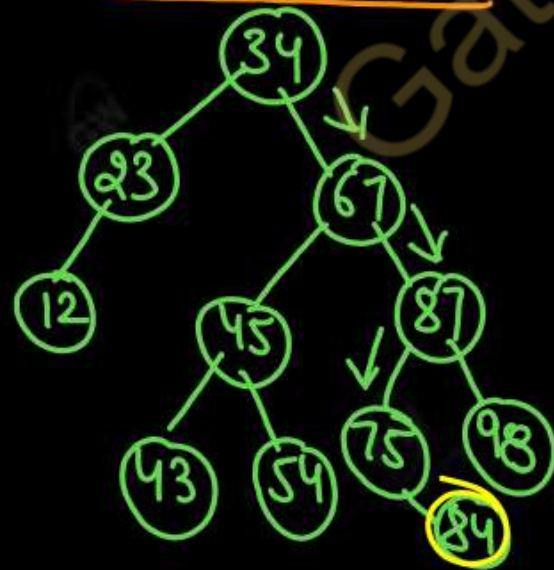
9) Insert 98



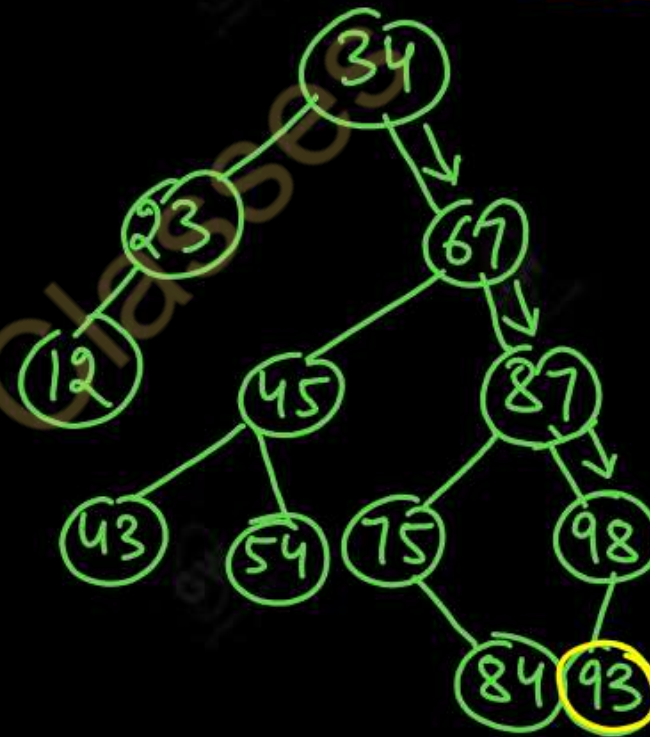
10) Insert 75



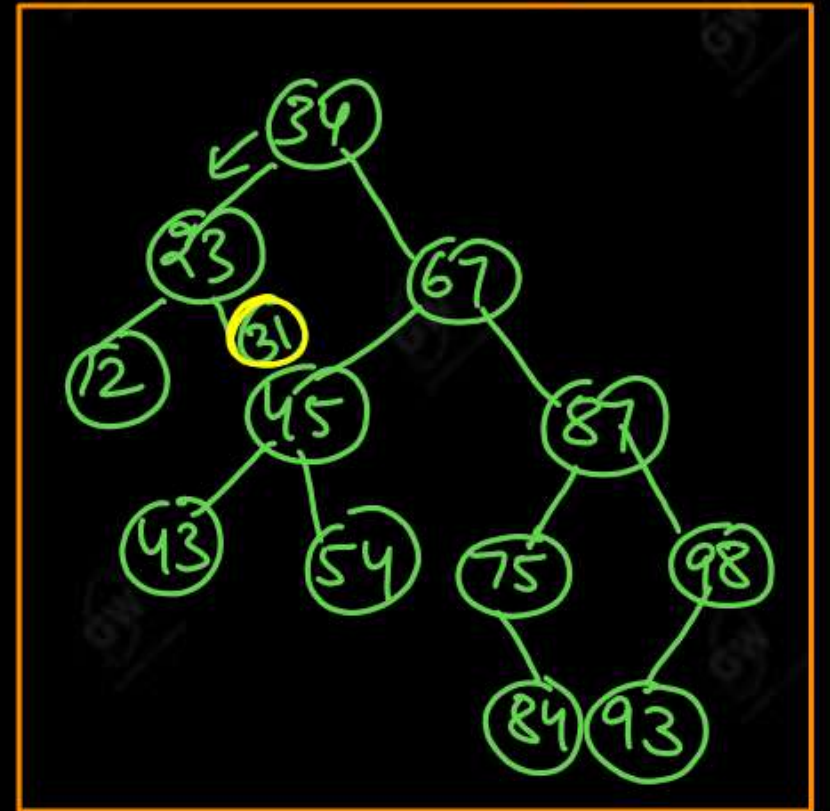
11) Insert 84



12) Insert 93



13) Insert 31



Q: Suppose the following list of letters is inserted in order into empty binary search tree:

J, R, D, G, T, E, M, H, P, A, F, Q (AKTU 2018-19)

- (i) Construct the binary Search Tree
- (ii) Find the in-order, Pre-order and Post-order Traversal of the BST created.

Ans.

Pre-order

In-order

Post-order

J, D, A, G, E, F, H, R, M, P, Q, T

A, D, E, F, G, H, J, M, P, Q, R, T

A, F, E, H, G, D, Q, P, M, T, R, J

Insert

J, R, D, G, T, E, M, H, P, A, F, Q

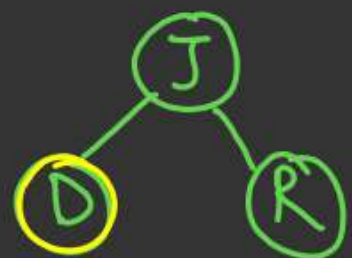
1) Insert J



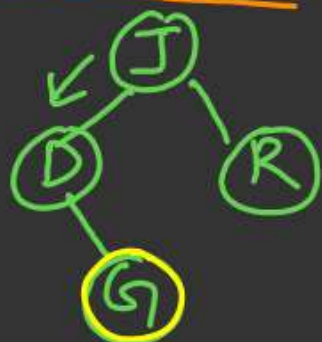
2) Insert R



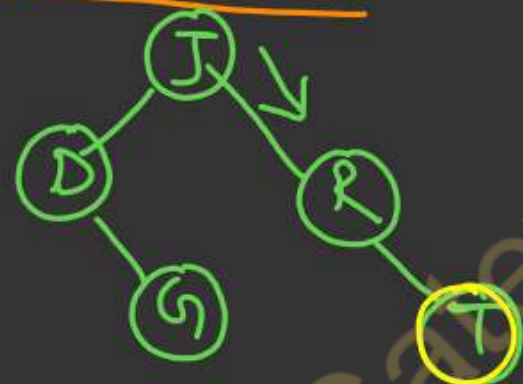
3) Insert D



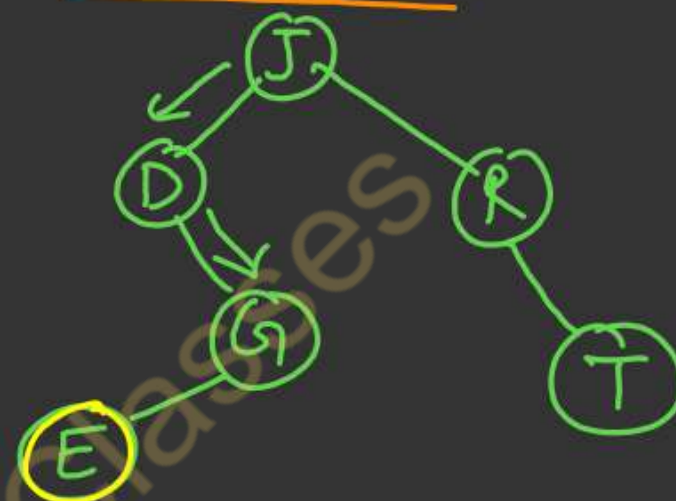
4) Insert G



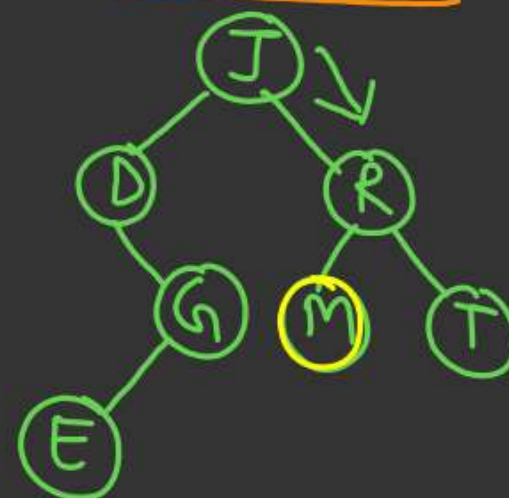
5) Insert T



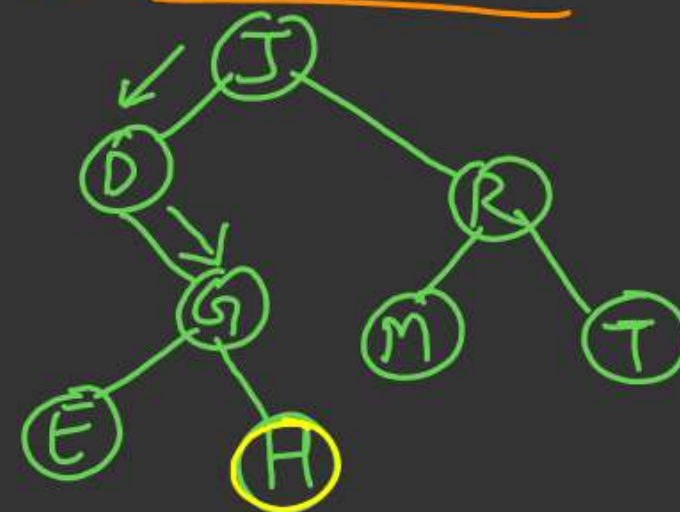
6) Insert E



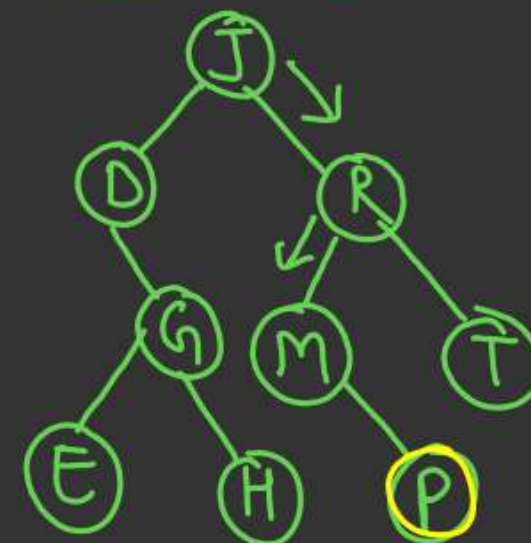
7) Insert M



8) Insert H

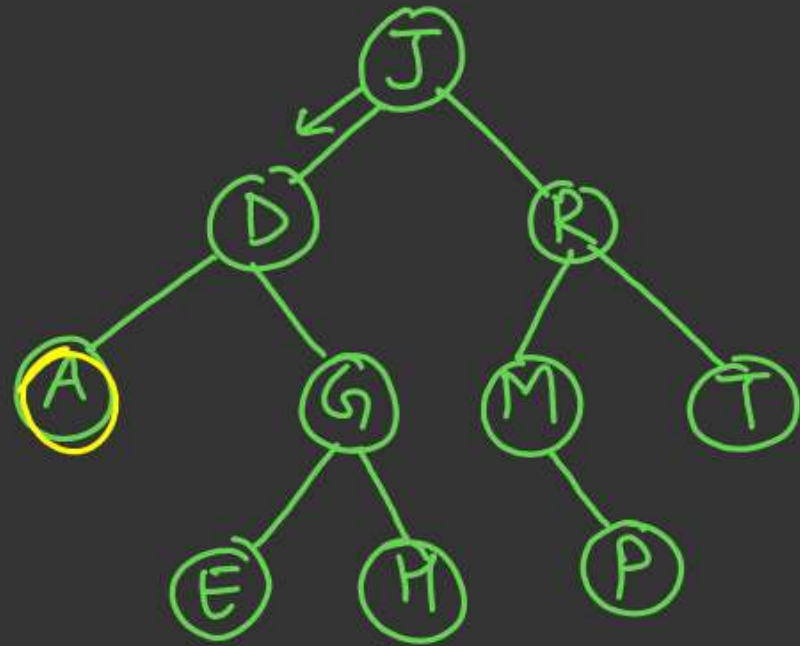


9) Insert P

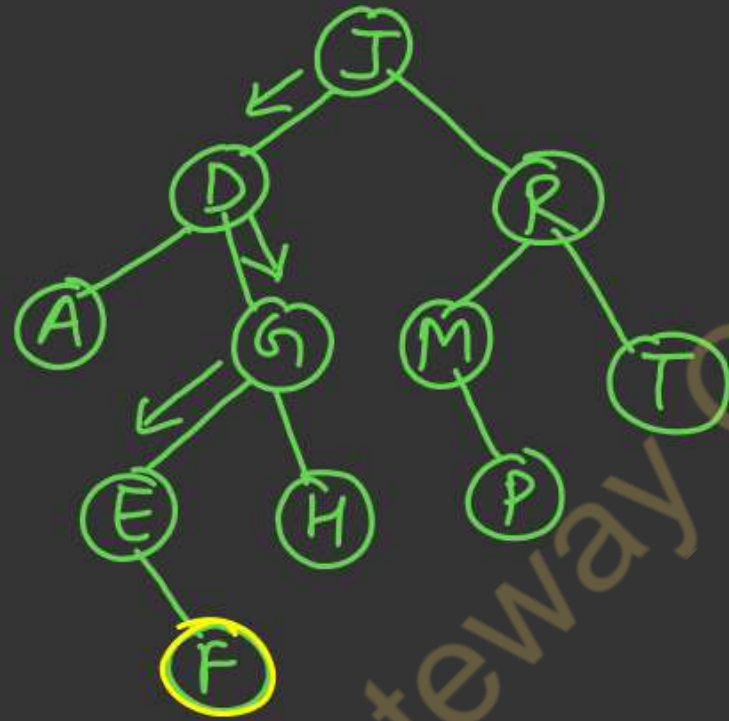


Insert J, R, D, G, T, E, M, H, P, A, F, Q

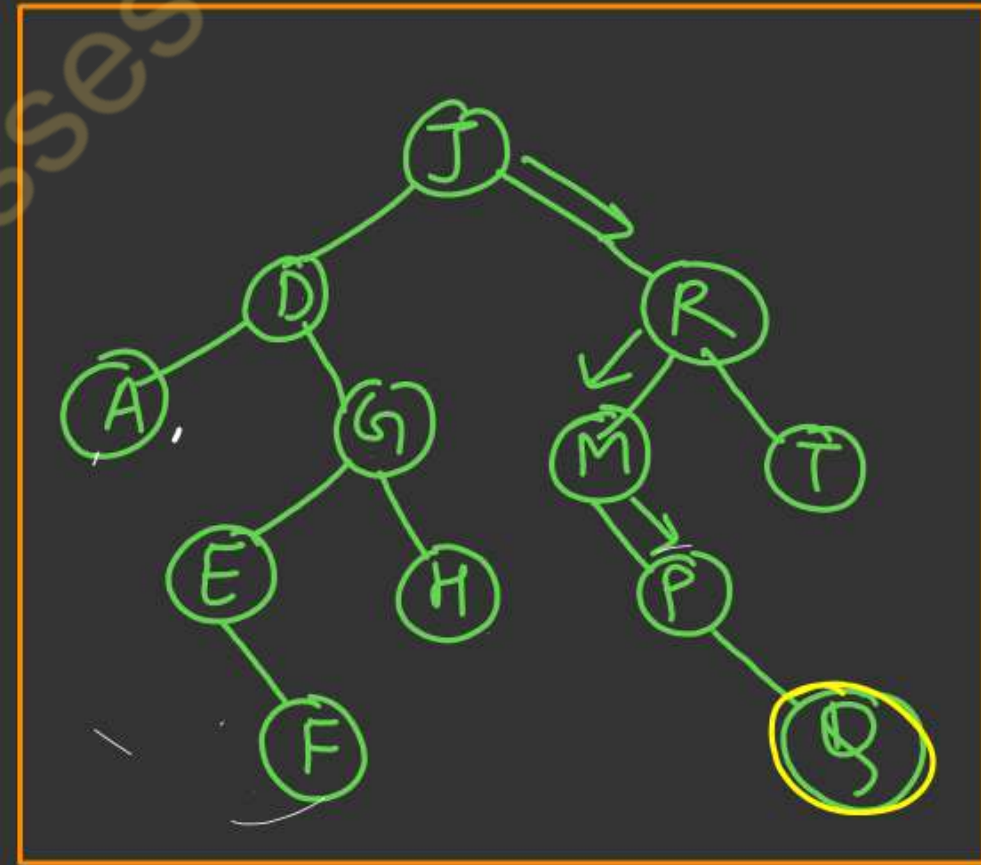
(10) Insert A



(11) Insert F



(12) Insert Q



In-order — A D E F G H J M P Q R T

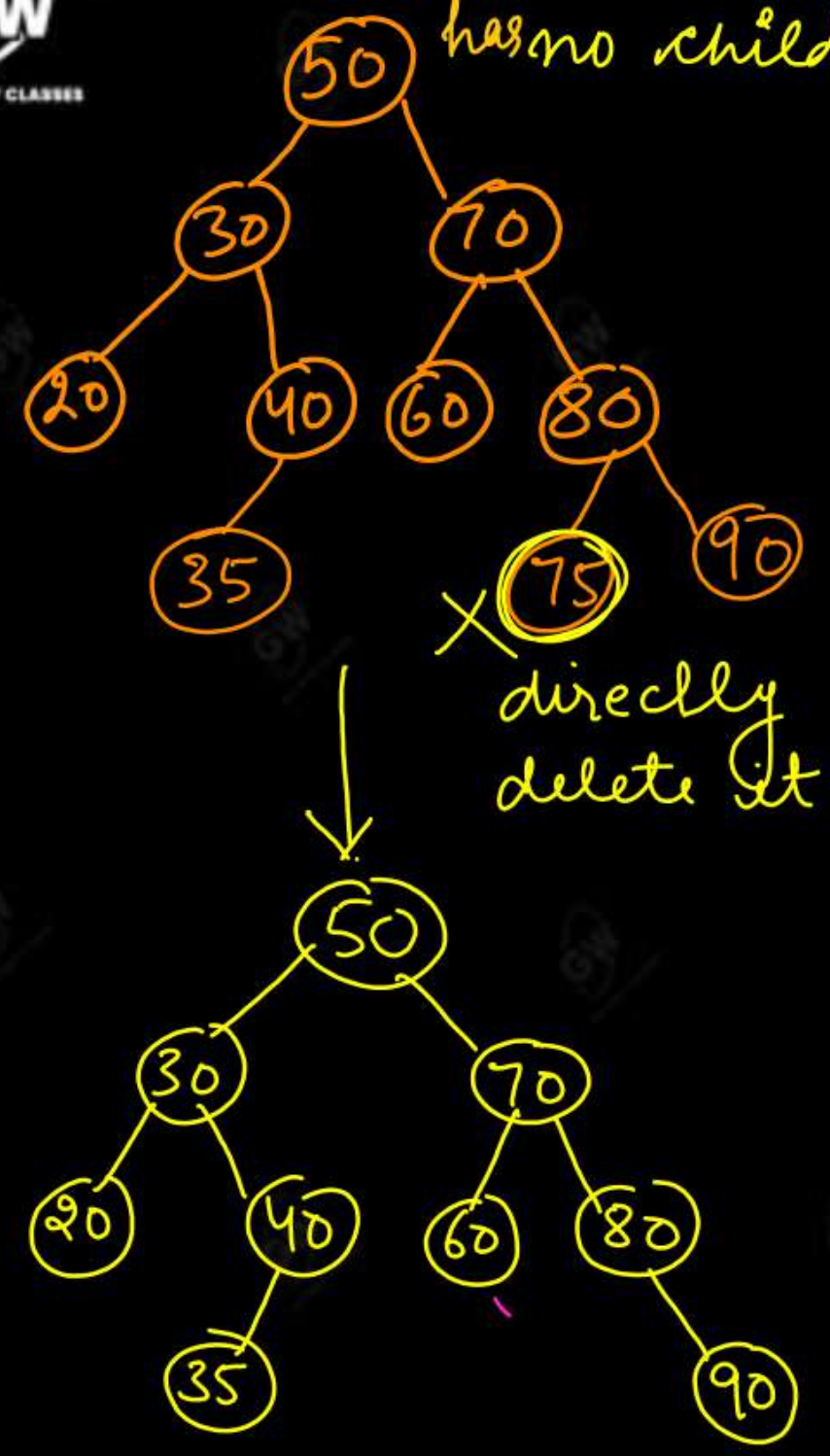
Pre-order — J D A G E F H R M P Q T

Post-order — A F E H G D Q P M T R J

Delete Operations

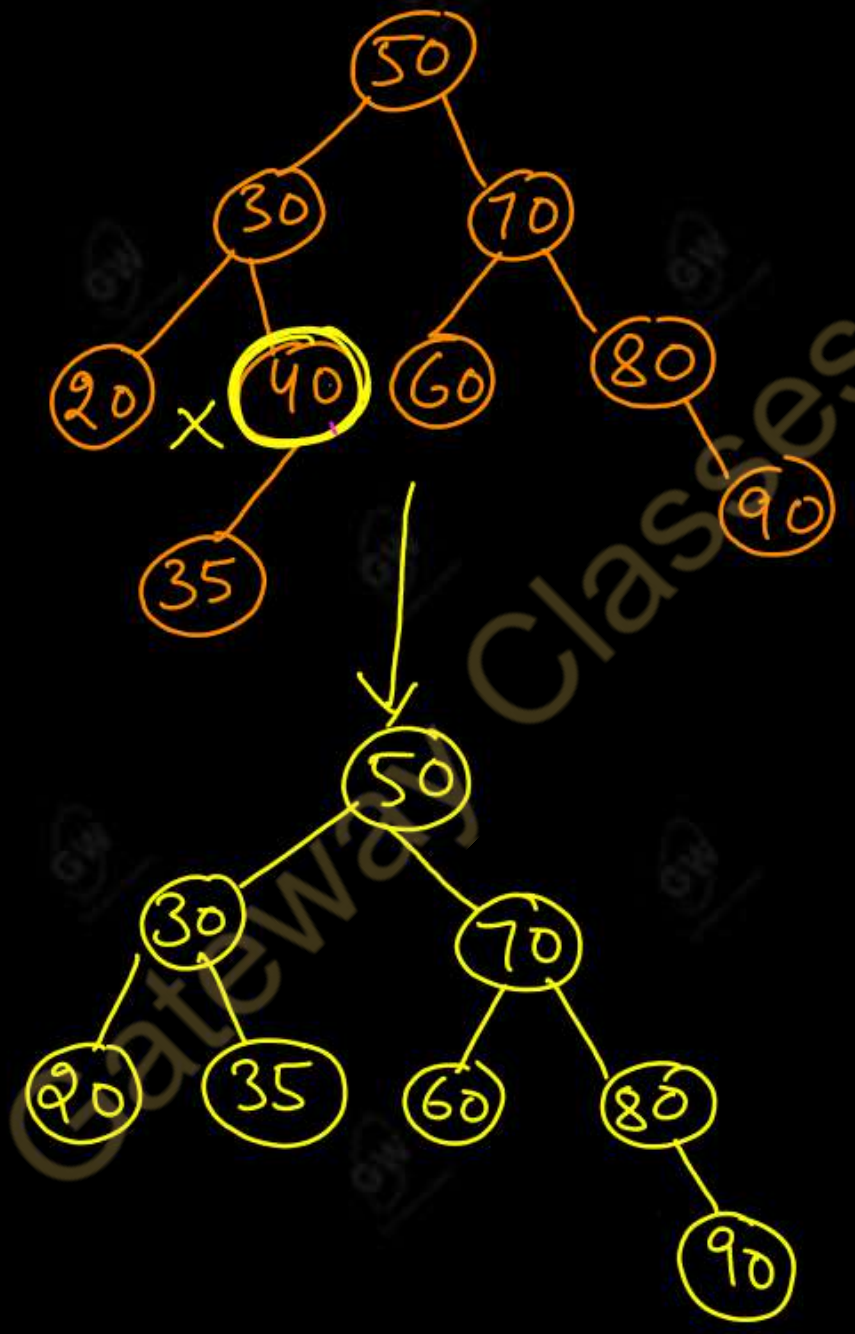
- **Case 1- Node with zero children(leaf node):** this is the easiest situation, you just need to delete the node which has no further children on the right or left.
- **Case 2 – Node with one child:** once you delete the node, simply connect its child node with the parent node of the deleted value.
- **Case 3- Node with two children:**
 - **Case 3a – In Order Predecessor:** Replace it with the largest value on left-subtree of deleted node
 - **Case 3b – In Order Successor:** Replace it with the ^{smallest} ~~largest~~ value on right-subtree of deleted node

Case-1 Node to be deleted has no child.

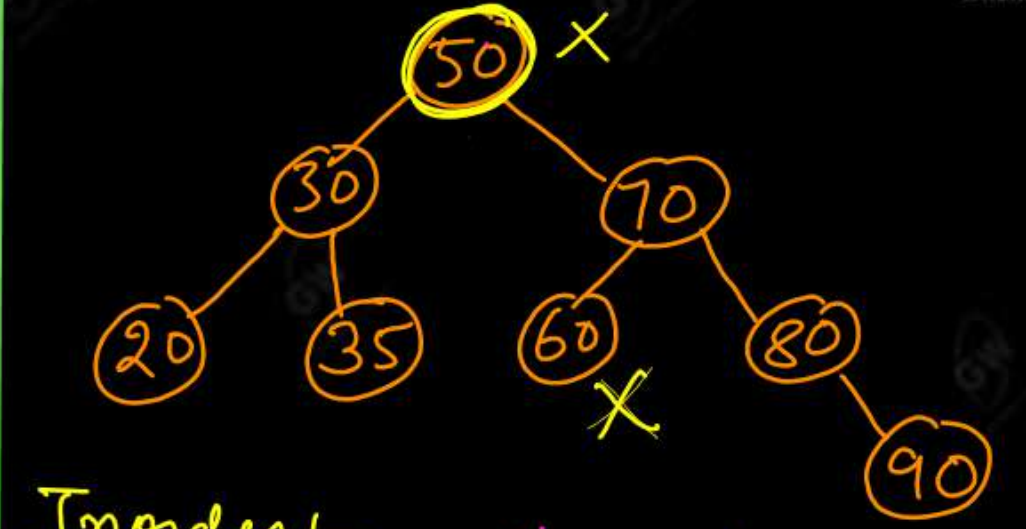


directly delete it

Case-2 → Node to be deleted has one child



Case-3 Node to be deleted has two children

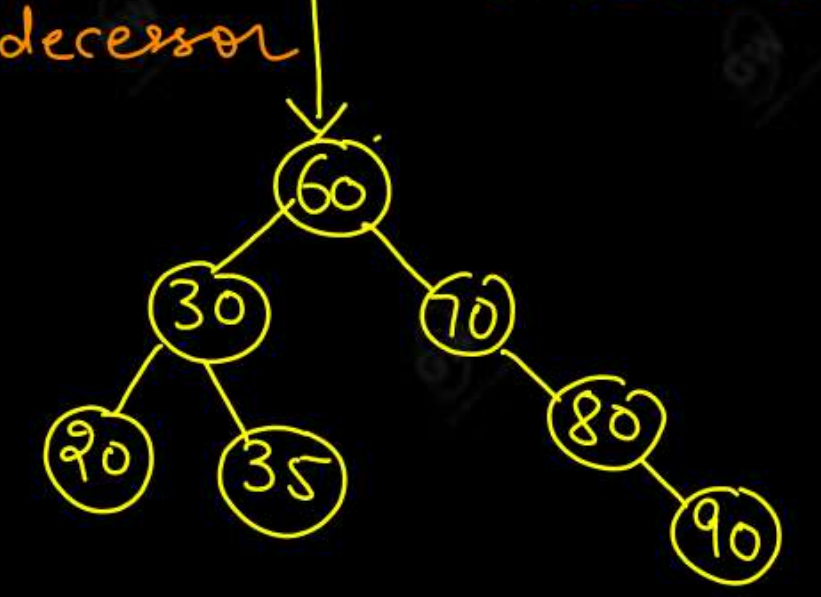


Inorder:-

20, 30, 35, 50, 60, 70, 80, 90

↑ In-order predecessor

↘ Inorder Successor

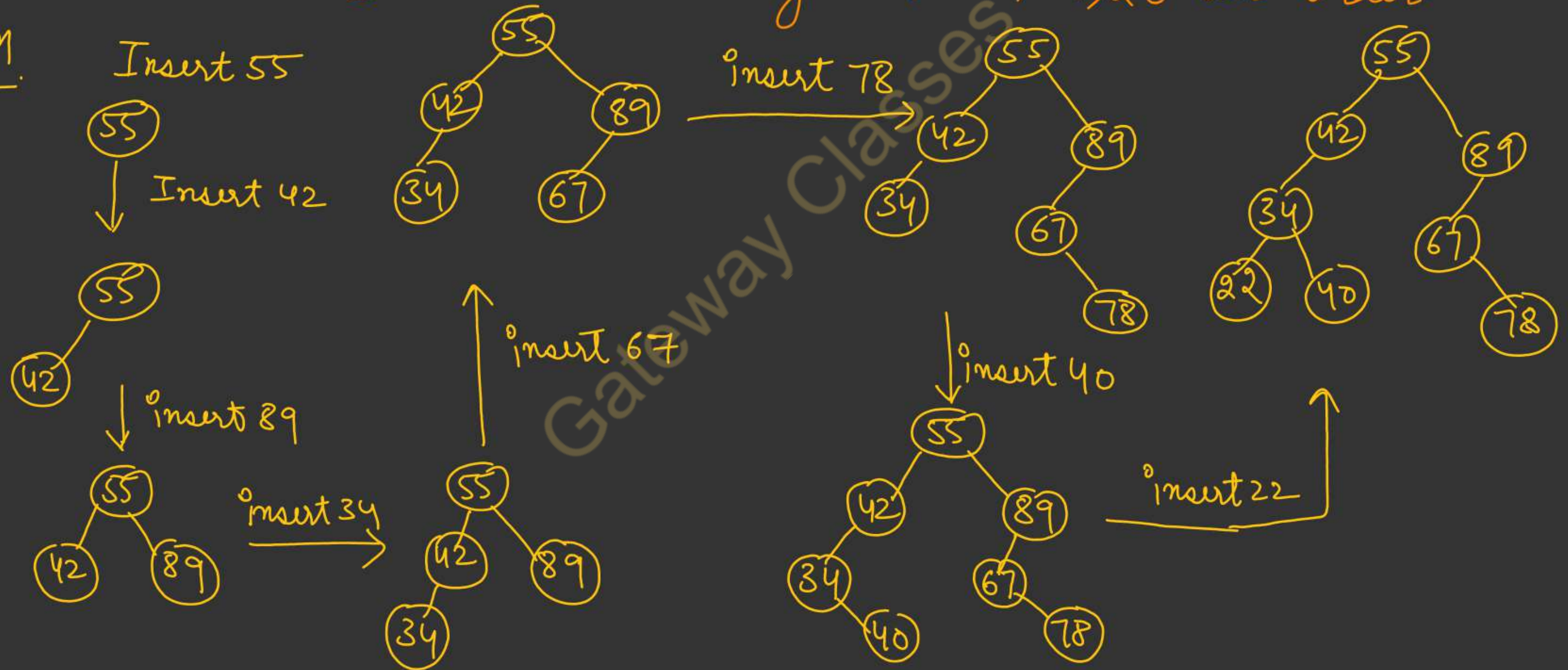


Q- Construct a BST having following keys inserted one by one.

55, 42, 89, 34, 67, 78, 40, 22, 12, 56, 88, 65, 26, 98, 102

From the resulting BST, delete the keys 56, 55, 67, 26 in order.

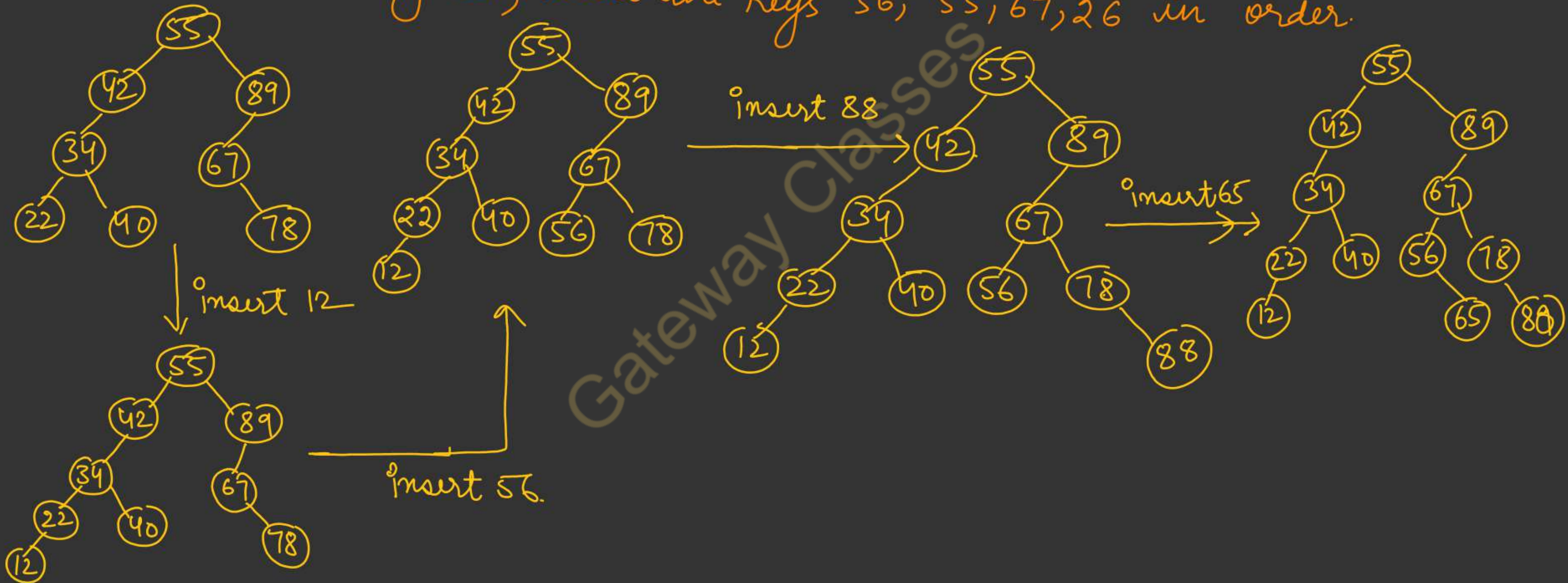
Solⁿ.



Q- Construct a BST having following keys inserted one by one.

55, 42, 89, 34, 67, 78, 40, 22, 12, 56, 88, 65, 26, 98, 102

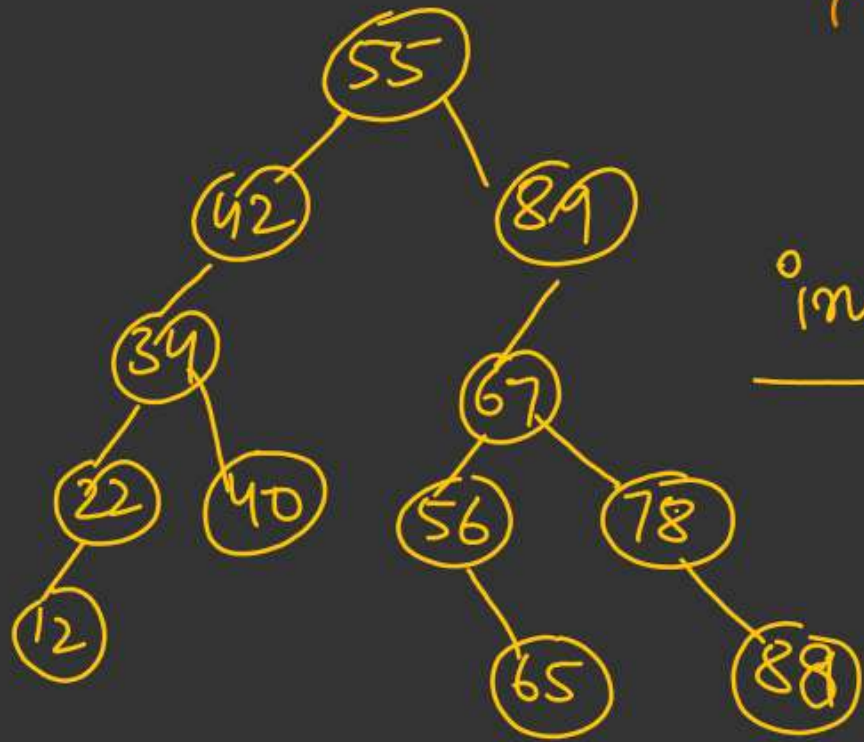
From the resulting BST, delete the keys 56, 55, 67, 26 in order.



Q- Construct a BST having following keys inserted one by one.

55, 42, 89, 34, 67, 78, 40, 22, 12, 56, 88, 65, 26, 98, 102

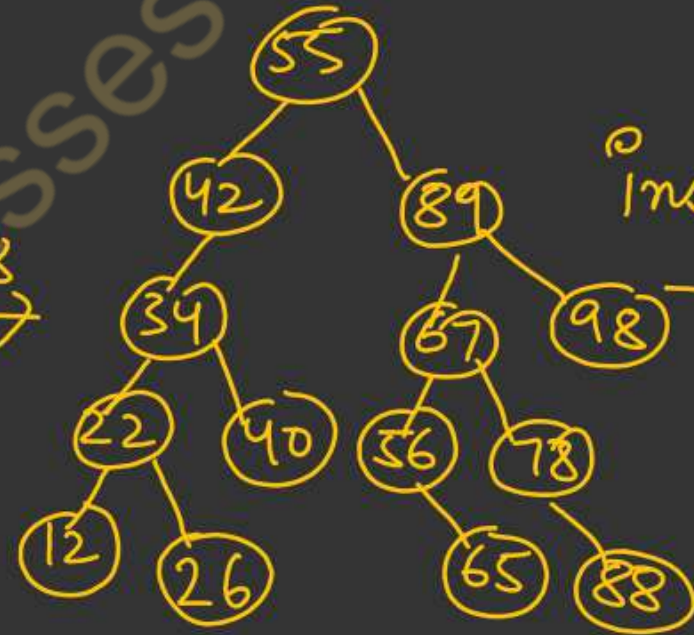
From the resulting BST, delete the keys 56, 55, 67, 26 in order.



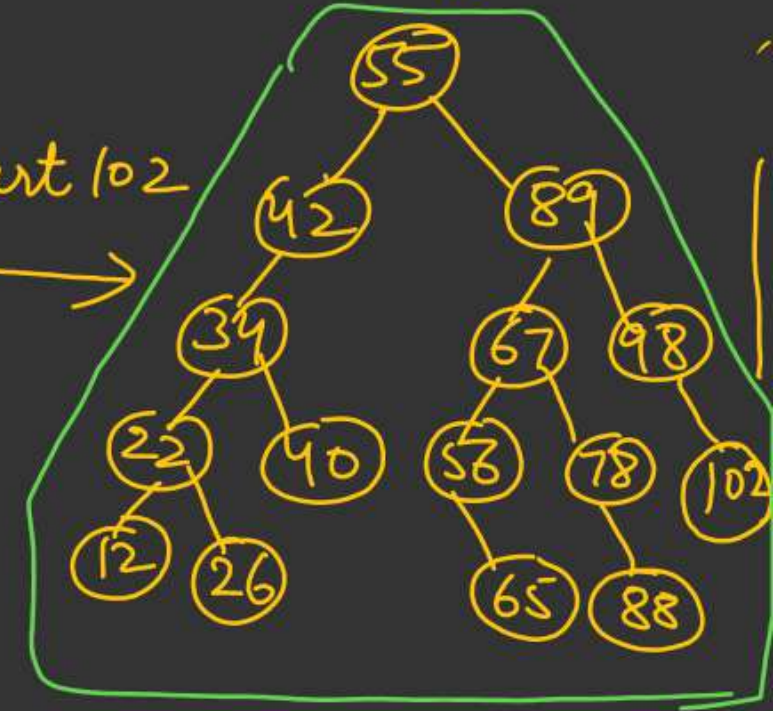
insert 26



insert 98



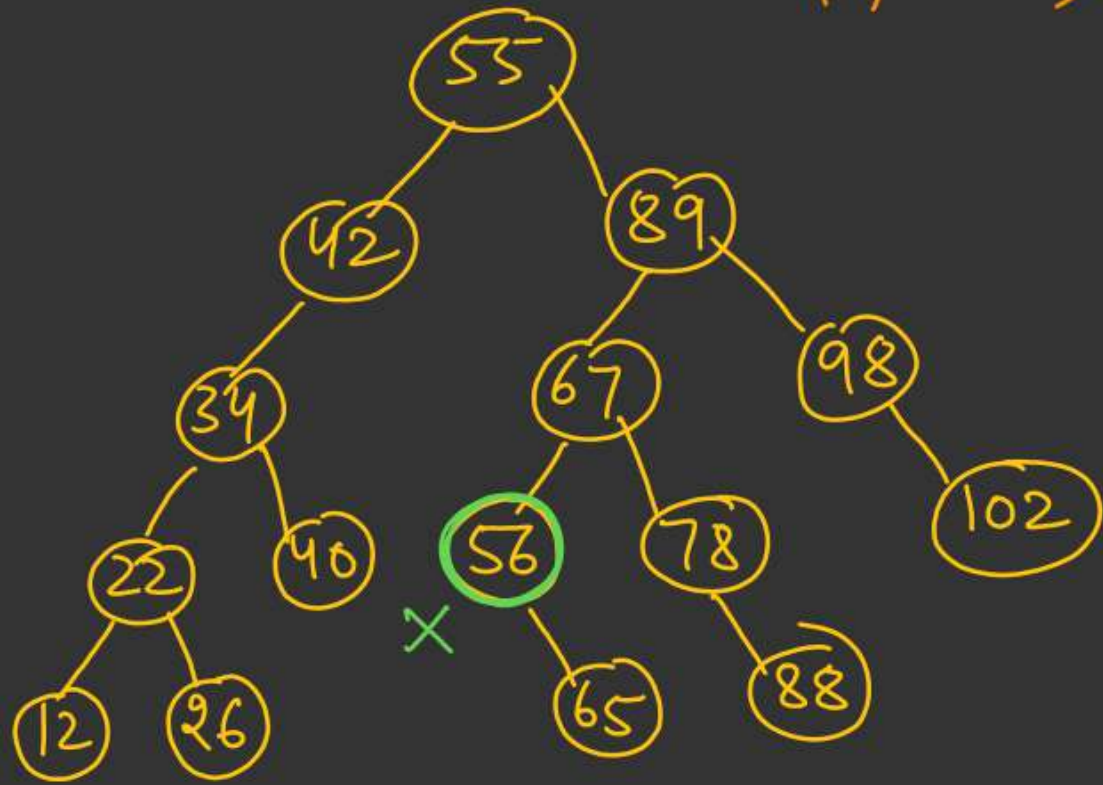
insert 102



Q- Construct a BST having following keys inserted one by one.

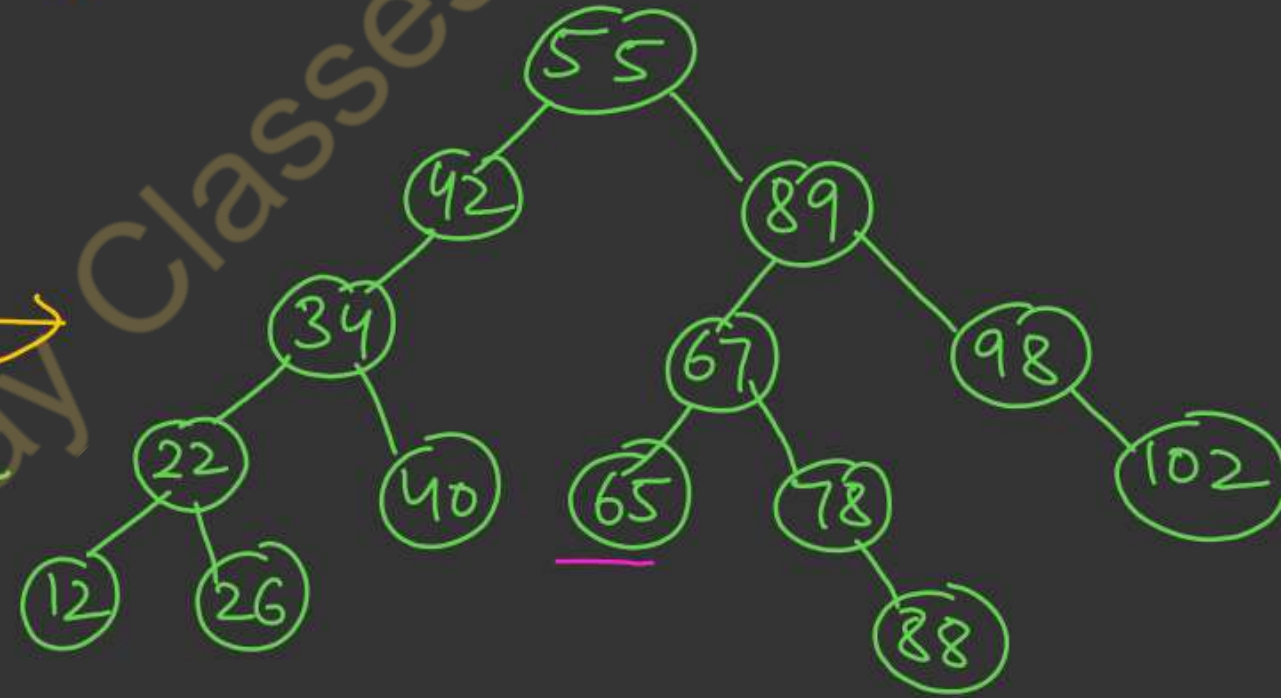
55, 42, 89, 34, 67, 78, 40, 22, 12, 56, 88, 65, 26, 98, 102

From the resulting BST, delete the keys 56, 55, 67, 26 in order.



Delete 56

Case-2



Q- Construct a BST having following keys inserted one by one.

55, 42, 89, 34, 67, 78, 40, 22, 12, 56, 88, 65, 26, 98, 102

From the resulting BST, delete the keys 56, 55, 67, 26 in order.

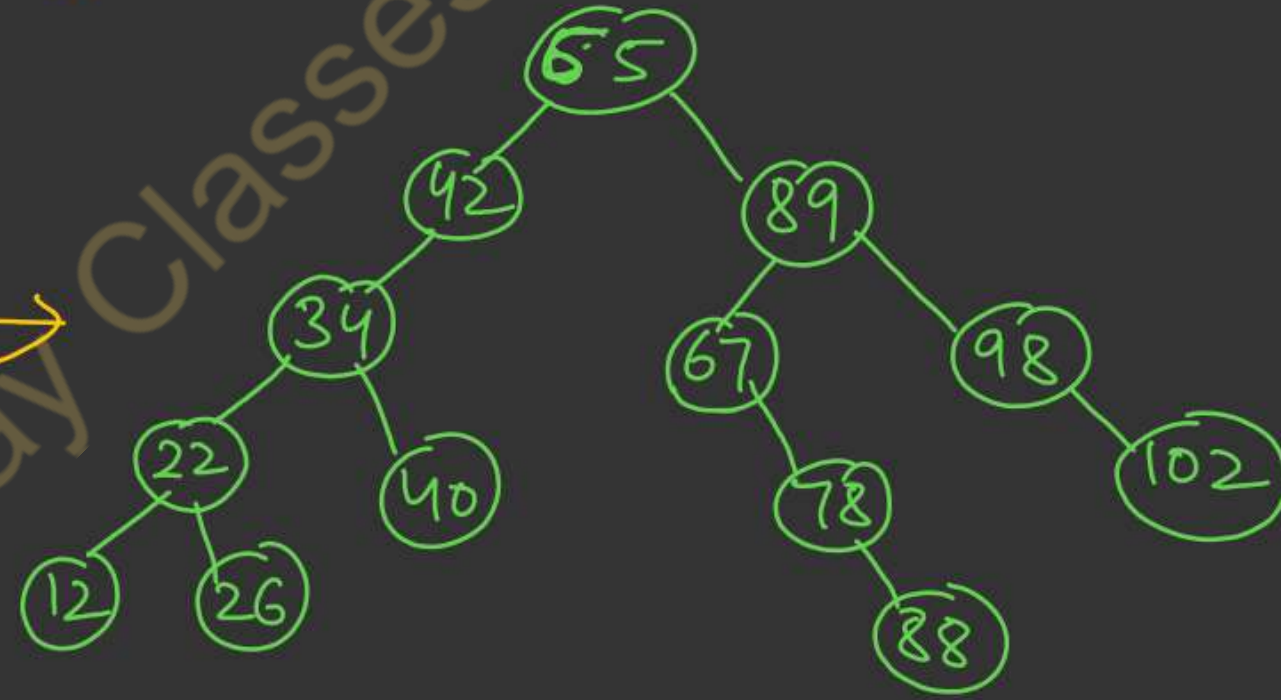


In-order
predecessor $\Rightarrow$ 42

In-order successor
 $\Rightarrow$ 65

Delete 55

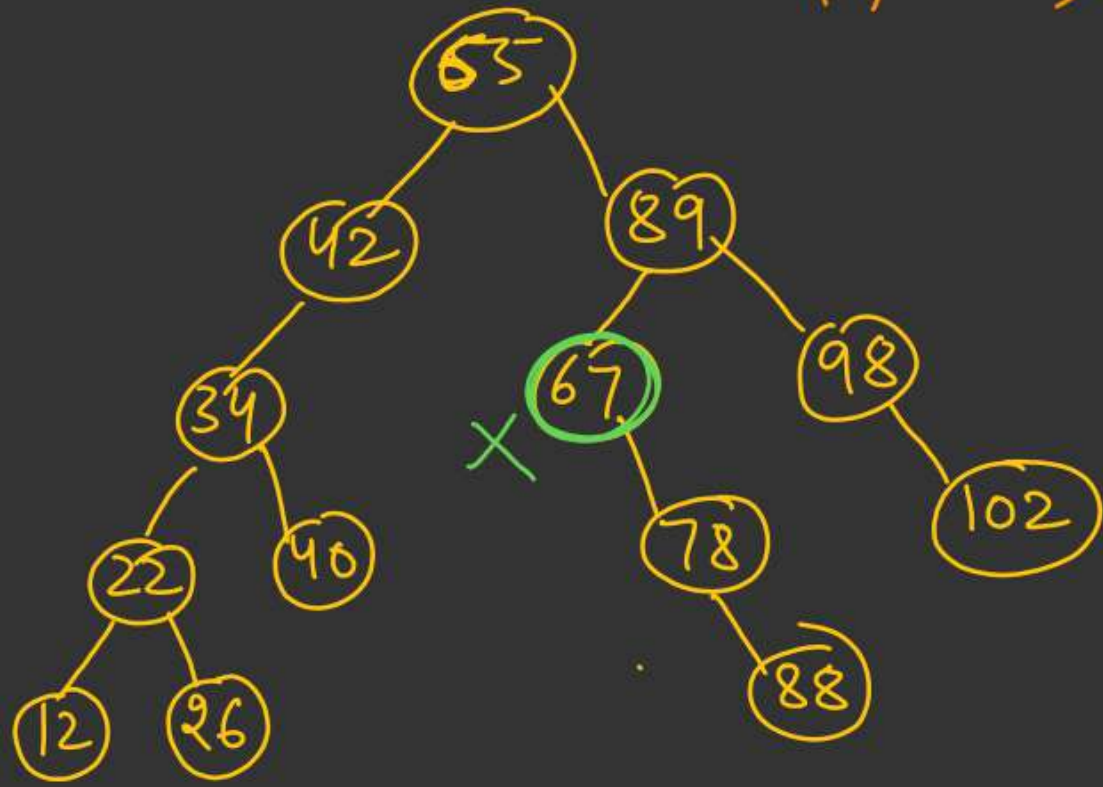
Case - 3.



Q- Construct a BST having following keys inserted one by one.

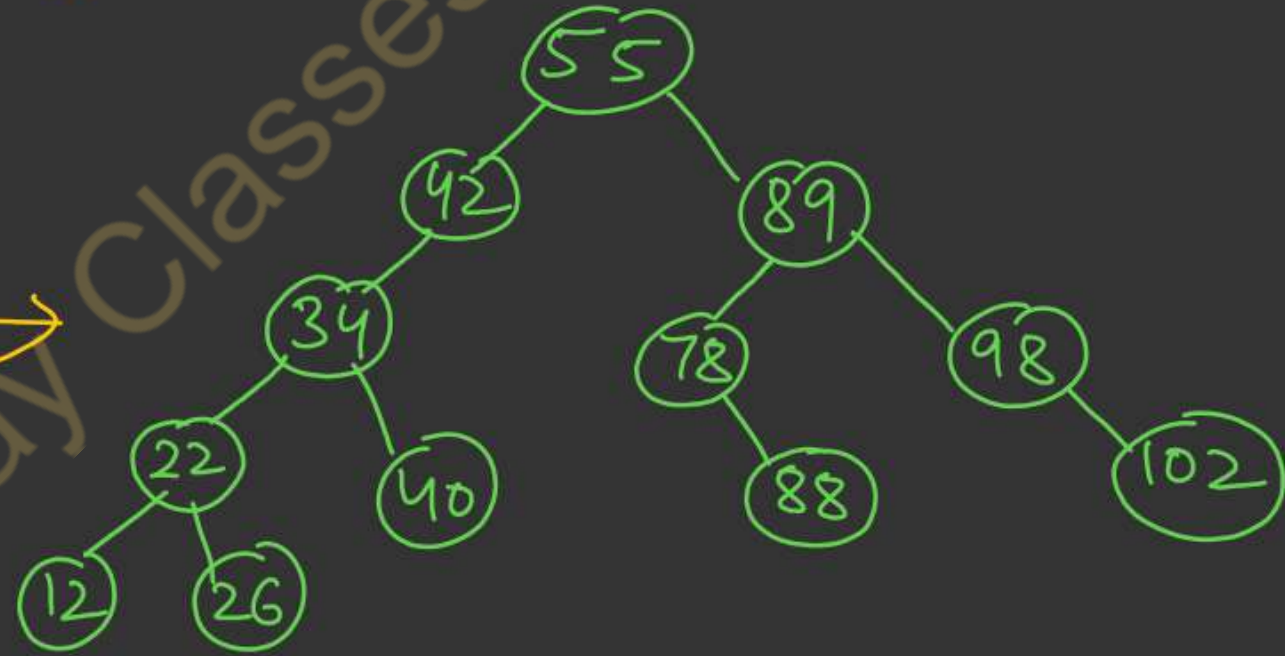
55, 42, 89, 34, 67, 78, 40, 22, 12, 56, 88, 65, 26, 98, 102

From the resulting BST, delete the keys 56, 55, 67, 26 in order.



Delete 67

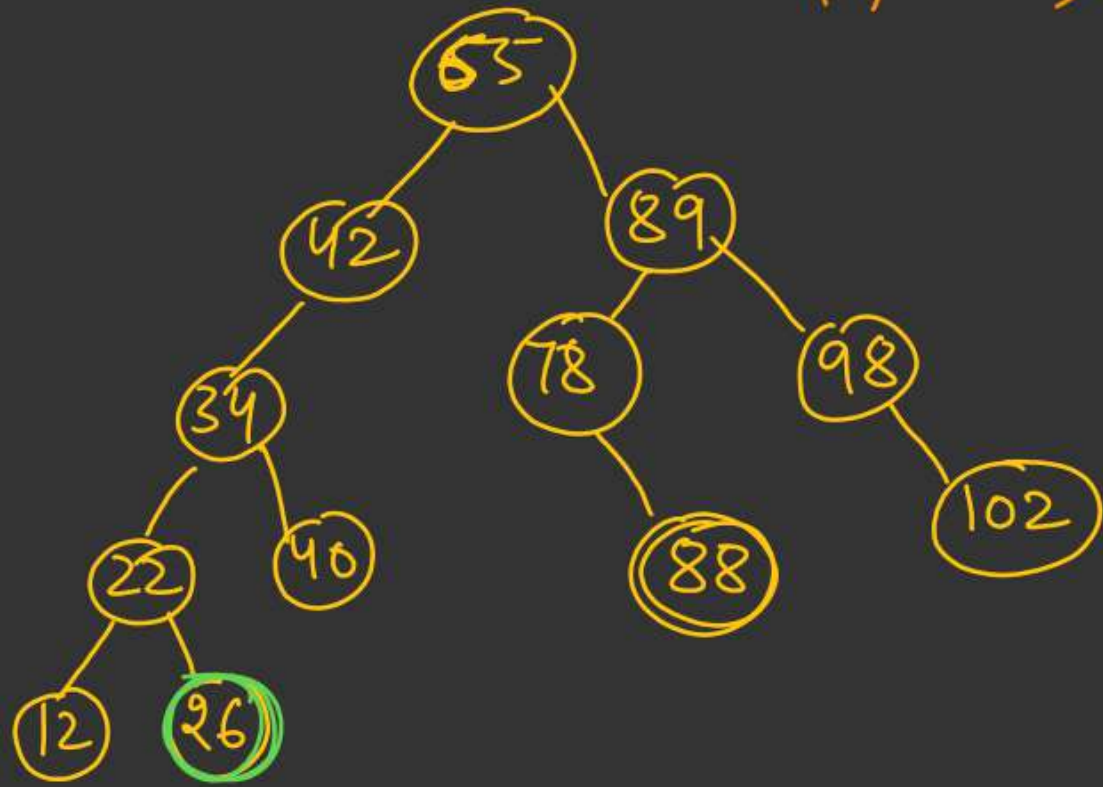
Case-2



Q- Construct a BST having following keys inserted one by one.

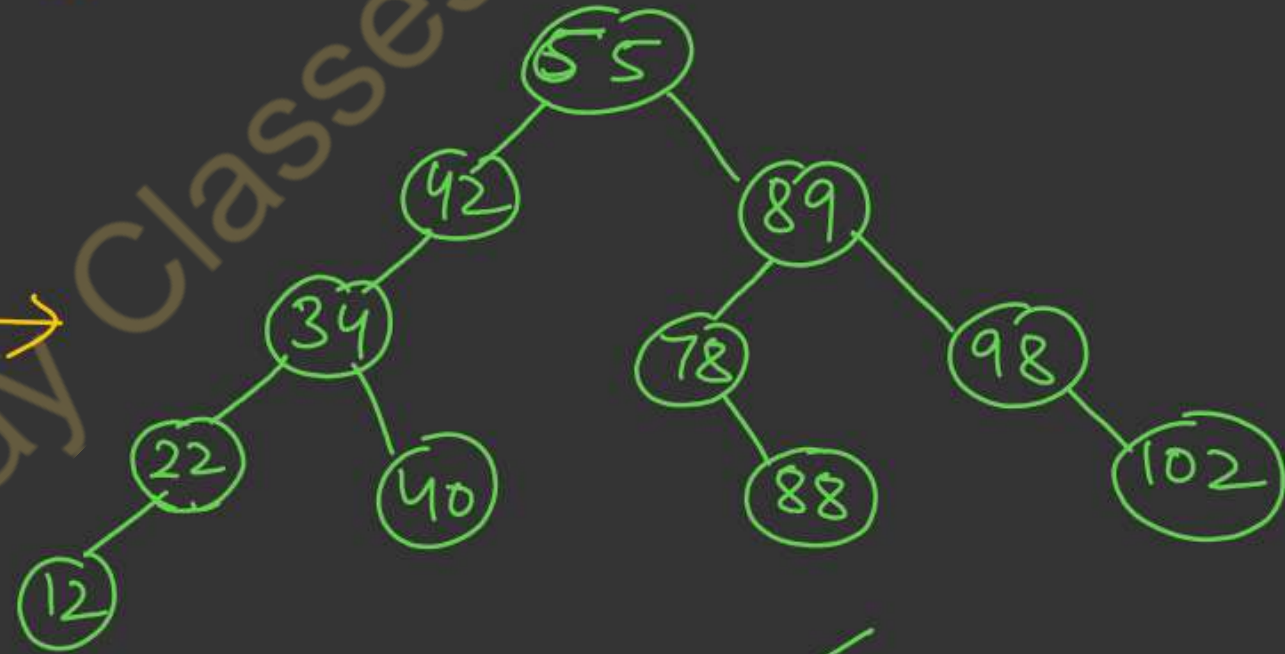
55, 42, 89, 34, 67, 78, 40, 22, 12, 56, 88, 65, 26, 98, 102

From the resulting BST, delete the keys 56, 55, 67, 26 in order.



Delete 26

Case - 1



Pseudo Code for Deleting a Node (AKTU 2017-18)

Tree-Delete(T, z)

```

1  if  $z.left == NIL$  or  $z.right == NIL$ 
2     $y = z$  ✓
3  else  $y = \text{Tree-Successor}(z)$ 
4  if  $y.left \neq NIL$  ✗
5     $x = y.left$ 
6  else  $x = y.right$ 
7  if  $x \neq NIL$ 
8     $x.parent = y.parent$ 
9  if  $y.parent == NIL$ 
10    $T.root = x$ 
11 else if  $y == y.parent.left$  // y is the left child of its parent
12    $y.parent.left = x$ 
13 else  $y.parent.right = x$ 
14 if  $y \neq z$ 
15    $z.key = y.key$ 
16   copy  $y$ 's satellite data into  $z$ 
17 return  $y$ 

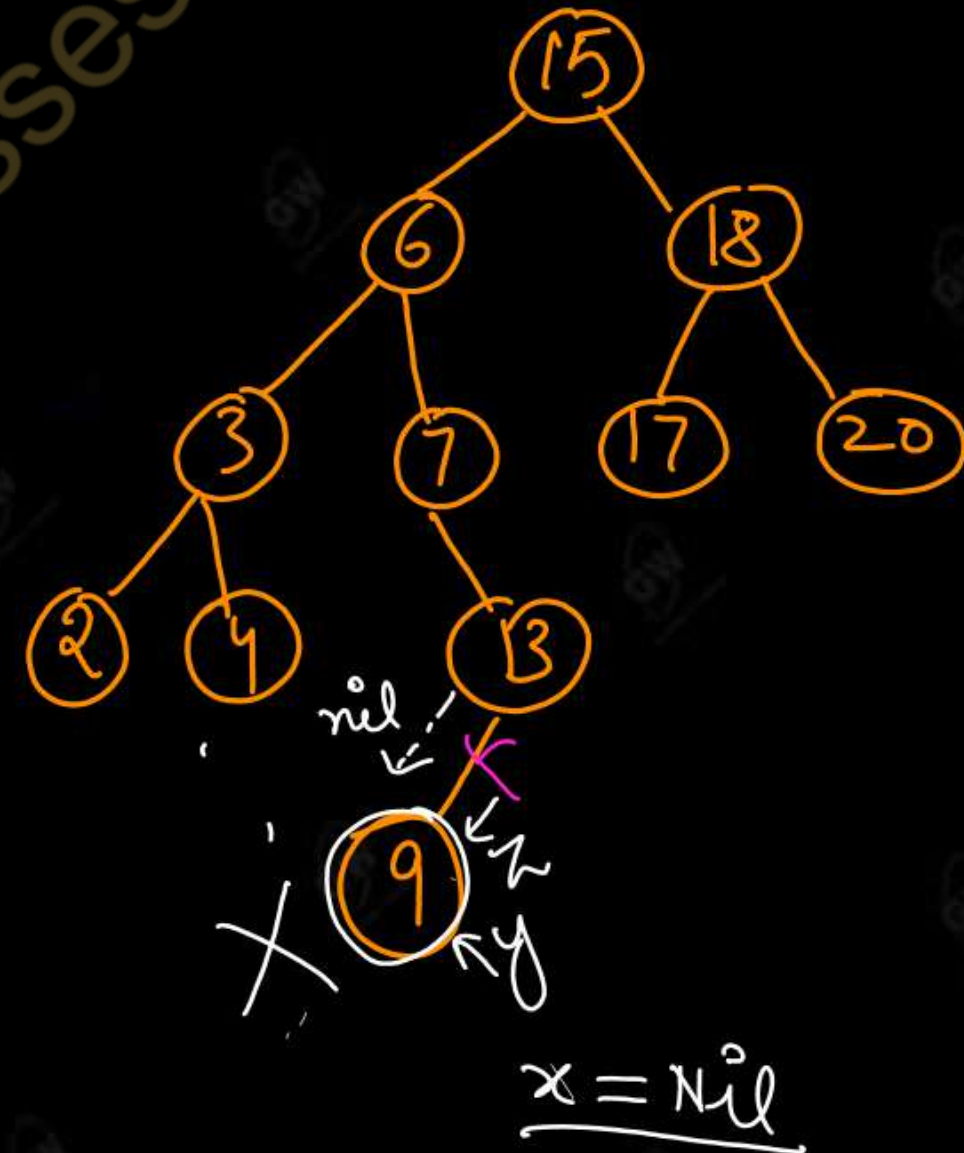
```

z = user-selected node to delete

y = node actually removed (either z or successor of z)

x = child that replaces y in the BST

Case - 1



Pseudo Code for Deleting a Node (AKTU 2017-18)

Tree-Delete(T, z)

1 if $z.left == NIL$ or $z.right == NIL$

2 $y = z$ ✓

3 else $y = \text{Tree-Successor}(z)$ ✗

4 if $y.left \neq NIL$ ✓

5 $x = y.left$ ✓

6 else $x = y.right$ ✗

7 if $x \neq NIL$ ✓

8 $x.parent = y.parent$ ✓

9 if $y.parent == NIL$ ✗

10 $T.root = x$ ✗

11 else if $y == y.parent.left$ ✗ // y is the left child of its parent

12 $y.parent.left = x$ ✗

13 else $y.parent.right = x$ ✓

14 if $y \neq z$

15 $z.key = y.key$

16 copy y 's satellite data into z

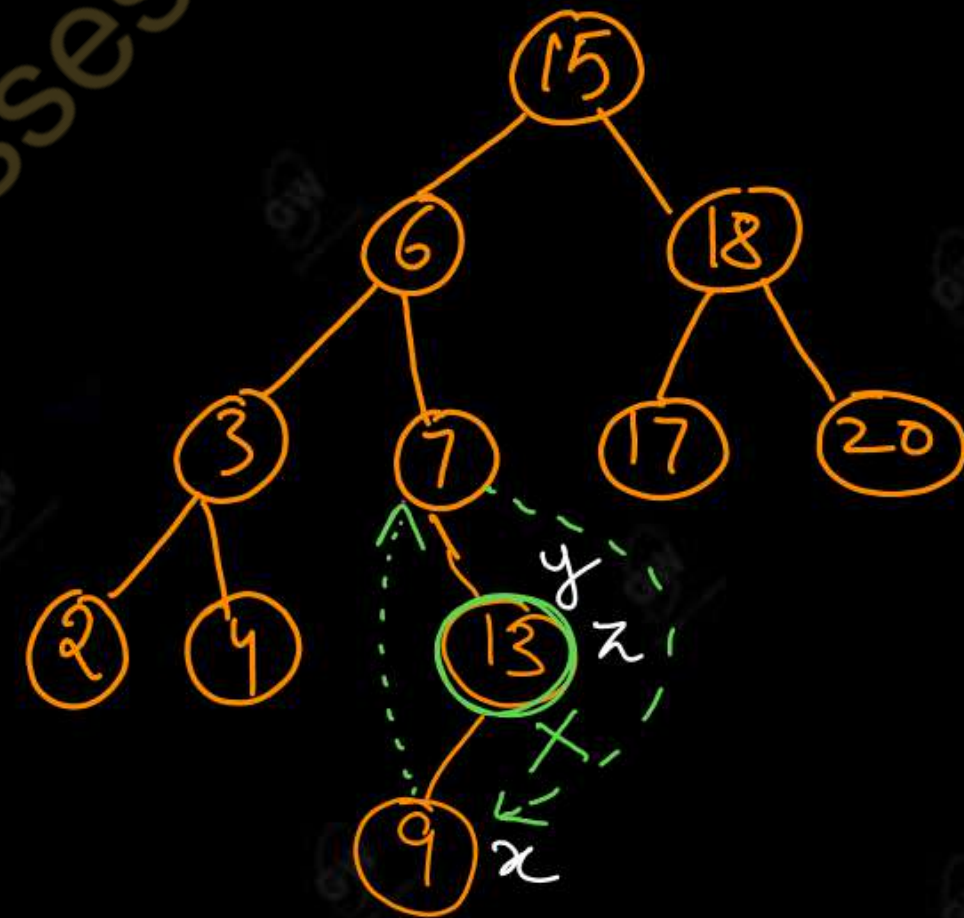
17 return y

z = user-selected node to delete

y = node actually removed (either z or successor of z)

x = child that replaces y in the BST

Case - 2



Pseudo Code for Deleting a Node (AKTU 2017-18)

Tree-Delete(T, z)

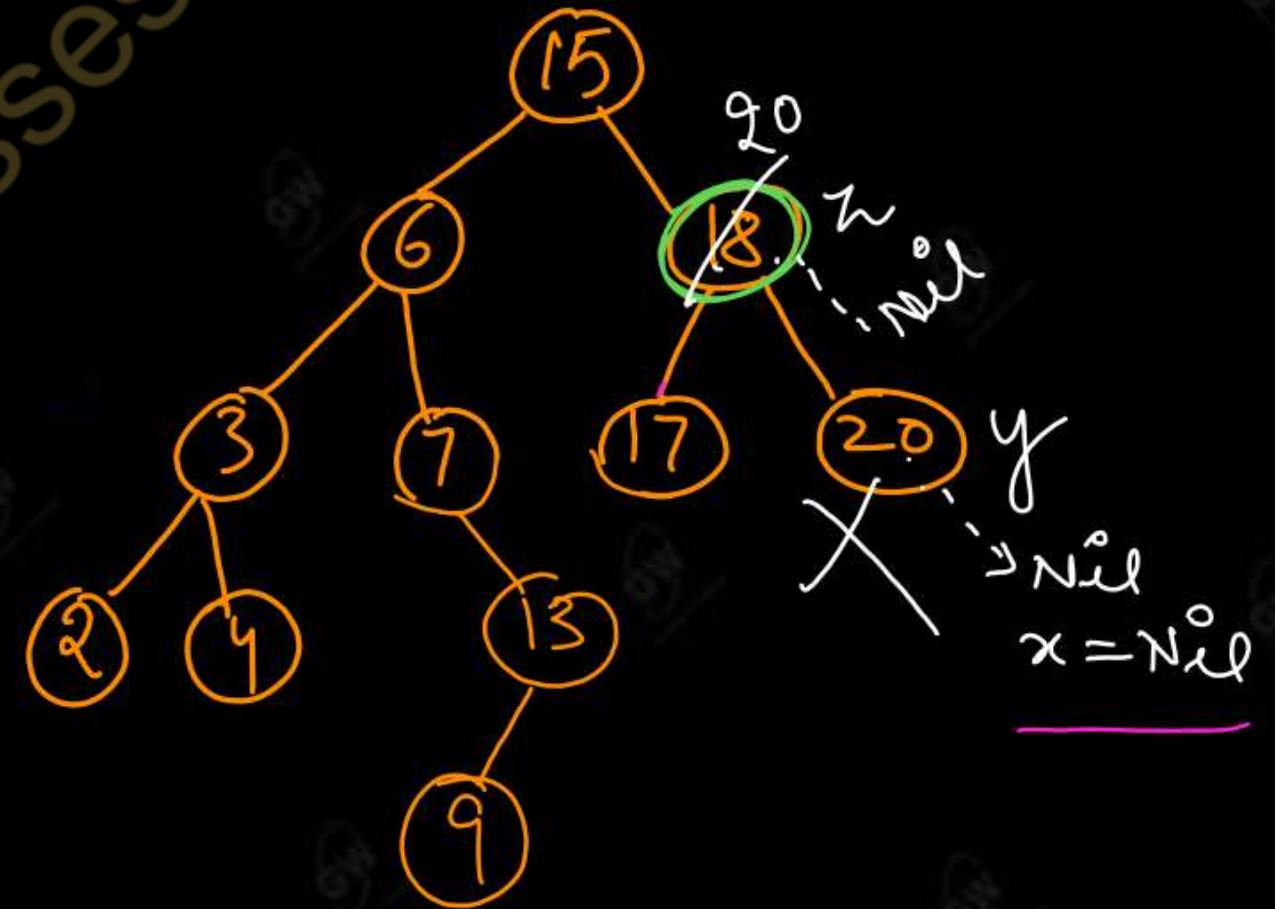
- 1 if $z.\text{left} == \text{NIL}$ or $z.\text{right} == \text{NIL}$ ✗
- 2 $y = z$ ✗
- 3 else $y = \text{Tree-Successor}(z)$
- 4 if $y.\text{left} != \text{NIL}$ ✗
- 5 $x = y.\text{left}$ ✗
- 6 else $x = y.\text{right}$ ✓
- 7 if $x != \text{NIL}$ ✗
- 8 $x.\text{parent} = y.\text{parent}$ ✗
- 9 if $y.\text{parent} == \text{NIL}$ ✗
- 10 $T.\text{root} = x$ ✗
- 11 else if $y == y.\text{parent}.\text{left}$ ✗
- 12 $y.\text{parent}.\text{left} = x$ ✗
- 13 else $y.\text{parent}.\text{right} = x$ ✓
- 14 if $y \neq z$ ✓
- 15 $z.\text{key} = y.\text{key}$ ✓
- 16 copy y 's satellite data into z ✓
- 17 return y

z = user-selected node to delete

y = node actually removed (either z or successor of z)

x = child that replaces y in the BST

Case - 3

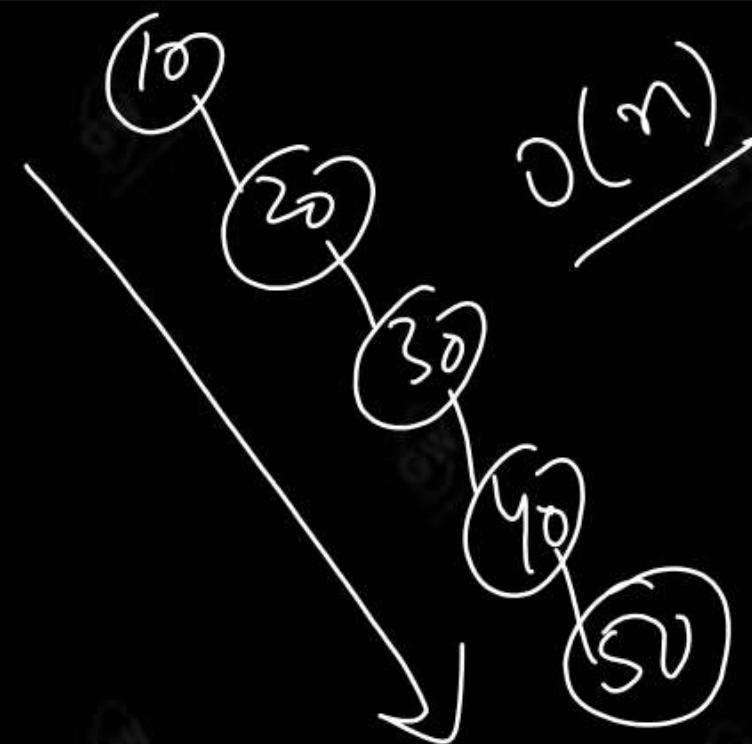


Time Complexity:

Operation	Best Case Complexity	Average Case Complexity	Worst Case Complexity
Search	$O(\log n)$	$O(\log n)$	$O(n)$
Insertion	$O(\log n)$	$O(\log n)$	$O(n)$
Deletion	$O(\log n)$	$O(\log n)$	$O(n)$

Space Complexity:

- The space complexity for all the operations: $O(n)$



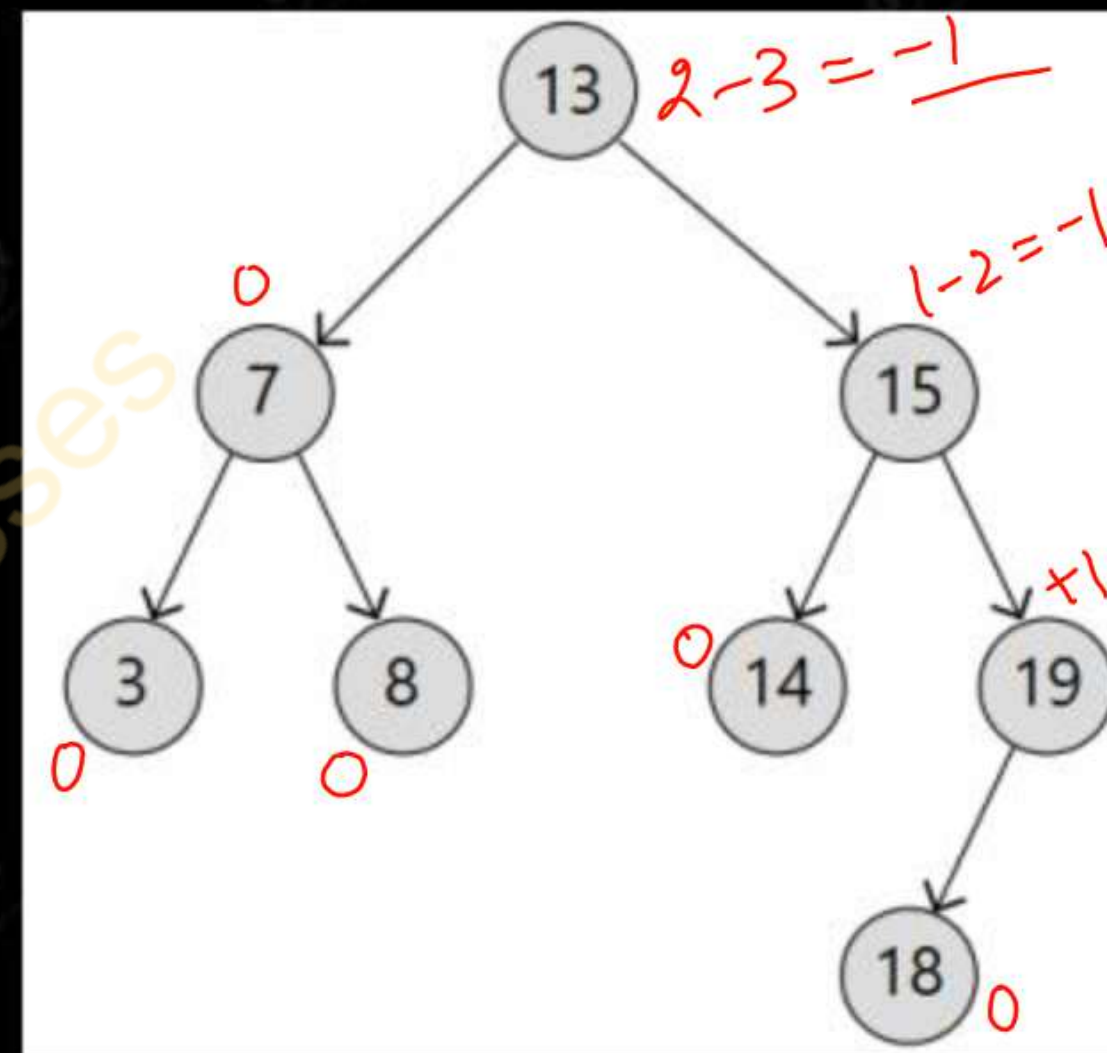
BST Compared to Arrays and Linked Lists.

Data Structure	Searching for a value	Delete / Insert leads to shifting in memory
Sorted Array	$O(\log n)$ ✓	Yes ✓
Linked List	$O(n)$ ✓	No ✓
Binary Search Tree	$O(\log n)$ ✓	No ✓

- Searching in a BST is as fast as [Binary Search](#) on an array, with the same time complexity $O(\log n)$.
- Deleting and inserting new values can be done without shifting elements in memory, just like with Linked Lists.

BST Balance and Time Complexity

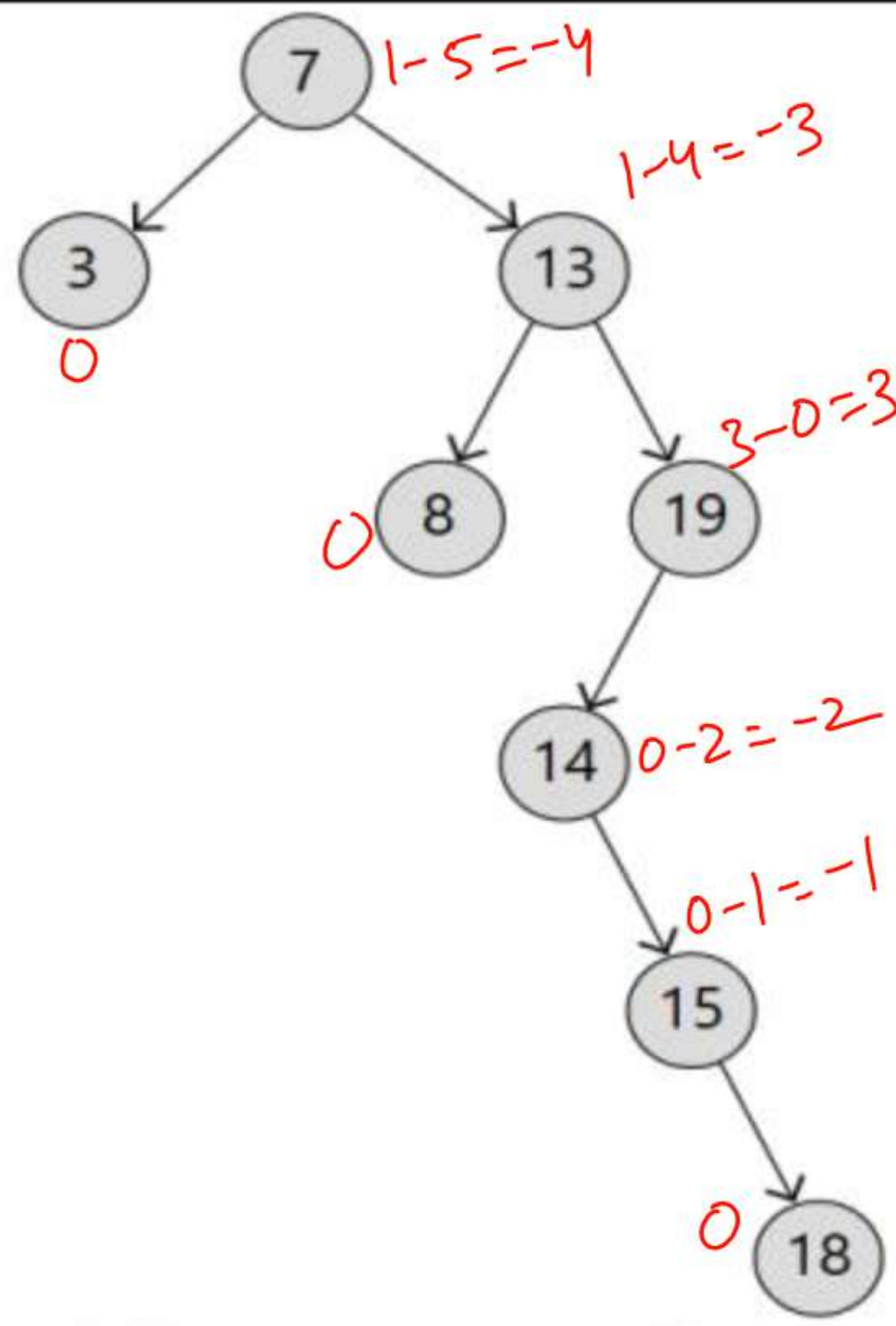
- On a BST, operations like inserting/deleting/searching for a node are actually $O(h)$. That means that the higher the tree is (h), the longer the operation will take. We know that $h = \log n$
- But for that the tree should be "balanced" (approximately the same number of nodes on the left and right side of the tree.)



$-1/+1/0$

BST is balanced if height of the left and right subtrees of any node only differs by one.

- For a balanced BST, with a large number of nodes (big n), we get height $h \approx \log_2 n$, so time complexity for searching, deleting, or inserting a node can be written as $O(h) = O(\log n)$.
- But, if BST is unbalanced, the height of the tree is approximately the same as the number of nodes, $h \approx n$, and we get time complexity $O(h) = O(n)$ for searching, deleting, or inserting a node.



To optimize operations on a BST, height must be minimized, and for that tree must be balanced.

Practical Applications of BST:

1. **Databases:** Many database systems use BSTs for indexing data, which allows for fast retrieval of records based on keys.
2. **Symbol Tables:** Compilers and interpreters use BSTs to store and search for identifiers (e.g., variable names) efficiently.
3. **File Systems:** BSTs are employed in file systems to organize directory structures and quickly locate files.
4. **Caches:** In computer architecture, BSTs are used in caches to efficiently manage and search for cached data.
5. **Network Routing:** Routing tables in network routers often use BSTs for efficient routing decisions.

1. Construct a Binary Search Tree (BST) using the following sequence of numbers: 50, 30, 70, 20, 40, 60, 80, 35, 45. Perform In-order traversal of Tree. (AKTU 2024-25)
2. Differentiate between binary search tree and a heap. (AKTU 2022-23)
3. Draw the binary search tree that results from inserting the following numbers in sequence starting with 11: 11, 47, 81, 9, 61, 10, 12 (AKTU 2021-22)
4. Write an iterative function to search a key in Binary Search Tree (BST). (AKTU 2022-23)
5. What is the difference between a binary search tree (BST) and heap? For a given sequence of numbers, construct a heap and a BST. 34, 23, 67, 45, 12, 54, 87, 43, 98, 75, 84, 93, 31. (AKTU 2019-20)

Data Structure

Unit-4 DPP-2

6. Define Binary search tree. How it is different from binary tree? (AKTU 2018-19)
7. Suppose the following list of letters is inserted in order into empty binary search tree: J, R, D, G, T, E, M, H, P, A, F, Q (AKTU 2018-19)
 - (i) Construct the binary Search Tree
 - (ii) Find the in-order, Pre-order and Post-order Traversal of the BST created.
8. What is binary search tree? Write the important applications of binary search tree. Write algorithm to delete a node from a binary search tree. (AKTU 2017-18)



AKTU

B.Tech III-Sem



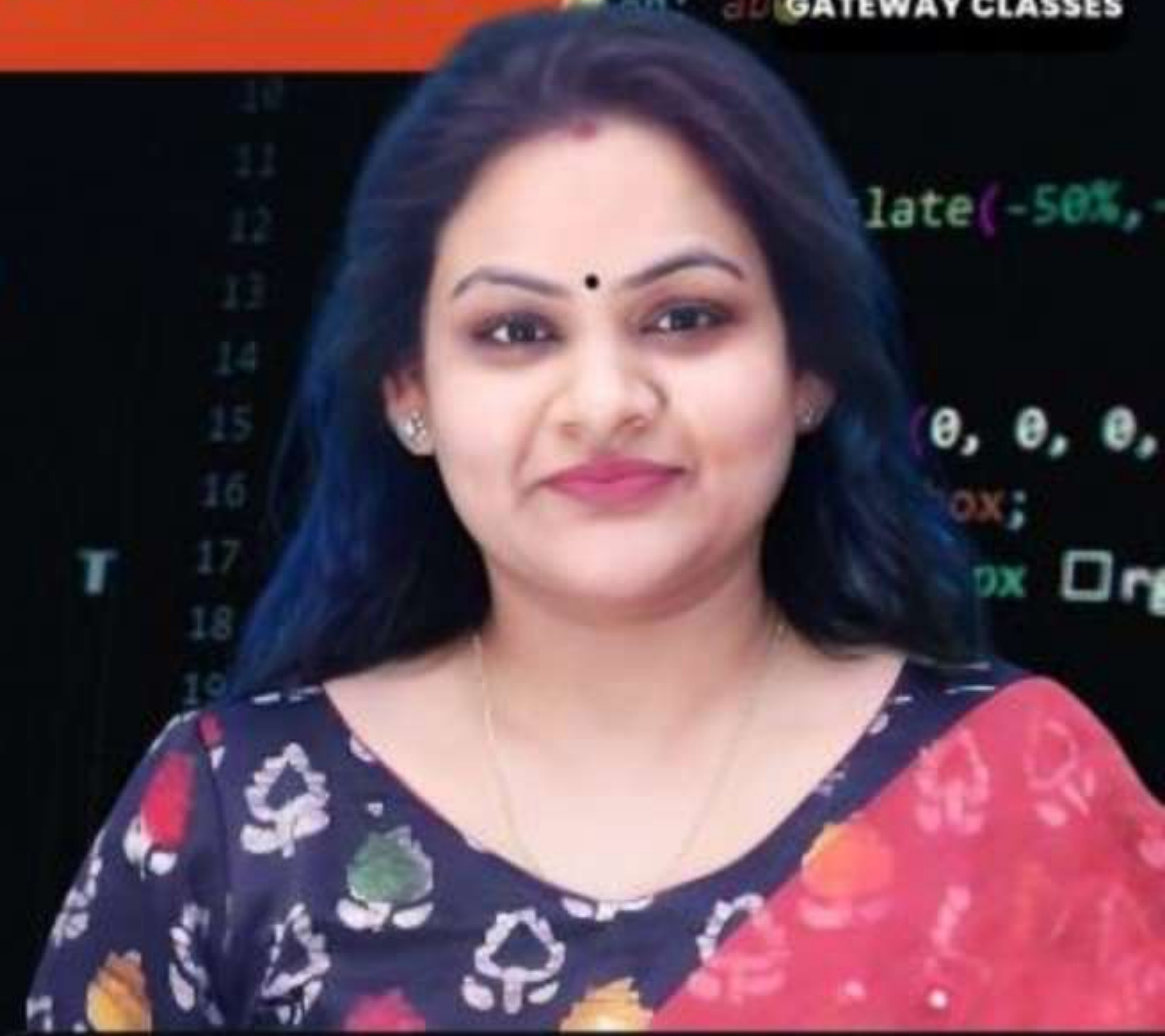
CS IT & CS Allied

DATA STRUCTURE

Unit-4 Lecture-4

Today's Target

- AVL tree ✓
- AKTU PYQs
- DPP

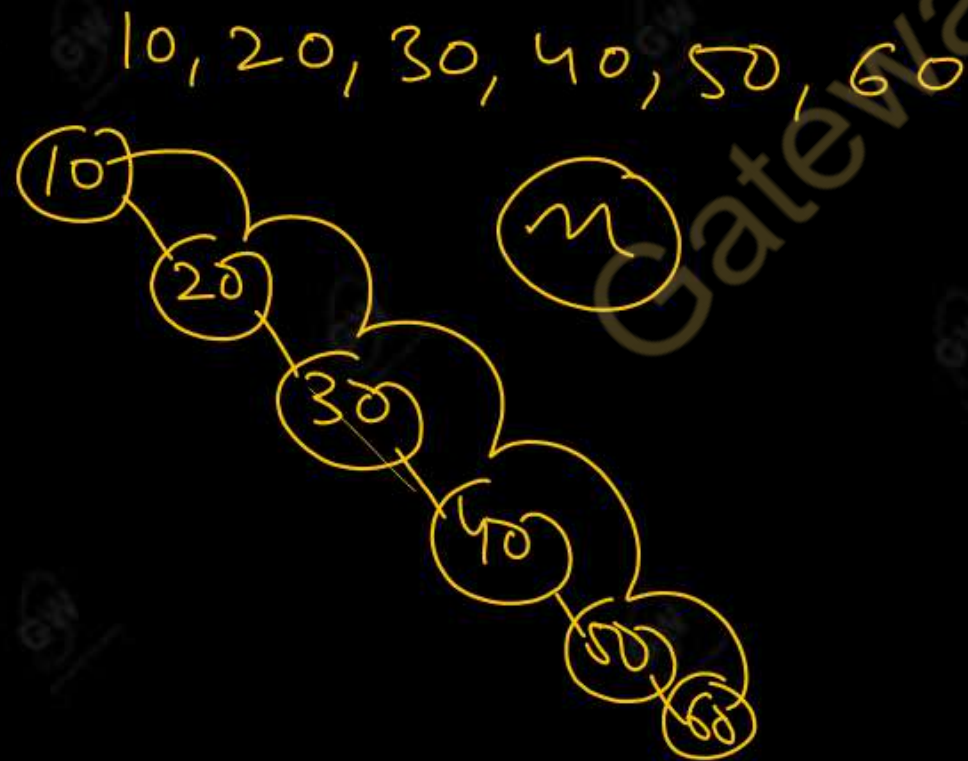


By Dr. Nidhi Parashar Ma'am

Ph.D (CSE)

- M.Tech Gold Medalist
- Net Qualified

- AVL Tree is a self-balancing binary search tree that can perform certain operations in logarithmic time.
- It exhibits height-balancing property by associating each node of the tree with a balance factor and making sure that it stays between -1 and 1 by performing certain tree rotations.
- This property prevents the Binary Search Trees from getting skewed, thereby achieving a minimal height tree that provides logarithmic time complexity for some major operations such as searching.

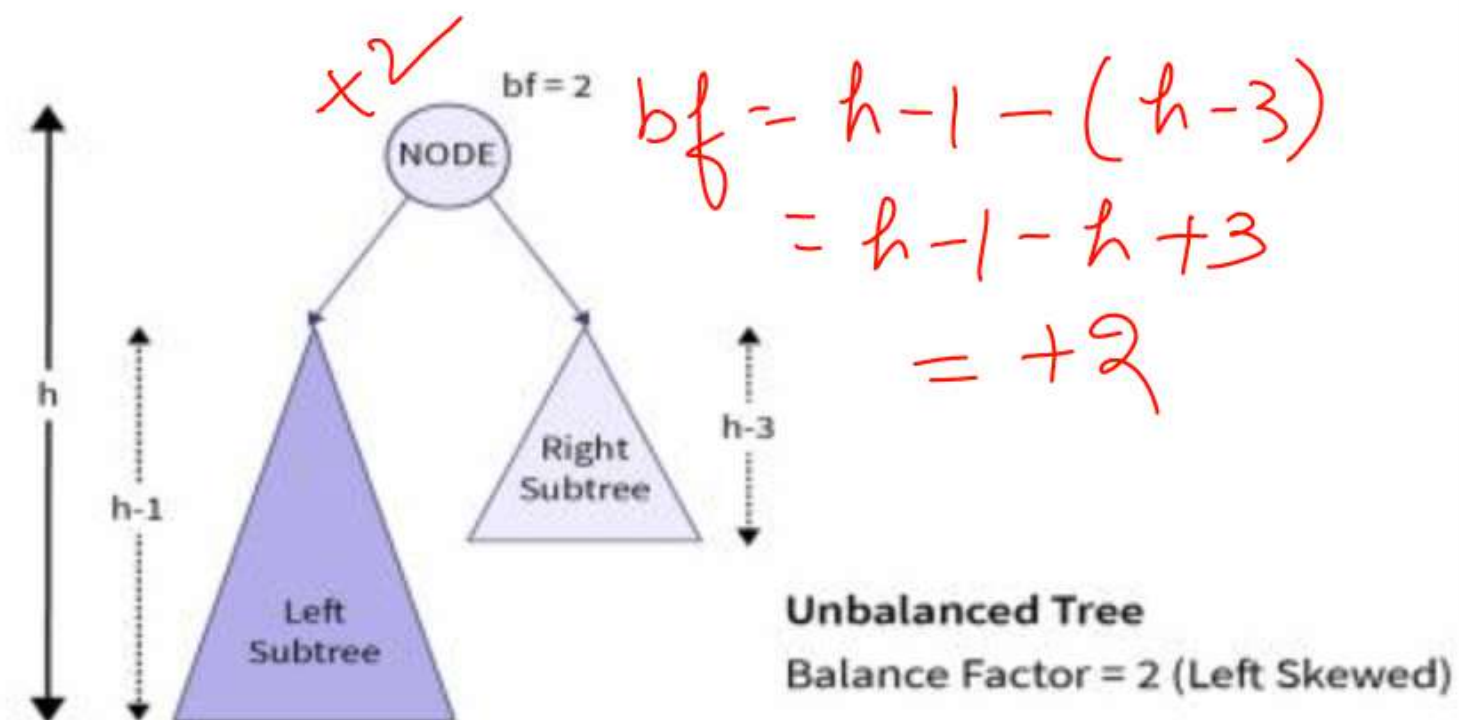
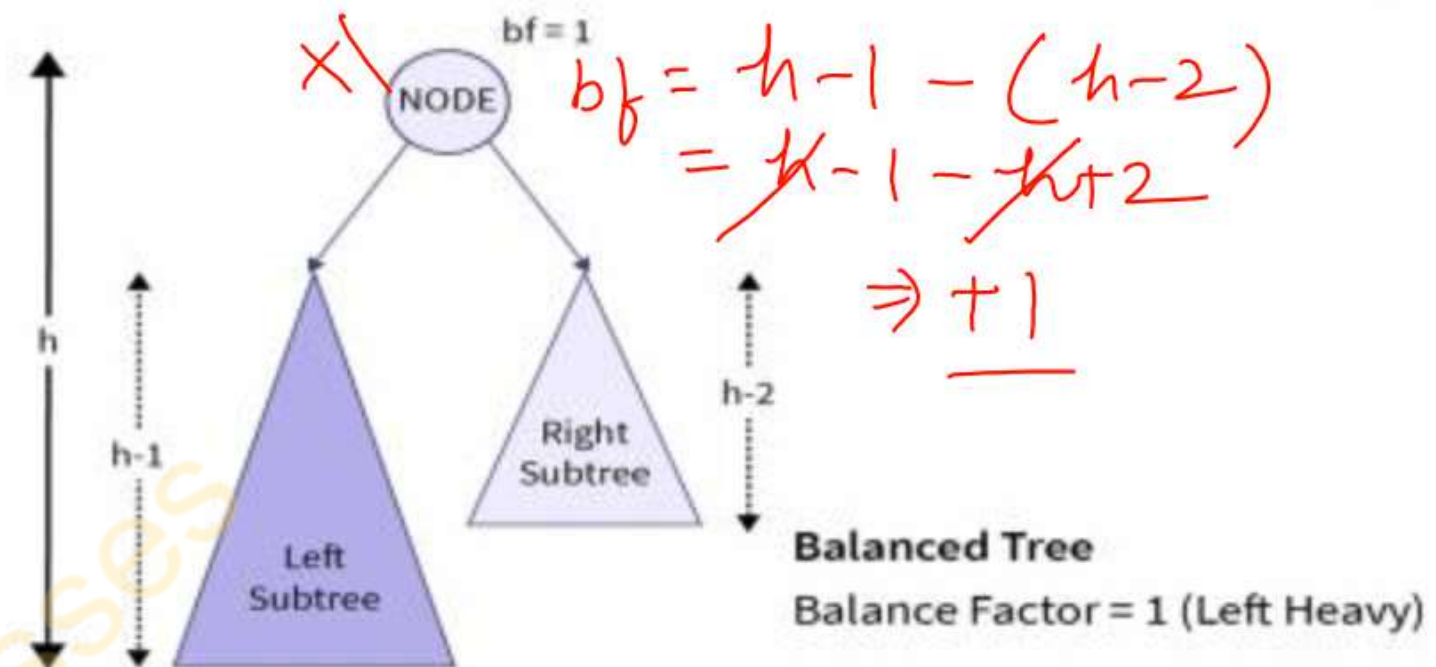


Balance Factor

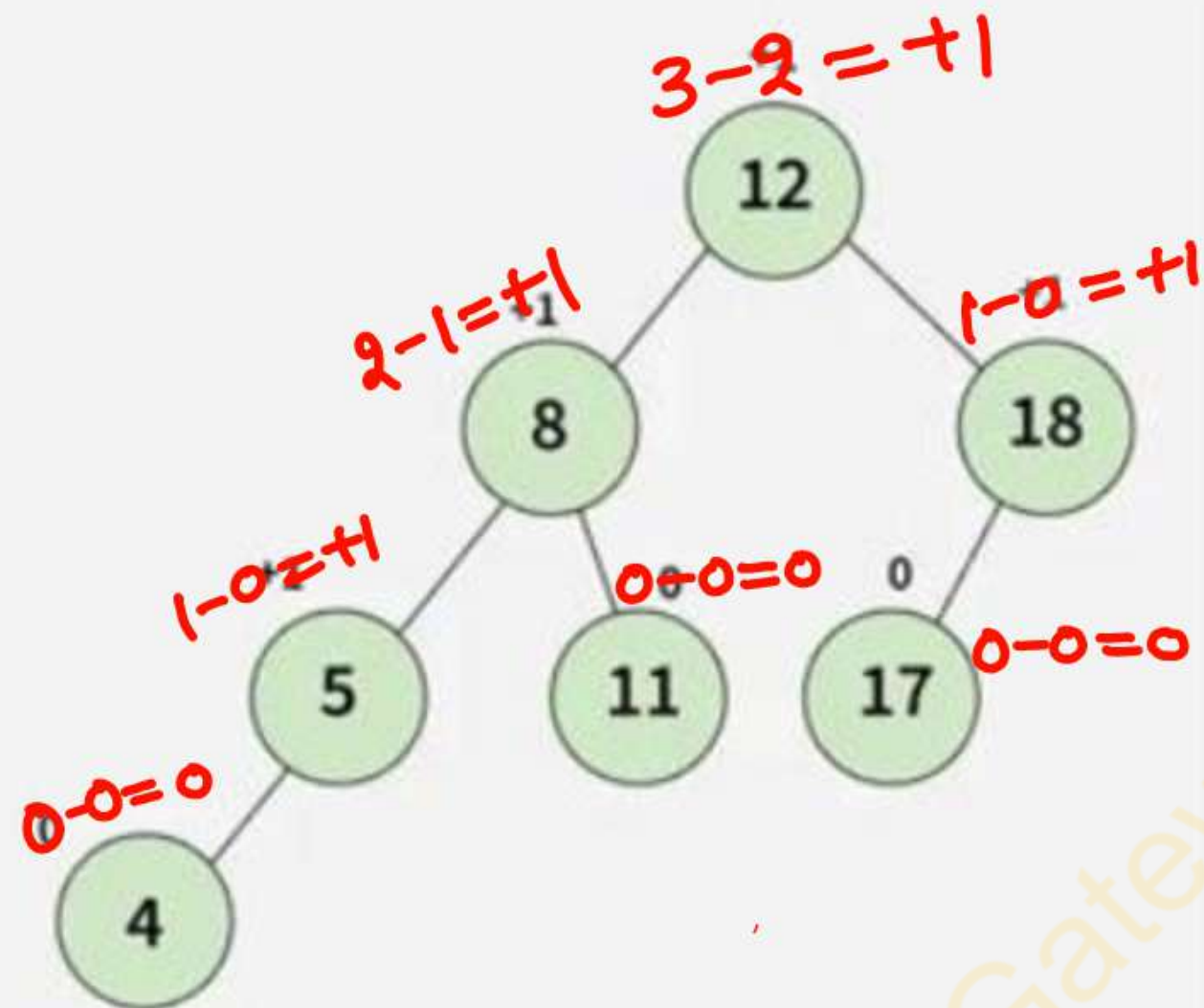
- In an AVL tree, the balance factor of a node is the height difference between its left and right subtrees. It ensures the tree remains balanced, with a balance factor of 0 indicating equilibrium.
- All nodes have balance factors within the range $[-1, 1]$, indicating a balanced AVL tree.
- Balance factor (BF) of a node N is given by:

$$bf = h_l - h_r$$

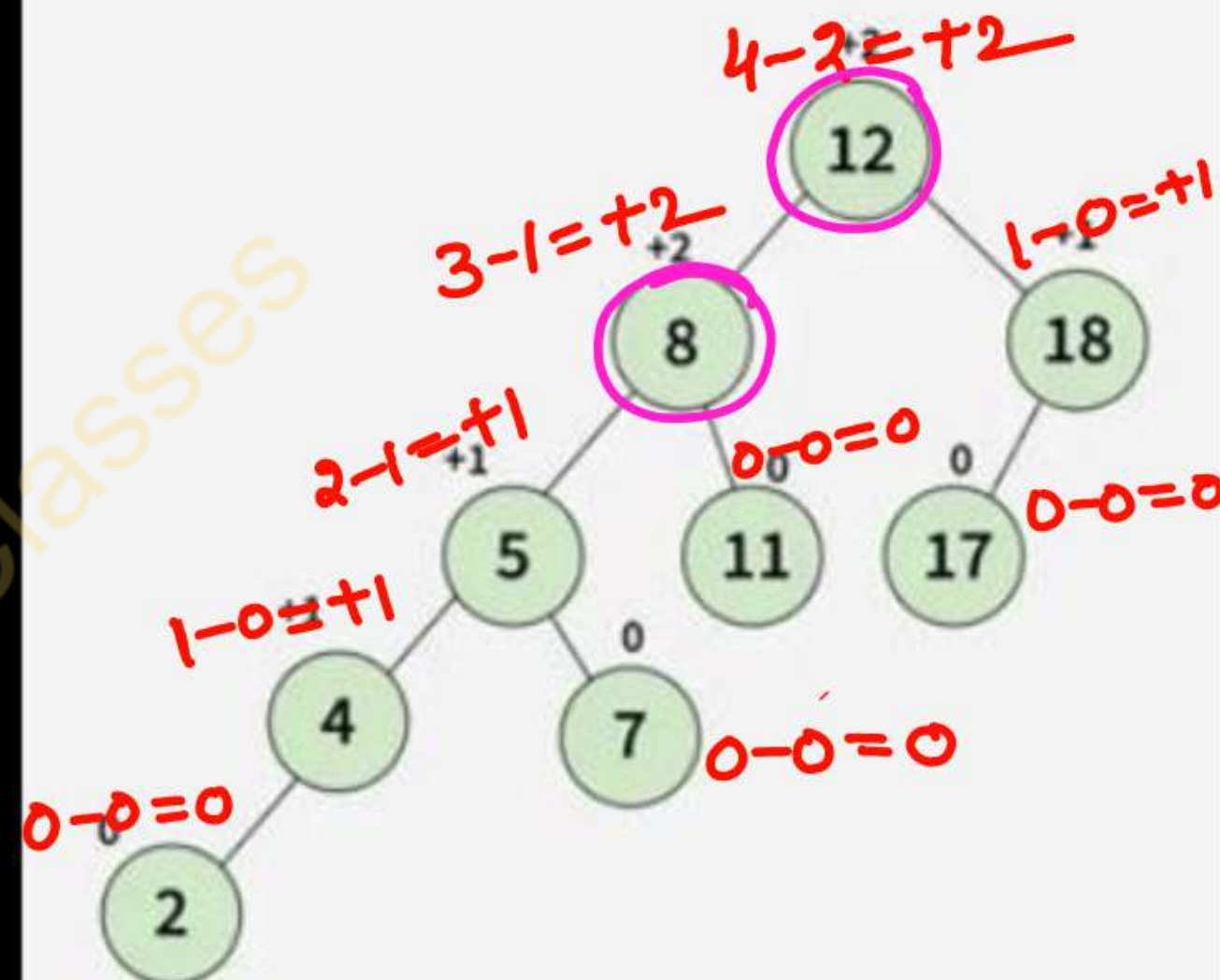
$$-1 \leq bf \leq 1$$



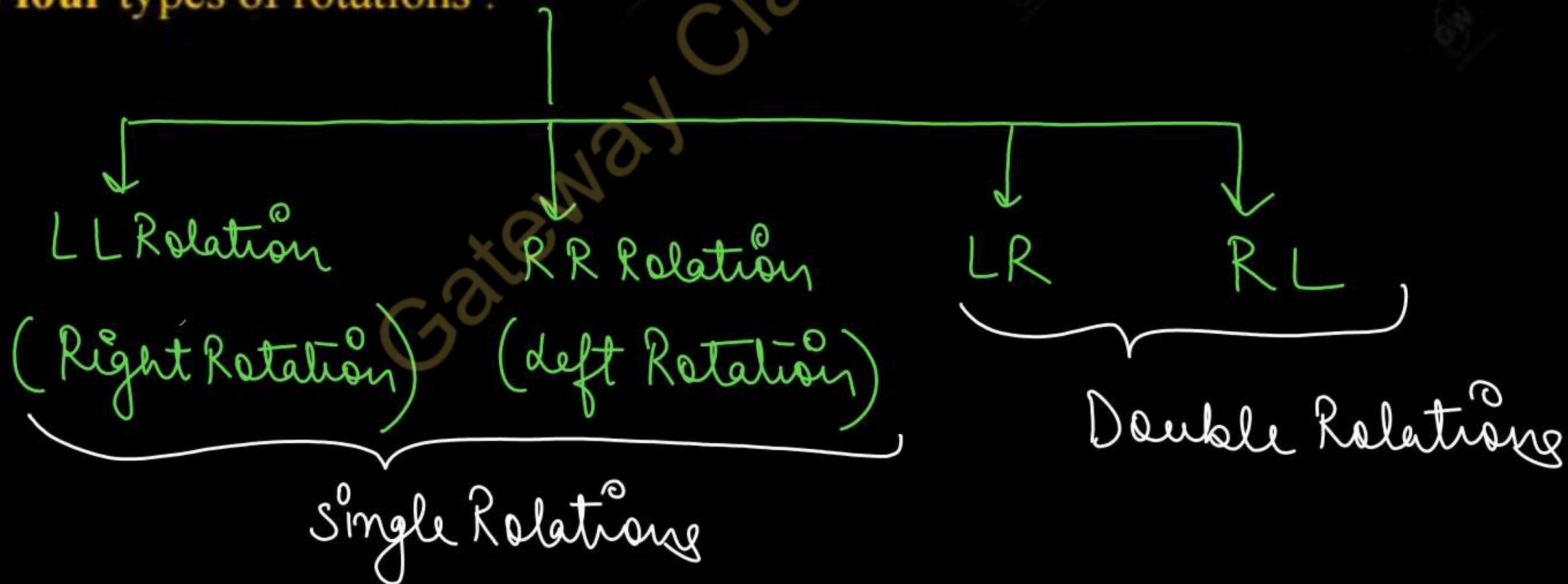
Balanced BST



Unbalanced BST



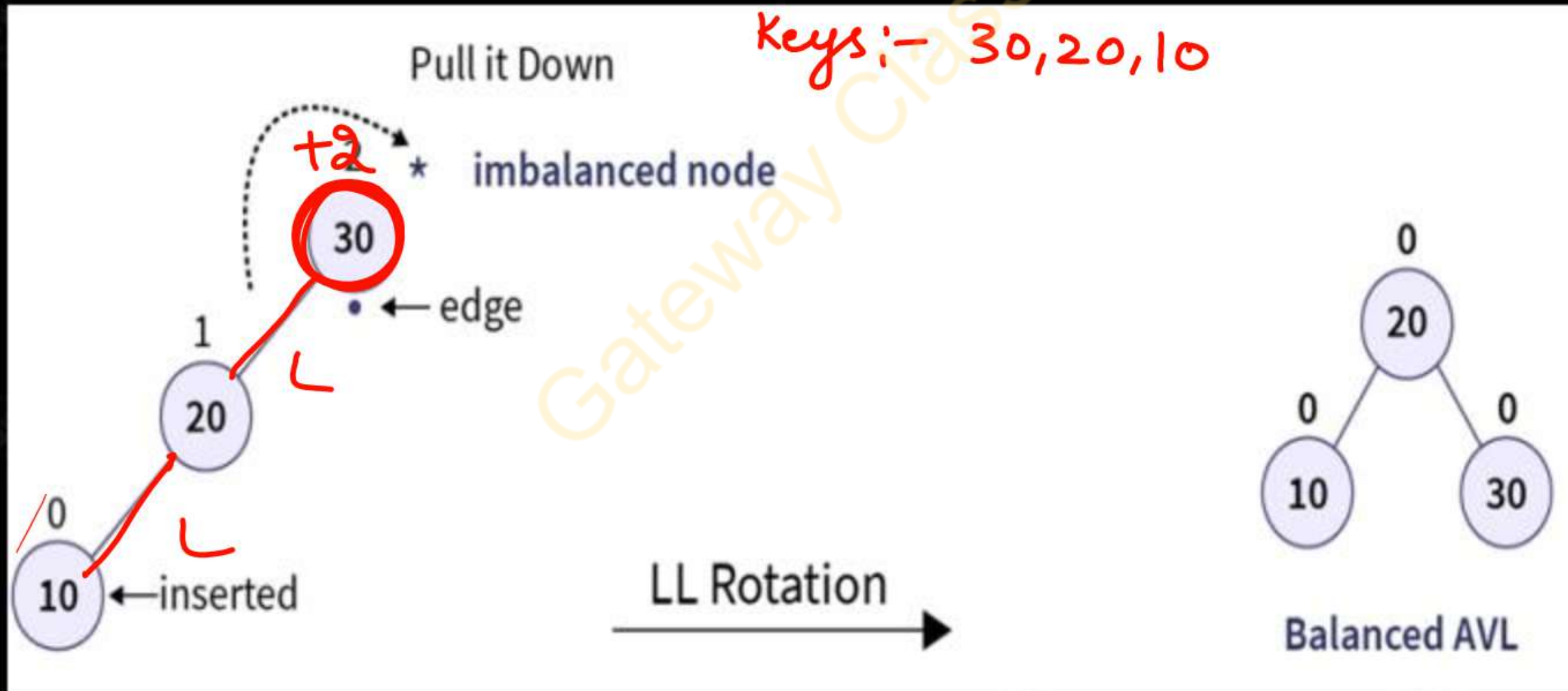
- AVL Trees use balance factor to determine whether a node is left-heavy, balanced, or right-heavy.
- When a node becomes unbalanced, specific tree rotations are performed to restore balance.
- These rotations involve rearranging the tree structure without changing the order of elements.
- **Rotations** restore balance in $O(1)$ time while ensuring overall time complexity remains $O(\log n)$.
- AVL Trees use **four** types of rotations :



1. LL Rotation

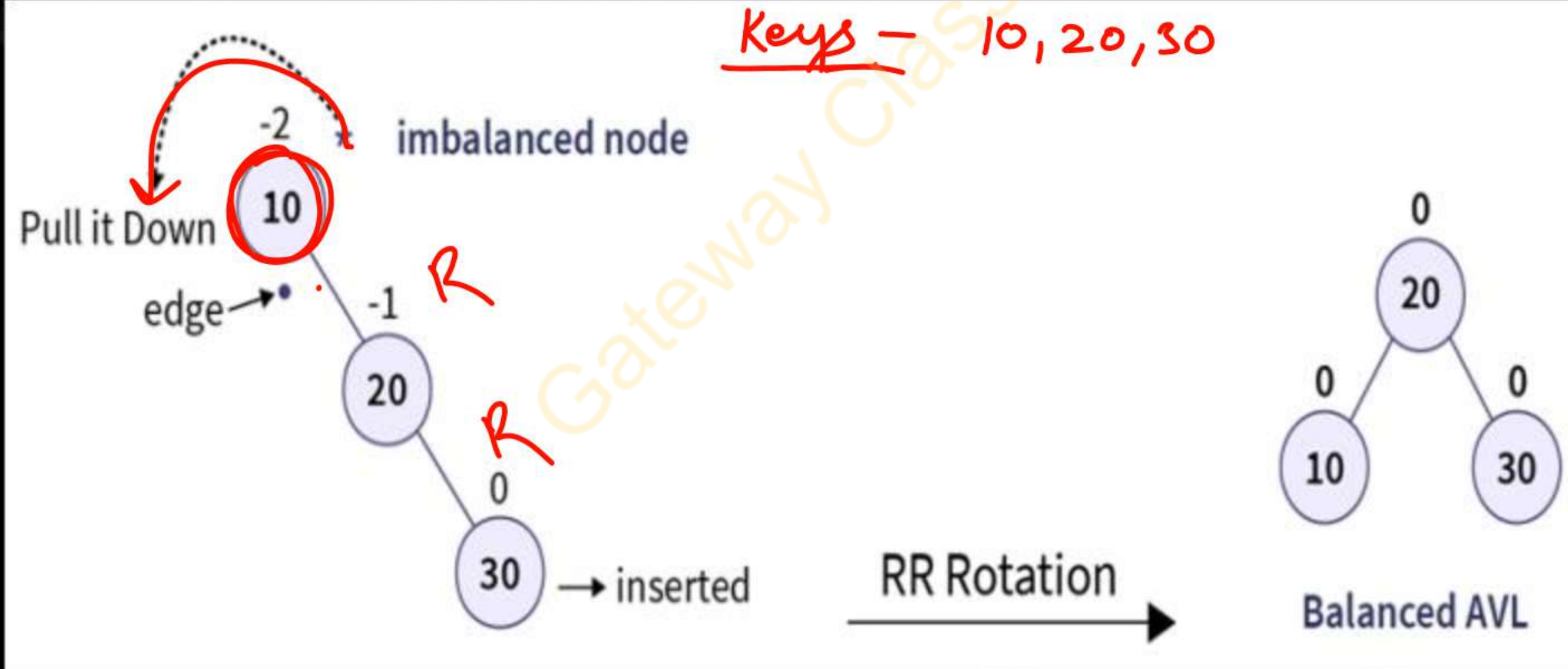
➤ LL Rotation is executed when tree becomes unbalanced due to **insertion of a node into the left subtree of the left child of the node causing the imbalance.**

➤ This imbalance indicates that the tree is heavy on the left side. Hence, a **right rotation** is applied such that this left heaviness imbalance is countered and the tree becomes balance.



2. RR Rotation

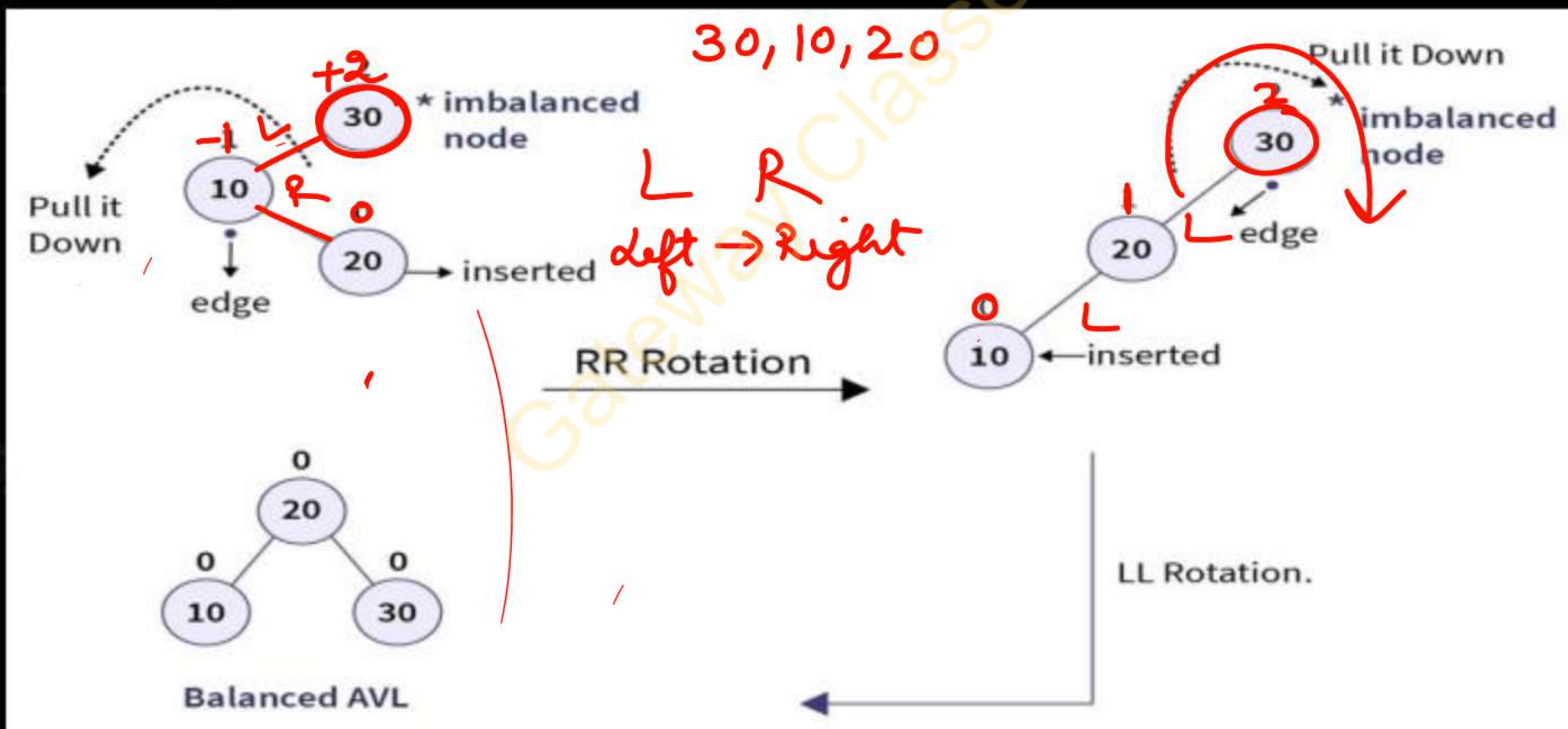
- Imbalance arises when a node is inserted into the right subtree of the right child of the node causing the imbalance, known as **Right-Right (RR)** insertion.
- Tree becomes right heavy and a **left rotation** is performed along the edge of the imbalanced node to counter this right skewness caused by the insertion operation.



3. LR Rotation (RR+LL)

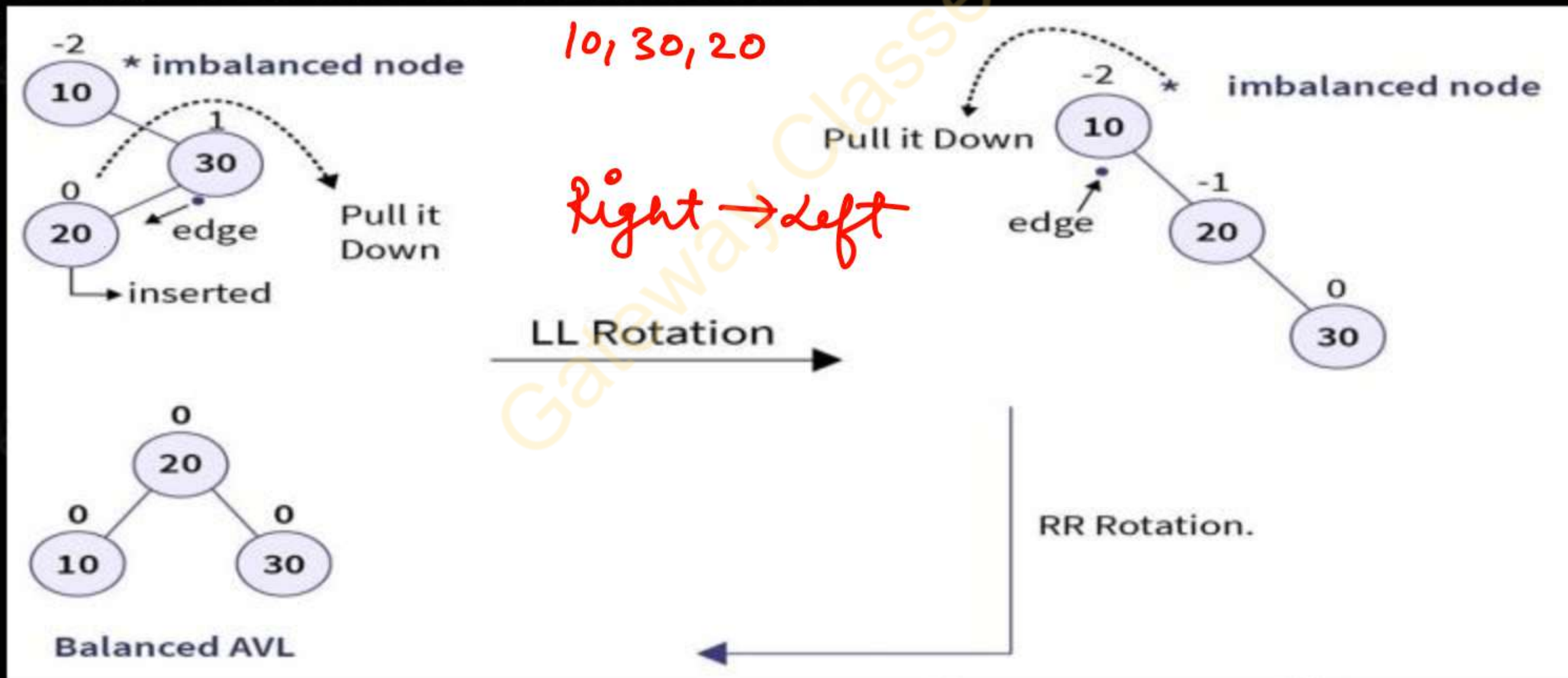
An imbalance arises when a node is inserted into the right subtree of the left child of a node.

- Upon applying the RR rotation the BST becomes left-skewed and is still unbalanced. This is now the case of LL rotation and by rotating the tree along the edge of the imbalanced node in the clockwise direction, we can retrieve a balanced BST.



4. RL Rotation (LL + RR)

It is performed when tree gets unbalanced, upon insertion of a node into left subtree of the right child of the imbalance node. LL (right rotation) is performed that converts tree into a right-skewed tree. After which, RR rotation (left rotation) is applied around the edge of imbalanced node to convert this right-skewed tree into a balanced BST.



Step 1 – Create a node

Step 2 – Check if the tree is empty

Step 3 – If the tree is empty, the new node created will become the root node of the AVL Tree.

Step 4 – If the tree is not empty, we perform the Binary Search Tree insertion operation and check the balancing factor of the node in the tree.

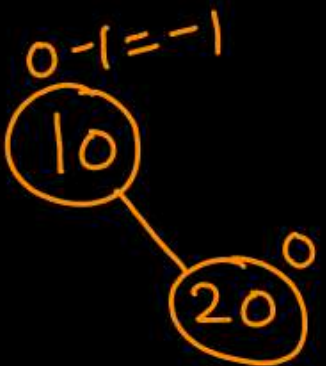
Step 5 – Suppose the balancing factor exceeds 1, we apply suitable rotations on the said node and resume the insertion from Step 4.

Q: Construct an AVL Tree by inserting following sequence of elements, starting with an empty tree: 10, 20, 30, 40, and 50.

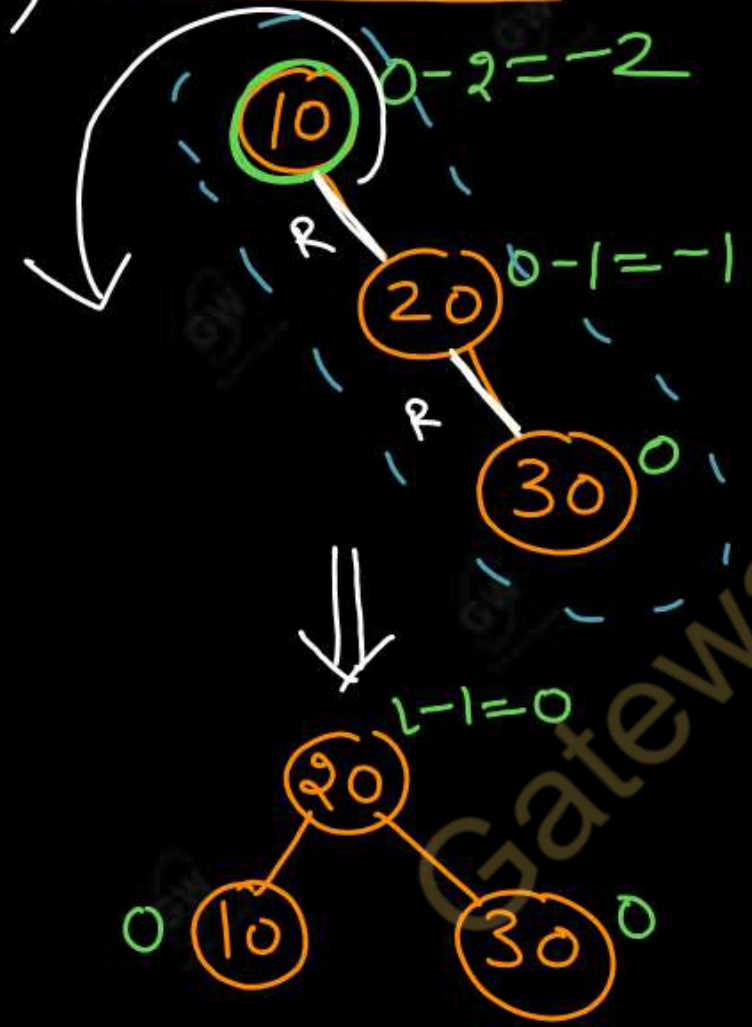
Solⁿ
1) Insert 10



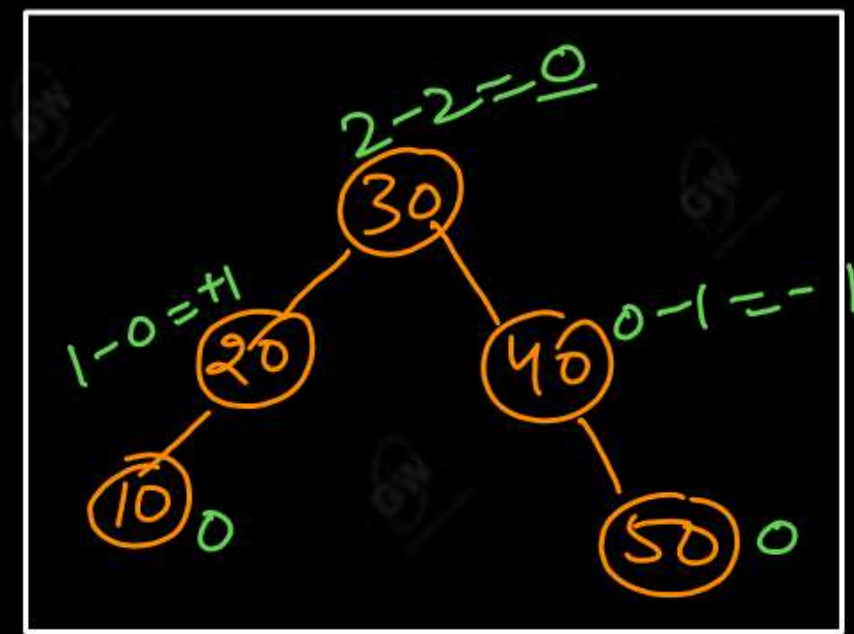
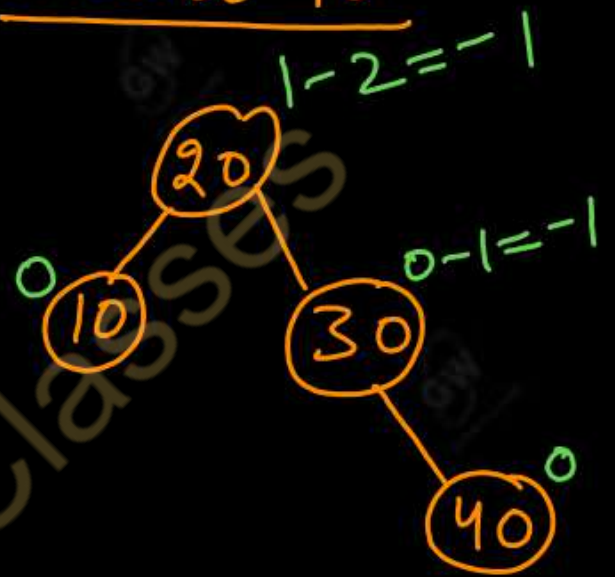
2) Insert 20



3) Insert 30

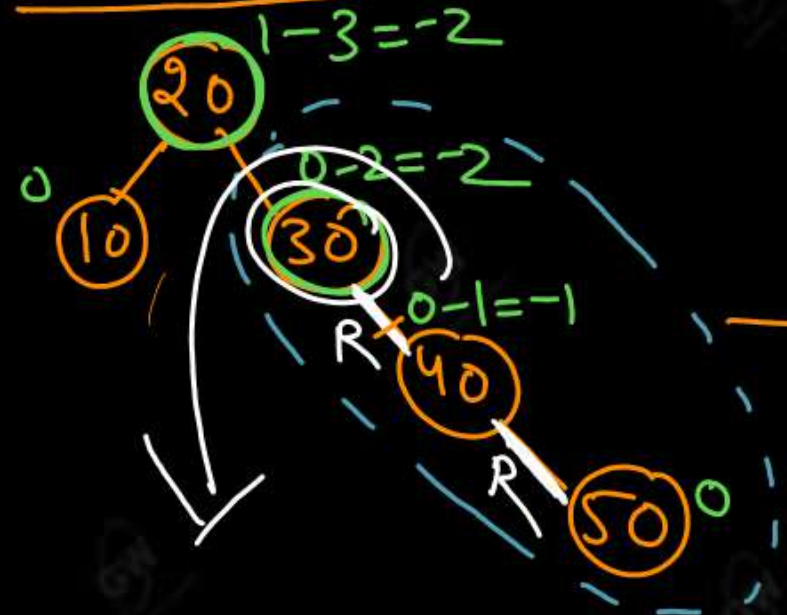


4) Insert 40



5)

Insert 50



Q: Construct an AVL Tree by inserting following sequence of elements, starting with an empty tree: 40, 20, 10, 25, 30, 22, 50.

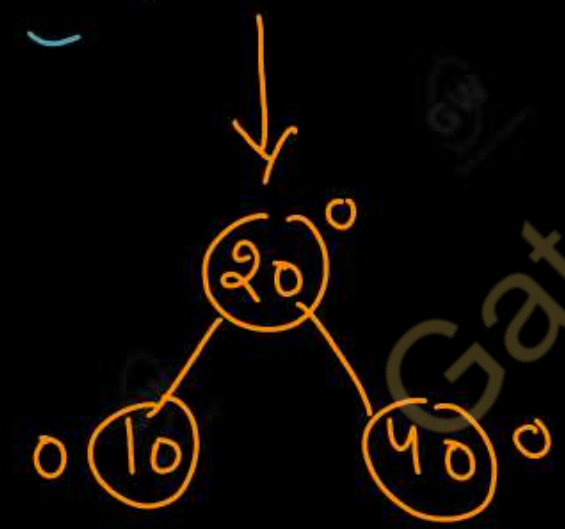
1) Insert 40



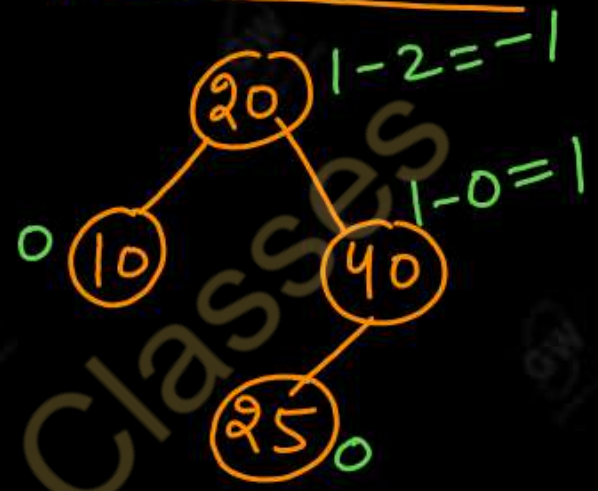
2) Insert 20



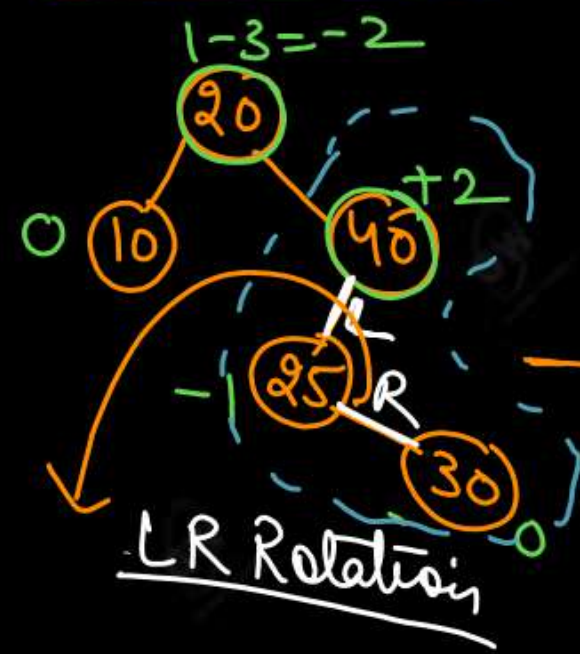
3) Insert 10



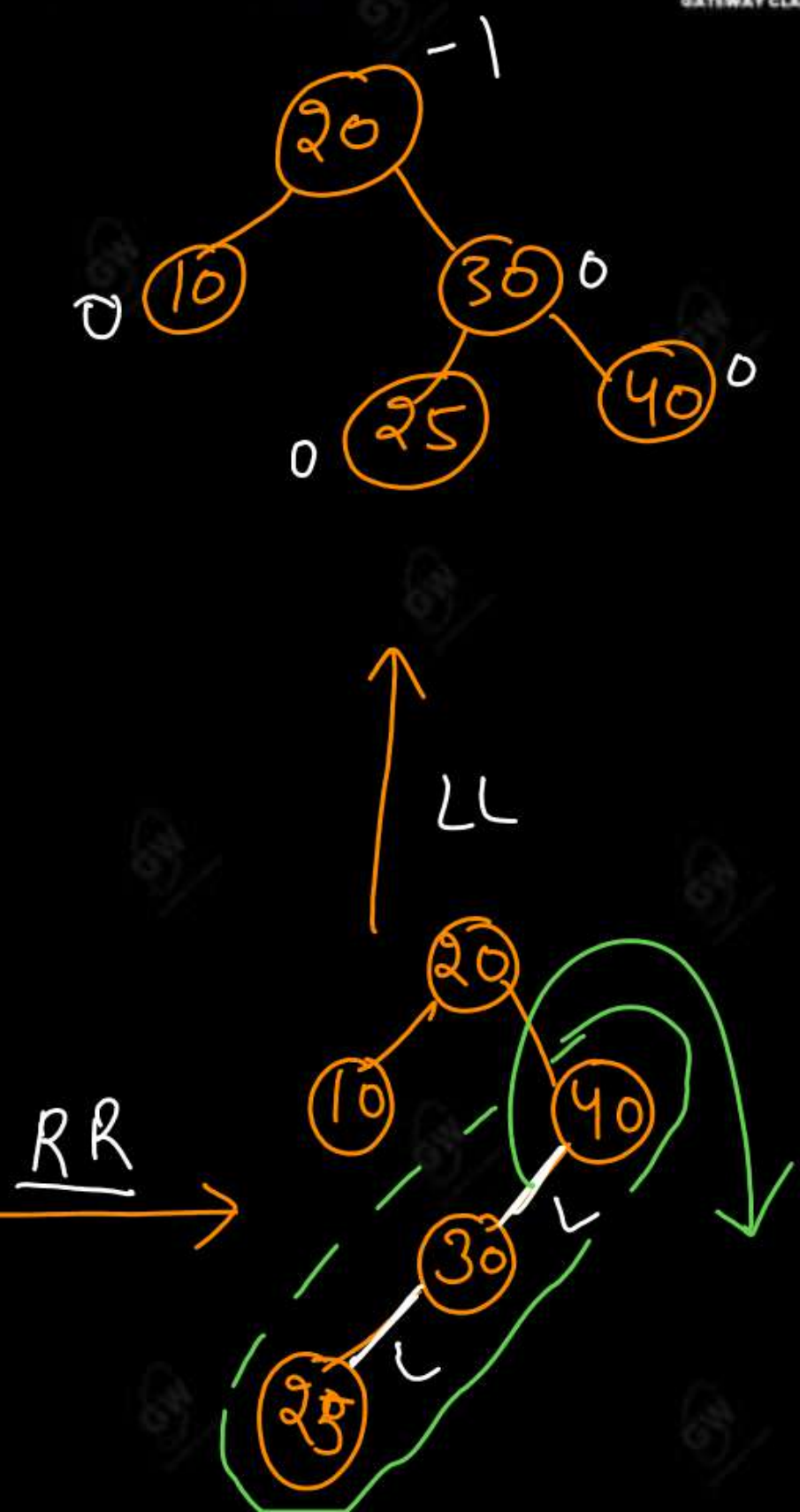
4) Insert 25



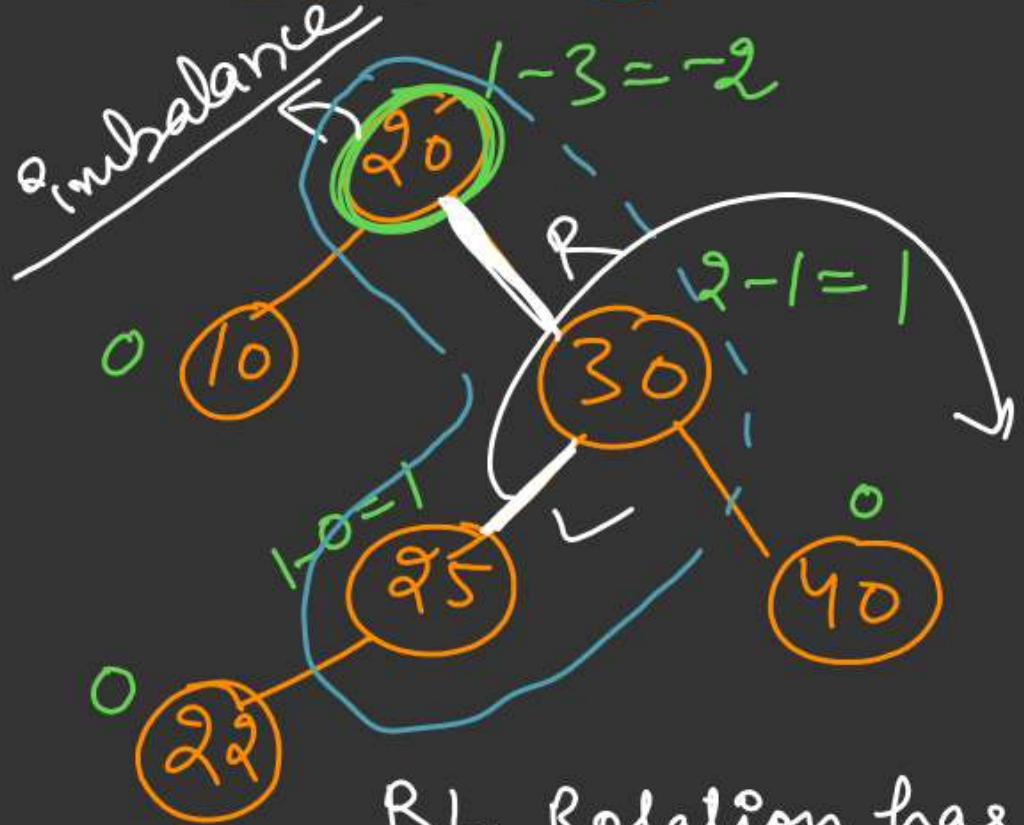
5) Insert 30



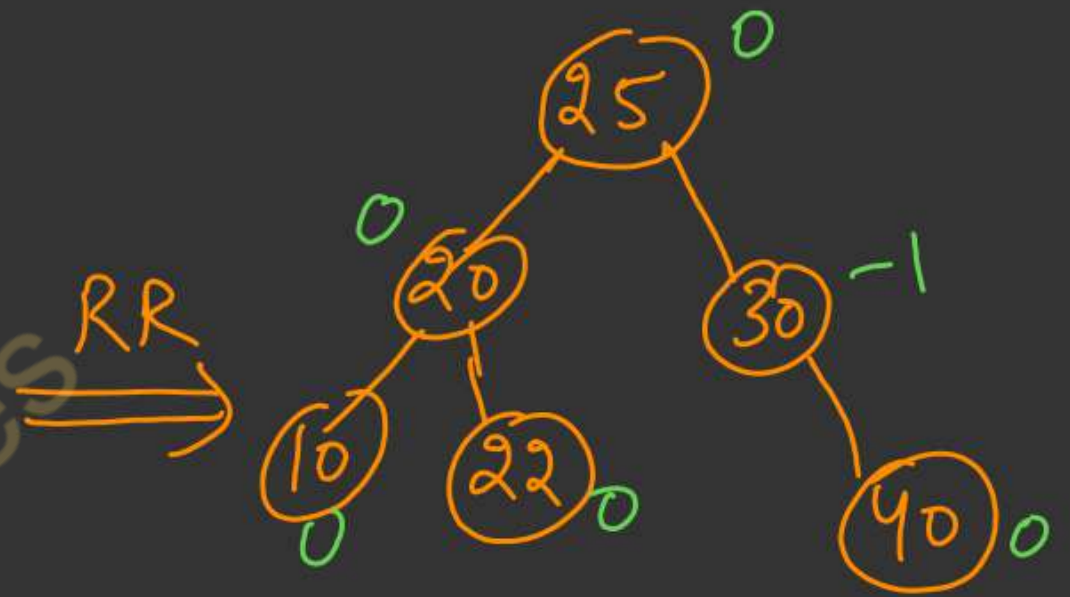
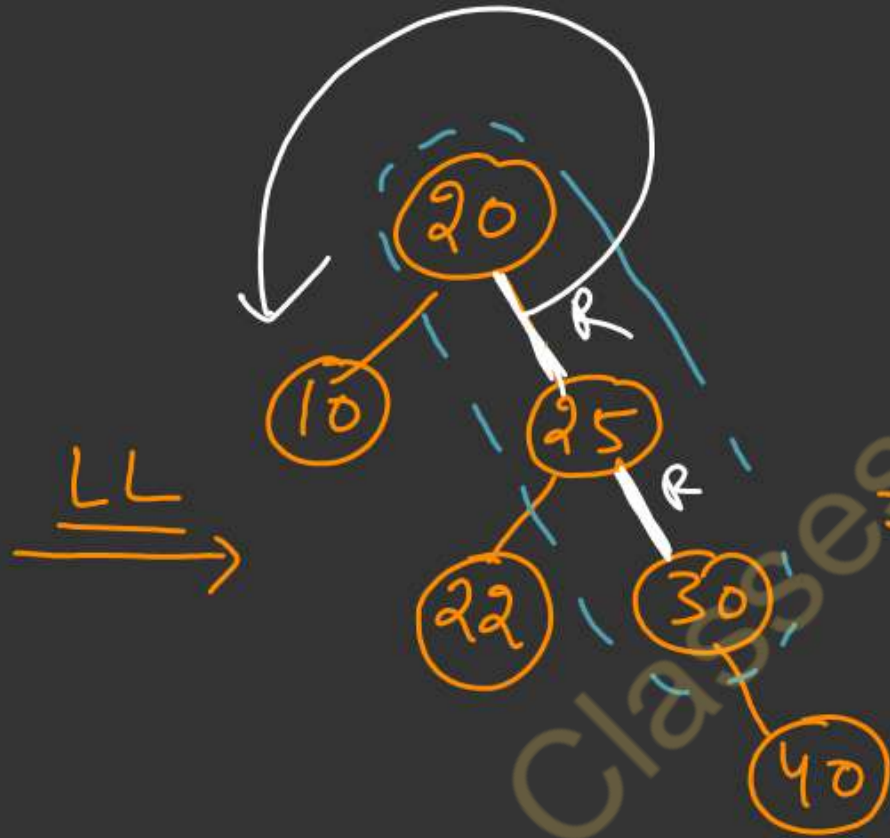
LR Rotation



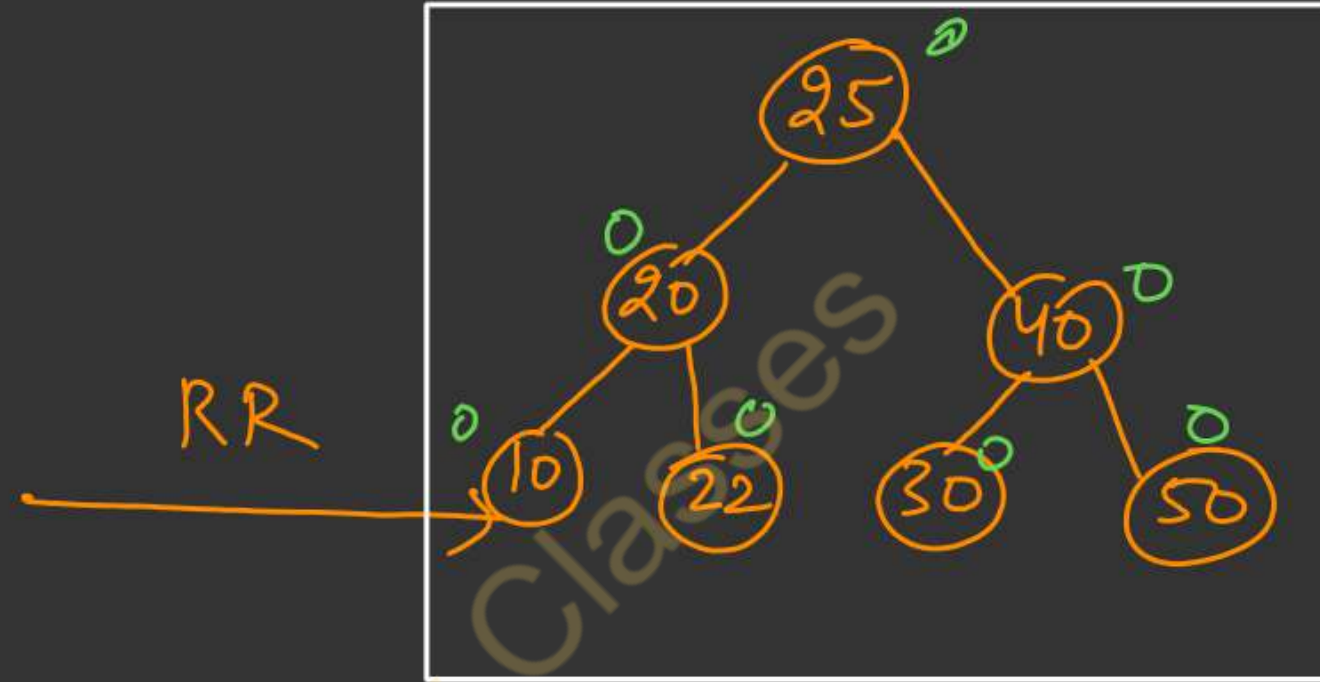
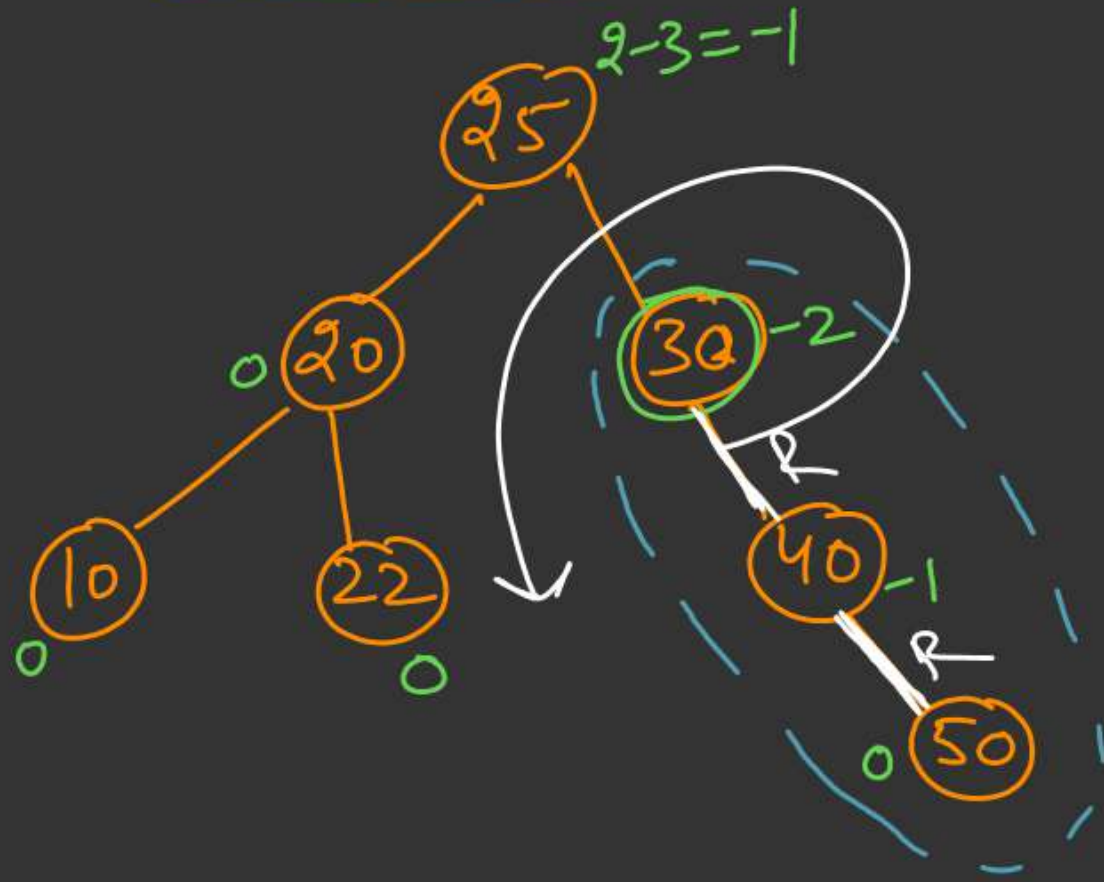
6) Insert 22



RL Relation has
to be performed



1) Insert 50

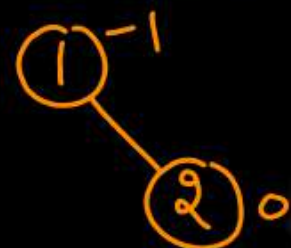


Q: Construct an AVL tree with the values 1 to 10 numbers into an initially empty tree.

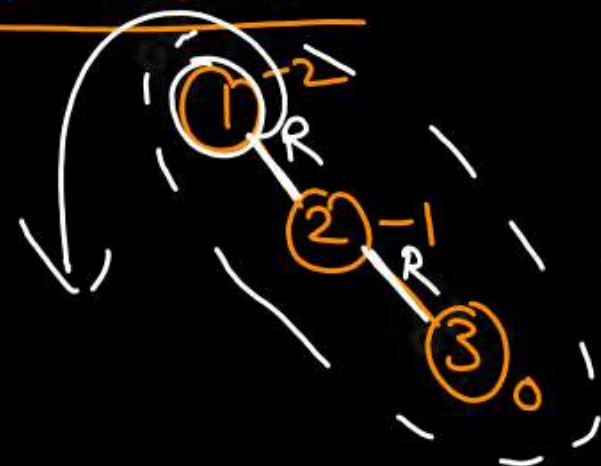
1) Insert 1



2) Insert 2



3) Insert 3

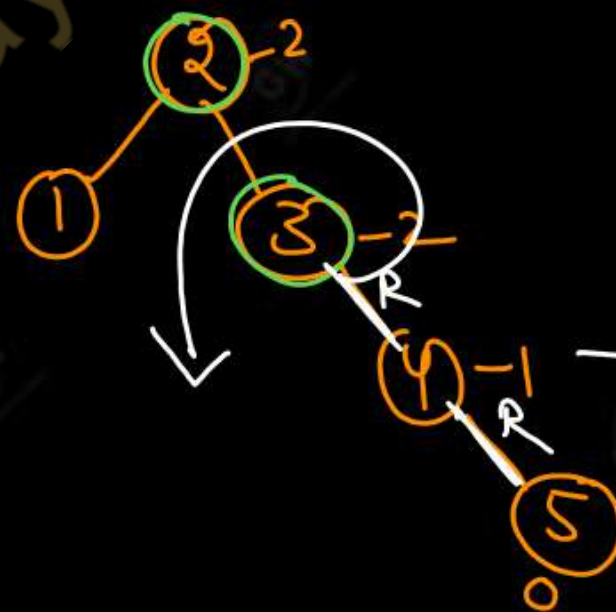


RR

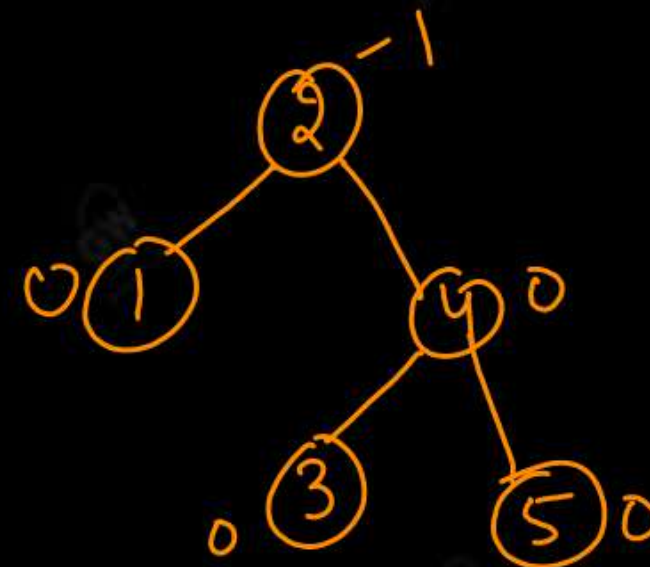
4) Insert 4



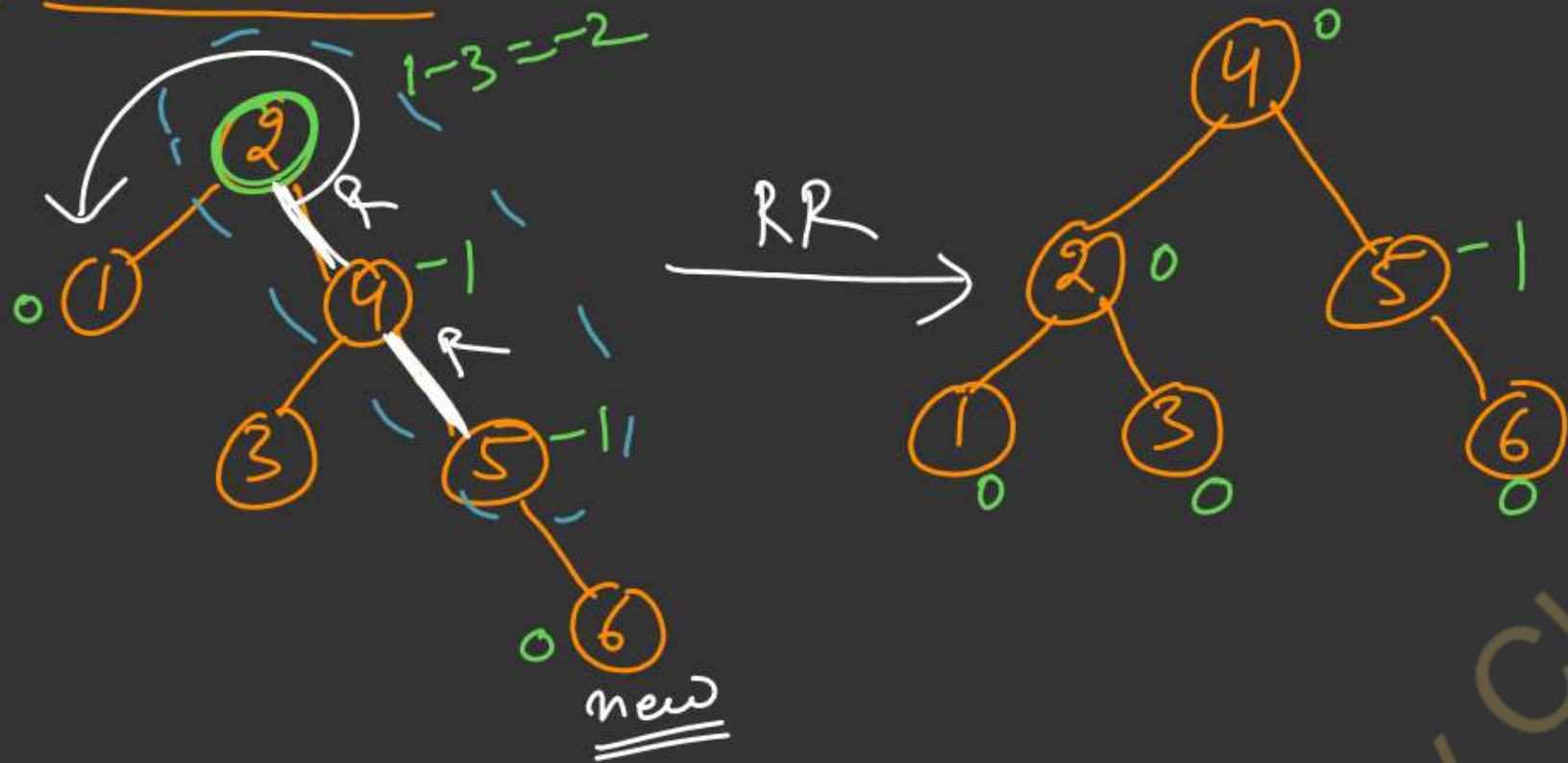
5) Insert 5



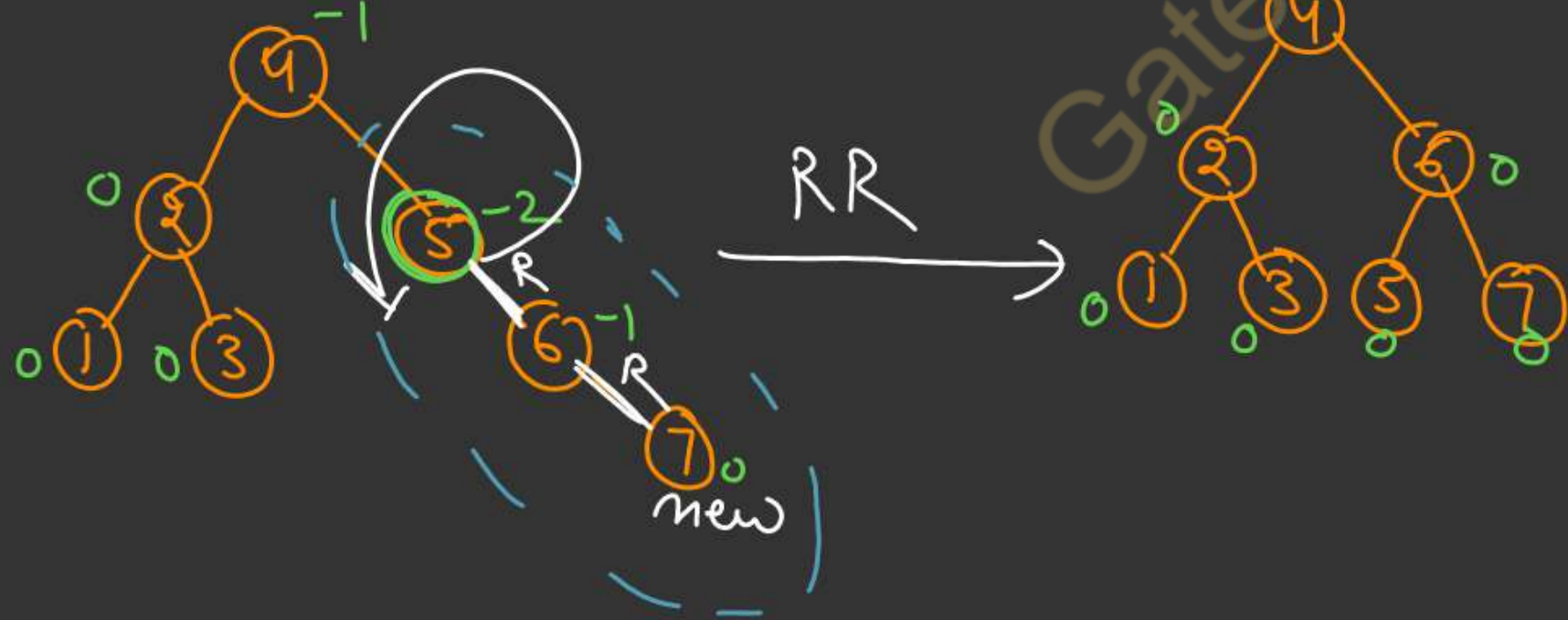
RR



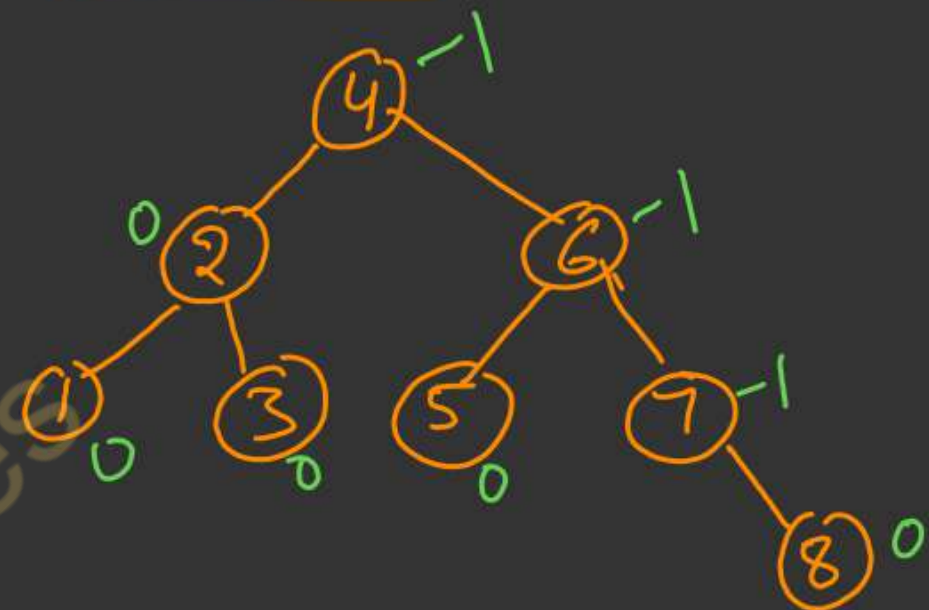
⑥ Insert 6



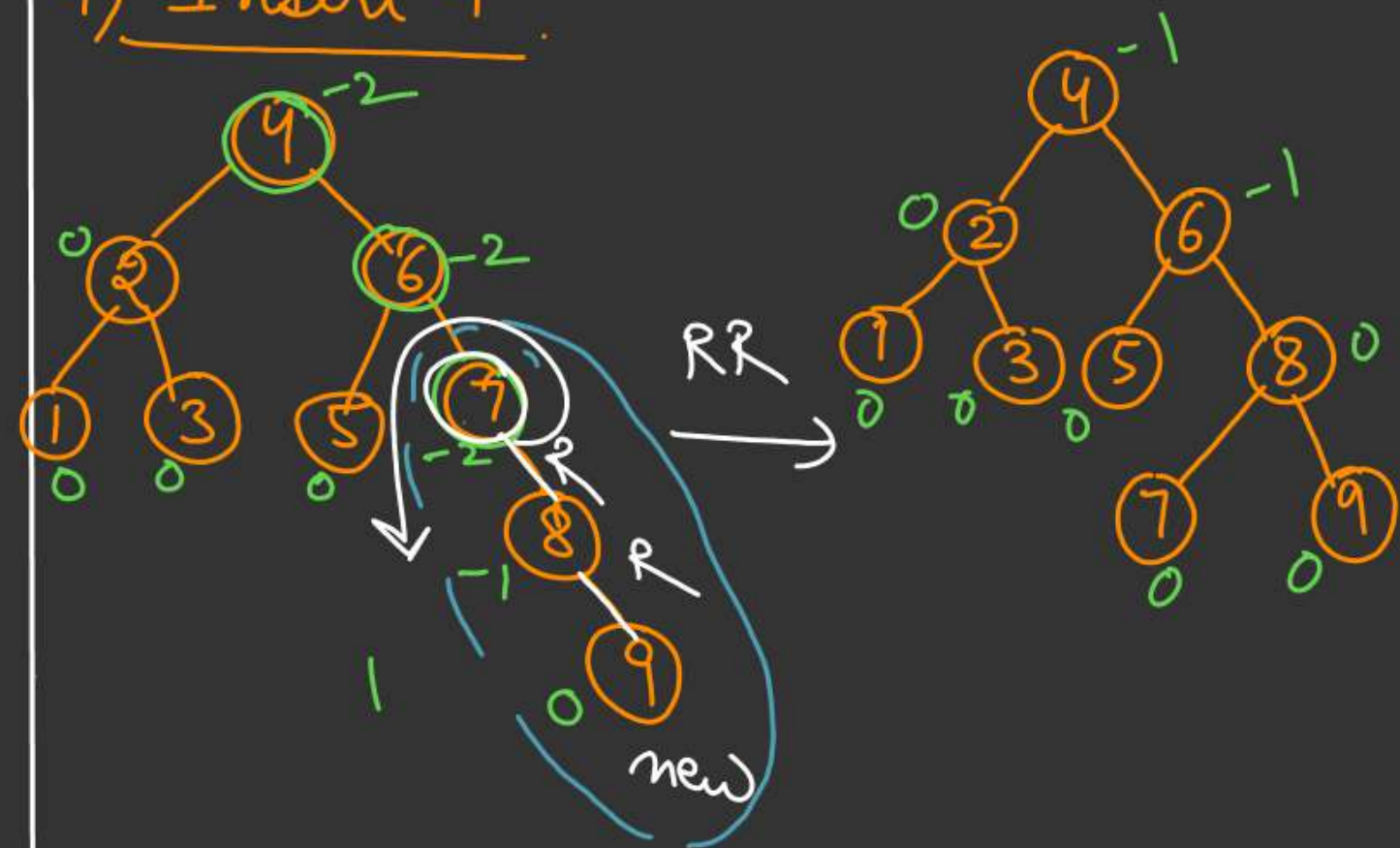
7) Insert 7



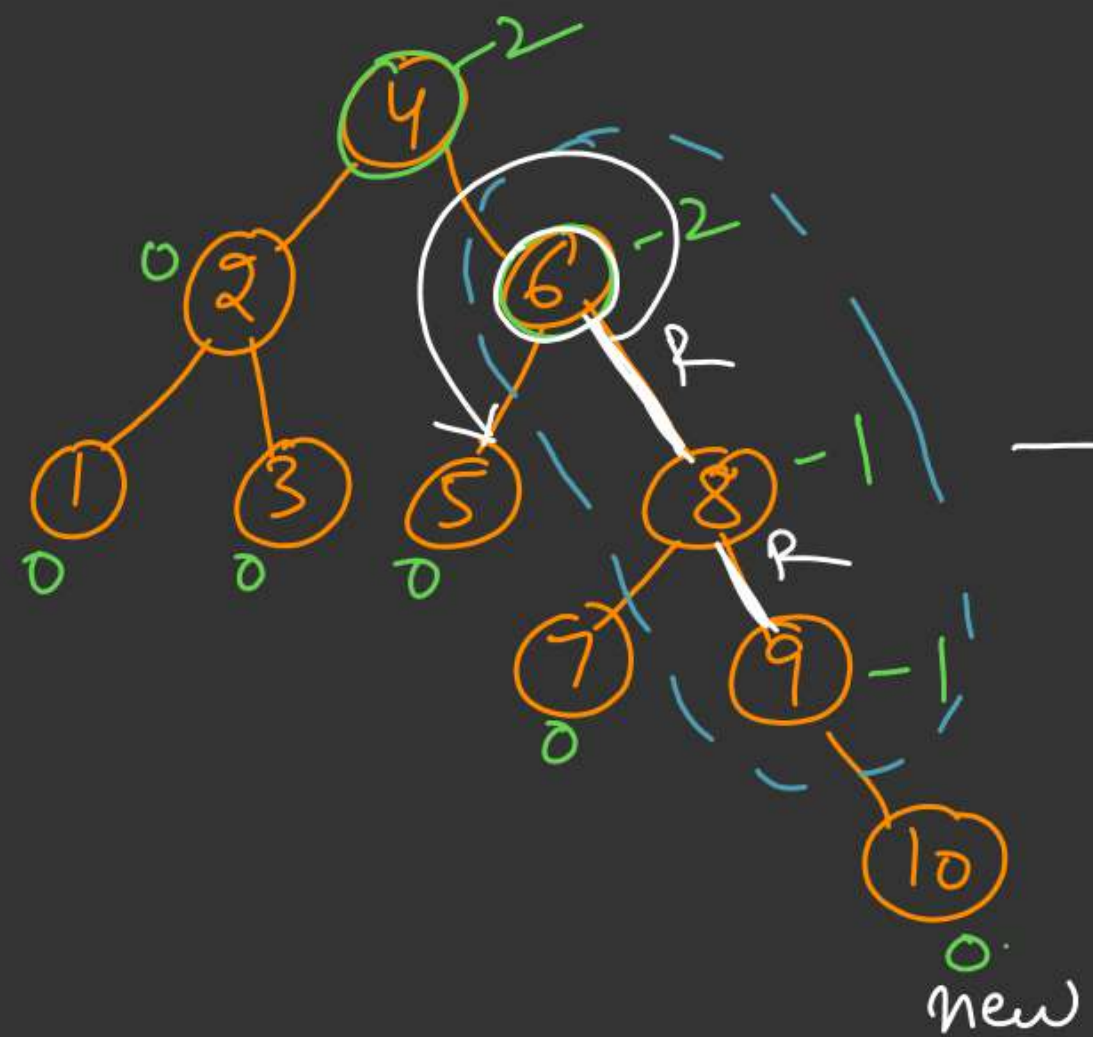
⑧ Insert 8



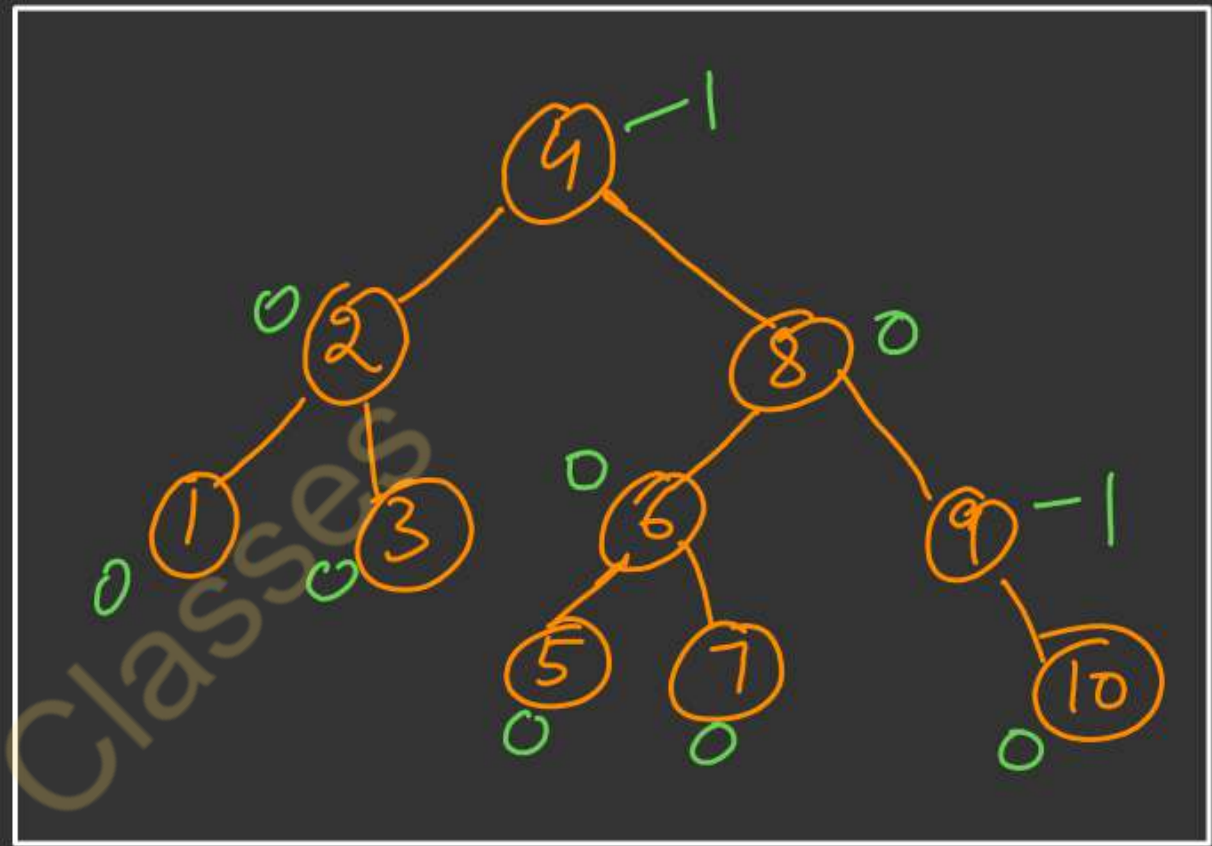
9) Insert 9



10) Insert 10



RR

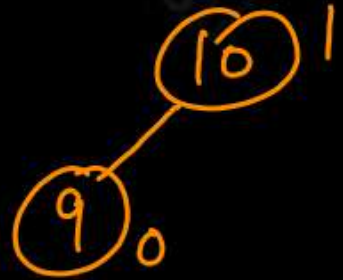


Q: Construct an AVL tree with the values 10 to 1 numbers into an initially empty tree.

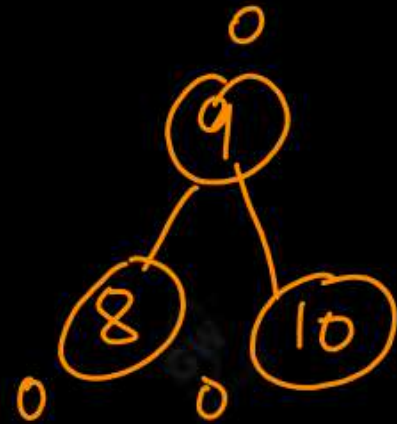
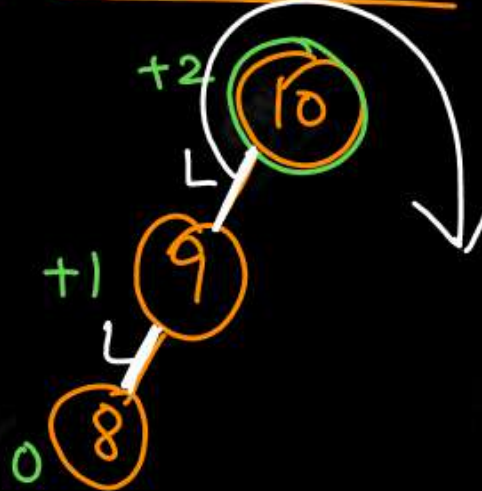
1) Insert 10



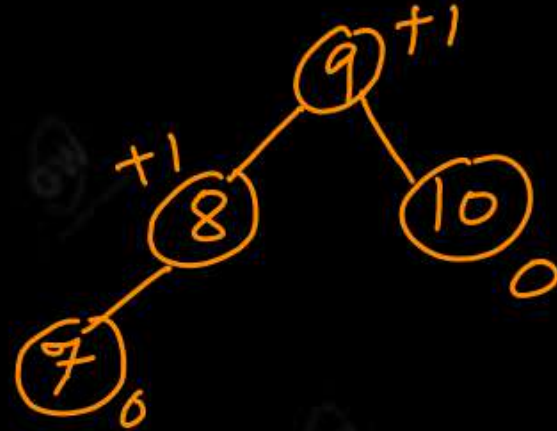
2) Insert 9



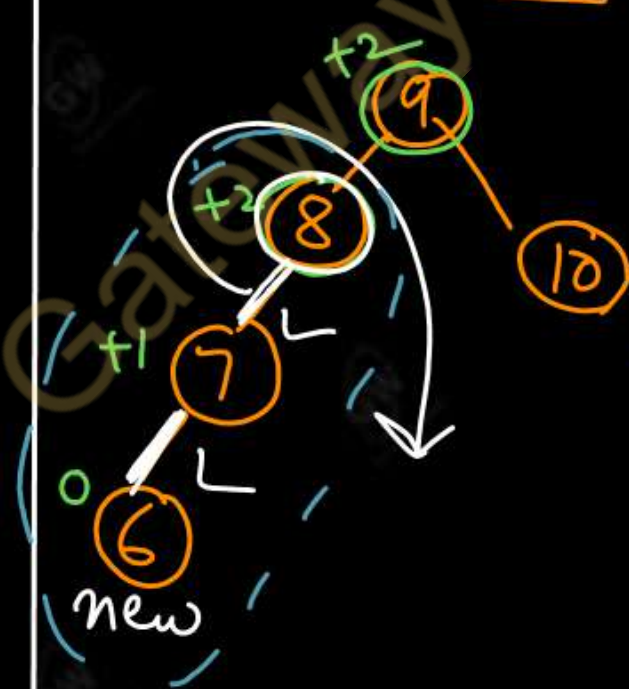
3) Insert 8



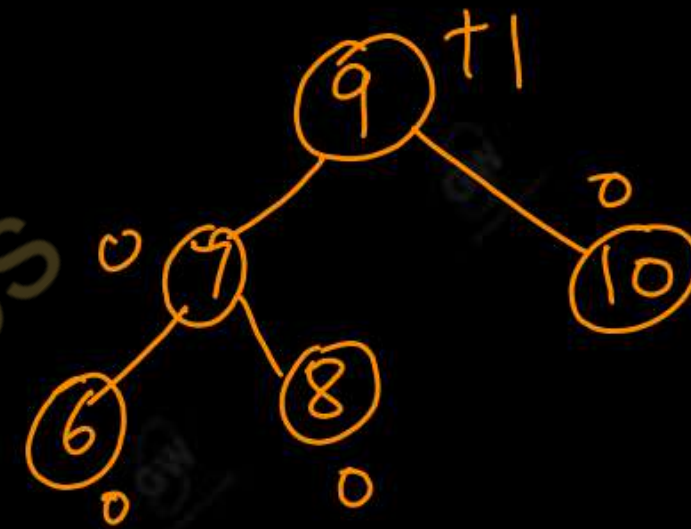
4) Insert 7



5) Insert 6

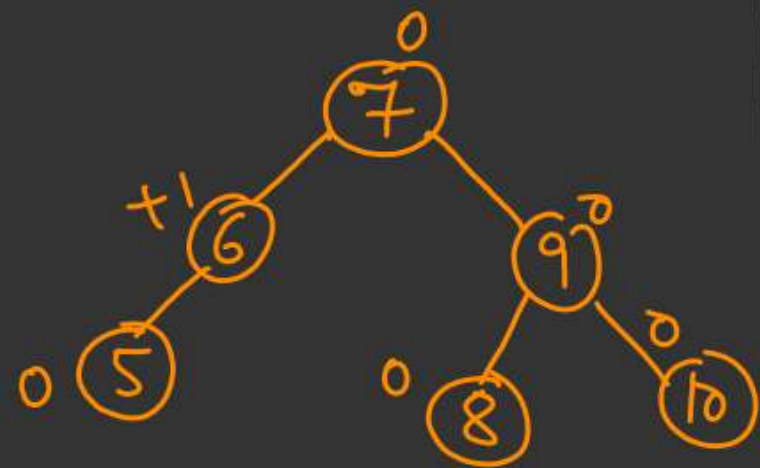
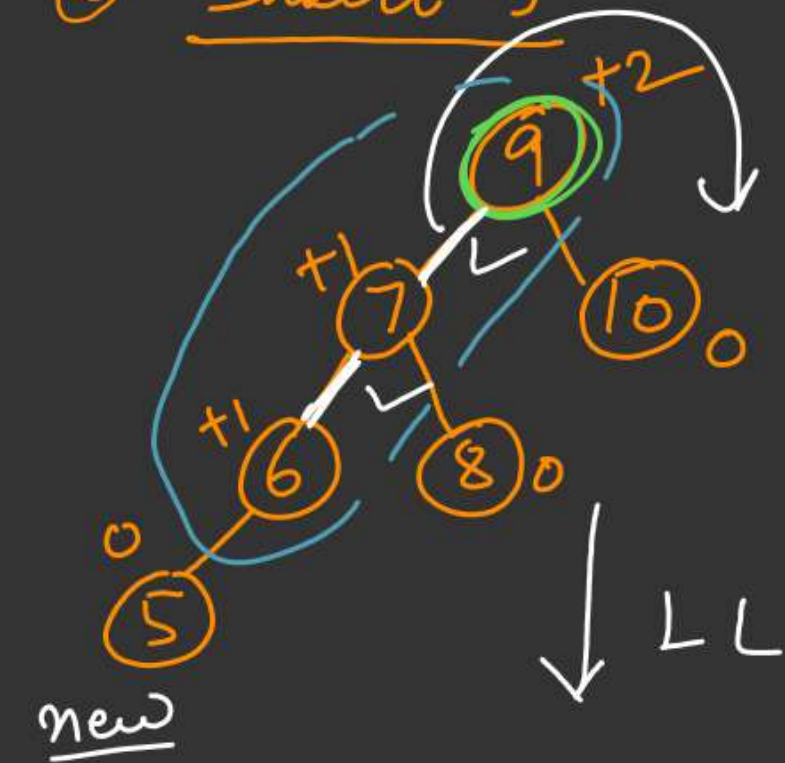


(AKTU 2022-23)

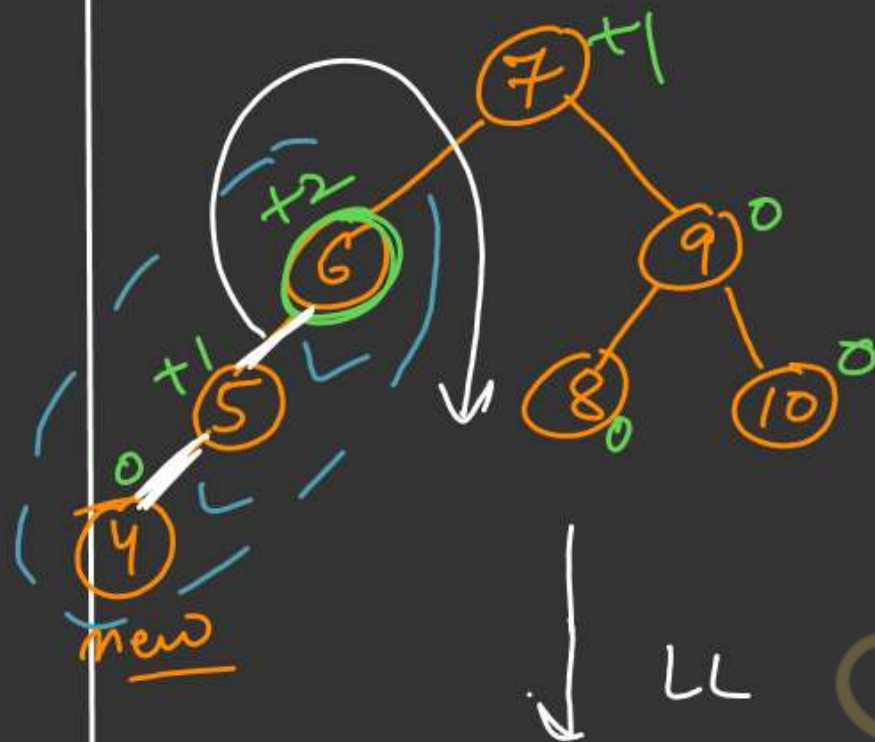


LL

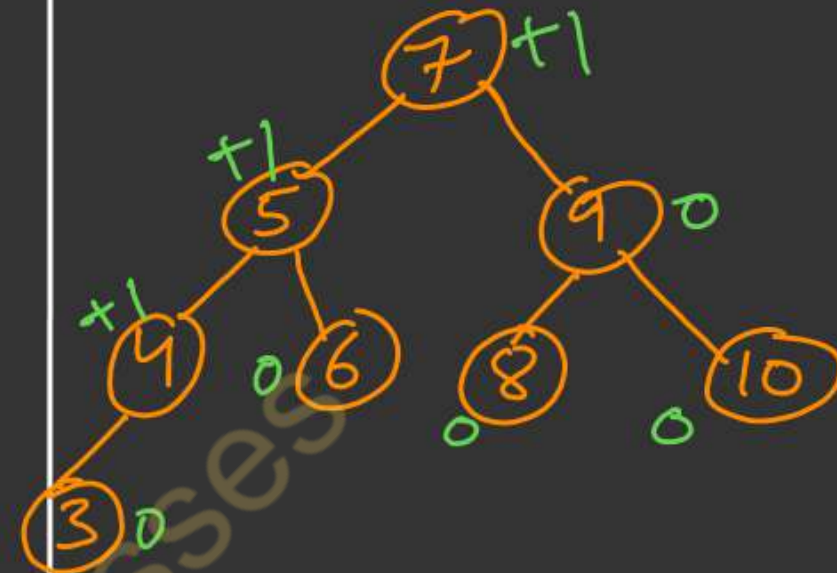
⑥ Insert 5



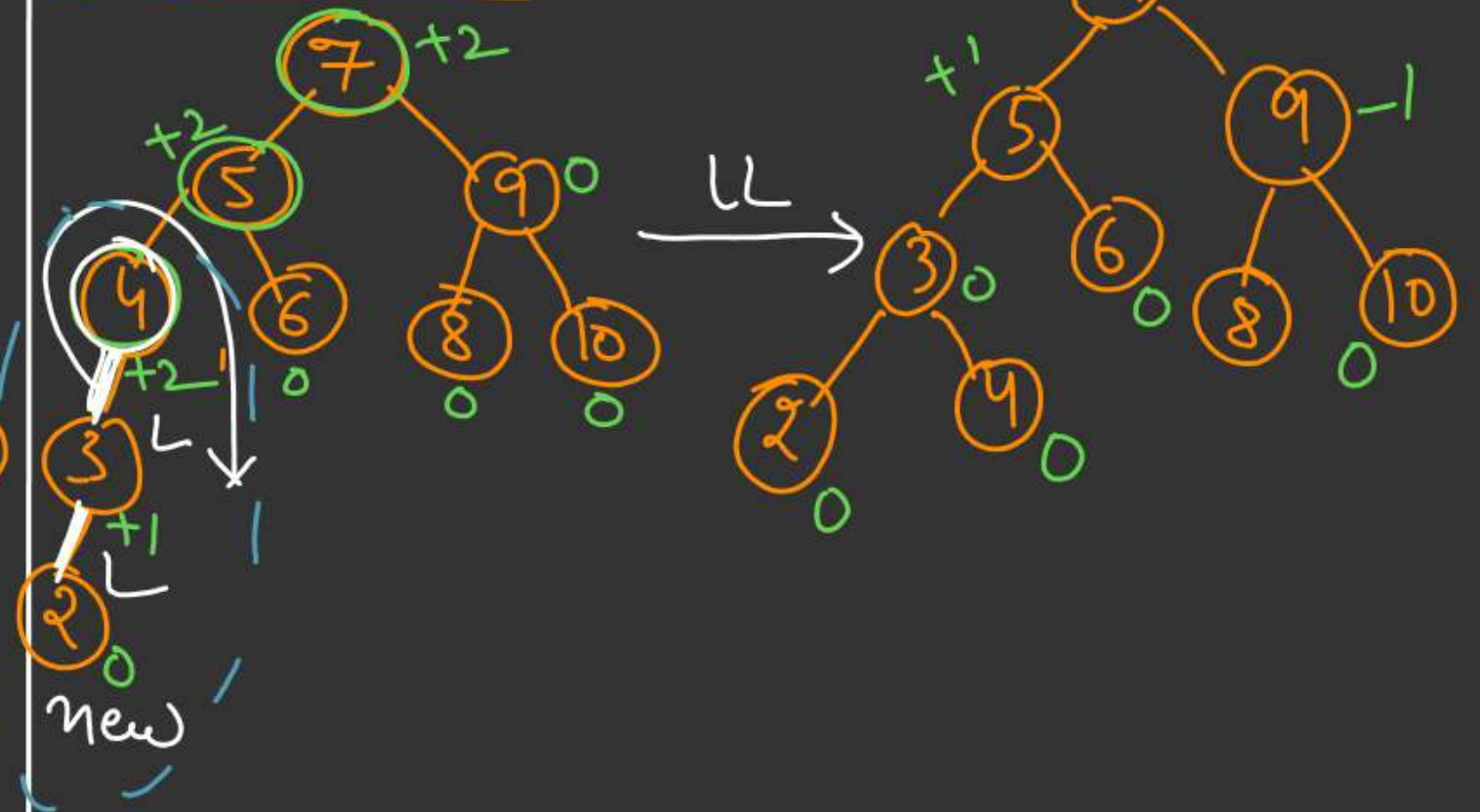
7) Insert 4



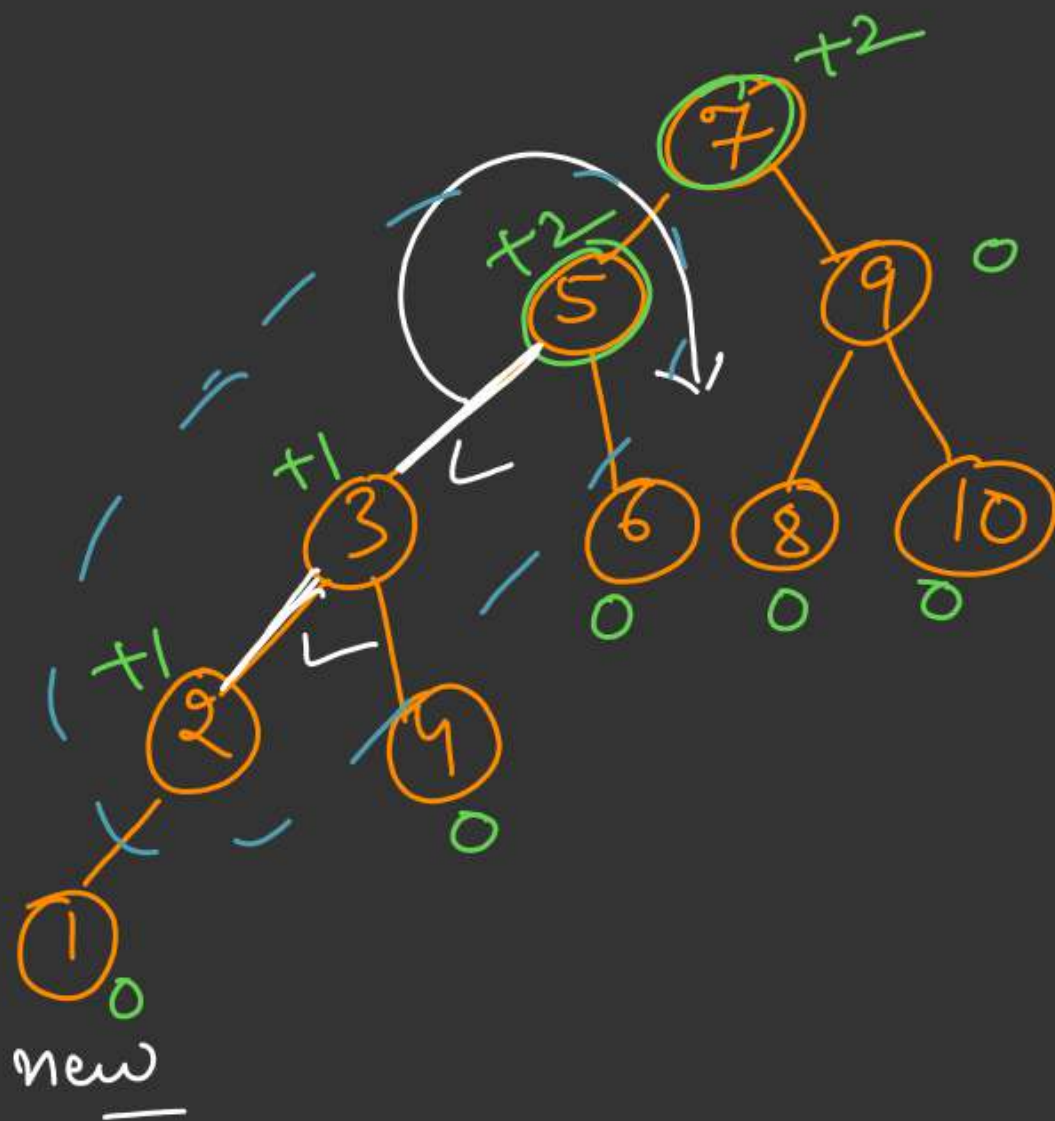
⑧ Insert 3



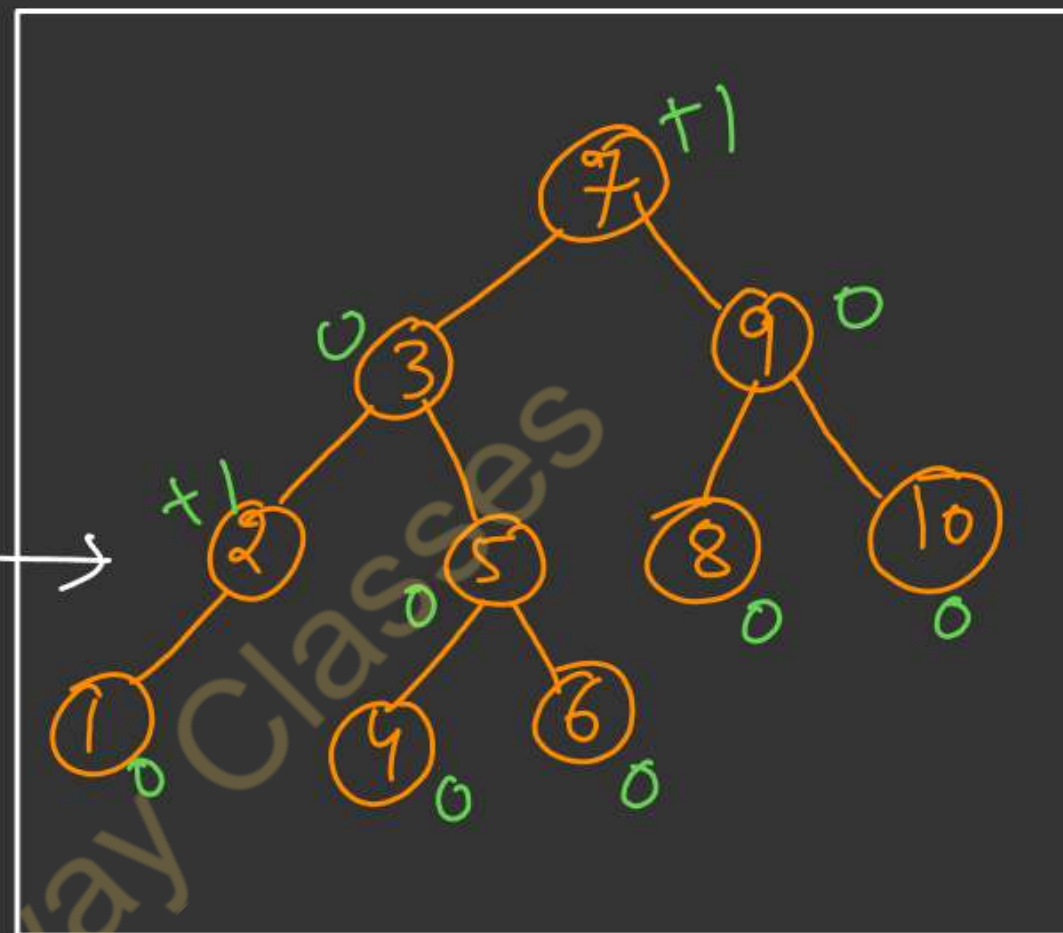
⑨ Insert 2



10) Insert 1



LL →



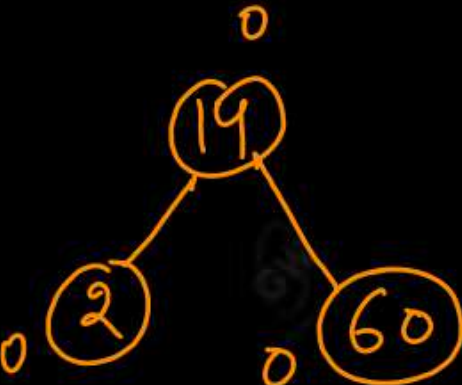
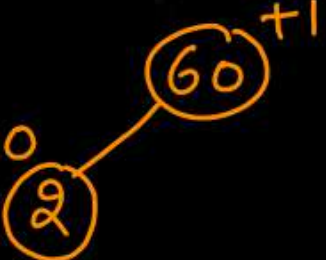
Q: Construct an AVL tree by inserting the following elements in the order of their occurrence:

✓✓✓✓✓✓
60, 2, 14, 22, 13, 111, 92, 86. (AKTU 2022-23)

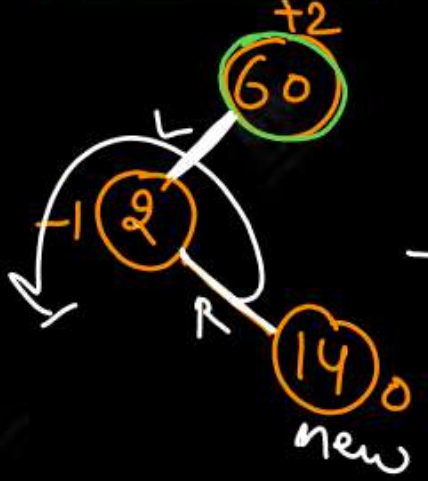
1) Insert 60



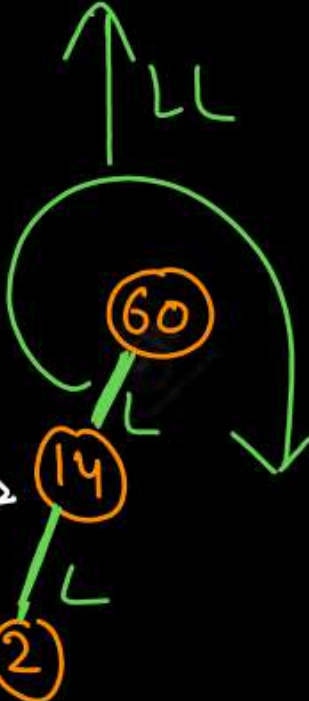
2) Insert 2



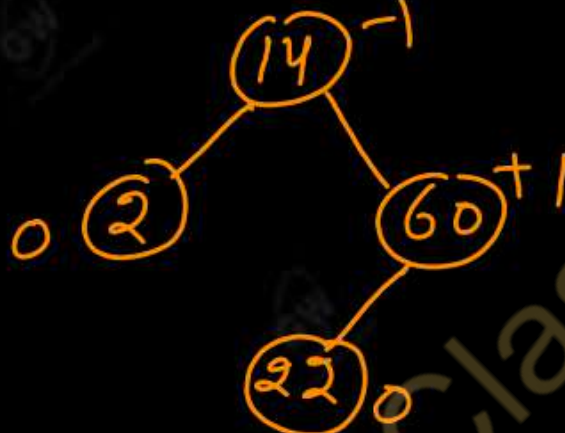
3) Insert 14



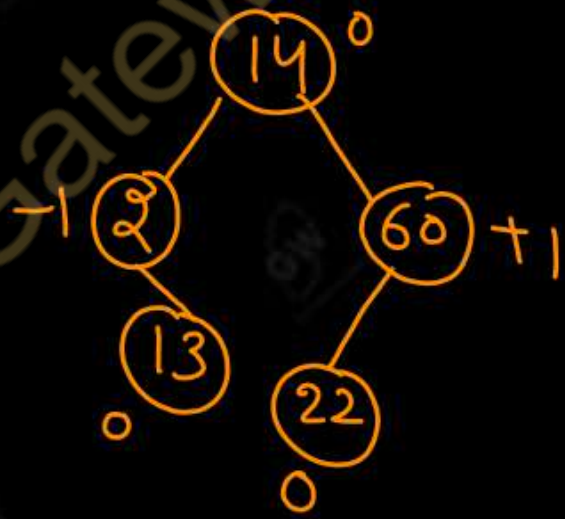
RR



4) Insert 22



5) Insert 13



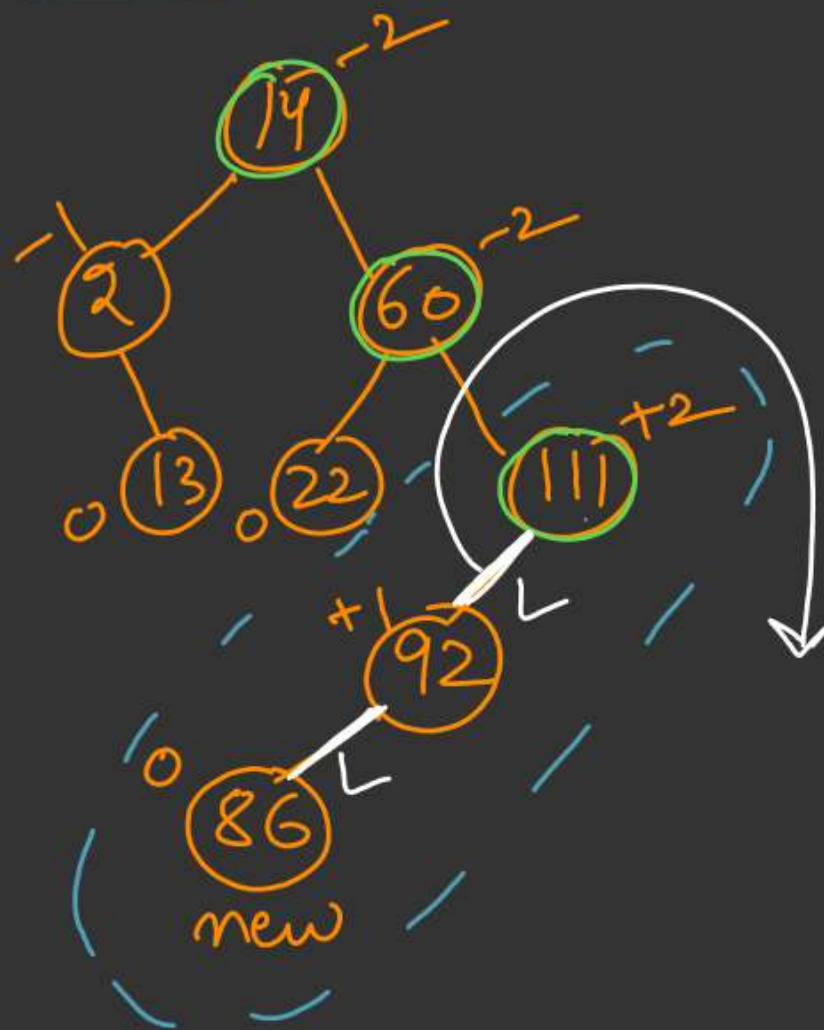
6)



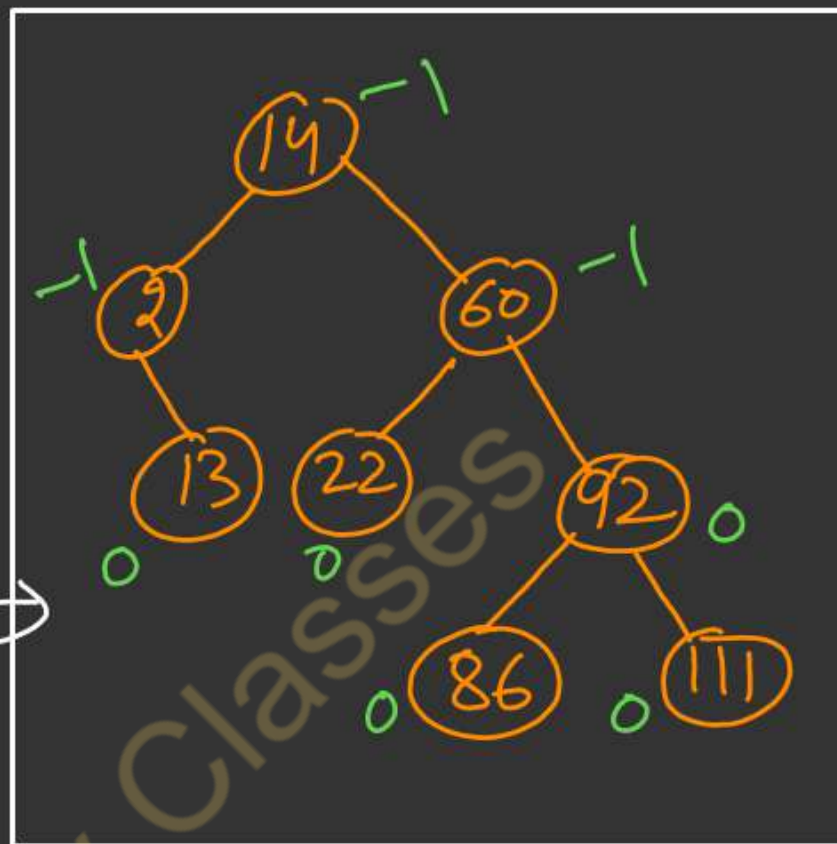
7)



8) Insert 86



LL

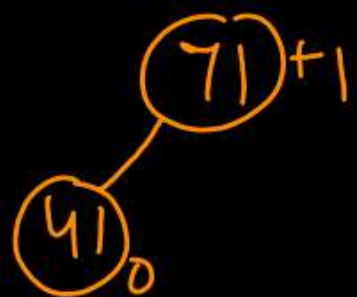


Q: Construct an AVL Tree by inserting following sequence of elements, starting with an empty tree: 71, 41, 91, 56, 60, 30, 40, 80, 50, 55. Also, find the minimum array size to represent this tree. (AKTU 2024-25, 2020-21)

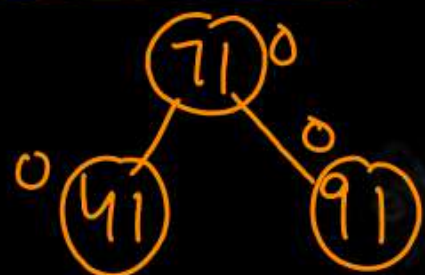
1) Insert 71



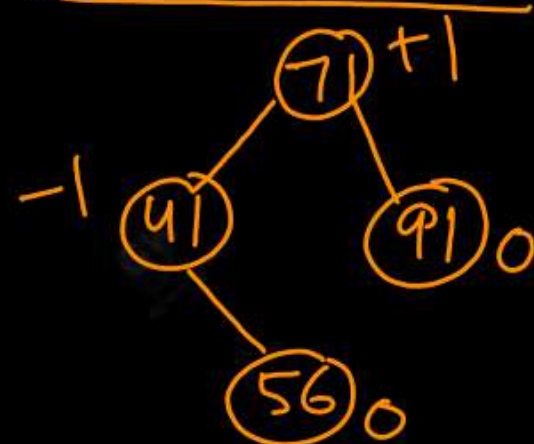
2) Insert 41



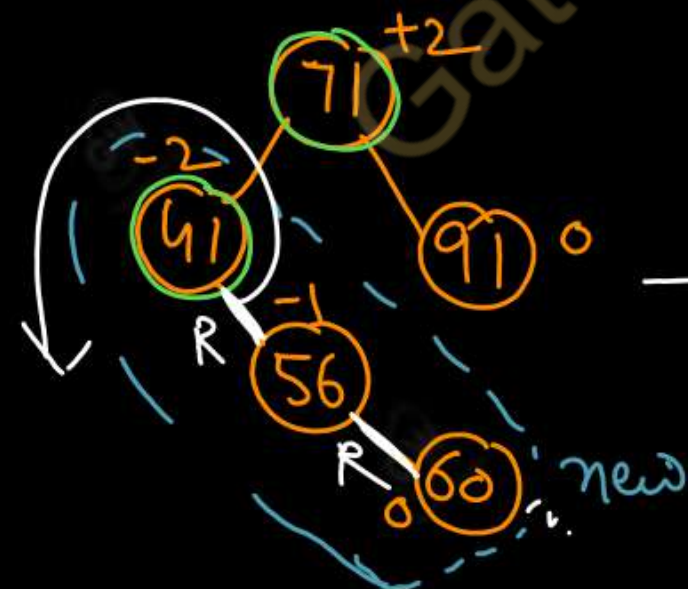
3) Insert 91



4) Insert 56

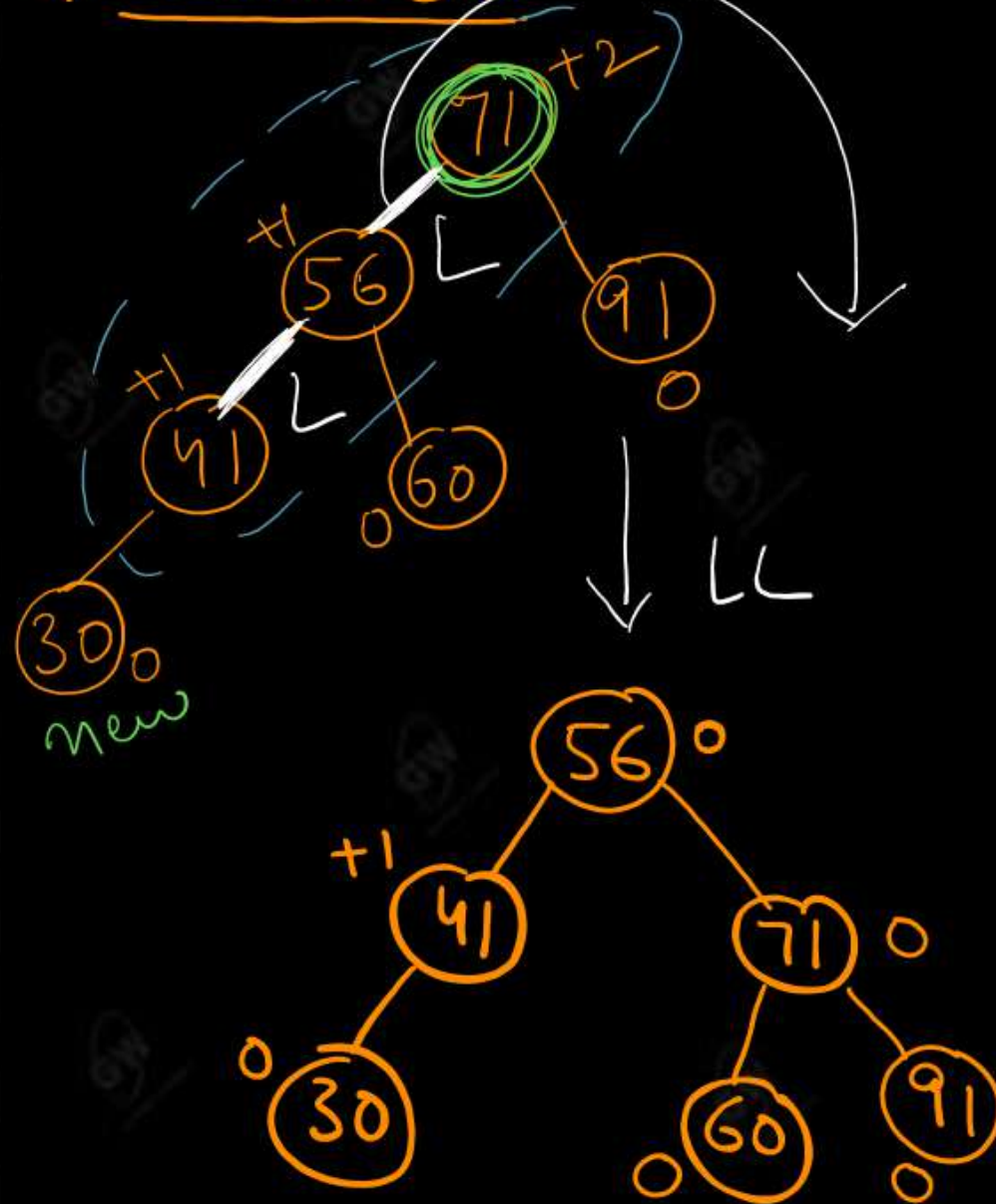


5) Insert 60



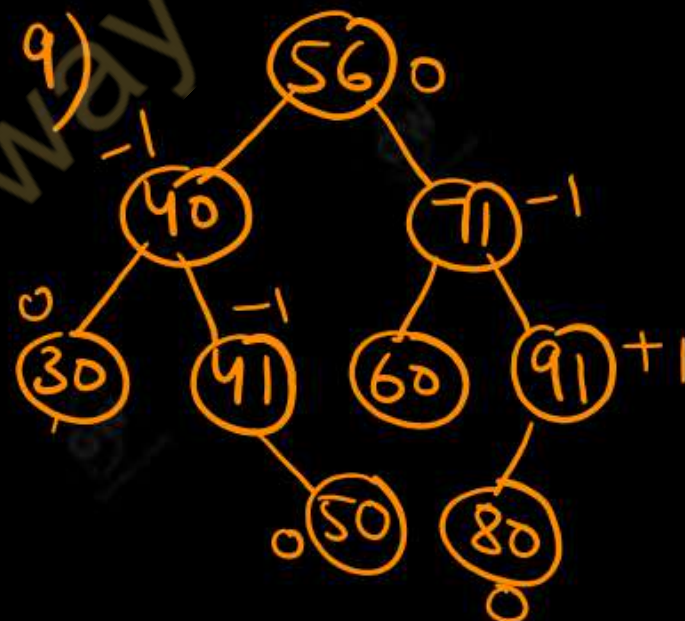
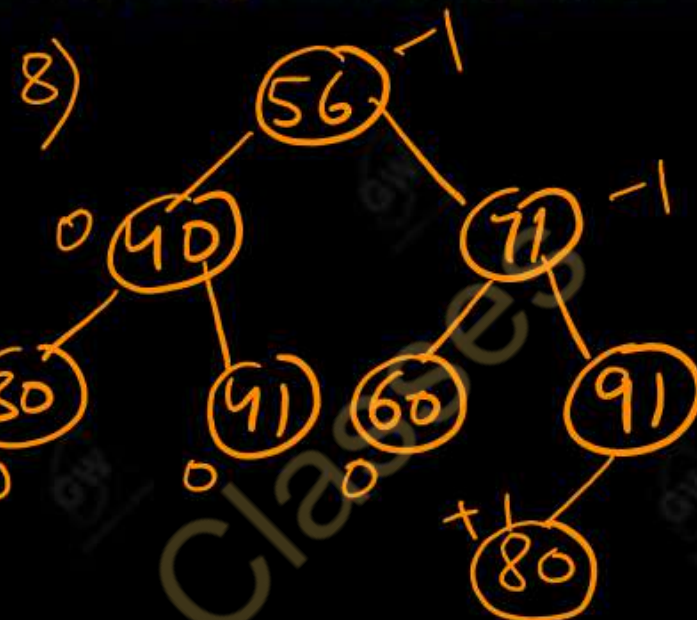
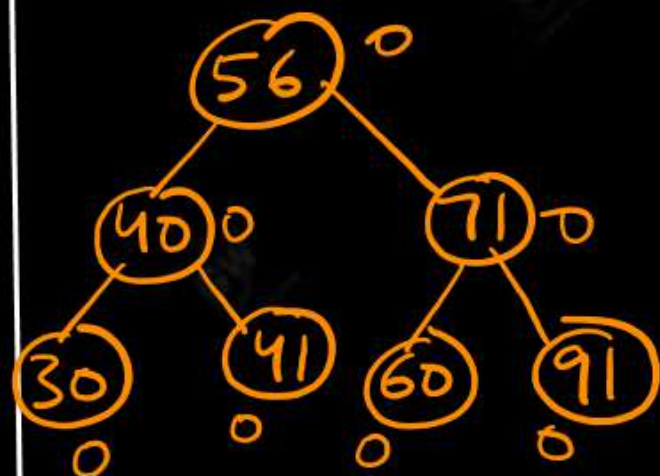
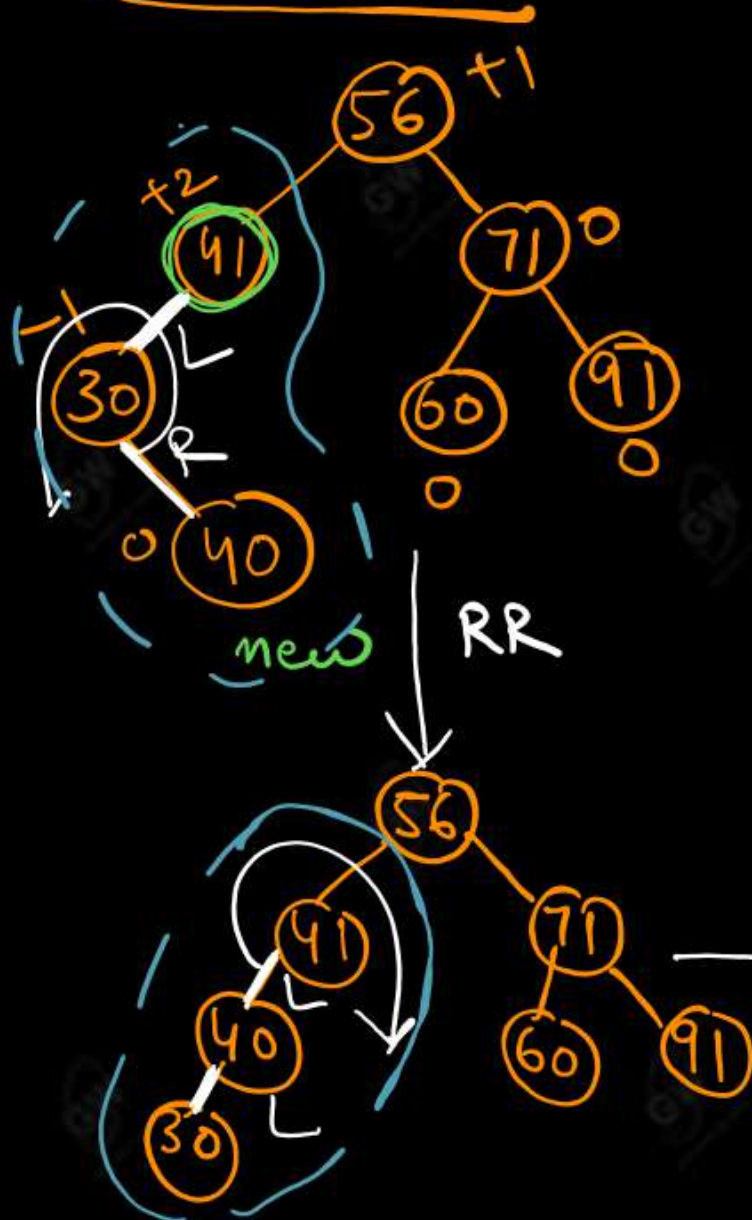
RR

6) Insert 30

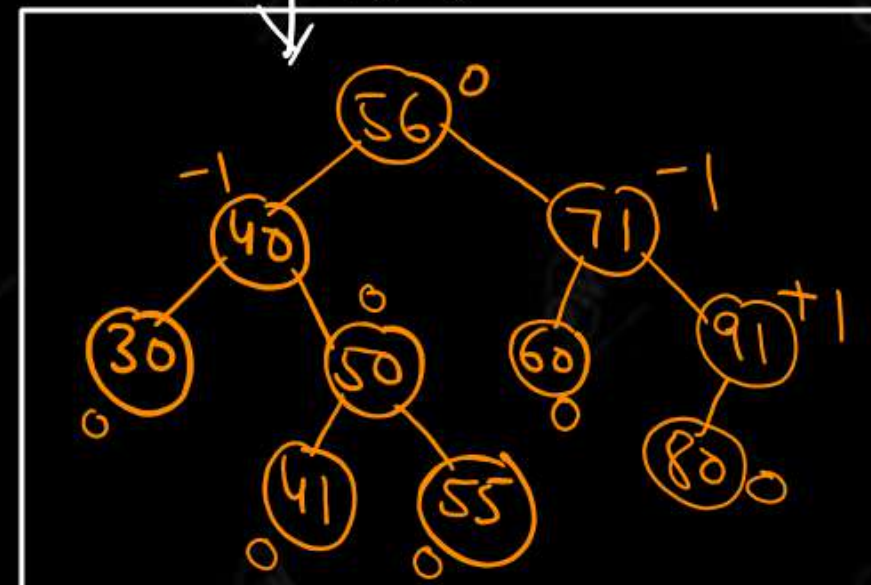
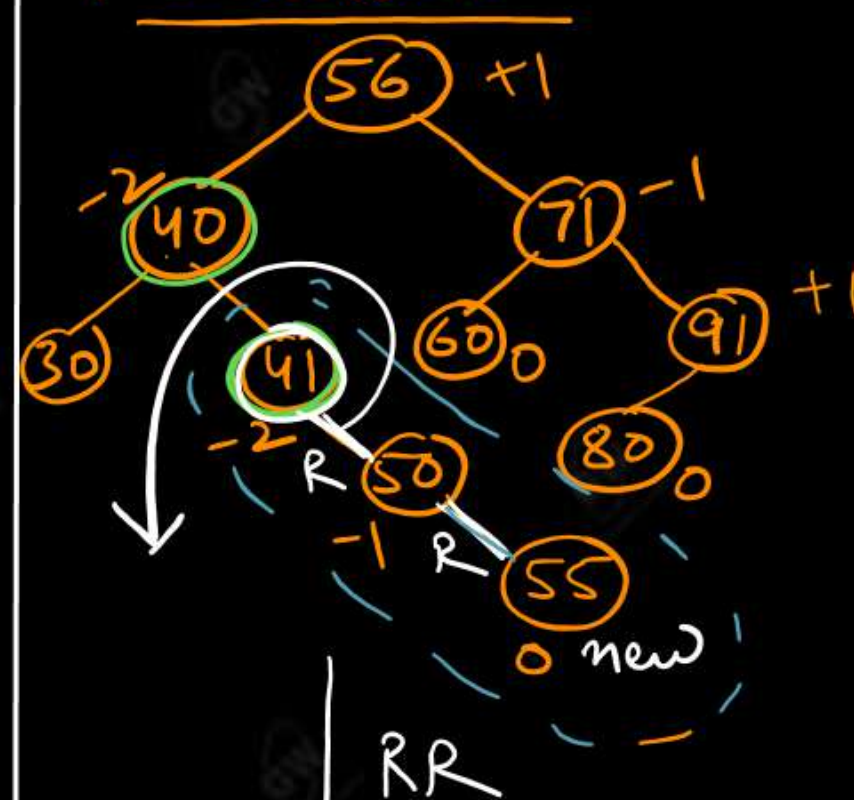


Q: Construct an AVL Tree by inserting following sequence of elements, starting with an empty tree: 71, 41, 91, 56, 60, 30, 40, 80, 50, 55. Also, find the minimum array size to represent this tree. (AKTU 2024-25, 2020-21)

7) Insert 40



10) Insert 55



Q- Insert following keys into initially empty AVL Tree.

50, 20, 60, 10, 8, 15, 32, 46, 11, 48

1) Insert 50



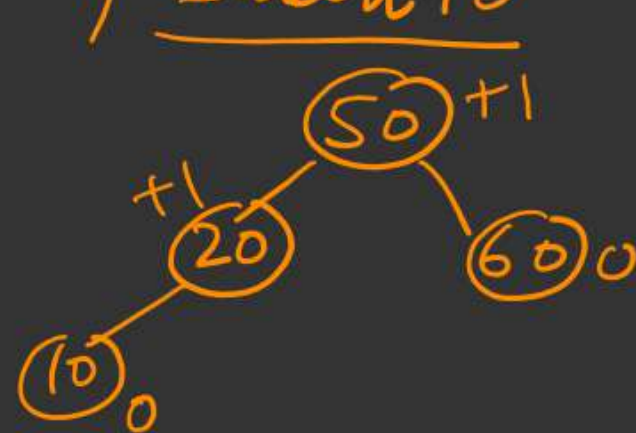
2) Insert 20



3) Insert 60



4) Insert 10

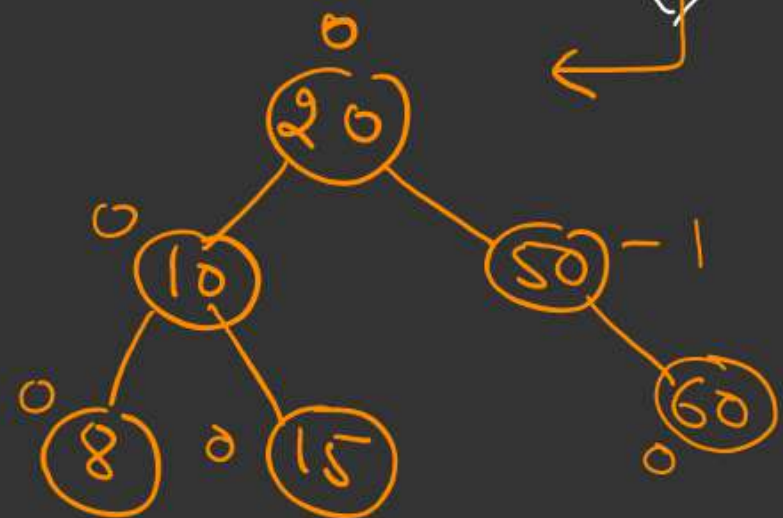


5) Insert 8



Insert 15

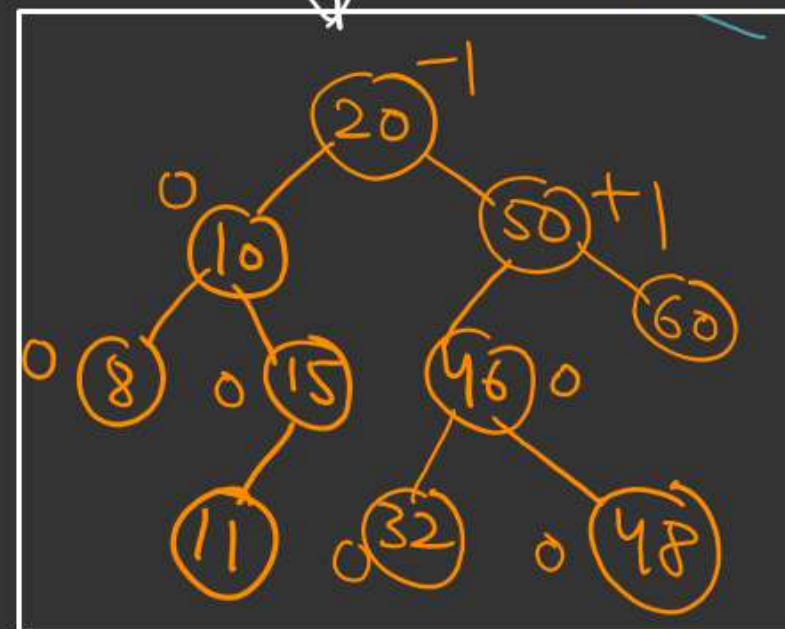
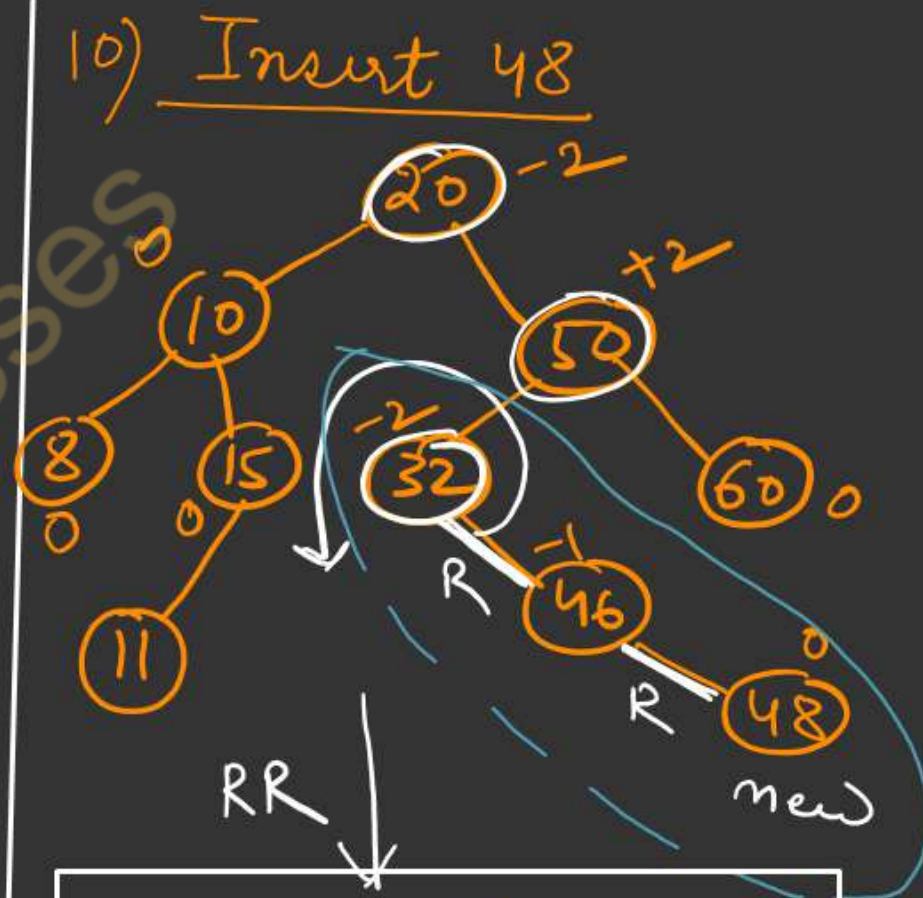
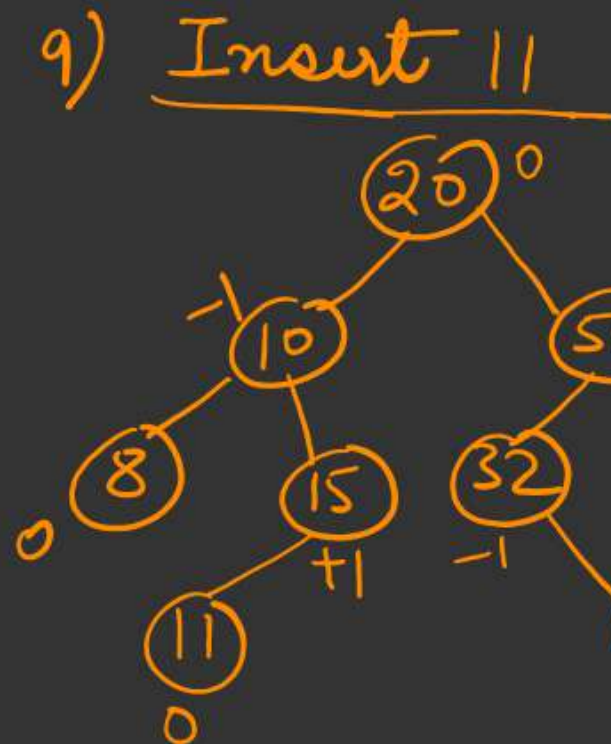
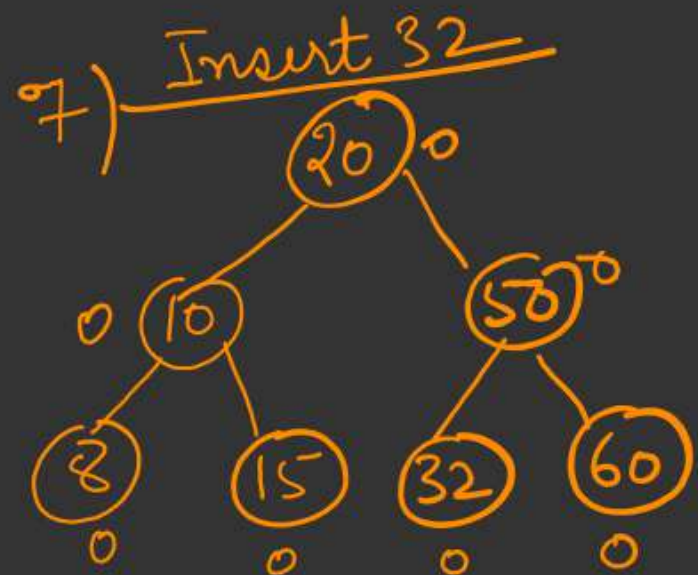
RR



LL

Q- Insert following keys into initially empty AVL Tree.

50, 20, 60, 10, 8, 15, 32, 46, 11, 48



Deletion in the AVL Trees take place in three different scenarios –

1. Deletion of a leaf node –

- If the node to be deleted is a leaf node, then it is deleted without any replacement . If balance factor gets disturbed, rotations are applied to restore it.

2. Deletion of a node with one child –

- If the node to be deleted has one child, replace the value in that node with the value in its child node. Then delete the child node. If the balance factor is disturbed, rotations are applied.

3. Deletion of a node with two child nodes –

- If the node to be deleted has two child nodes, find the inorder successor of that node and replace its value with the inorder successor value. Then delete the inorder successor node. If the balance factor is disturbed, rotations are applied.

- 1. Balanced Height:** AVL trees are self-balancing BST. Balanced height leads to efficient search, insert, and delete operations with a time complexity of $O(\log n)$ for all operations.
- 2. Guaranteed Performance:** Due to their balanced nature, AVL trees provide guaranteed logarithmic time complexity for all operations, making them suitable for applications requiring predictable performance.
- 3. Optimized for Search:** AVL trees are optimized for search operations, as balanced structure minimizes the number of comparisons required to find a specific element within the tree.
- 4. Suitable for Real-time Systems:** In real-time systems where predictable performance is crucial, AVL trees are preferred over BST.

Disadvantages:

Overhead of maintaining balance, which can lead to slightly slower insertion and deletion operations compared to regular BST.

1. Construct an AVL Tree by inserting following sequence of elements, starting with an empty tree:
71,41,91,56,60,30,40,80,50,55. (AKTU 2024-25) ✓
2. Discuss left skewed and right skewed binary tree. Construct an AVL tree by inserting the following elements in the order of their occurrence: 60, 2, 14, 22, 13, 111, 92, 86. (AKTU 2022-23)
3. Write advantages of AVL tree over Binary Search Tree (BST) (AKTU 2021-22)
4. Insert following sequence of elements into an AVL tree, starting with empty tree
71,41,91,56,60,30,40,80,50,55 also find minimum array size to represent this tree. (AKTU 2020-21)
5. Define AVL Trees. Explain its rotation operations with example. Construct an AVL tree with the values 10 to 1 numbers into an initially empty tree. (AKTU 2016-17) ✓



AKTU

B.Tech III-Sem



GATEWAY CLASSES

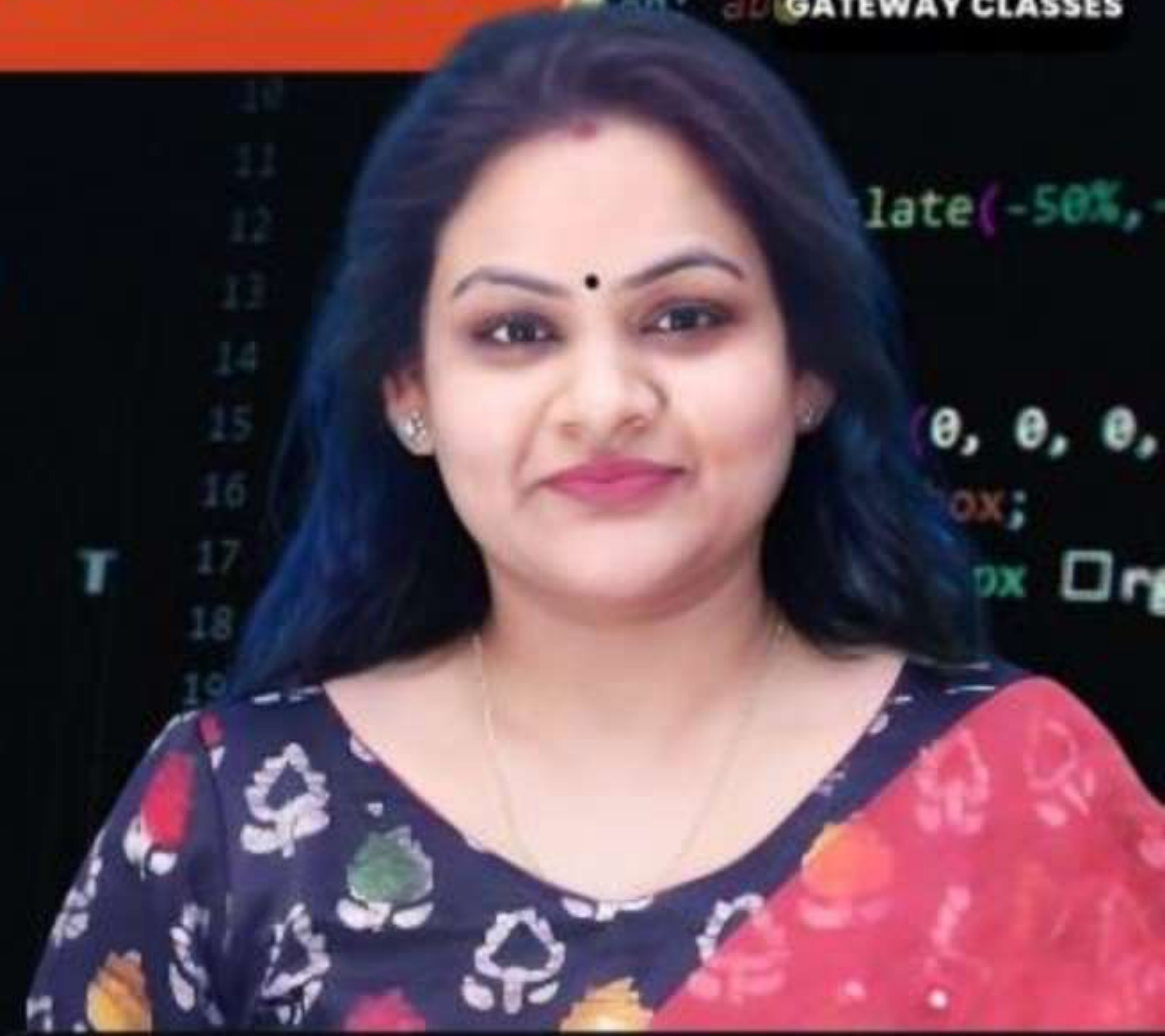
CS IT & CS Allied

DATA STRUCTURE

Unit-4 Lecture-5

Today's Target

- Introduction to B-tree ✓
 - B-Tree properties ✓
 - Insertion operation into B-tree ✓
- AKTU PYQs
- DPP

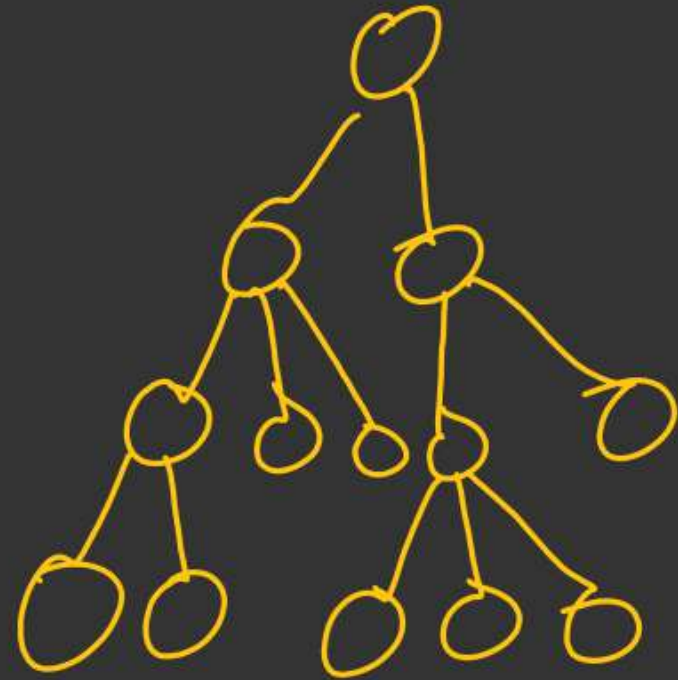


By Dr. Nidhi Parashar Ma'am

Ph.D (CSE)

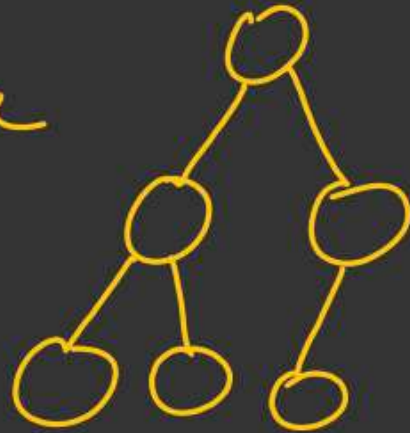
- **M.Tech Gold Medalist**
- **Net Qualified**

Tree



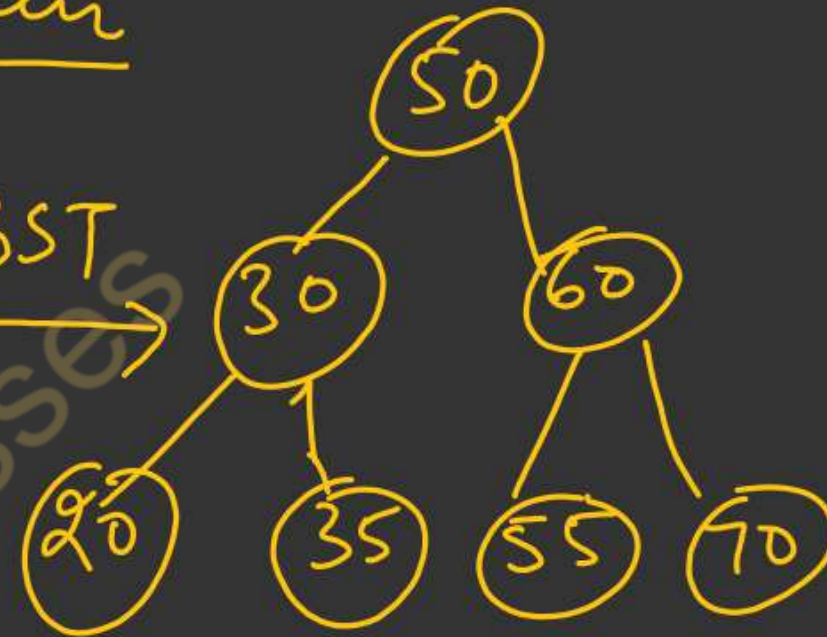
$0/1/2$
Max 2 children

Binary Tree



$\log n$

BST



$O(\log n)$

In-order

20, 30, 35, 50, 55, 60, 70

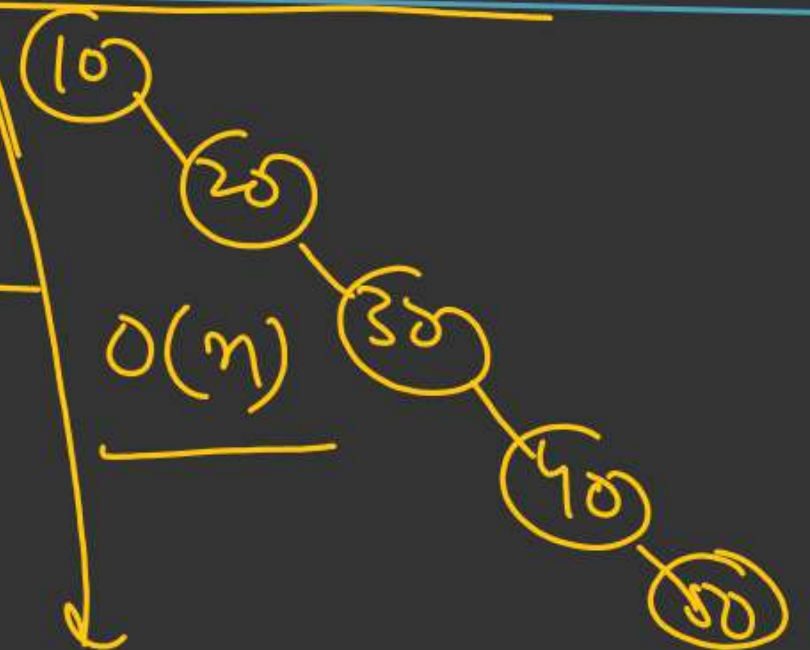
Store multiple
keys in a
single node

B-Tree

AVL tree

Height balanced
BST

LR RL LL RR



$O(n)$

In search trees like binary search tree, AVL Tree etc., every node contains only one value (key) and a maximum of two children. B-Tree is a special type of ^{binary} search tree in which a node contains more than one value (key) and more than two children.

B-Tree was initially called **Height Balanced m-way Search Tree**. Later it was named as B-Tree.

B-Tree can be defined as..

- B-Tree is a self-balanced M-way Search Tree in which every node contains multiple keys and has more than two children.
- Here, the number of keys in a node and number of children for a node depends on the order of B-Tree. Every B-Tree has an order.

Properties of B-Tree (Imp)

$$\left(\left\lceil \frac{M}{2} \right\rceil - 1 \right)$$

- All leaf nodes must be at same level.
- All nodes except root must have at least $\text{ceil}(M/2) - 1$ keys and maximum of $M - 1$ keys.
- All non leaf nodes except root (i.e. all internal nodes) must have at least $\text{ceil}(M/2)$ children.
- If the root node is a non leaf node, then it must have at least 2 children.
- A non leaf node with $n - 1$ keys must have n number of children.
- All the key values in a node must be in Ascending Order.
- B-Tree grows and shrinks from the root unlike Binary Search Tree. Binary Search Trees grow downward and also shrink from downward.
- Time complexity to search, insert and delete is $O(\log n)$.

$$M = 5$$

B tree Structure		In terms of order 'M'	In terms of minimum degree 't'
Maximum Keys	All nodes	M-1	2t-1
Maximum Children	All internal nodes (including root)	M.	2t
Minimum Keys	All nodes except root	Ceil(M/2)-1	t-1
	Root	1	1
Minimum children	All internal nodes (except root)	Ceil(M/2)	t
	Root	2	2

In a 2-3-4 tree ,any internal node can have either two, three, or four children. It is also called a 2-4 tree.

Example B-Tree of Order 5 (M=5)

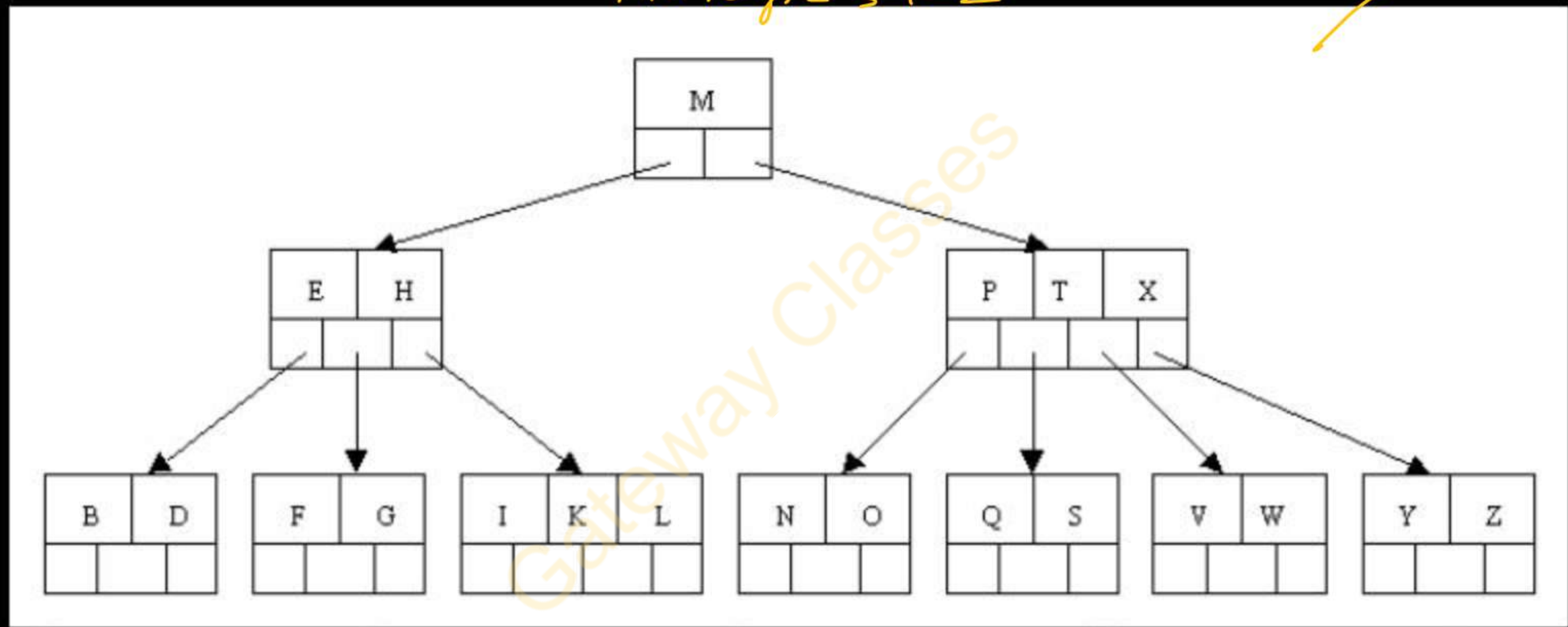
Max children $\Rightarrow 5$

Max Keys = $5 - 1 = 4$

Min children = $\lceil \frac{M}{2} \rceil \Rightarrow \lceil \frac{5}{2} \rceil = 3$

Min Keys = $3 - 1 = 2$

1 [0]



To insert a new key, we go down from root to leaf. Before traversing down to a node, we first check if the node is full. If the node is full, we split it to create space. Following is the complete algorithm.

- INSERT(x,k)
- Find the position i in the node x where k should be inserted:
 - Locate first key in x greater than k or move to the end if no such key exists.
 - If x is a leaf node, directly insert k at position i in sorted order.
 - If x is not a leaf node, proceed to the child node at position i:
 - Check if the child node is full (has maximum number of keys allowed).
 - If the child is full:
 - Split the child node into two nodes.
 - Move the middle key of the child node up to the parent node (x).
 - Adjust the position i if k is greater than the promoted key.
 - Recursively call INSERT procedure on appropriate child node to continue.
- The process ends when k is successfully inserted into a leaf node, ensuring the tree remains balanced and within its key limit.

Q: Insert the following letters into what is originally an empty B-tree of order 5:
A G F B K D H M J E S I R X C L N T U P. (AKTU 2021-22)

Solⁿ. $M=5$

Max children/node = 5

Max no. of Keys/node = $5-1=4$

Insert A:- [A]

Insert G:- [A|G]

Insert F:- [A|F|G]

Insert B:- [A|B|F|G]

Insert K:-

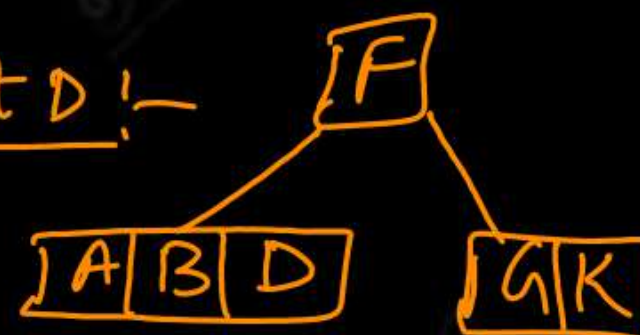
[A|B|F|G|K]

contains more than max. no. of keys

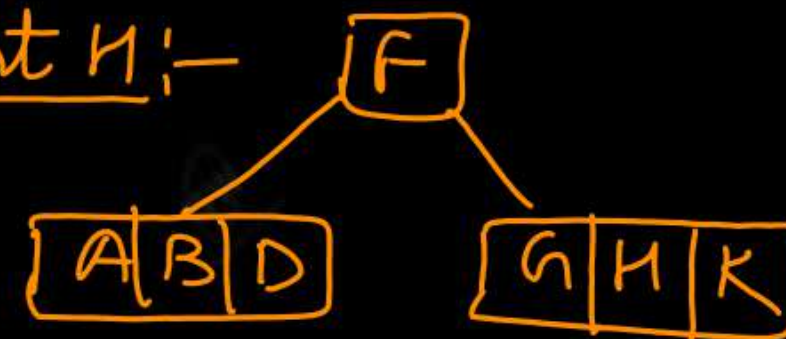
split this node



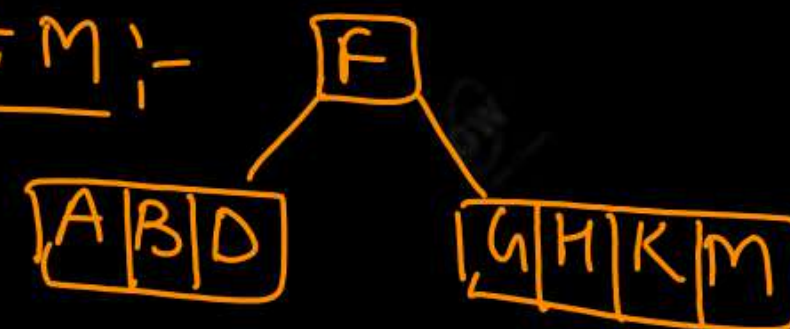
Insert D:-



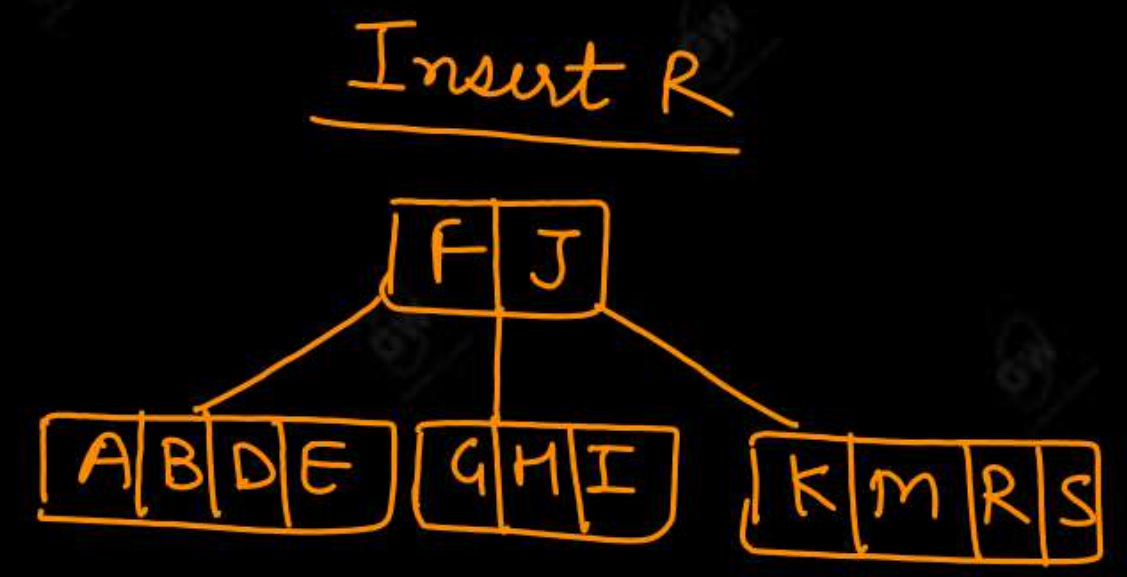
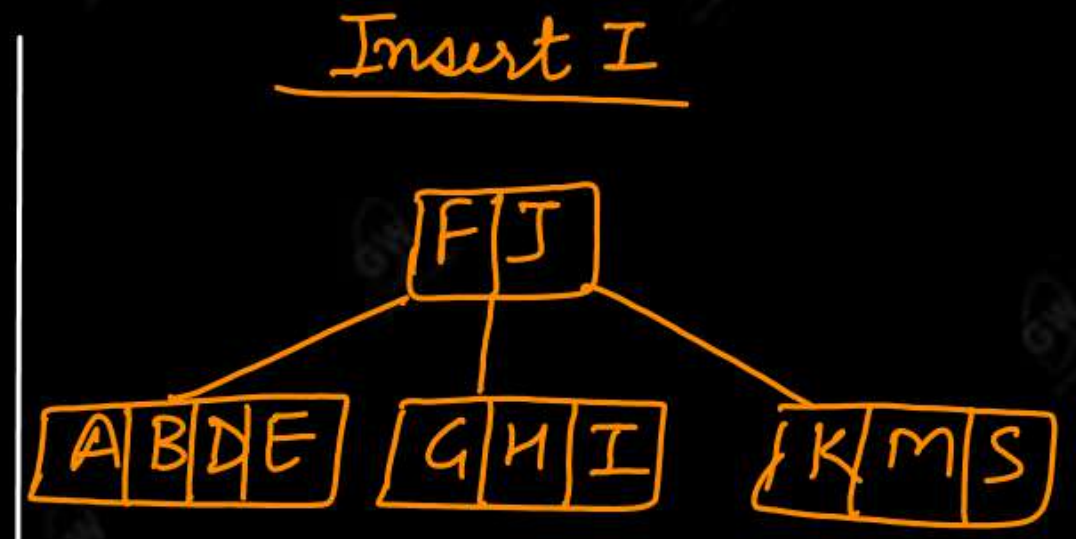
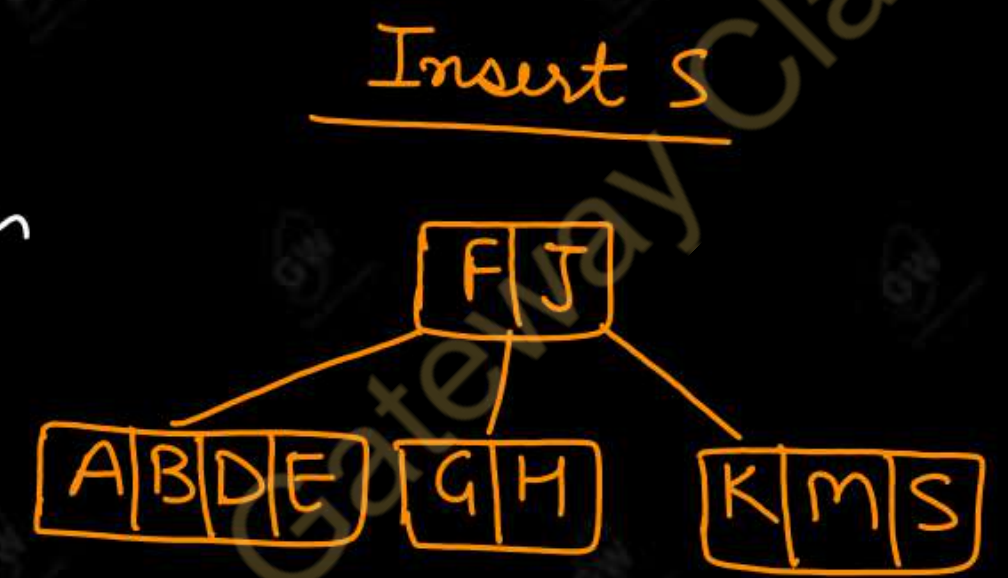
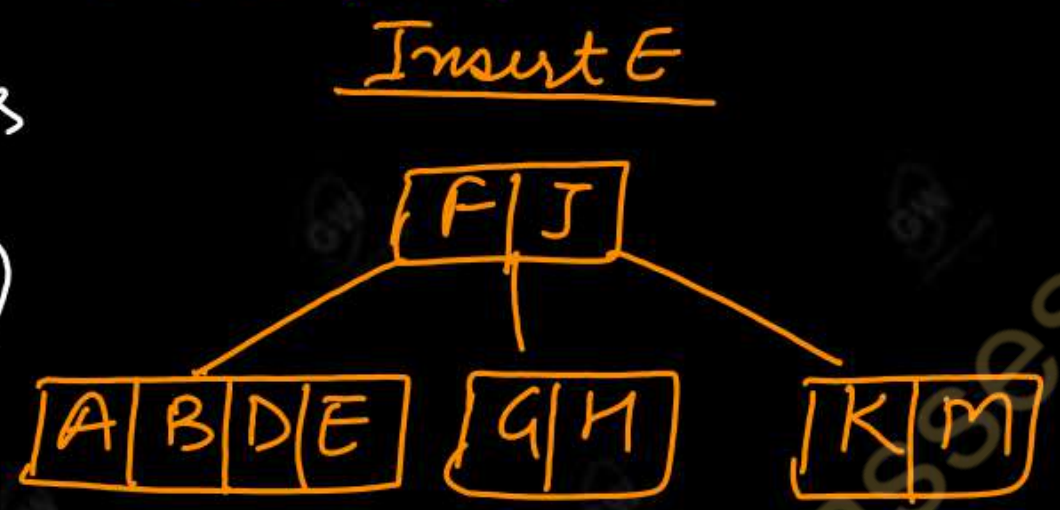
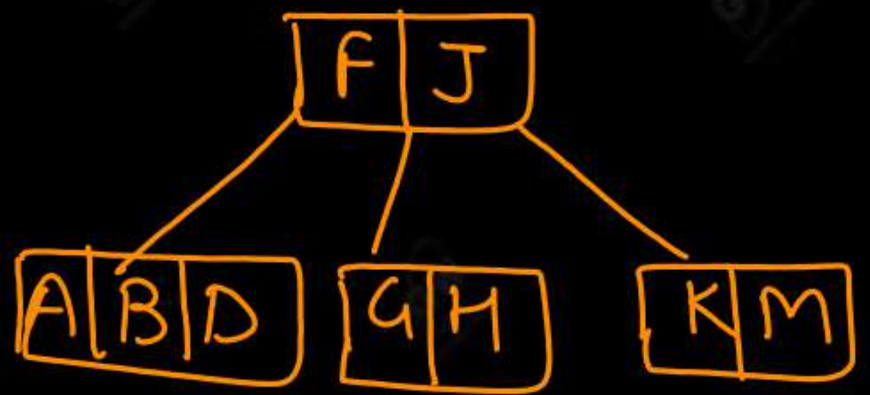
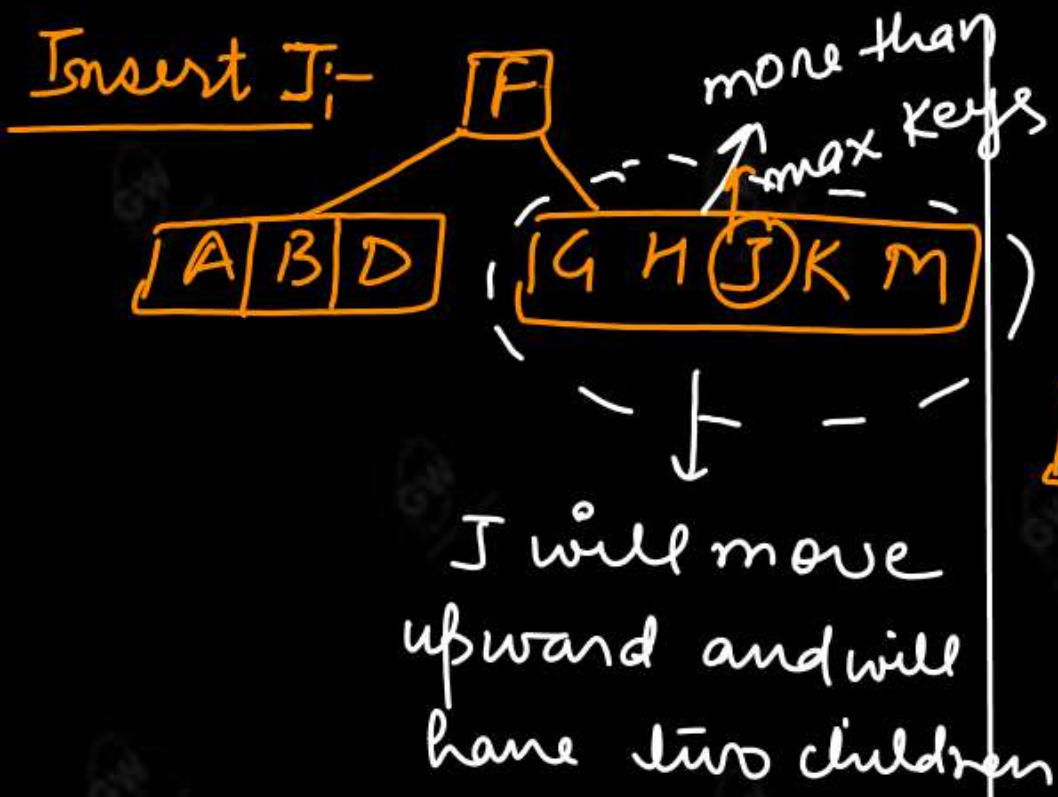
Insert H:-



Insert M:-

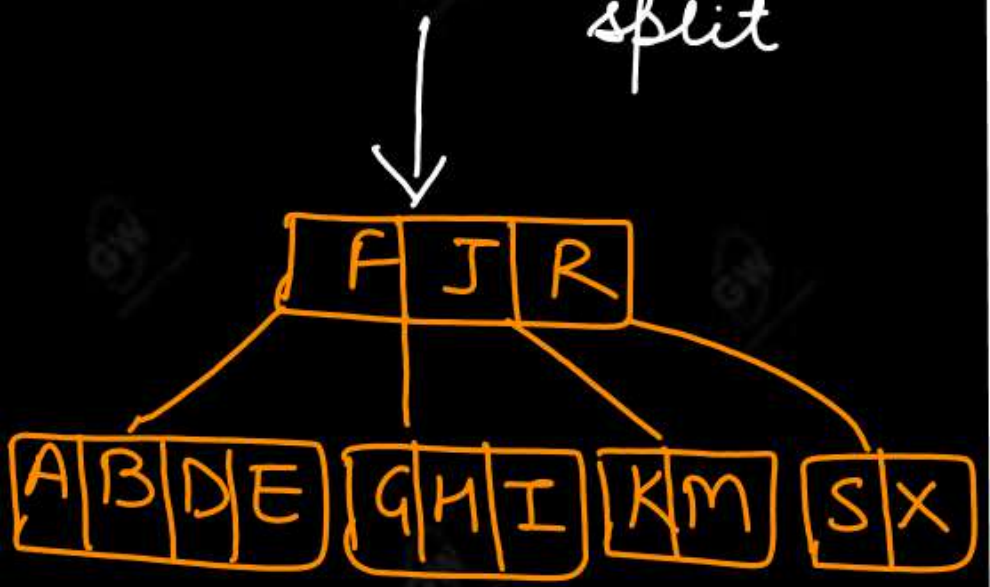
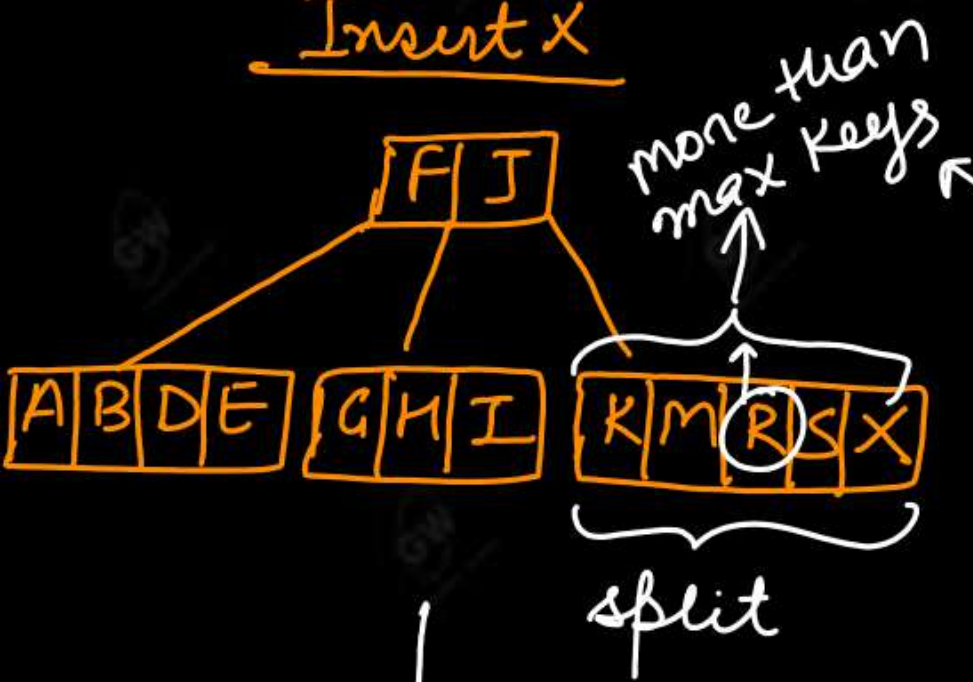


Q: Insert the following letters into what is originally an empty B-tree of order 5:
AGFBKDHMJESIRXCLNTUP. (AKTU 2021-22)

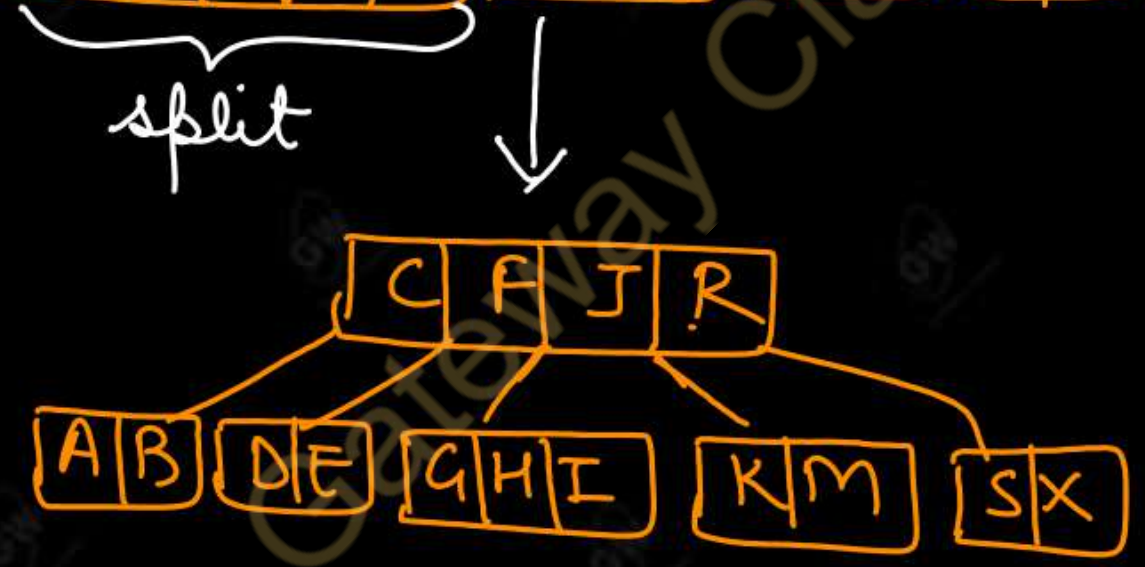
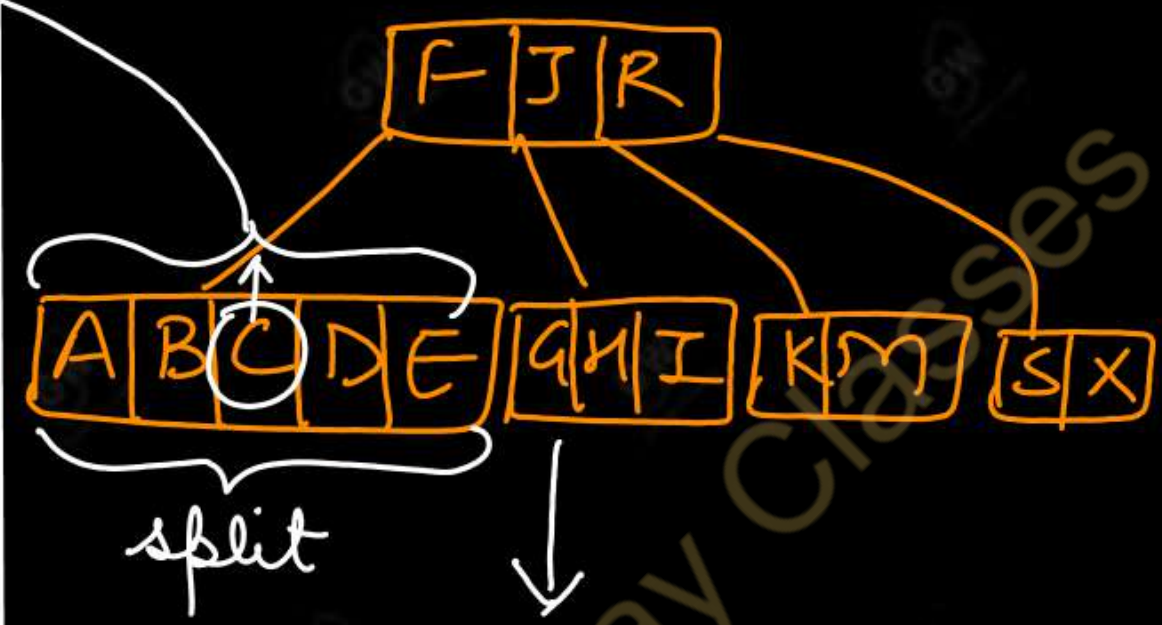


Q: Insert the following letters into what is originally an empty B-tree of order 5:
A G F B K D H M J E S I R X C L N T U P. (AKTU 2021-22)

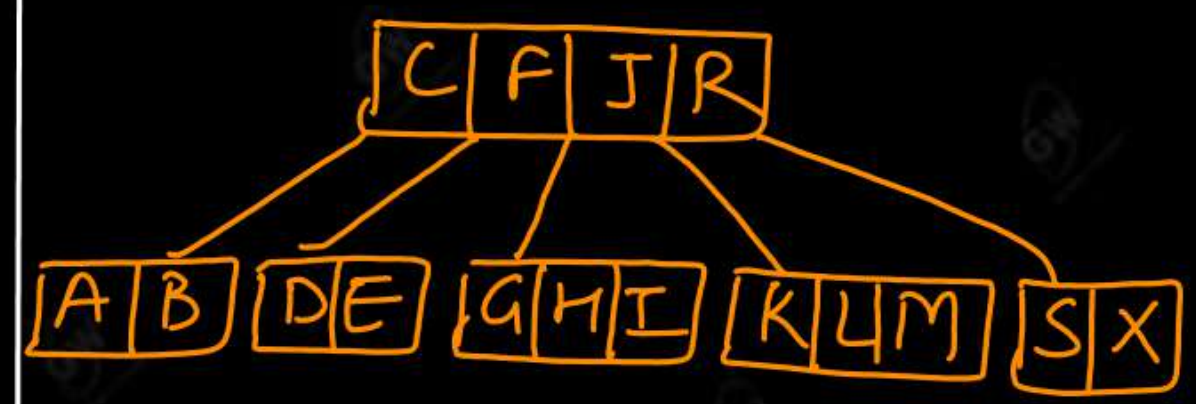
Insert X



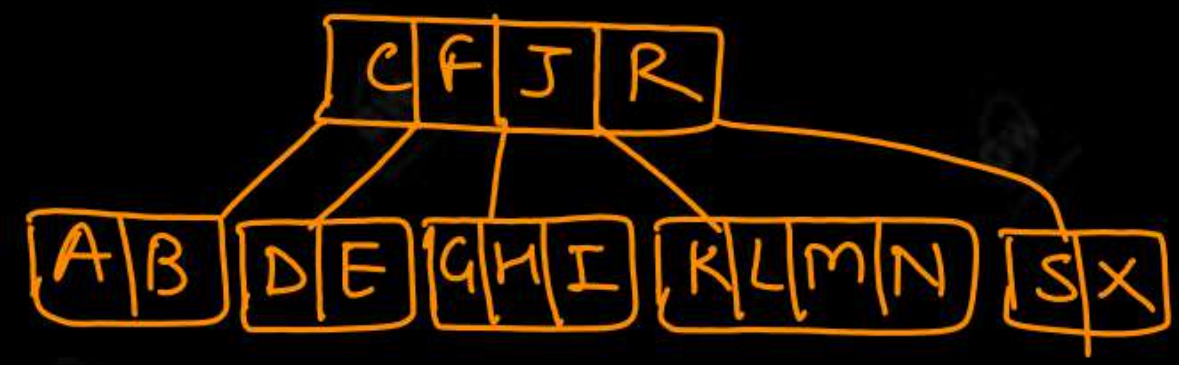
Insert C



Insert L



Insert N

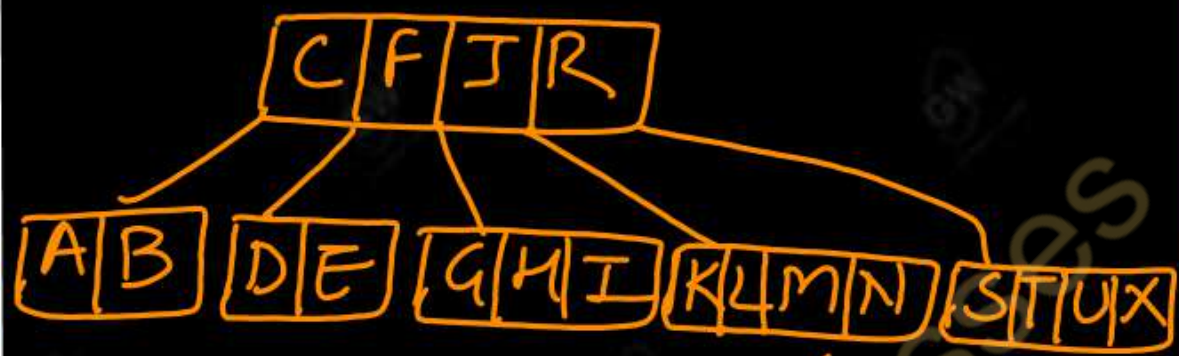


Q: Insert the following letters into what is originally an empty B-tree of order 5:
A G F B K D H M J E S I R X C L N T U P. (AKTU 2021-22)

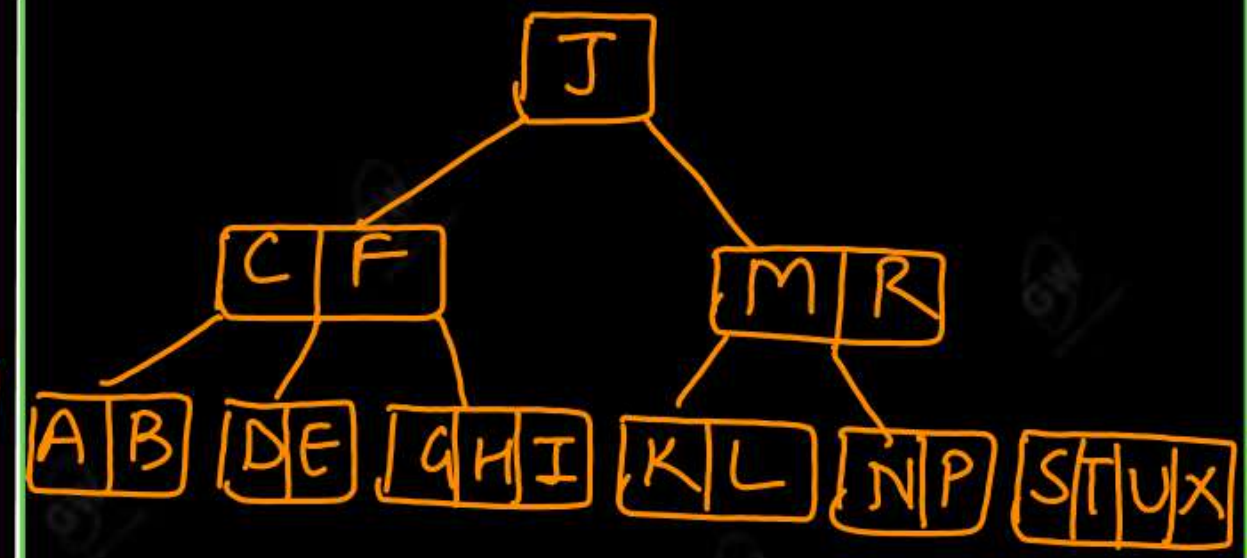
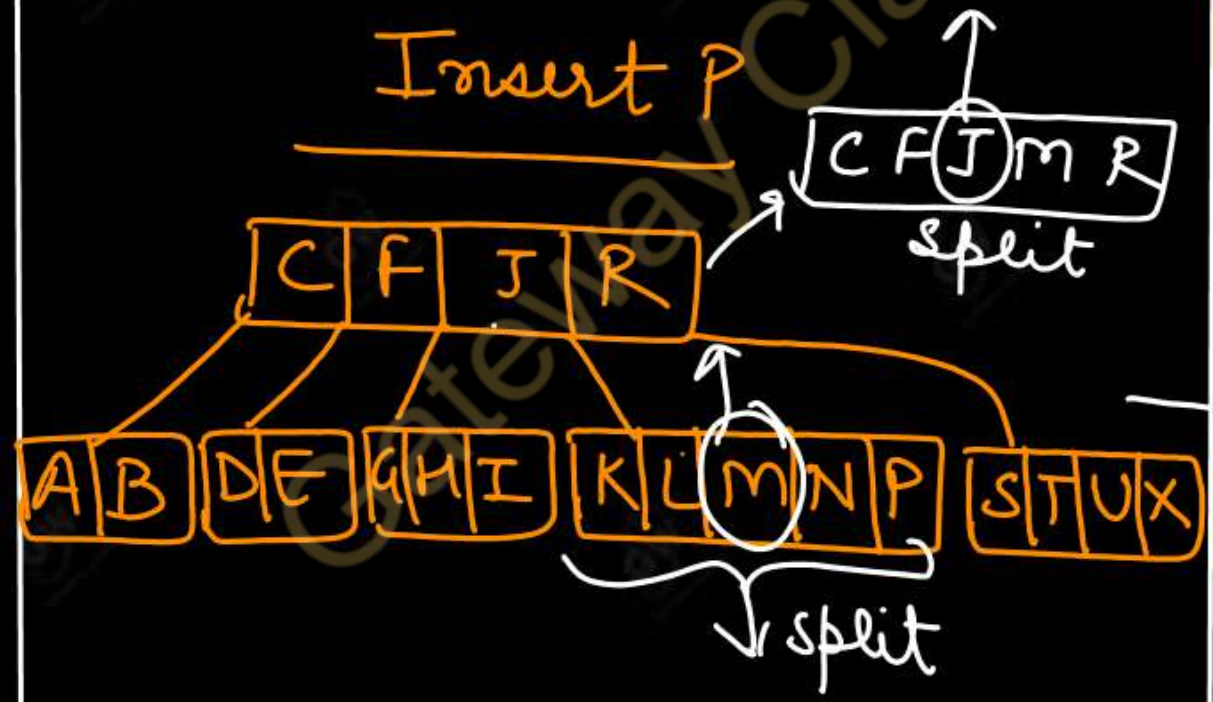
Insert T



Insert U



Insert P



↓
Final B-Tree

Q: Show the results of inserting the keys F, S, Q, K, C, L, H, T, V, W, M, R, N, P, A, B in order into an empty B-Tree of order 5. (AKTU 2022-23)

M=5. Max Keys = 4, Max children = 5

Insert F :-

F

Insert S :-

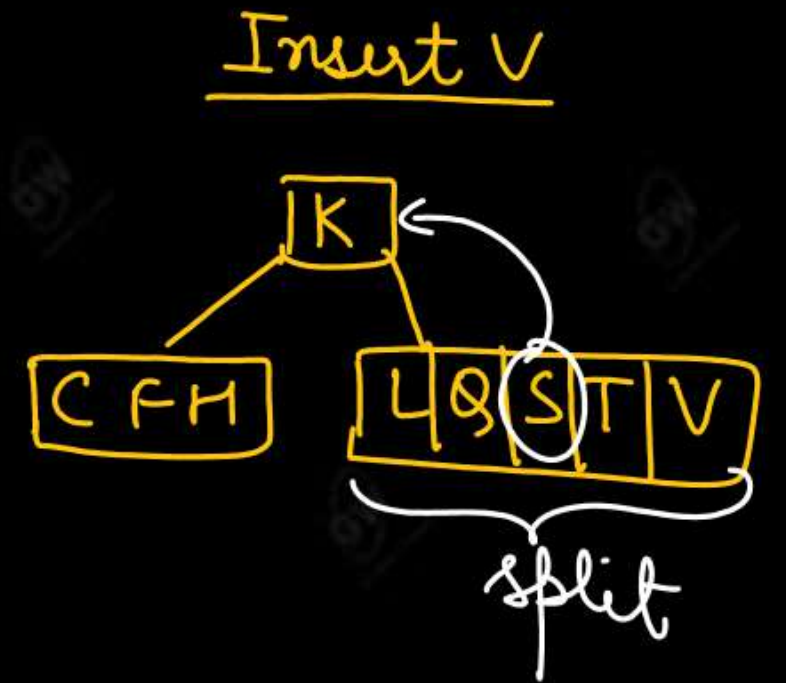
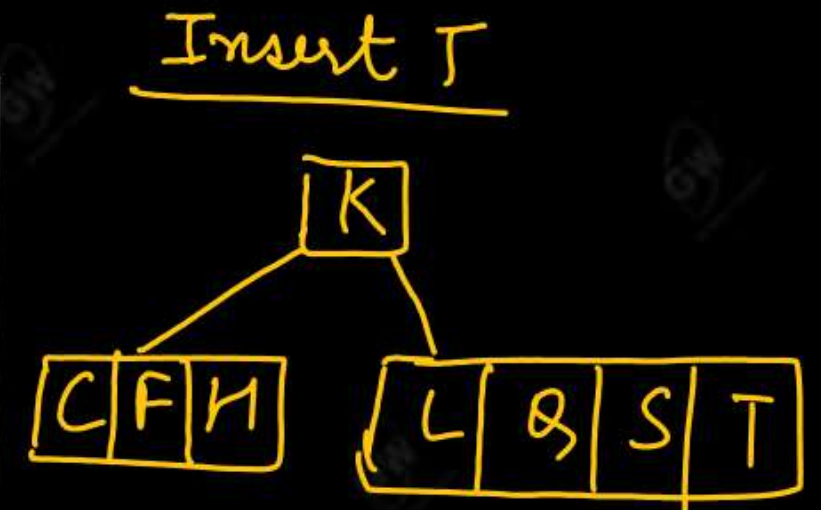
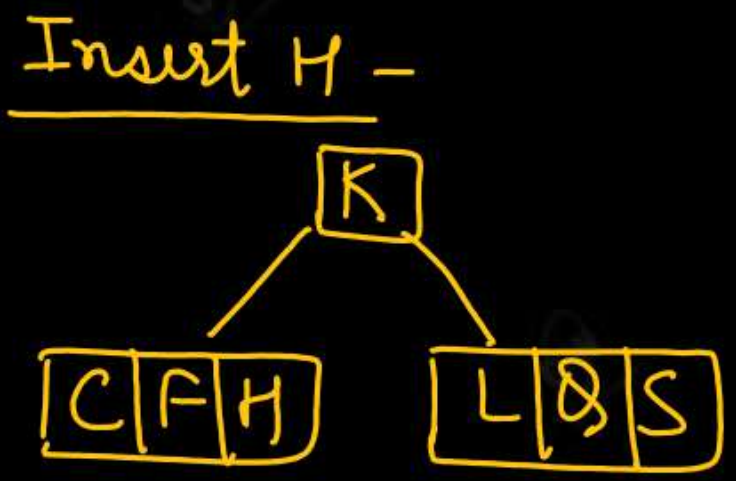
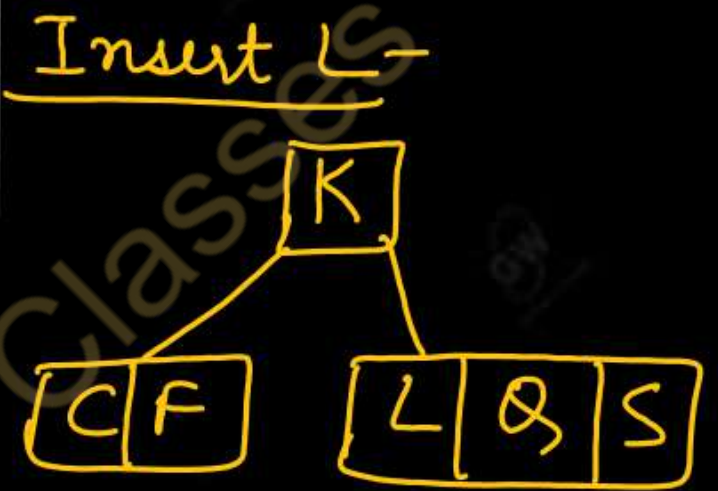
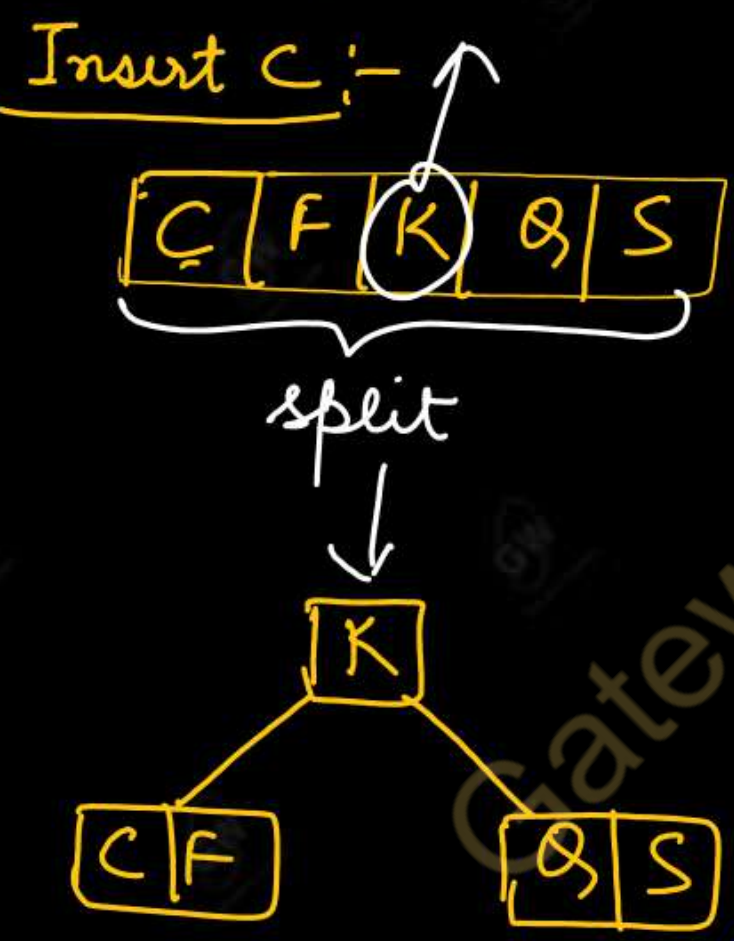
F	S
---	---

Insert Q :-

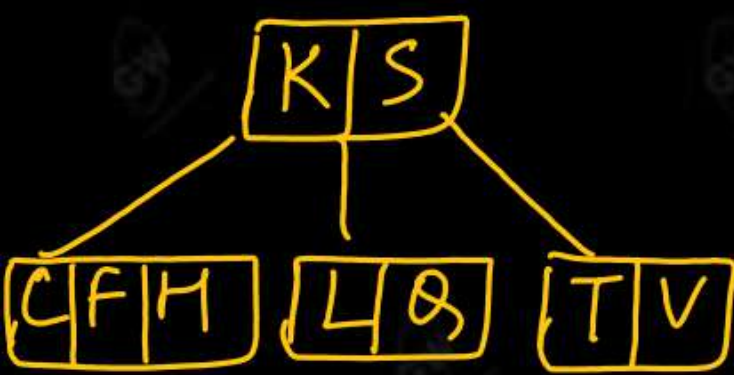
F	Q	S
---	---	---

Insert K -

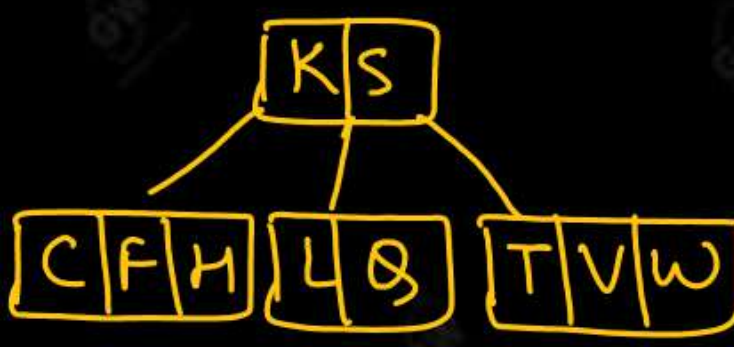
F	K	Q	S
---	---	---	---



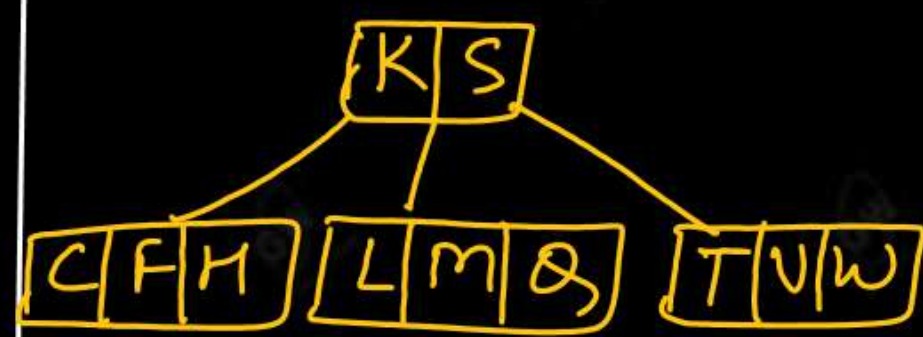
Q: Show the results of inserting the keys F, S, Q, K, C, L, H, T, V, W, M, R, N, P, A, B in order into a empty B-Tree of order 5. (AKTU 2022-23)



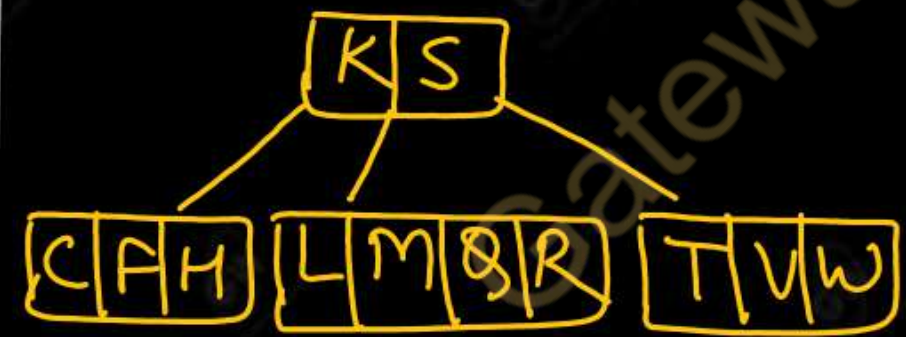
Insert W



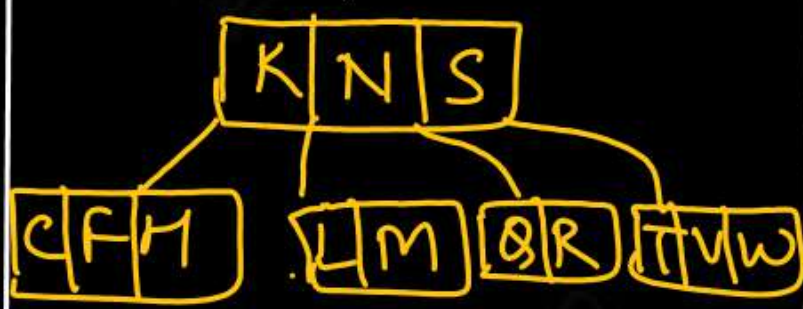
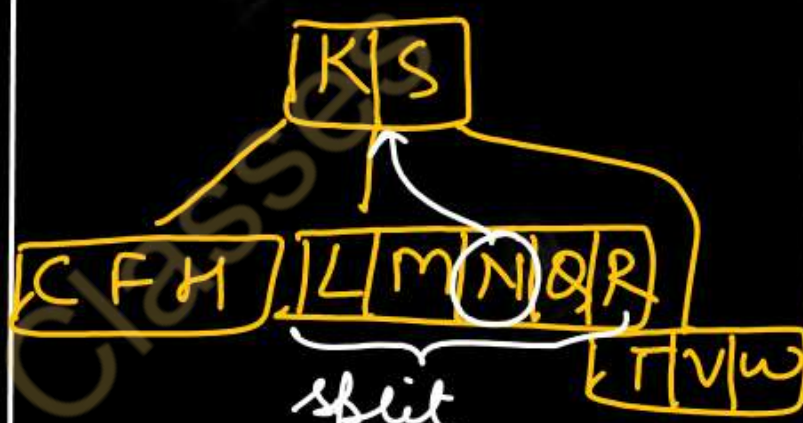
Insert M



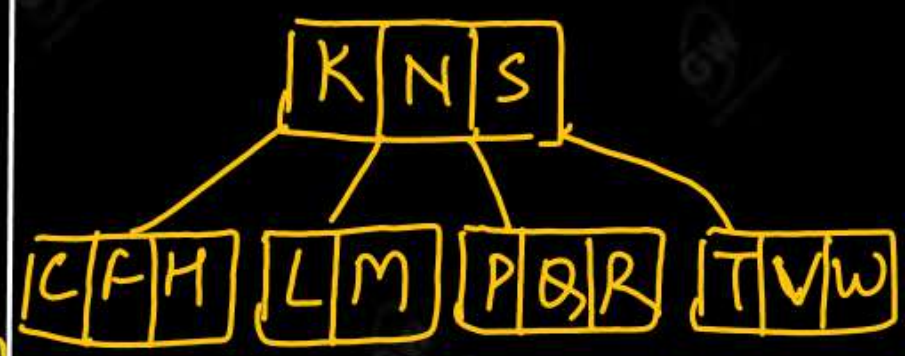
Insert R



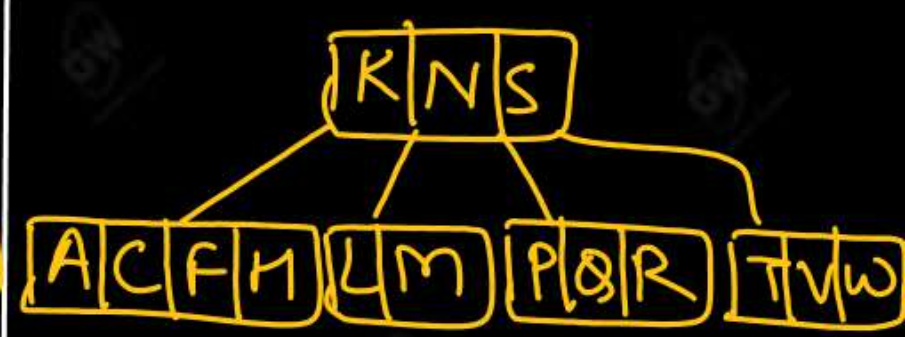
Insert N



Insert P

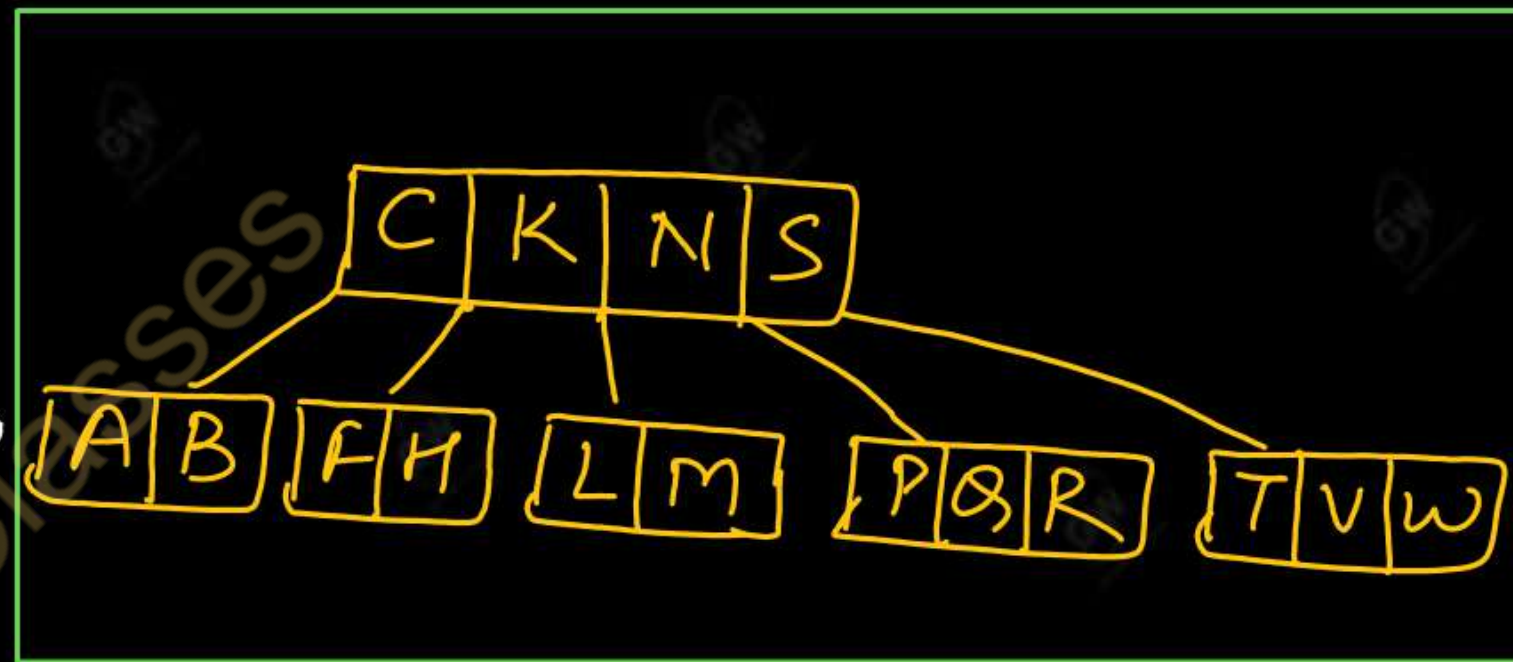
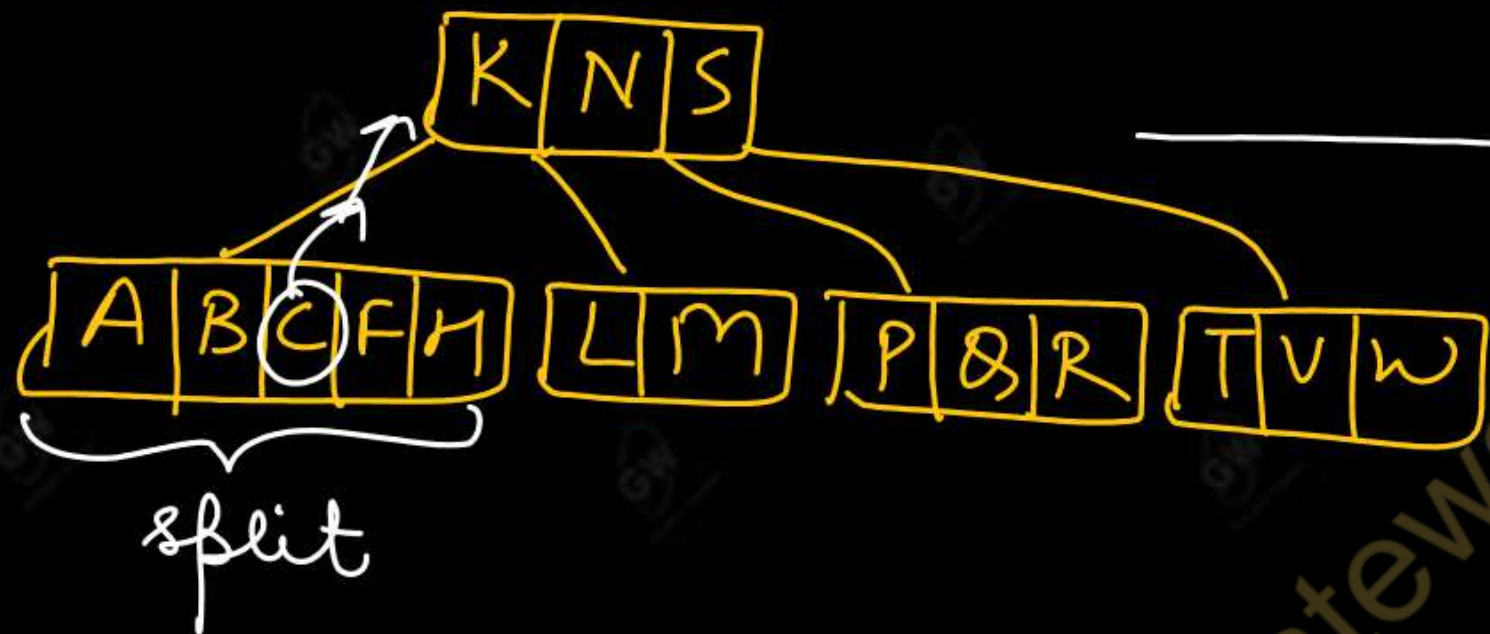


Insert A



Q: Show the results of inserting the keys F, S, Q, K, C, L, H, T, V, W, M, R, N, P, A, B in order into a empty B-Tree of order 5. (AKTU 2022-23)

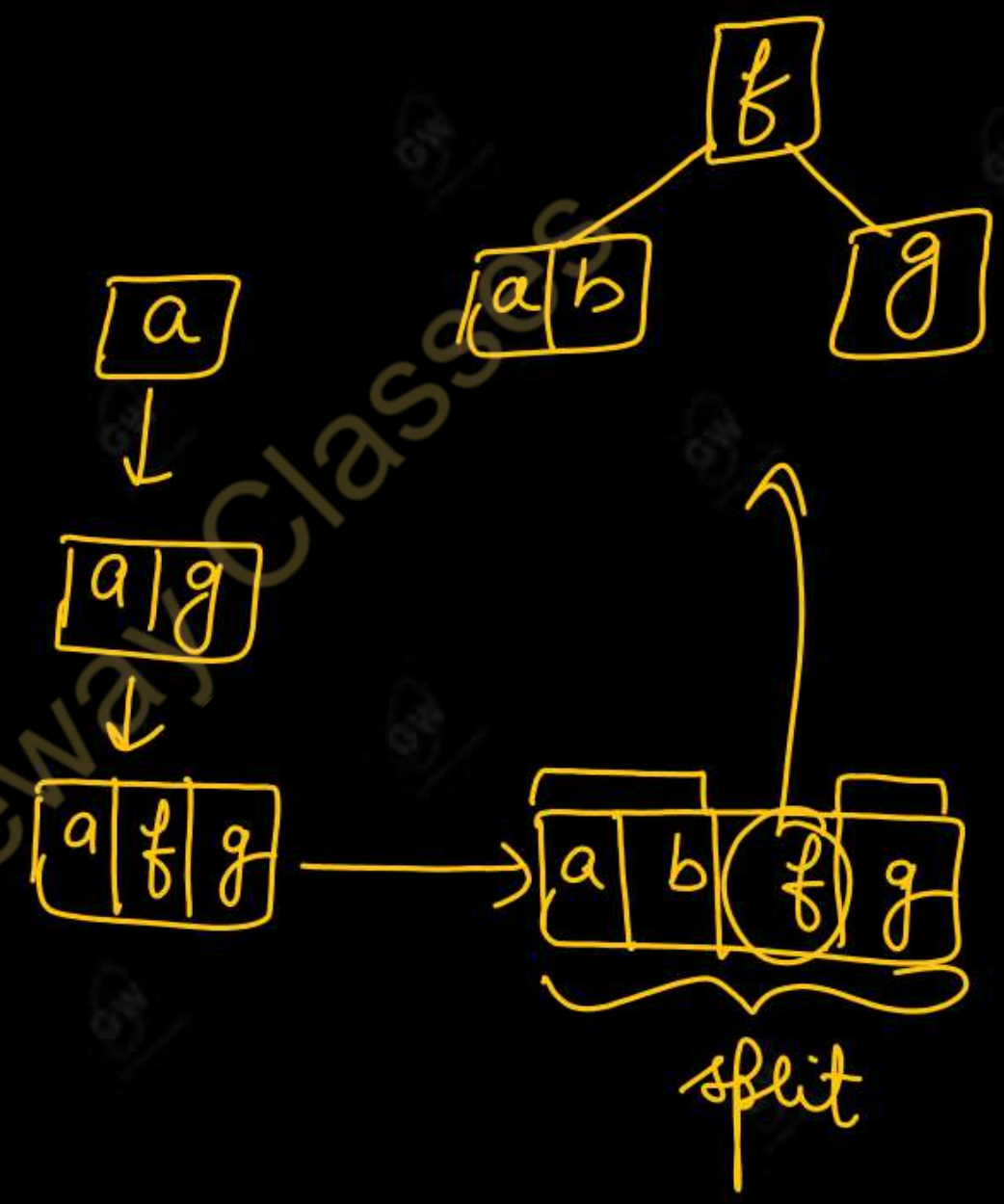
Insert B



Q: Construct a B-Tree of order 4 with the alphabets (letters) arrive in the sequence as follows: a g f b k d h m j e s i r x c l n t u p. (AKTU 2024-25, 2019-20)

$$m=4$$

$$\text{Max Keys} = 3$$



Why B-Tree search is better than Search in BST(Specialy for external storage)?

1. One B-tree node can contain many keys:

- Multiple keys per node → fewer levels
- Instead of going down one level per comparison as in a BST, you search inside a node (with binary search) and skip many keys at once.

2. Strict balancing:

- B-trees are perfectly balanced. All leaves are at same level, ensuring height remains logarithmic in number of keys.
- Optimized for disk I/O (where reads are expensive)
- In BST, each comparison may require loading another node from disk → multiple disk accesses. In B-trees, one node load brings in many keys. Searching inside a node is done in RAM, saving disk accesses.

1. Demonstrate B-Tree? Construct a B-Tree of order 4 with the alphabets (letters) arrive in the sequence as follows: a g f b k d h m j e s i r x c l n t u p. (AKTU 2024-25, 2019-20)
2. What is B-Tree? Write the various properties of B- Tree. Show the results of inserting the keys F, S, Q, K, C, L, H, T, V, W, M, R, N, P, A, B in order into a empty B-Tree of order 5. (AKTU 2022-23)
3. (i) Why does time complexity of search operation in B-Tree is better than BST?
 (ii) Insert keys into an initially empty B-tree of order 5 a, g, f, b, k, d, h, m, j, e, s, i, r, x, c, l, n, t, u, p
 (iii) What will be the resultant B-Tree after deleting keys j, t and d in sequence? (AKTU 2021-22)

4. Discuss the advantages of using B-Tree. Insert the following Information 86, 23, 91, 4, 67, 18, 32, 54, 46, 96, 45 into an empty B-Tree with order 4. (AKTU 2018-19)
5. Write short notes on B-Tree (AKTU 2017-18)



AKTU

B.Tech III-Sem



GATEWAY CLASSES

CS IT & CS Allied

DATA STRUCTURE

Unit-4 Lecture-6

Today's Target

- Some more examples of B-tree insertion
- B-tree deletion
- AKTU PYQs
- DPP



By Dr. Nidhi Parashar Ma'am

Ph.D (CSE)

- **M.Tech Gold Medalist**
- **Net Qualified**

Q: Construct a B-Tree of order 4 with the alphabets (letters) arrive in the sequence as follows: a g f b k d h m j e s i r x c l n t u p. (AKTU 2024-25, 2019-20)

Solⁿ: $M=4$, $\text{Max Keys/node} = 4-1=3$, $\text{Max children/node} = 4$

Insert a - [a]

insert g - [a | g]

insert f - [a | f | g]

insert b - [a | b | f | g] {split}

↓

[f]

↙ ↘

[a | b] [g]

insert k →

[f]

↙ ↘

[a | b] [g | k]

insert d -

[f]

↙ ↘

[a | b | d] [g | k]

insert h -

[f]

↙ ↘

[a | b | d] [g | h | k]

Insert m -

[f]

↙ ↘

[a | b | d] [g | h | k | m]

↑

split

[f | k]

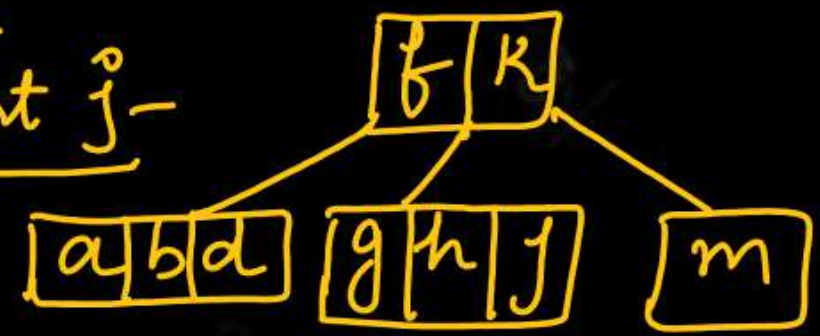
↙ ↘ ↘

[a | b | d] [g | h] [m]

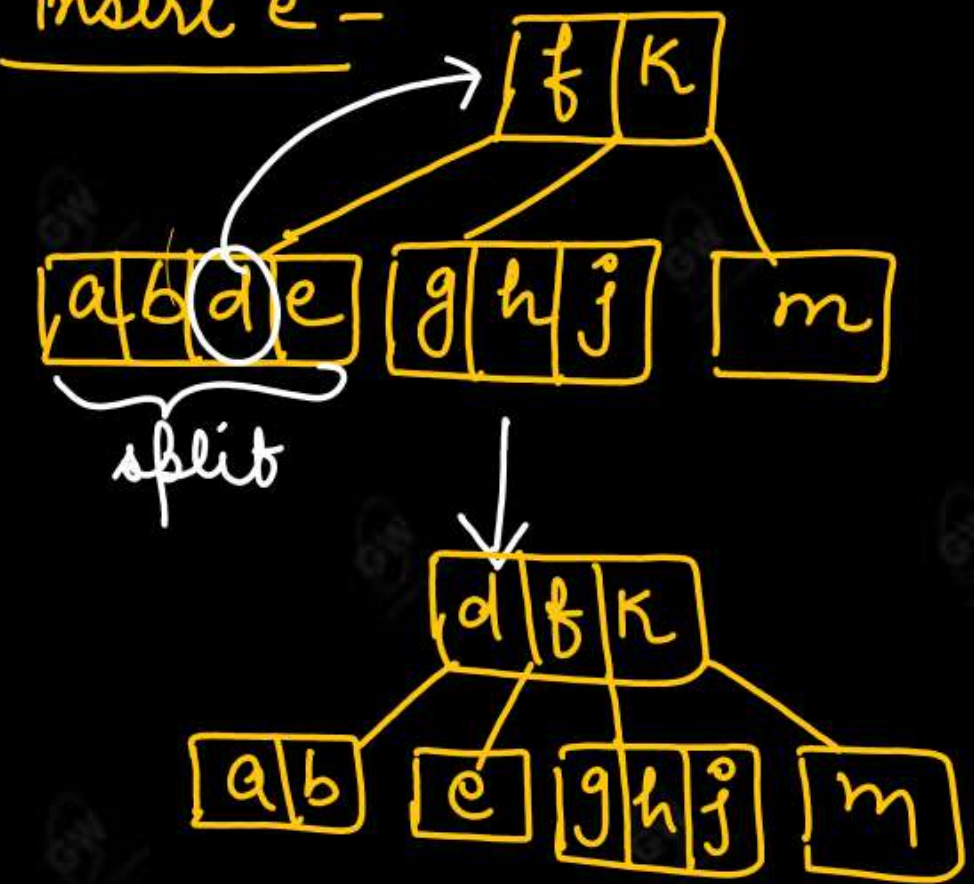
Q: Construct a B-Tree of order 4 with the alphabets (letters) arrive in the sequence as follows: a g f b k d h m j e s i r x c l n t u p. (AKTU 2024-25, 2019-20)

Solⁿ

Insert j -



Insert e -



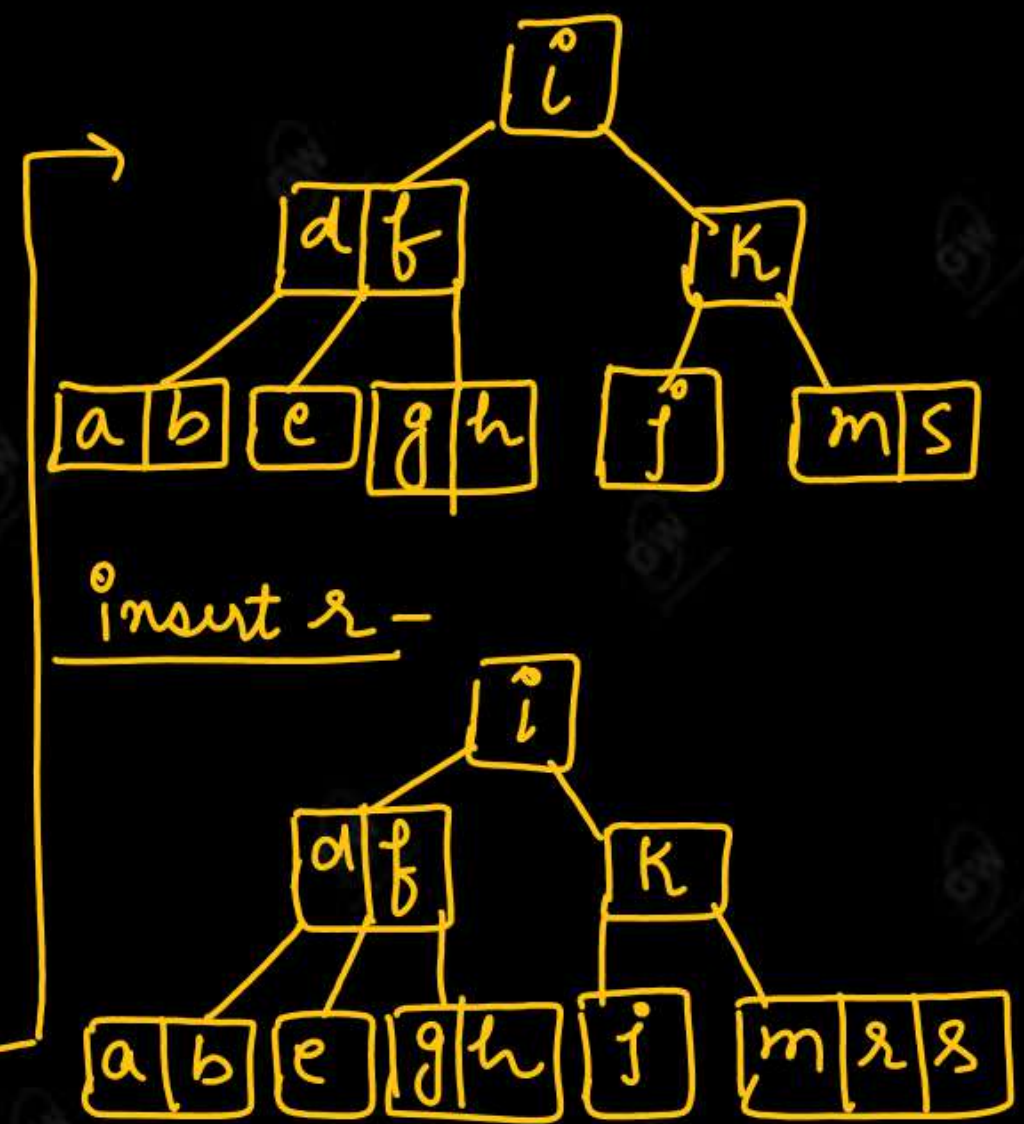
Insert s -



Insert i -

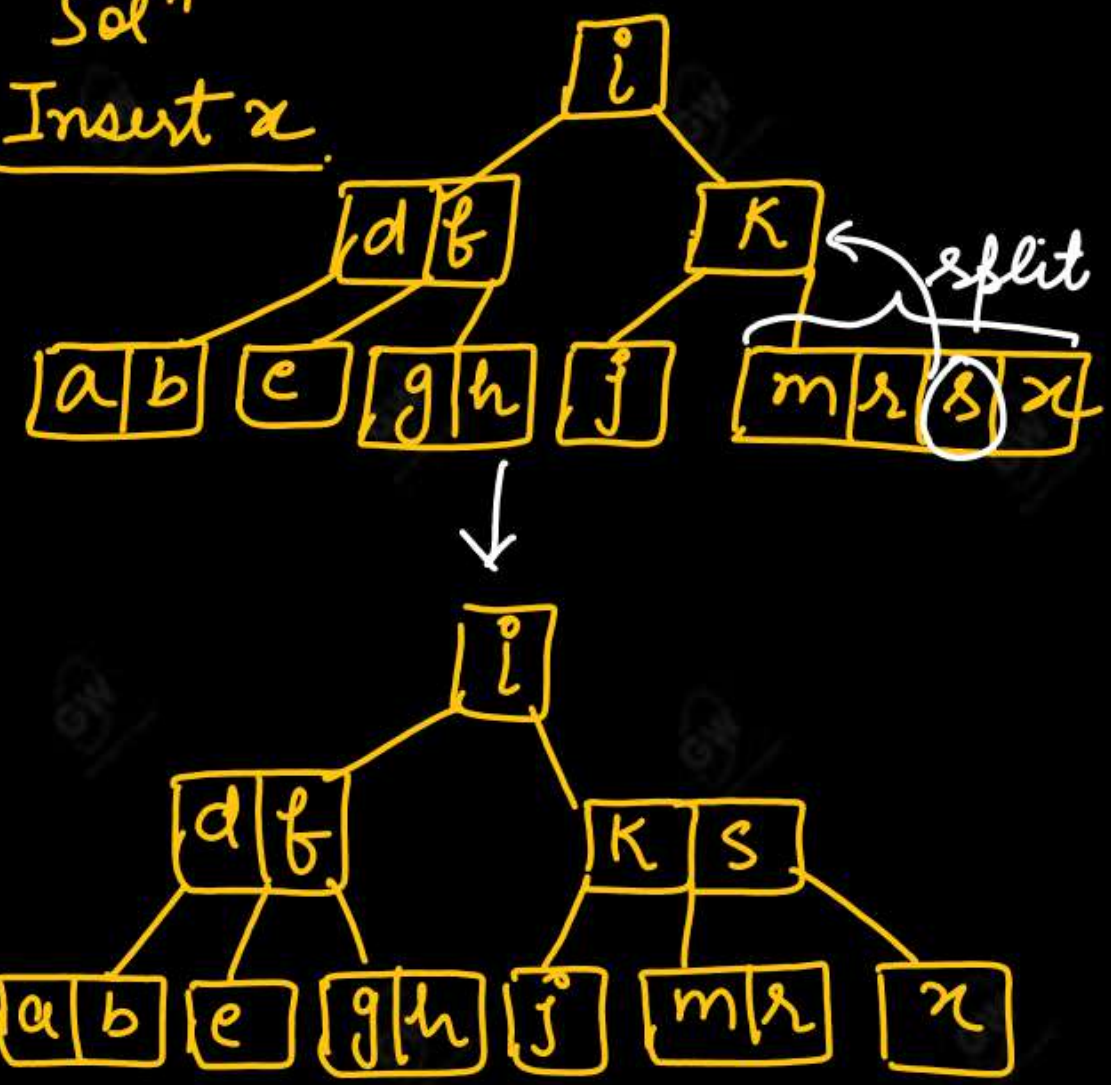


Insert r -

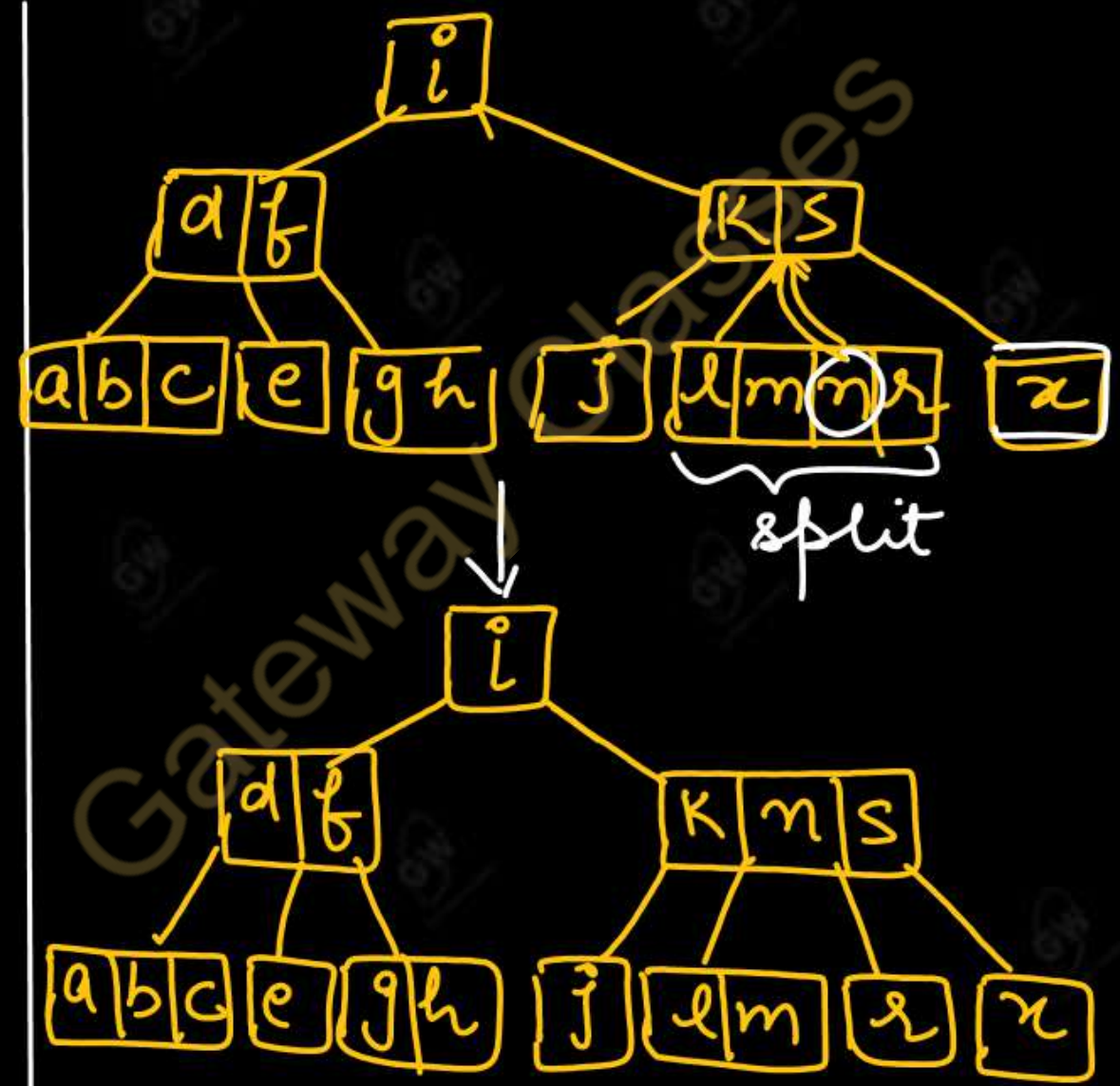


Q: Construct a B-Tree of order 4 with the alphabets (letters) arrive in the sequence as follows: a g f b k d h m j e s i r x c l n t u p. (AKTU 2024-25, 2019-20)

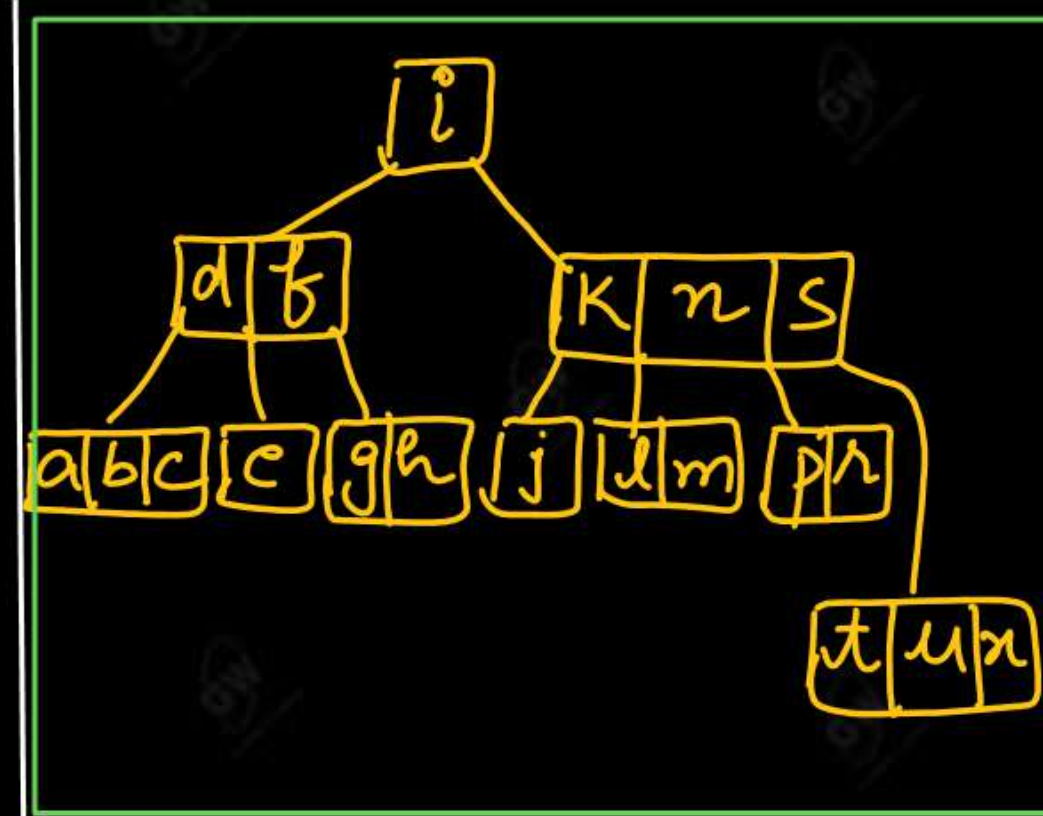
Solⁿ
Insert x



Insert c, l, n



Insert t, u, p



✓

Q: Insert the following Information 86, 23, 91, 4, 67, 18, 32, 54, 46, 96, 45 into an empty B-

Tree with order 4. (AKTU 2018-19)

Insert 86 -

86

Insert 23 -

23	86
----	----

Insert 91 -

23	86	91
----	----	----

Insert 4 -

4	23	86	91
---	----	----	----

 } split

↓

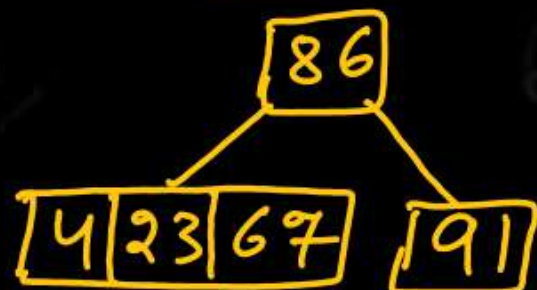
86

↙ ↘

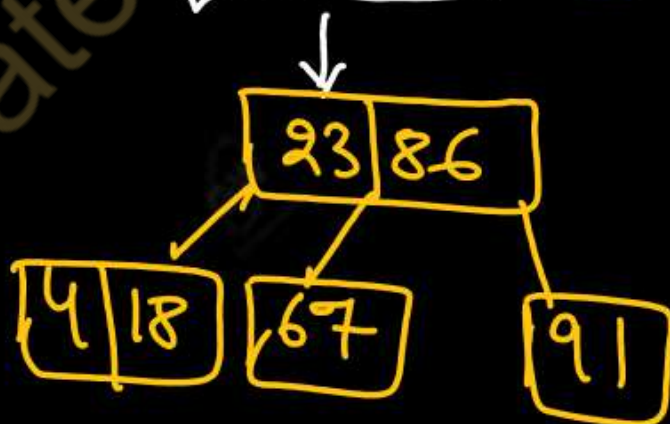
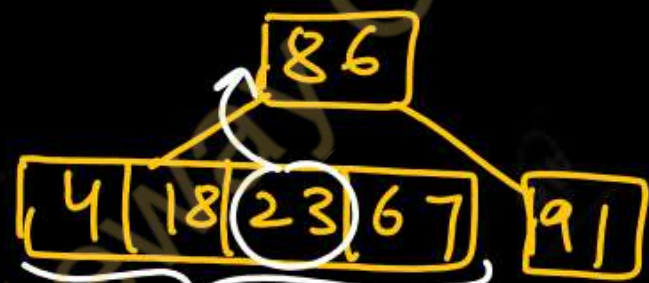
4	23
---	----

91

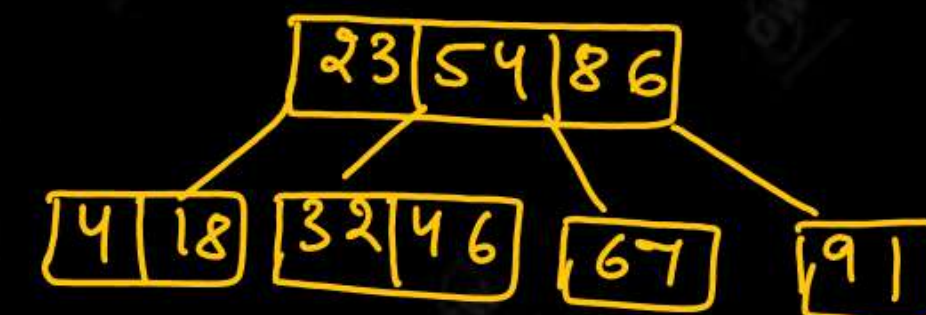
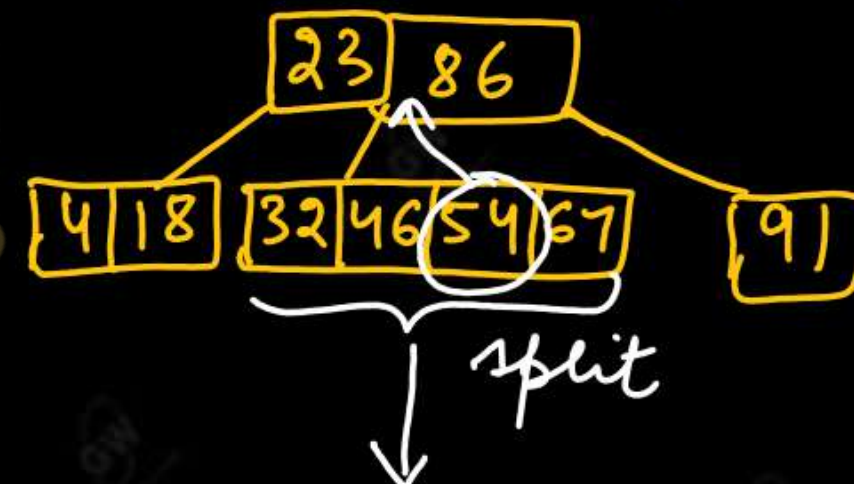
Insert 67 -



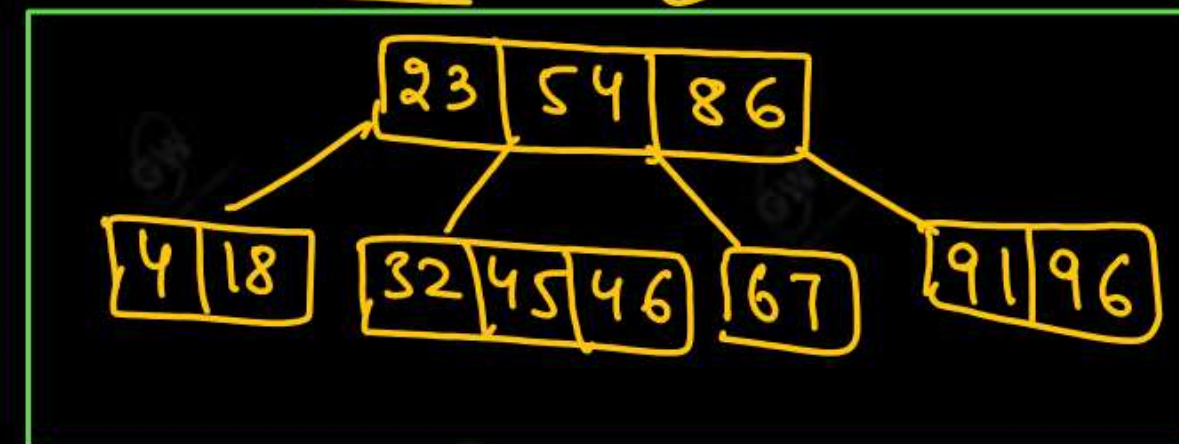
Insert 18 -



Insert 32, 54, 46 -



Insert 96, 45 -



Case 1: x is a leaf node and contains the target key to be deleted.

- 1.1. Leaf node contain more than min no. key then simply delete that key from node.
- 1.2. Leaf node contain min number of keys then 3 subcases:
 - 1.2.1. Borrow from left sibling iff that sibling contain more than min number of keys.
Move max key of left sibling to parent node and parent key to target node and then delete target key.
 - 1.2.2. Borrow from right sibling iff that sibling contain more than min number of keys.
Move minimum key of right sibling to parent node and parent key to target node and then delete target key.
 - 1.2.3. Neither left nor right sibling contain more than min keys then merge both the siblings with parent and then delete target key.

Case 2: x is an internal node and contains the target key. There are 3 sub-cases:

- 2.1. predecessor child node has more than min no. of keys then replace the key with its in-order predecessor
- 2.2. successor child node has more than min no. of keys then replace the key with its in-order successor
- 2.3. Neither predecessor nor successor child has more than min no. of keys then merge the both the predecessor and successor node with parent and then delete the key. After merging if the parent node has less than the minimum number of keys then, look for the siblings as in Case 1.

Q- Delete 64, 23, 72, 65, 20, 70, 95, 77, 80, 100, 6, 27, 60, 15, and 50 from the following B-Tree. $M=5$ $\text{min keys} - \lceil \frac{m}{2} \rceil - 1 \Rightarrow \lceil \frac{5}{2} \rceil - 1 \Rightarrow 2$



delete 64 →

directly delete it as it is a leaf node and has more than min no. of keys.

delete 23 -

node has already min no. of keys, so it will take help from its sibling (left / right)

Q- Delete 64, 23, 72, 65, 20, 70, 95, 77, 80, 100, 6, 27, 60, 15, and 50 from the following B-Tree. $M=5$ min keys - $\lceil \frac{m}{2} \rceil - 1 \Rightarrow \lceil \frac{5}{2} \rceil - 1 \Rightarrow 2$



Delete 72 - node has min no. of keys
so right sibling will help

Q- Delete 64, 23, 72, 65, 20, 70, 95, 77, 80, 100, 6, 27, 60, 15, and 50 from the following B-Tree. $M=5$ $\text{min keys} - \lceil \frac{m}{2} \rceil - 1 \Rightarrow \lceil \frac{5}{2} \rceil - 1 \Rightarrow 2$



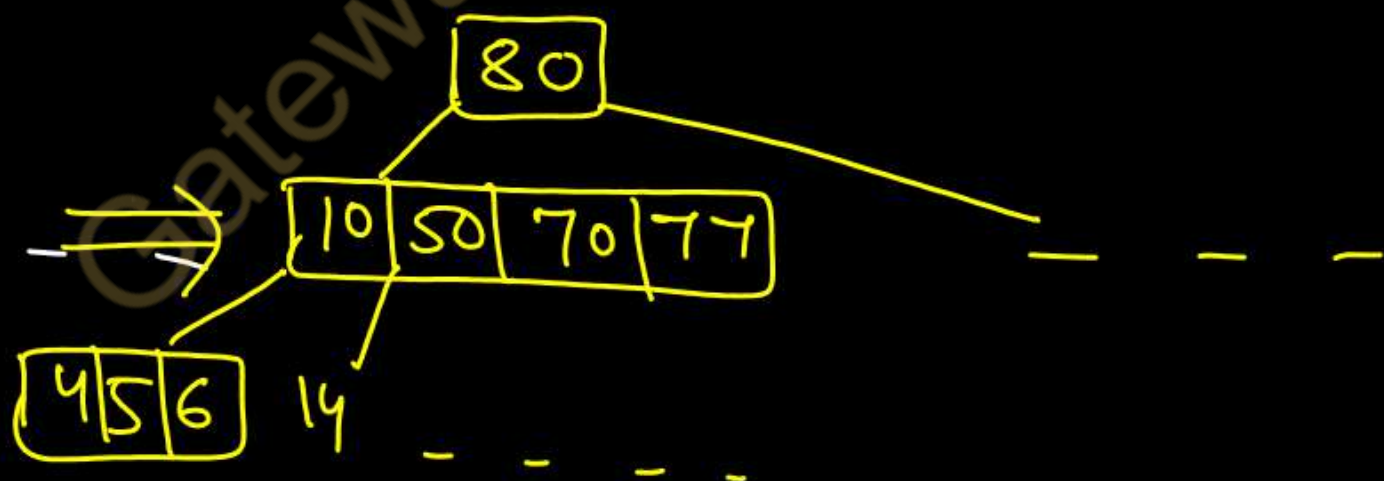
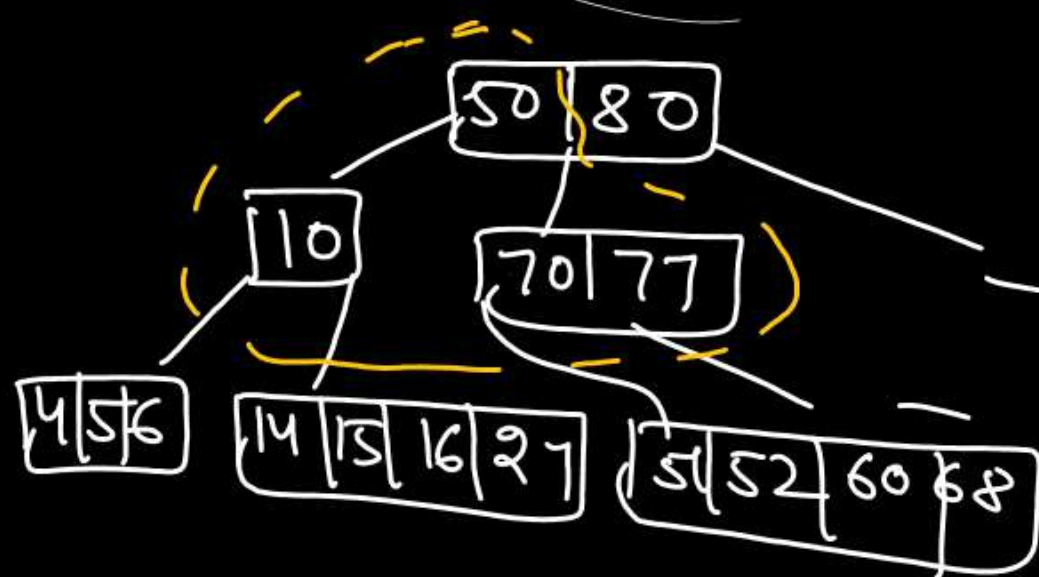
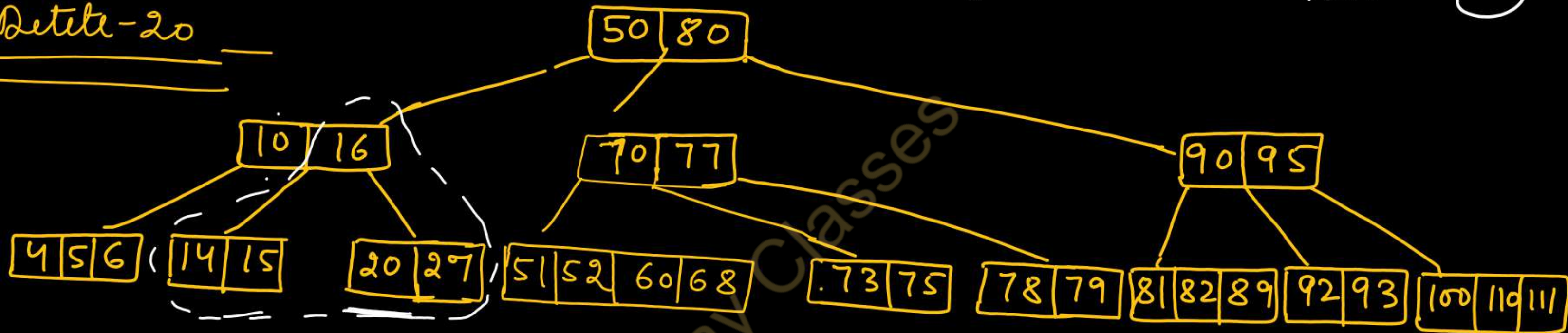
Delete 65 -

Neither left / right sibling has extra keys.

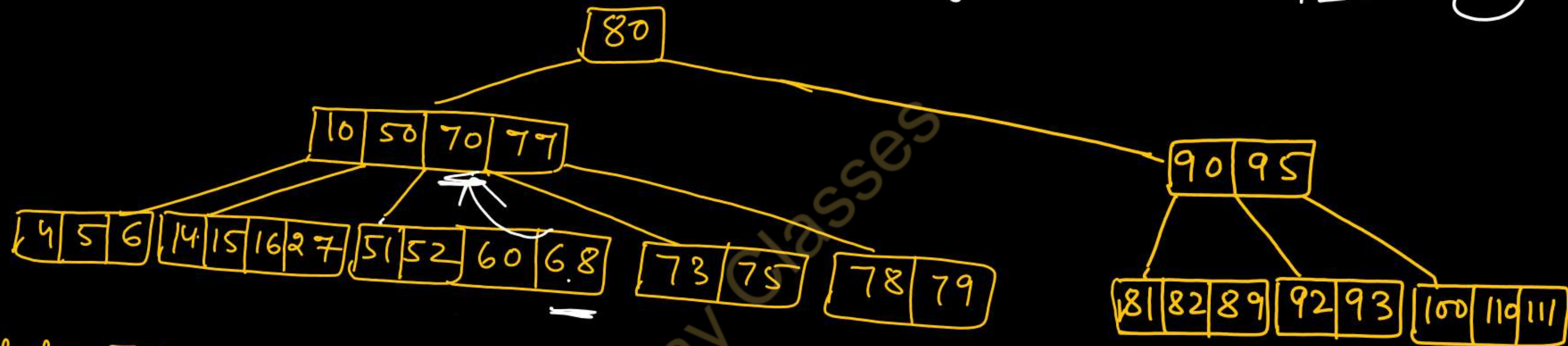
So parent will move downward and merge with both children to make a single node.

Q- Delete 64, 23, 72, 65, 20, 70, 95, 77, 80, 100, 6, 27, 60, 15, and 50 from the following B-Tree. $M=5$ Min Keys - $\lceil \frac{m}{2} \rceil - 1 \Rightarrow \lceil \frac{5}{2} \rceil - 1 \Rightarrow 2$

Delete-20



Q- Delete 64, 23, 72, 65, 20, 70, 95, 77, 80, 100, 6, 27, 60, 15, and 50 from the following B-Tree. $M=5$ min keys - $\lceil \frac{m}{2} \rceil - 1 \Rightarrow \lceil \frac{5}{2} \rceil - 1 \Rightarrow 2$



Delete 70 -

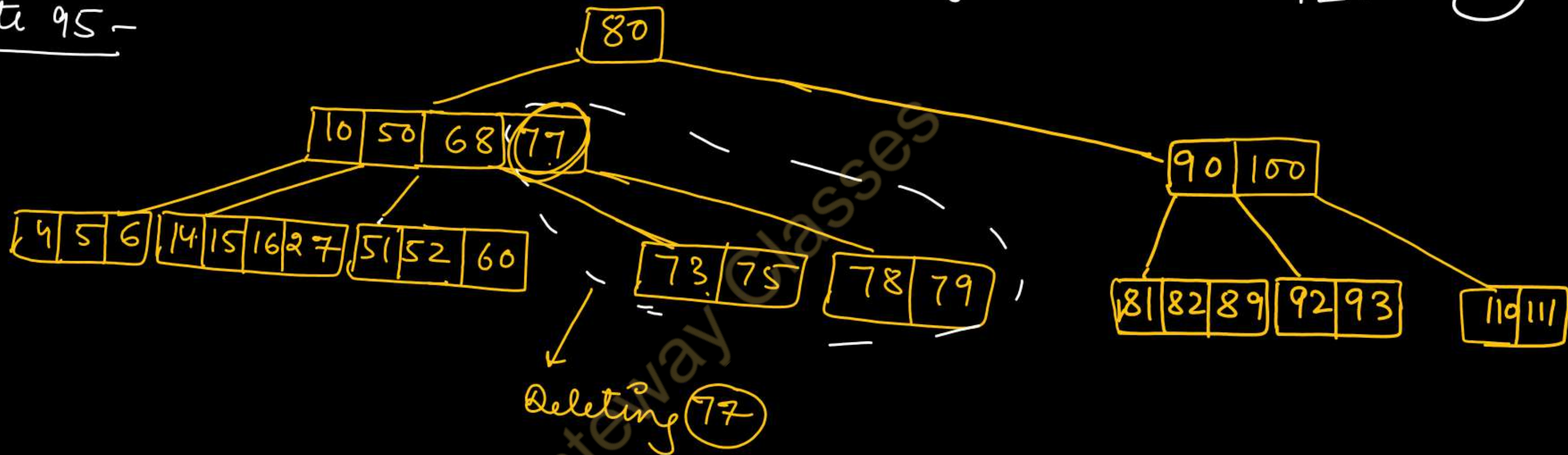
The key is in internal node

Now check if any child node can help.

Here left child has extra keys so in-order pred. (68) will take 70's place.

Q- Delete 64, 23, 72, 65, 20, 70, 95, 77, 80, 100, 6, 27, 60, 15, and 50 from the following B-Tree. $M=5$ Min Keys - $\lceil \frac{m}{2} \rceil - 1 \Rightarrow \lceil \frac{5}{2} \rceil - 1 \Rightarrow 2$

Delete 95 -

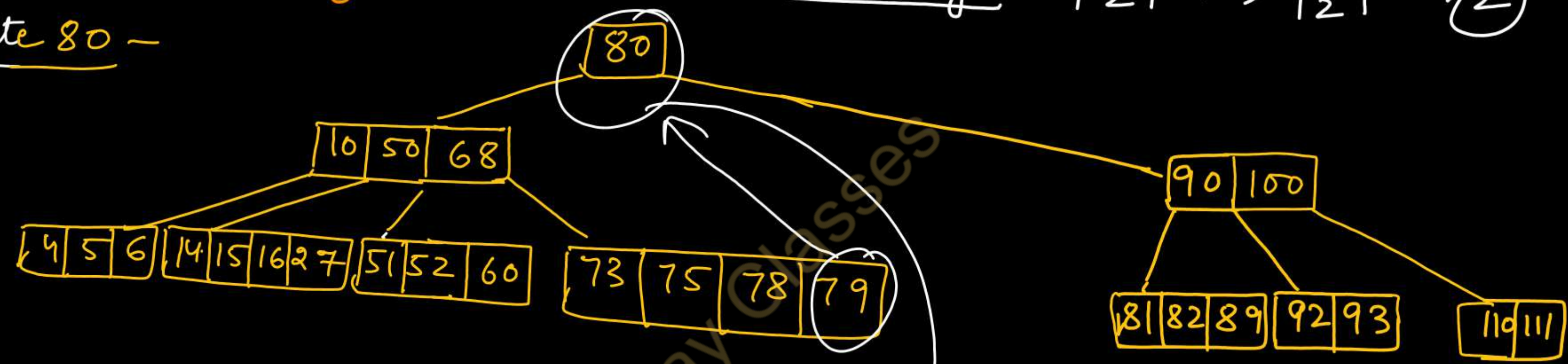


Q- Delete 64, 23, 72, 65, 20, 70, 95, 77, 80, 100, 6, 27, 60, 15, and 50 from the following B-Tree.

$$M=5$$

$$\text{min keys} - \left\lceil \frac{m}{2} \right\rceil - 1 \Rightarrow \left\lceil \frac{5}{2} \right\rceil - 1 \Rightarrow 2$$

Delete 80 -



To delete 80, either in-order predecessor / in-order successor will take its place

Q- Delete 64, 23, 72, 65, 20, 70, 95, 77, 80, 100, 6, 27, 60, 15, and 50 from the following B-Tree.

$$M=5$$

$$\text{Min Keys} - \left\lceil \frac{m}{2} \right\rceil - 1 \Rightarrow \left\lceil \frac{5}{2} \right\rceil - 1 \Rightarrow 2$$

Delete 100

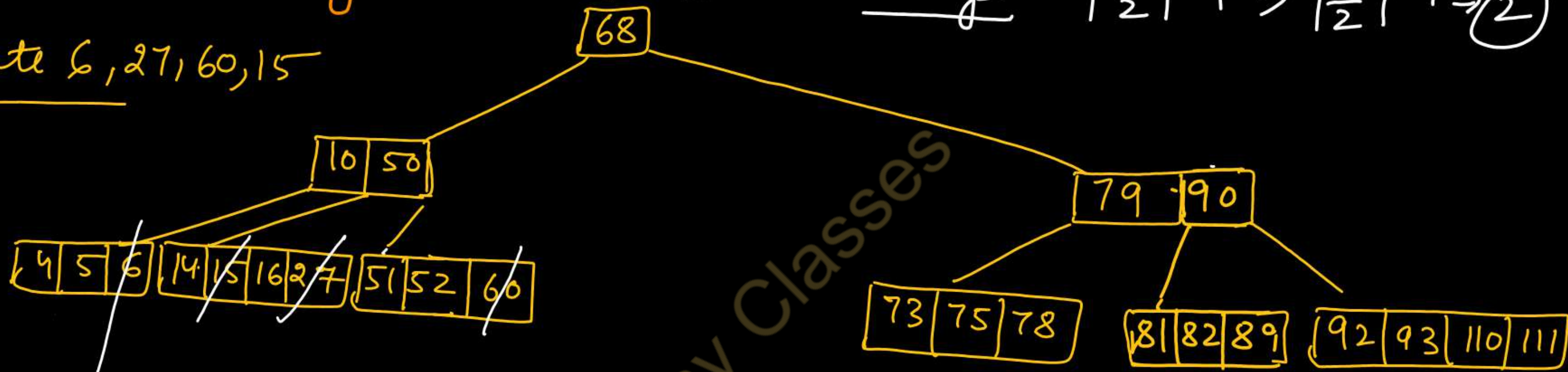


Q- Delete 64, 23, 72, 65, 20, 70, 95, 77, 80, 100, 6, 27, 60, 15, and 50 from the following B-Tree.

$$M=5$$

$$\text{Min Keys} - \left\lceil \frac{m}{2} \right\rceil - 1 \Rightarrow \left\lceil \frac{5}{2} \right\rceil - 1 \Rightarrow 2$$

Delete 6, 27, 60, 15

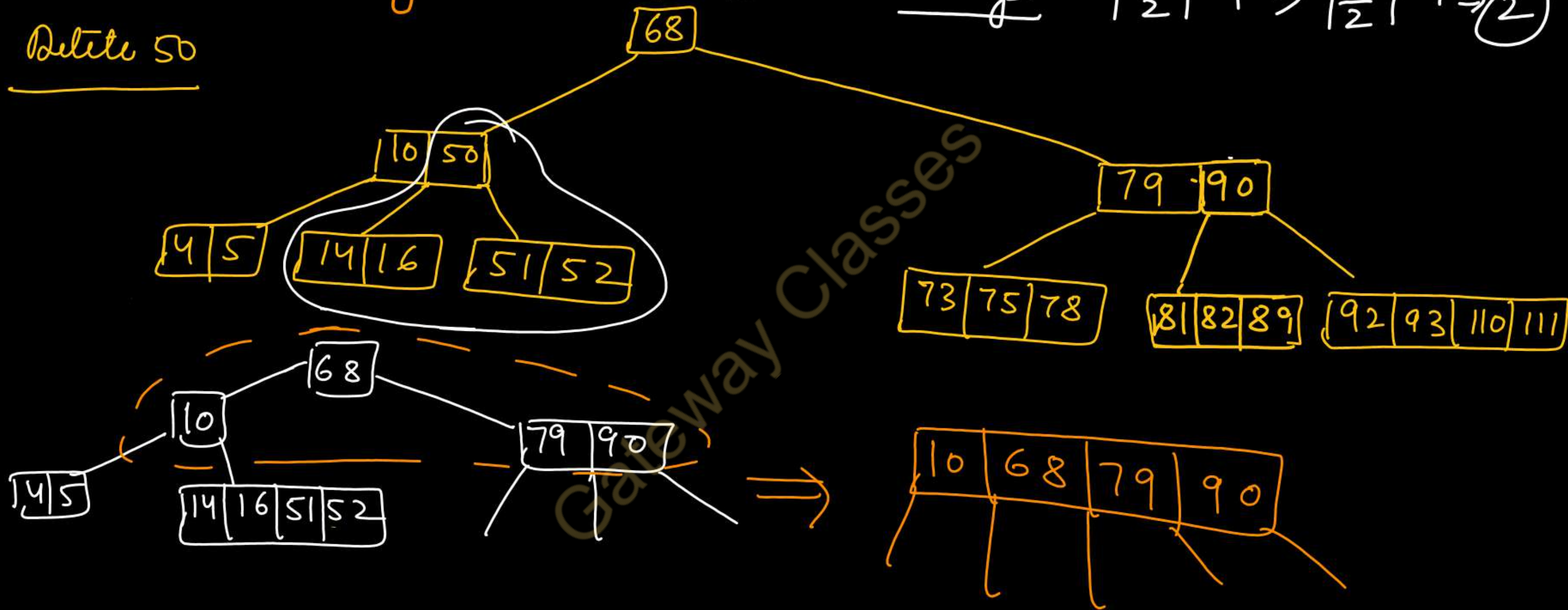


Q- Delete 64, 23, 72, 65, 20, 70, 95, 77, 80, 100, 6, 27, 60, 15, and 50 from the following B-Tree.

$$M=5$$

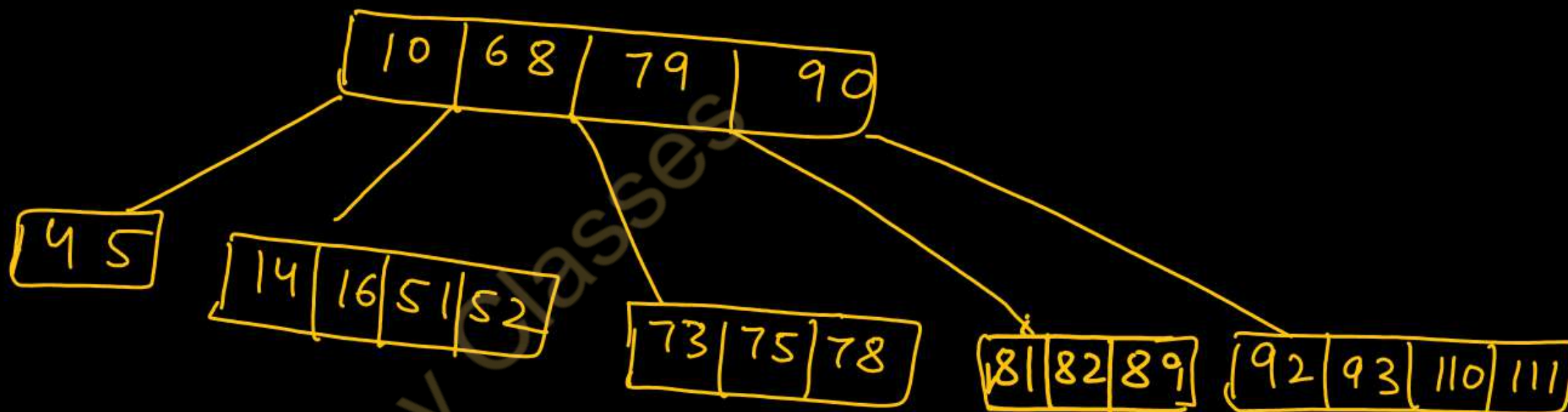
$$\text{min keys} - \left\lceil \frac{m}{2} \right\rceil - 1 \Rightarrow \left\lceil \frac{5}{2} \right\rceil - 1 \Rightarrow 2$$

Delete 50



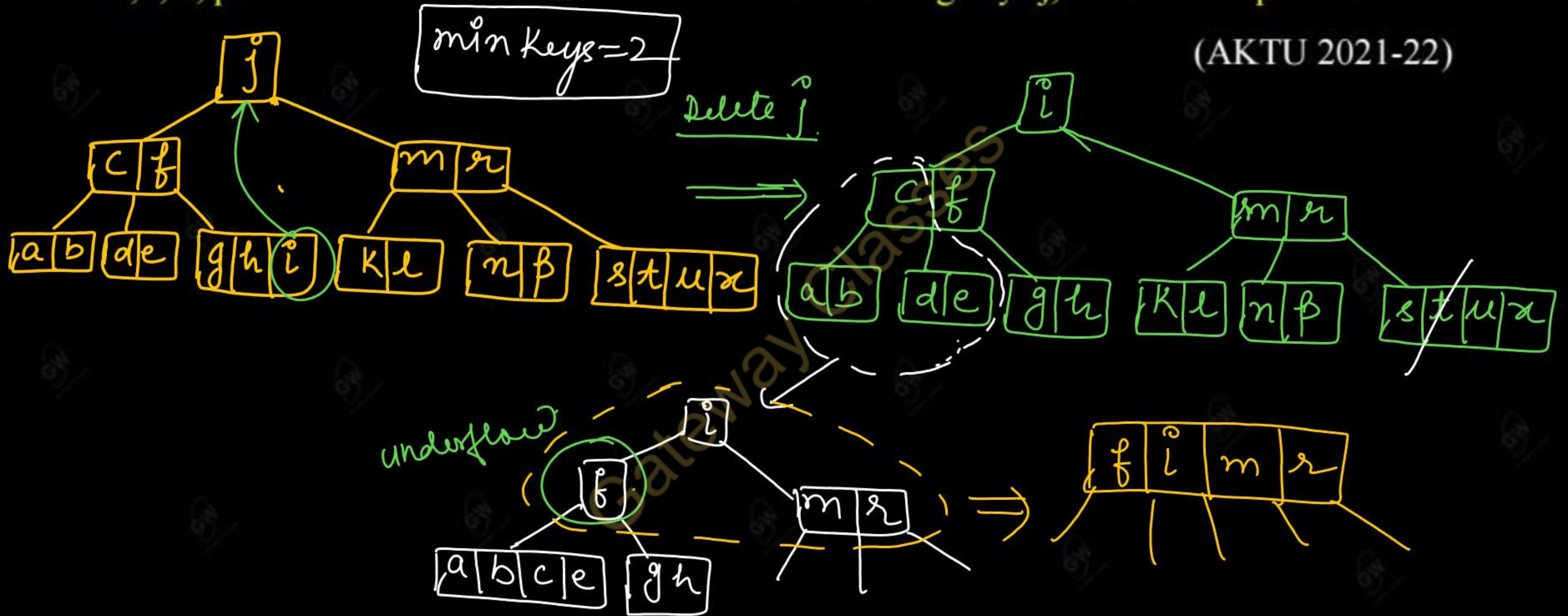
Q- Delete 64, 23, 72, 65, 20, 70, 95, 77, 80, 100, 6, 27, 60, 15, and 50 from the following B-Tree. $\boxed{M=5}$ min keys - $\lceil \frac{m}{2} \rceil - 1 \Rightarrow \lceil \frac{5}{2} \rceil - 1 \Rightarrow \textcircled{2}$

Delete 50



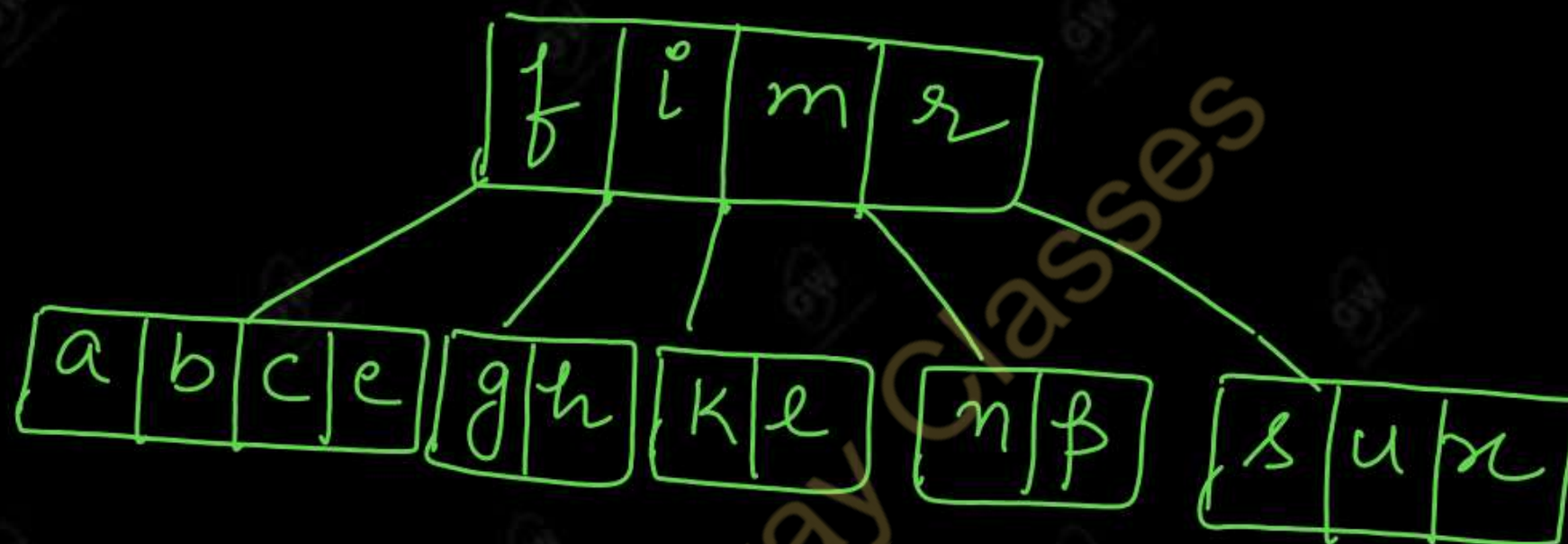
Q: Insert keys into an initially empty B-tree of order 5 a, g, f, b, k, d, h, m, j, e, s, i, r, x, c, l, n, t, u, p. What will be the resultant B-Tree after deleting keys j, t and d in sequence?

(AKTU 2021-22)



Q: Insert keys into an initially empty B-tree of order 5 a, g, f, b, k, d, h, m, j, e, s, i, r, x, c, l, n, t, u, p . What will be the resultant B-Tree after deleting keys j, t and d in sequence?

(AKTU 2021-22)





AKTU

B.Tech III-Sem



GATEWAY CLASSES

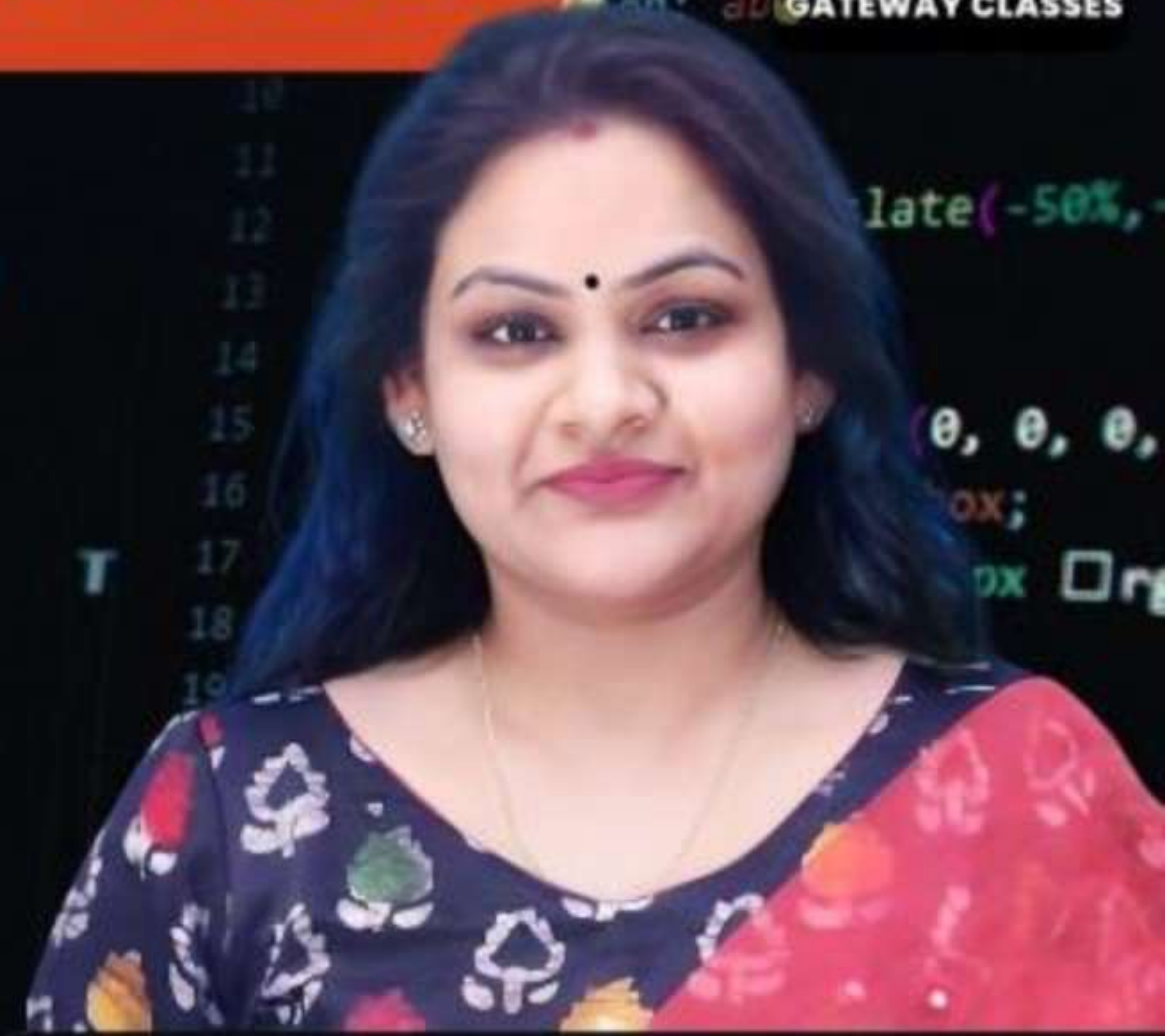
CS IT & CS Allied

DATA STRUCTURE

Unit-4 Lecture-7

Today's Target

- Threaded Binary Tree ✓
- Huffman coding ✓
- AKTU PYQs
- DPP



By Dr. Nidhi Parashar Ma'am

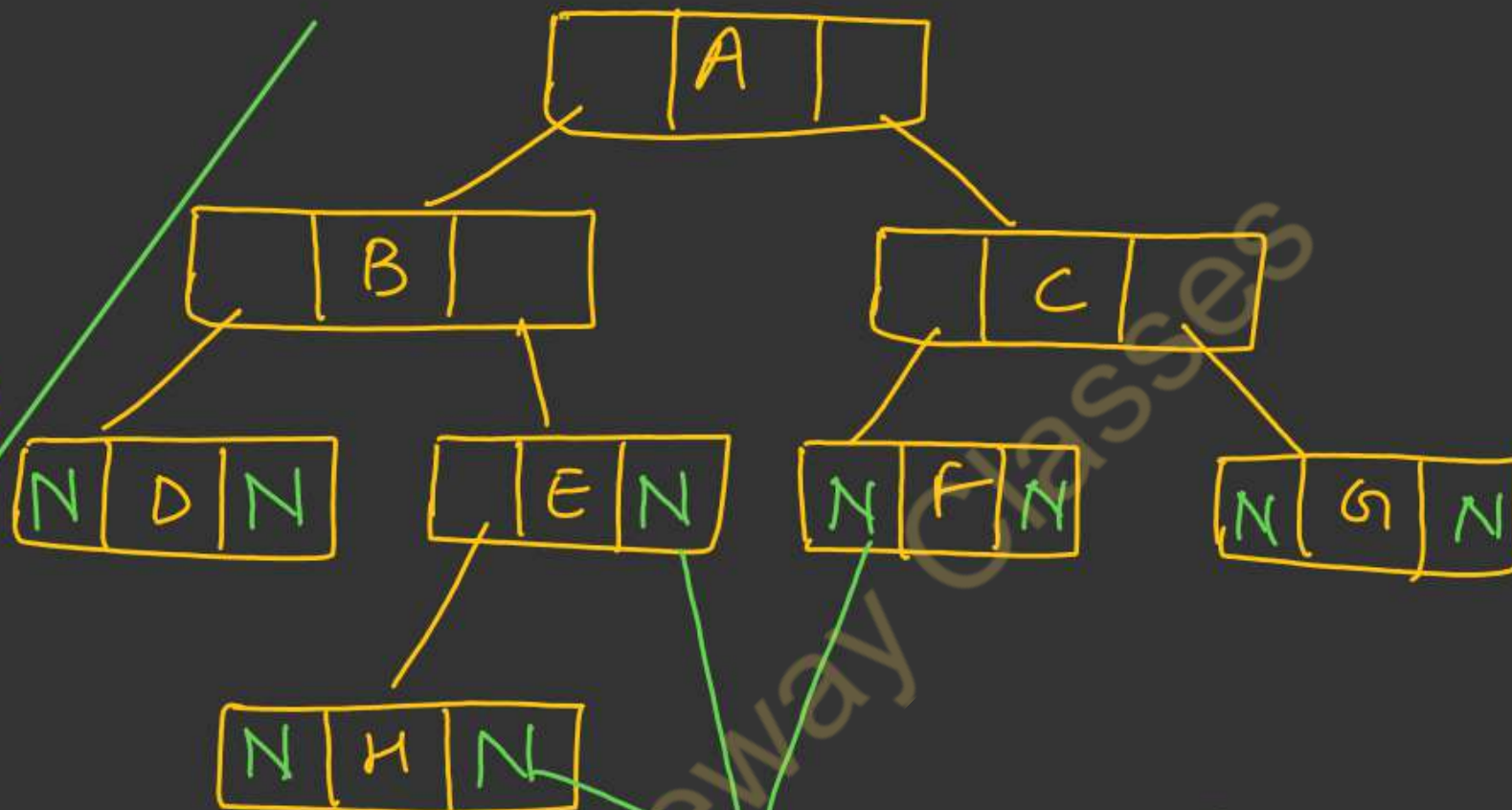
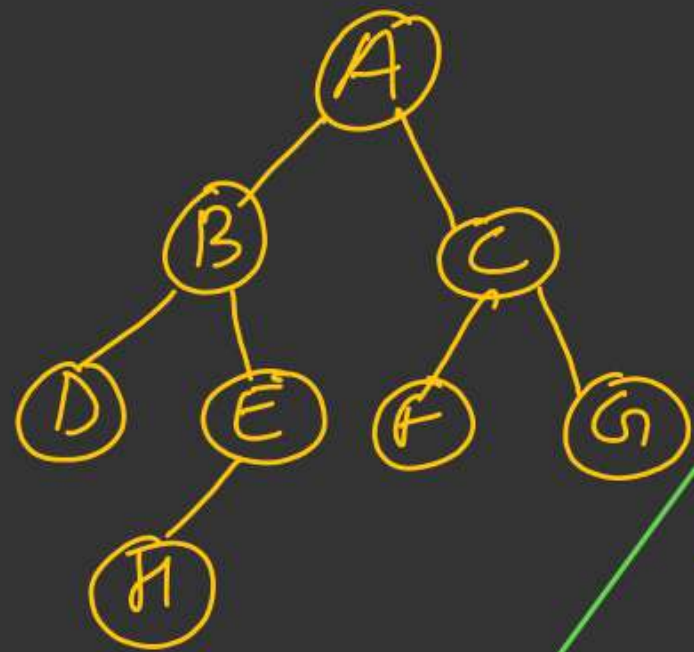
Ph.D (CSE)

- M.Tech Gold Medalist
- Net Qualified

Threaded Binary Tree

- Inorder traversal of a Binary tree can either be done using recursion or using a stack.
- The idea of threaded binary trees is to make inorder traversal faster without stack or recursion.
- A binary tree is made threaded by making all right child pointers that would normally be NULL point to the inorder successor of the node (if it exists).
- Each node of any binary tree stores the three fields.
 - The left field stores the left thread value and the right field stores the right thread value.
 - The middle field contains the actual value of the node i.e., data.

Why threaded Binary tree?

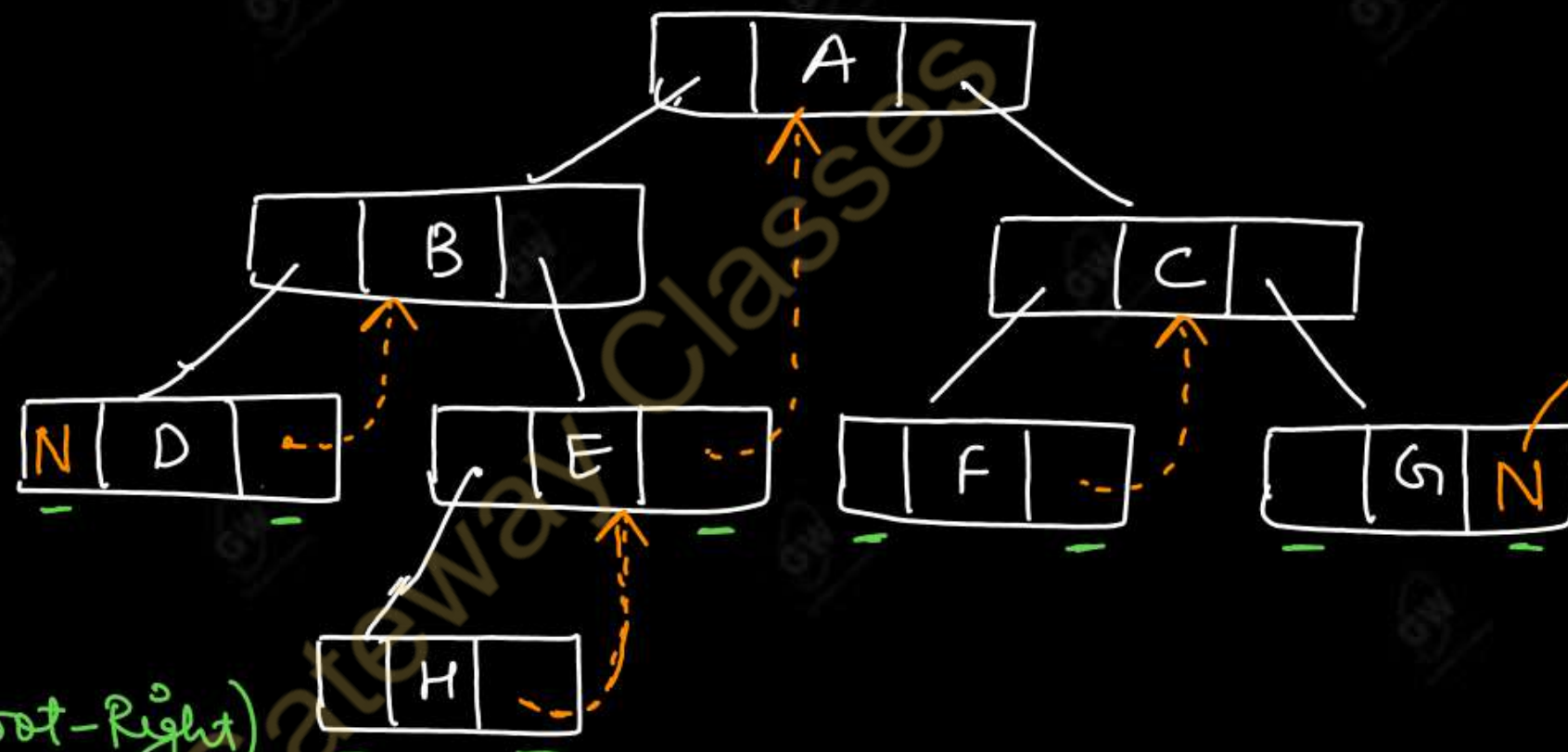
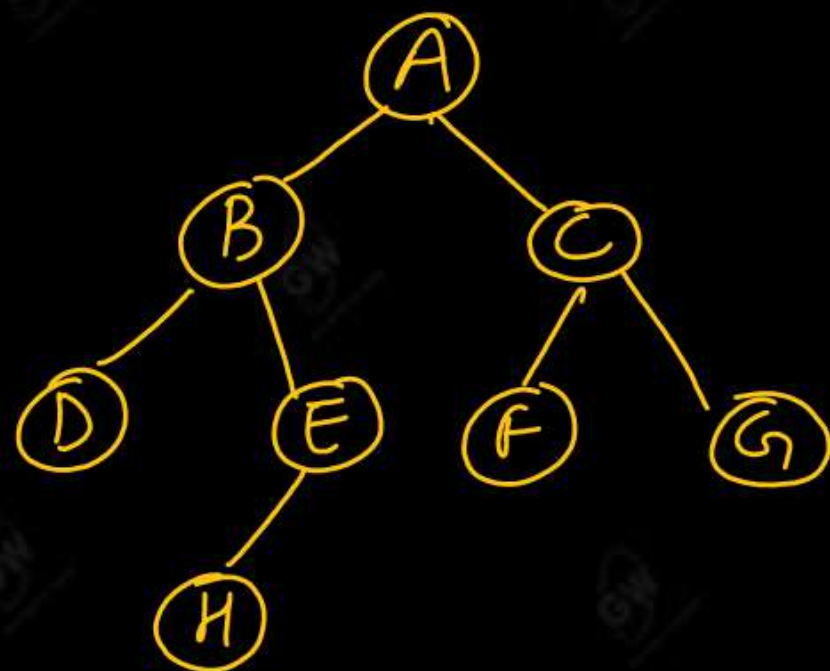


Unused pointers
which takes extra space

↓
It's better to utilize them for traversals

1. Single Threaded:

➤ Here a NULL right pointer is made to point to the inorder successor (if successor exists)



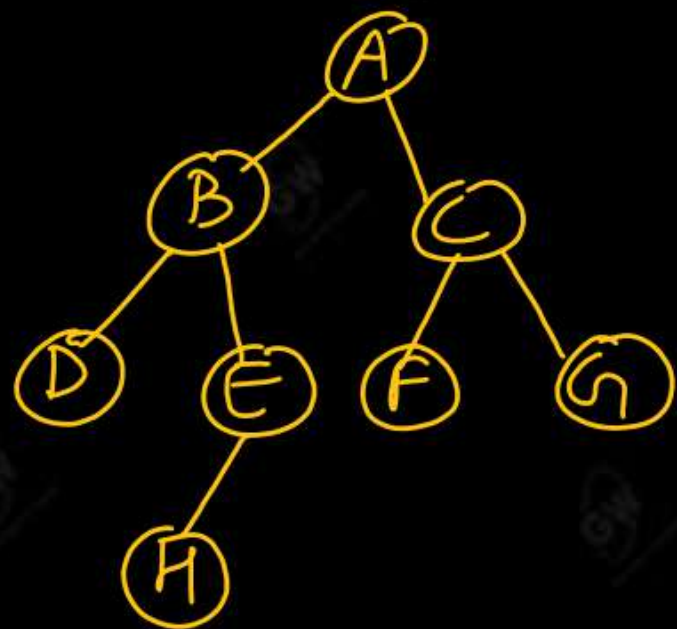
As G's successor does not exist

In-order Traversal (L-Root-Right)

DBHEAFCG

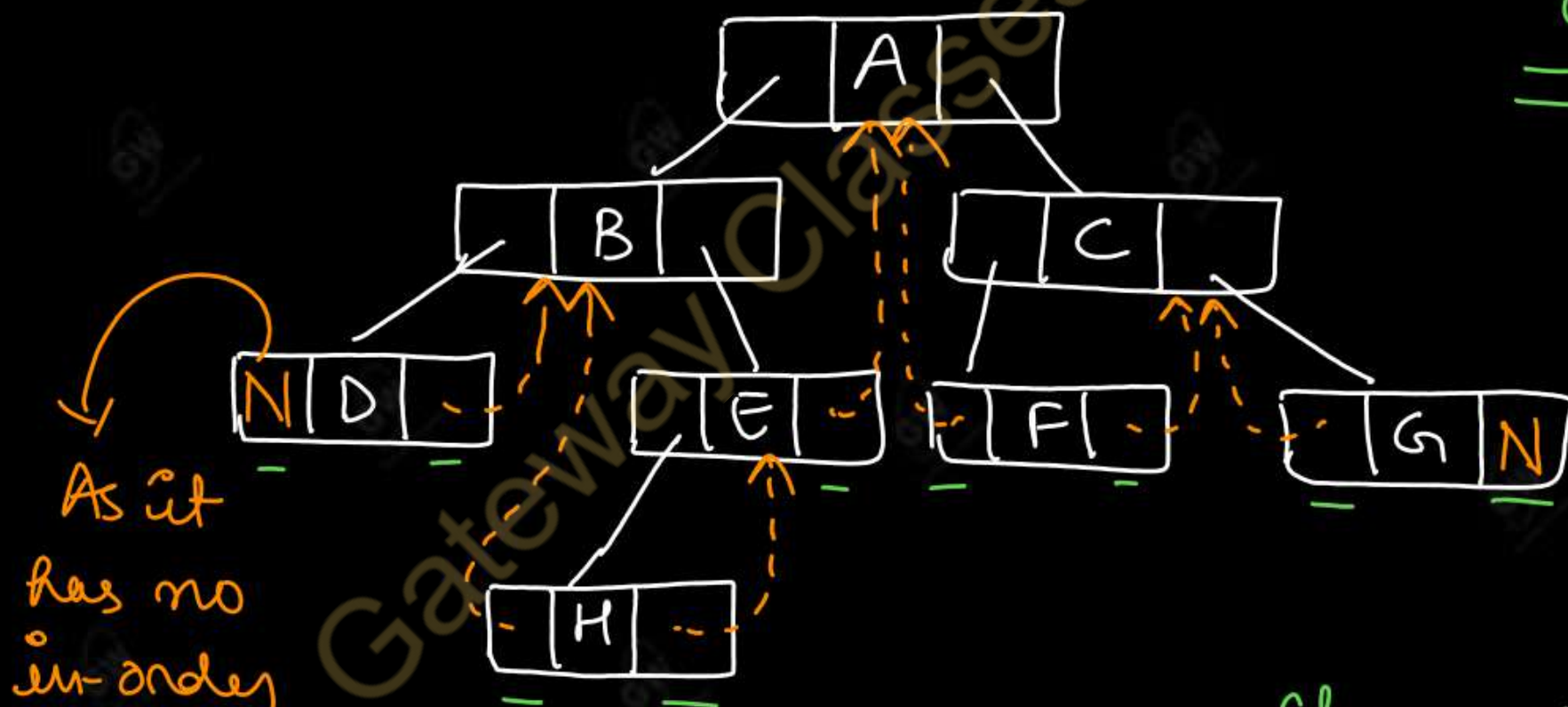
2. Double Threaded:

- Here both left and right NULL pointers are made to point to inorder predecessor and inorder successor respectively.
- The predecessor threads are useful for reverse inorder traversal and postorder traversal.



In-order Traversal

DBHEAFCG



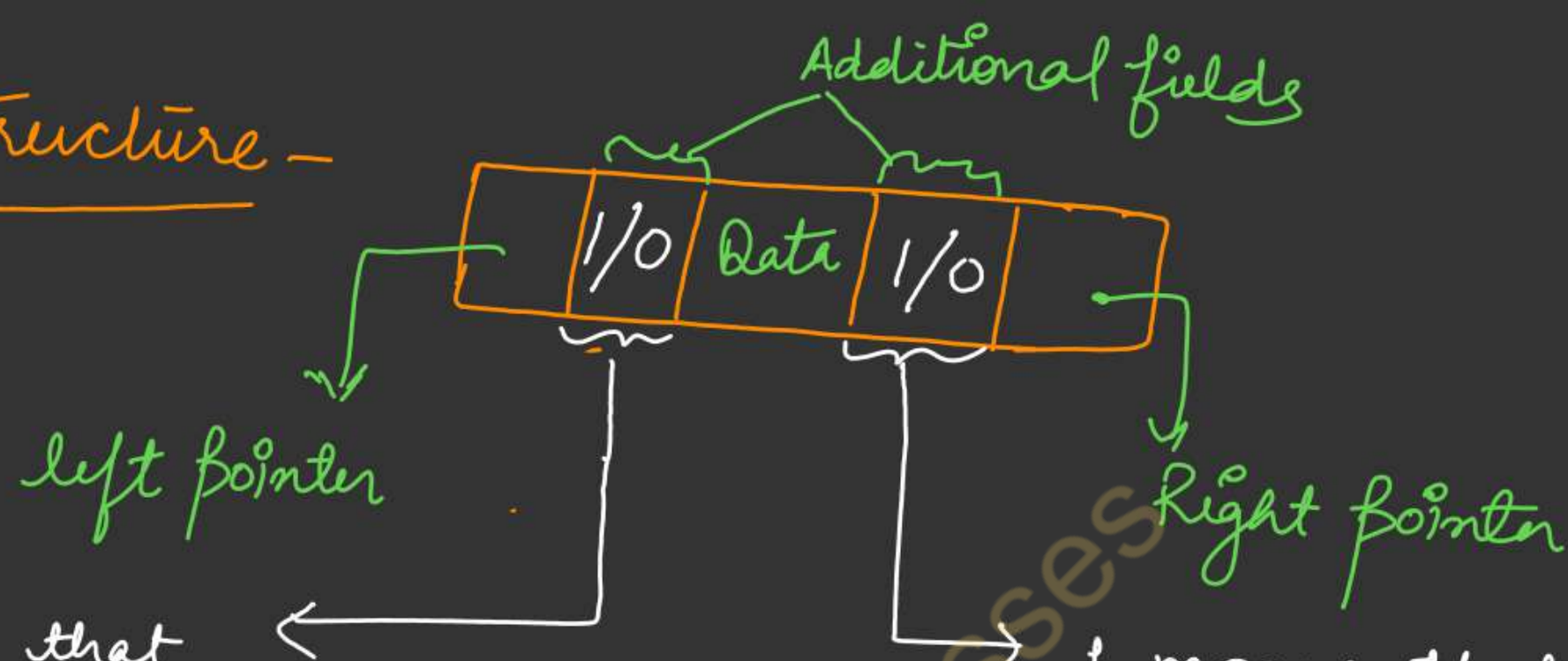
As it
has no
in-order
predecessor

Challenge

How to determine
that pointer fields
point to child nodes
or in-order pred
or in-order succ

Change the
node structure

Modified Node Structure -



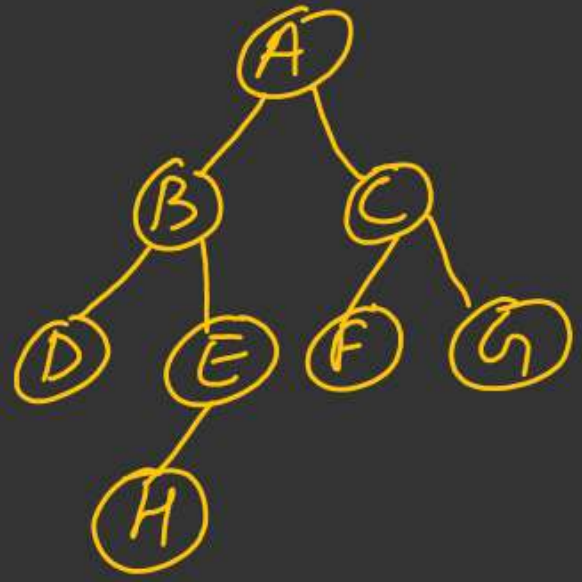
⇒ 1 means that left pointer holds address of the child node.

⇒ 0 means left pointer holds address of inorder predecessor node.

⇒ 1 means that right pointer holds address of the child node

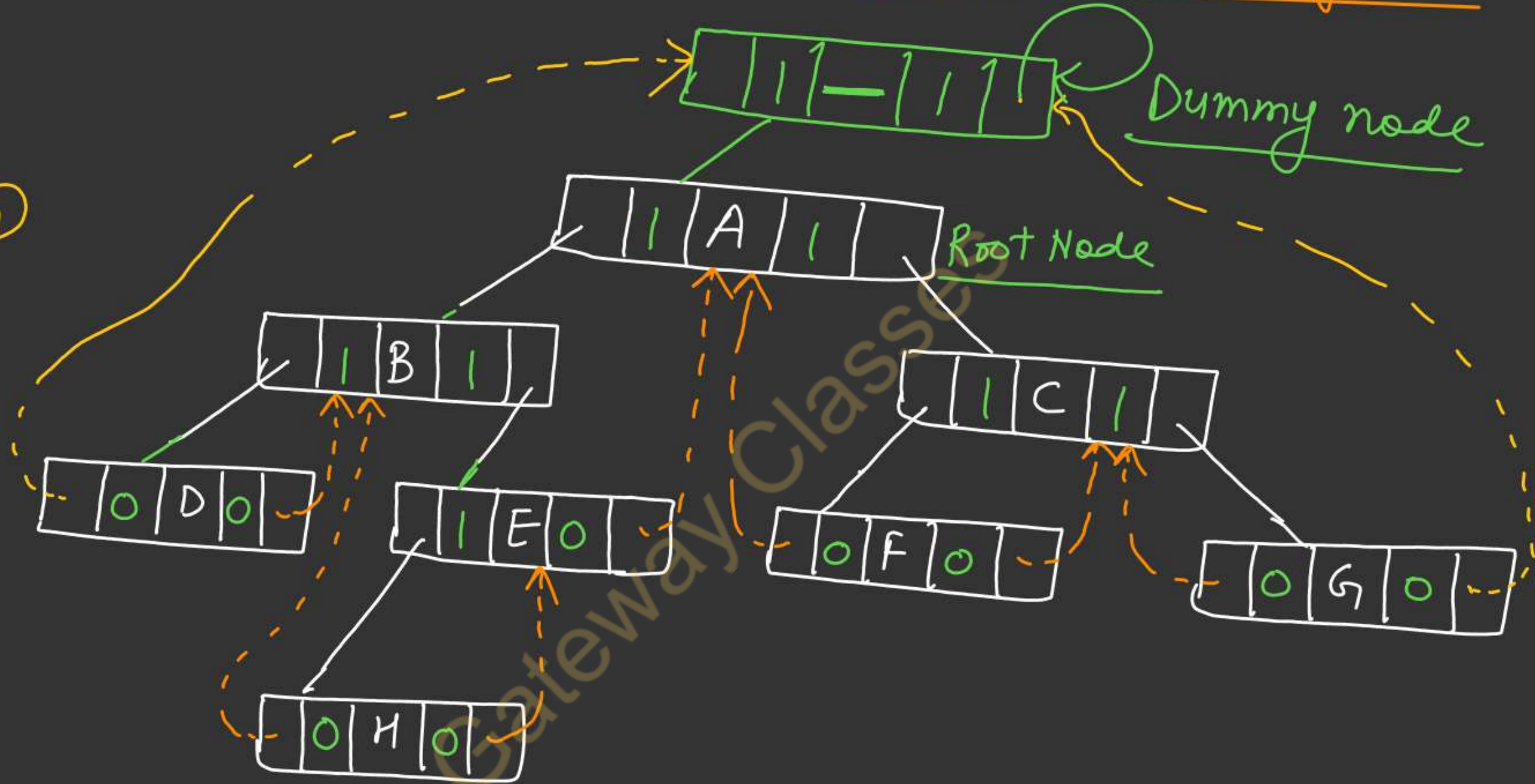
⇒ 0 means right pointer holds address of In-order successor node.

Optimized Double Threaded Binary Tree



In-order

DBHEAFCG



- Huffman Coding is a technique of compressing data to reduce its size without losing any of details.
- Generally useful to compress data in which there are frequently occurring characters.

How Huffman Coding works?

Suppose string to be sent over a network:

B	C	A	A	D	D	D	C	C	A	C	A	C	A	C
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

- Each character occupies 8 bits. There are a total of 15 characters in the above string. Thus, a total of $8 * 15 = \underline{120}$ bits are required to send this string.
- Using the Huffman Coding technique, we can compress the string to a smaller size.
- Huffman coding first creates a tree using the frequencies of the character and then generates code for each character.
- Once the data is encoded, it has to be decoded. Decoding is done using the same tree.

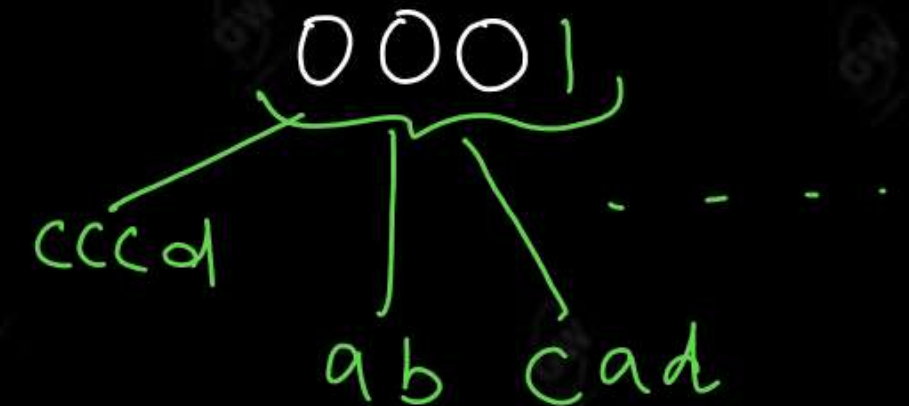
Huffman coding is a lossless data compression algorithm.

- It assigns variable-length codes to input characters, based on their frequencies.
- The codes are assigned in such a way that code assigned to one character is not prefix of code assigned to any other character **ensuring there is no ambiguity when decoding generated bitstream.**
- Let there be four characters a, b, c and d, and their corresponding variable length codes be 00, 01, 0 and 1. This coding leads to ambiguity because code assigned to c is the prefix of codes assigned to a and b. If the compressed bit stream is 0001, the de-compressed output may be "cccd" or "ccb" or "acd" or "ab".

There are mainly two major parts in Huffman Coding:

1. **Build a Huffman Tree from input characters.**
2. **Traverse the Huffman Tree and assign codes to characters.**

a	00
b	01
c	0
d	1



Compress the following message using huffman algorithm

ABBCDBCCDAABBE~~EE~~BEAB

Char	Freq	Code
A	4	
B	7	
C	3	
D	2	
E	4	

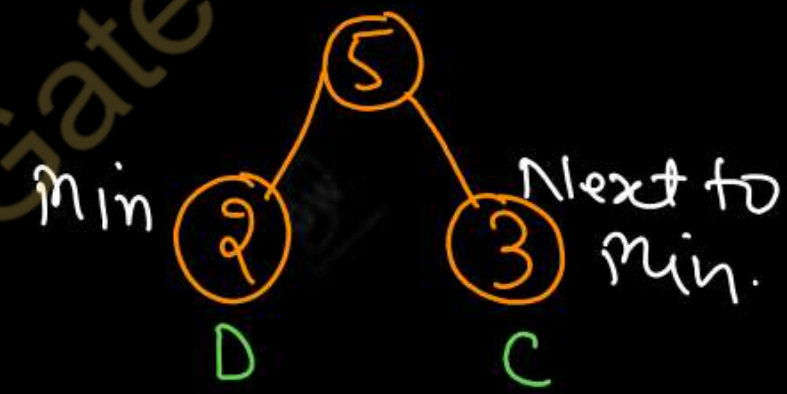
Step-1

Arrange all characters in ascending order according to their frequencies

D	C	A	E	B
2	3	4	4	7

Step-2

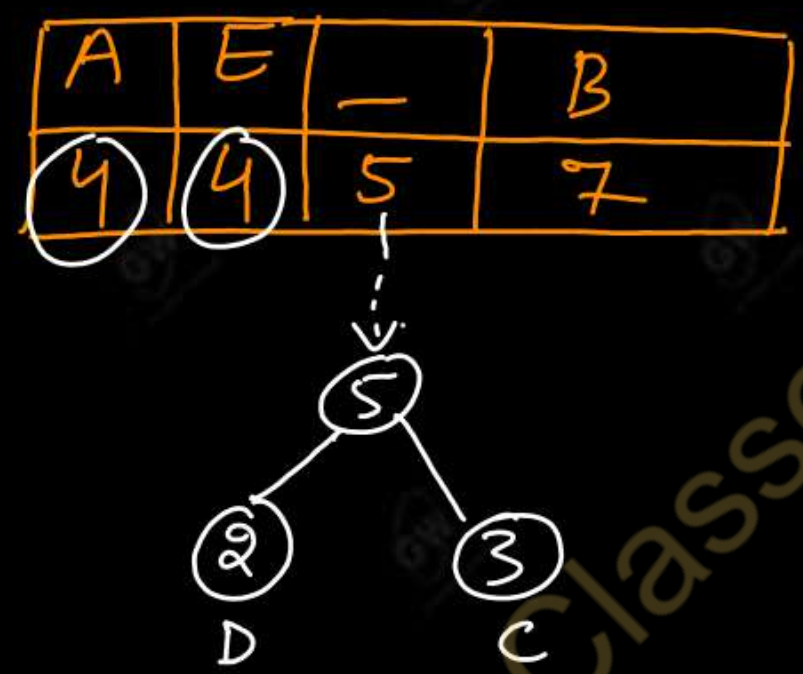
Pick two minimum frequent characters.



Compress the following message using huffman algorithm

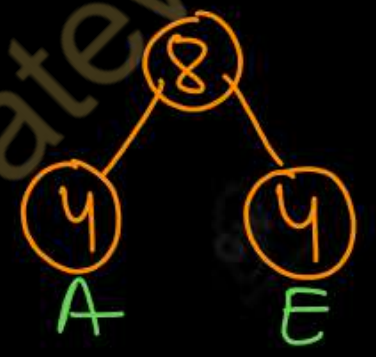
ABBCDBCCDAABBEEEBEAB

Char	Freq	Code
A	4	
B	7	
C	3	
D	2	
E	4	



Now again pick Two mins →

A	E
4	4



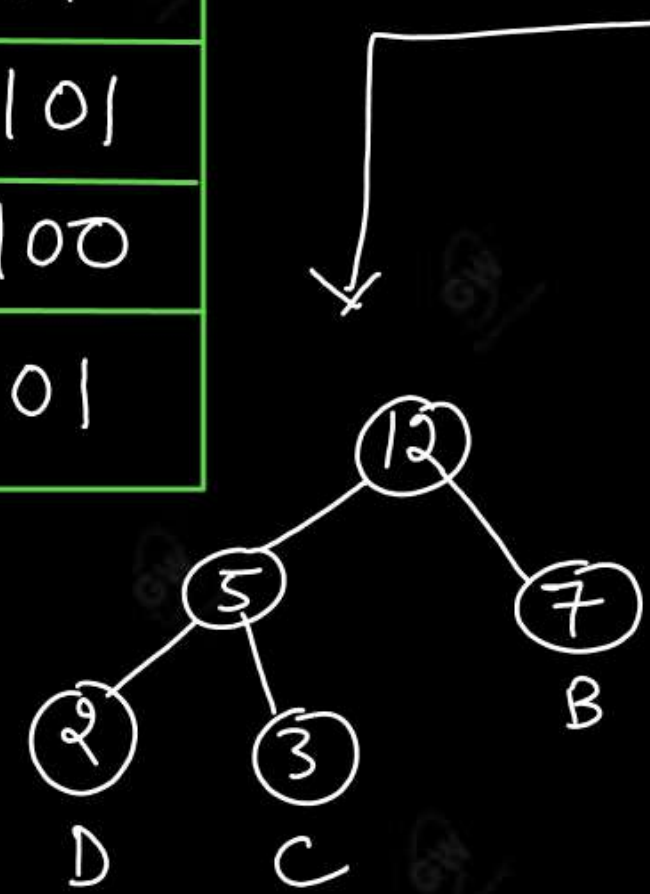
Compress the following message using huffman algorithm

ABBCDBCCDAABBEEEBEAB

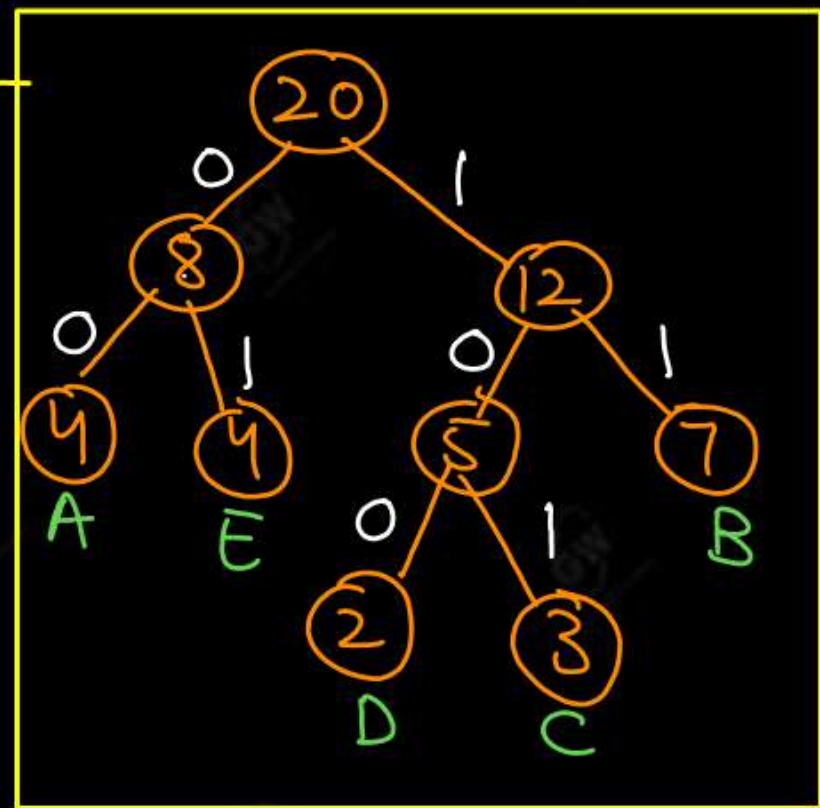
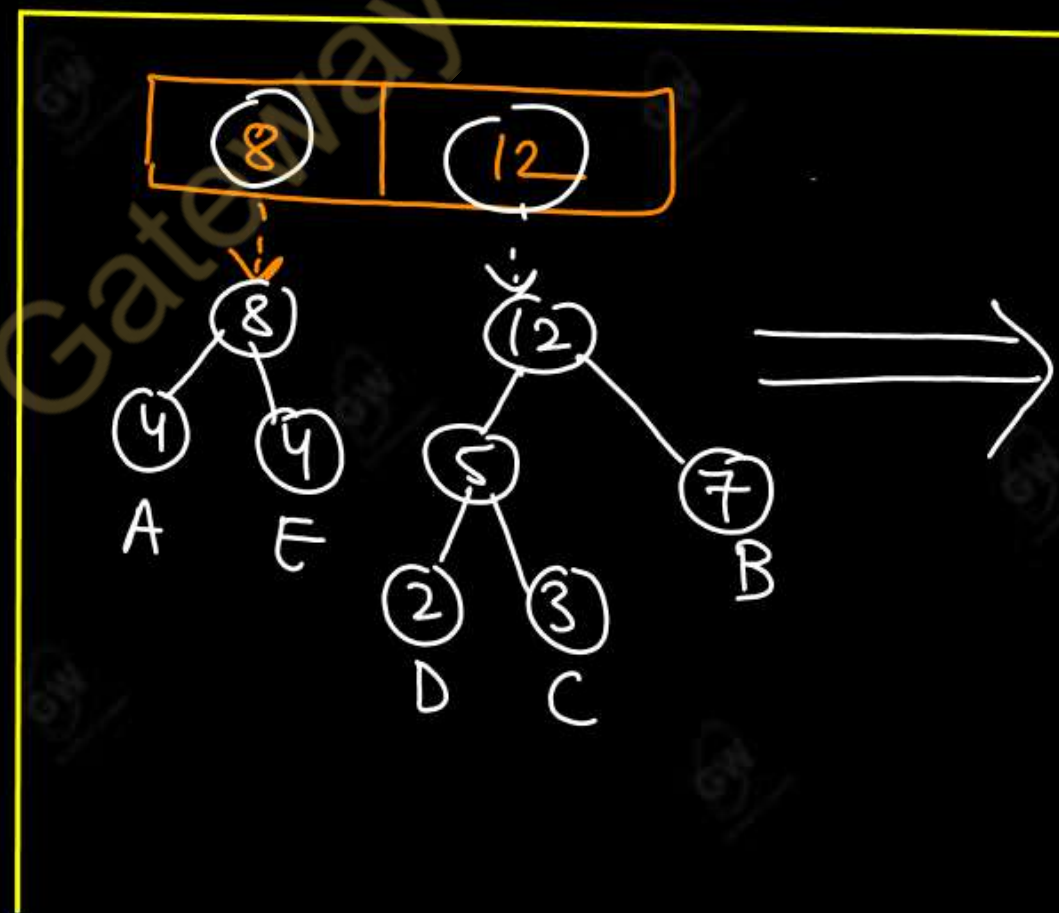
Char Freq Code

A	4	00
B	7	11
C	3	101
D	2	100
E	4	01

-	B	
5	7	8



Final Huffman Tree



Compress the following message using huffman algorithm

ABBCDBCCDAABBEEEBEAB

Char Freq Code

A	4	00
B	7	11
C	3	101
D	2	100
E	4	01

length of the message sent -

No. of bits used in encoded character

$$4 \times \underline{2} + 7 \times \underline{2} + 3 \times \underline{3} + 2 \times \underline{3} + 4 \times \underline{2}$$

$$8 + 14 + 9 + 6 + 8$$

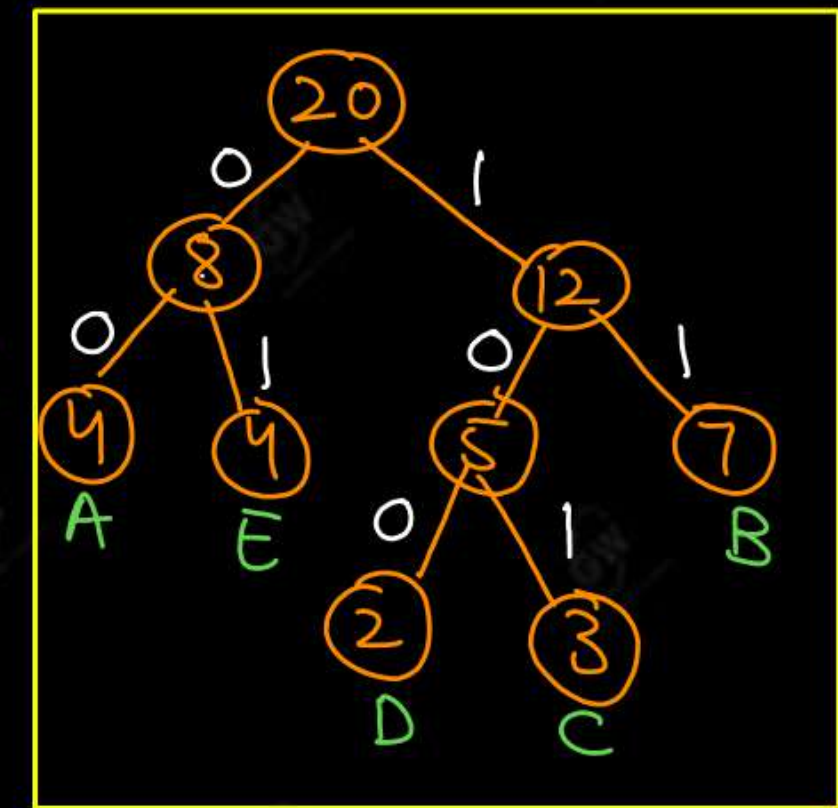
= 45 bits are sent.

Avg bits/char

$$\Rightarrow \frac{45}{4+7+3+2+4}$$

At receiver side. How to decode?
 → Either send Huffman tree
 → Or send the above table.

Final Huffman Tree



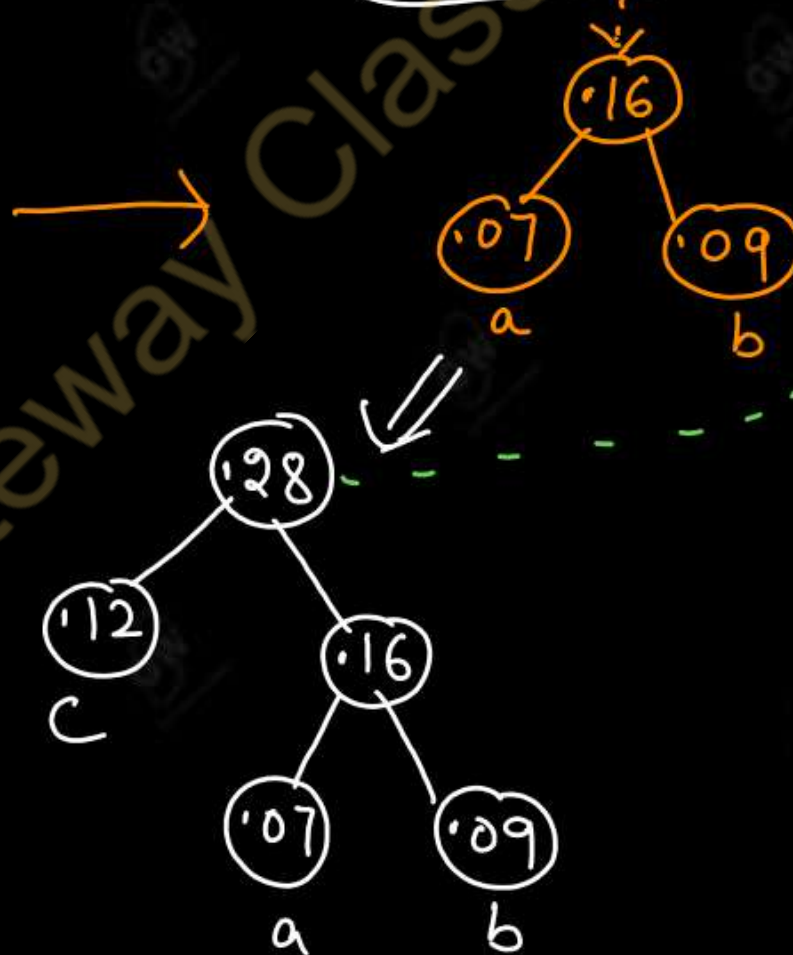
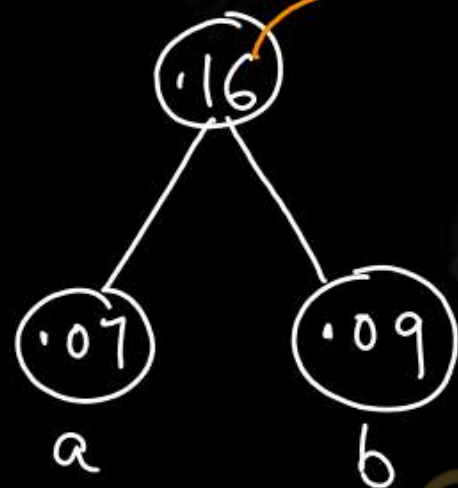
Q: With Suppose character a, b, c, d, e, f has probabilities 0.07, 0.09, 0.12, 0.22, 0.23, 0.27 respectively. Find an optional Huffman code and draw the Huffman tree. What is the

average code length? (AKTU 2023-24)

char	Prob	code
a	0.07	
b	0.09	
c	0.12	
d	0.22	
e	0.23	
f	0.27	

a	b	c	d	e	f
.07	.09	.12	.22	.23	.27

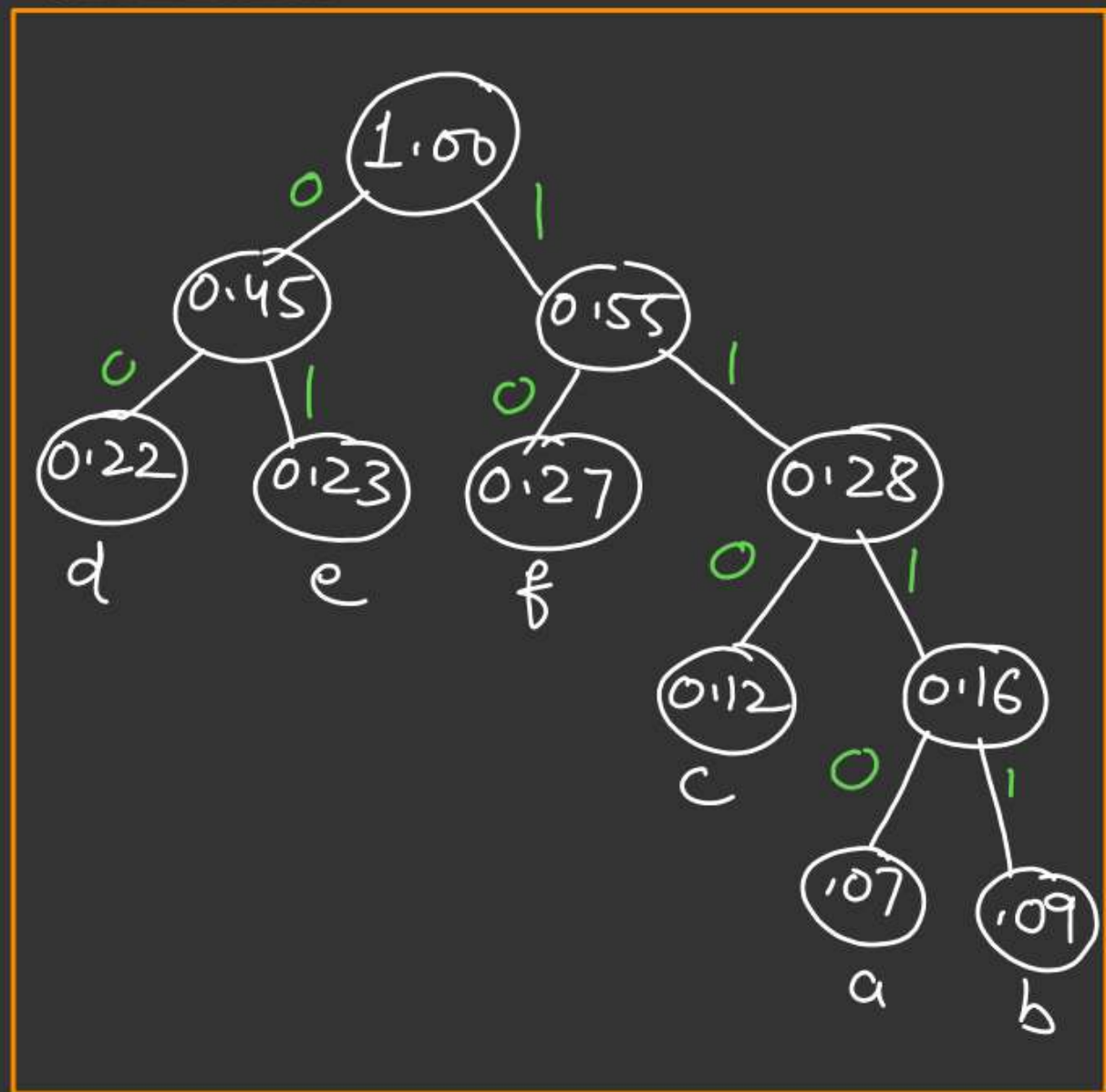
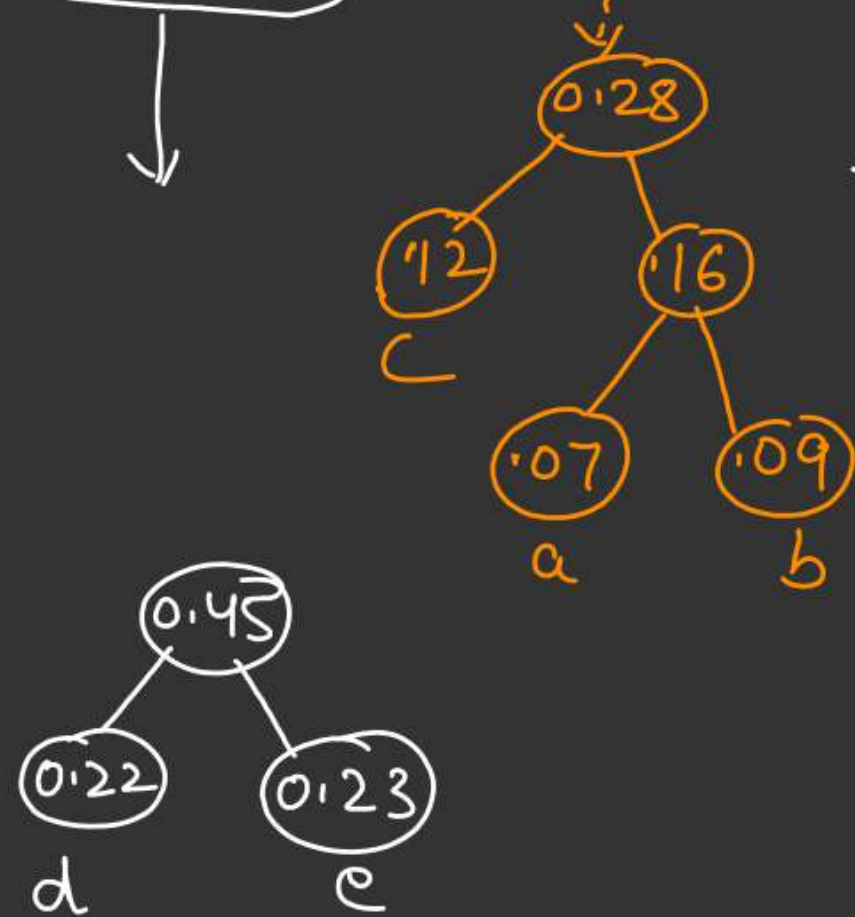
c	-	d	e	f
.12	.16	.22	.23	.27



d	e	f	—
.22	.23	.27	.28

f	—	—
.27	.28	.45

—	—
.45	.55



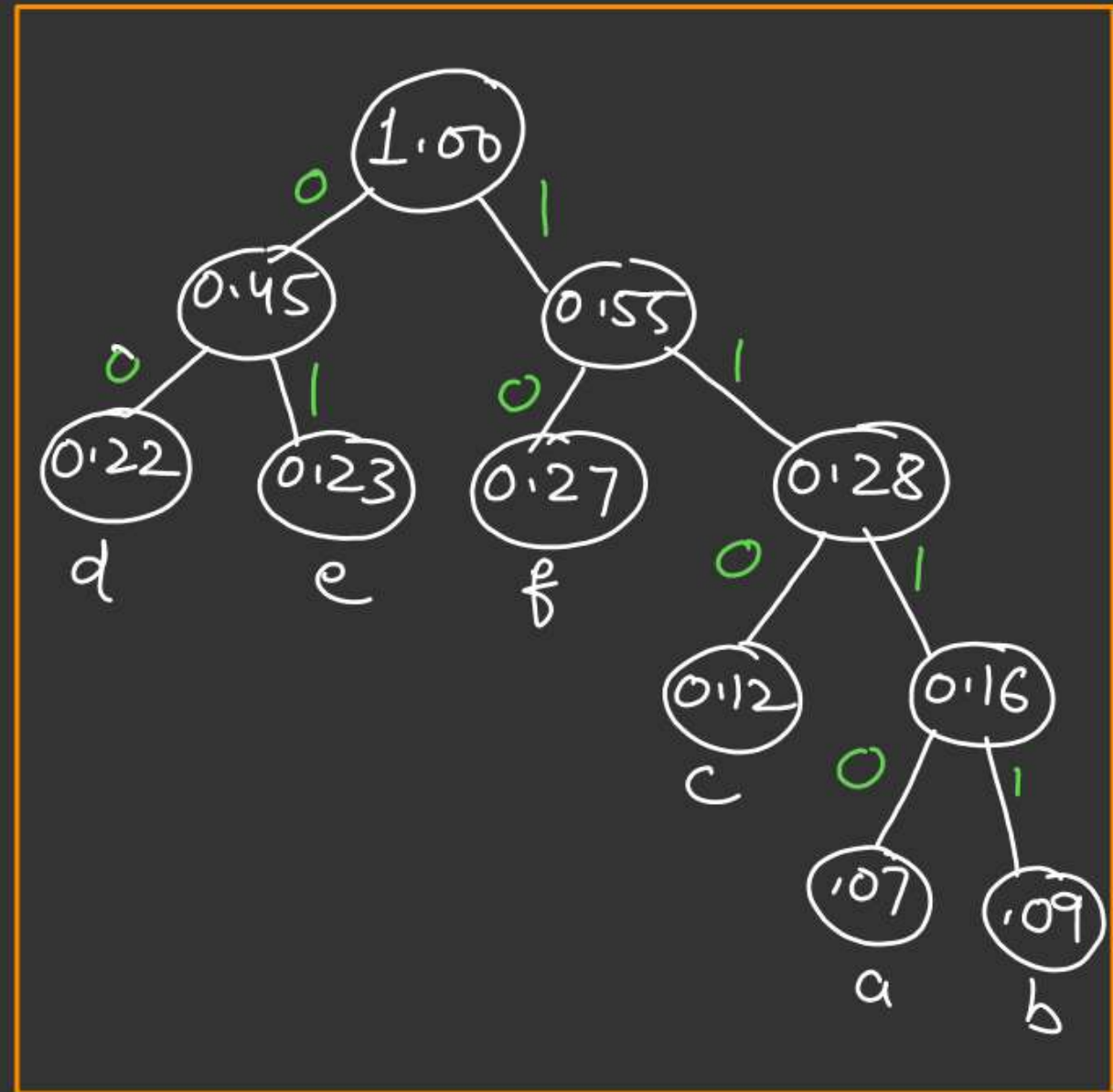
char	prob	codes	length
a	.07	1110	4
b	.09	1111	4
c	.12	110	3
d	.22	00	2
e	.23	01	2
f	.27	10	2

Average code length

$$= .07 \times 4 + .09 \times 4 + .12 \times 3 + .22 \times 2 + .23 \times 2 + .27 \times 2$$

$$= .28 + .36 + .36 + .44 + .46 + .54$$

$$= \boxed{2.44} \text{ Ans.}$$



Huffman Algorithm:

1. create a priority queue Q consisting of each unique character.
2. sort then in ascending order of their frequencies.
3. for all the unique characters:
 - create a newNode
 - extract minimum value from Q and assign it to leftChild of newNode
 - extract minimum value from Q and assign it to rightChild of newNode
 - calculate the sum of these two minimum values and assign it to the value of newNode
 - insert this newNode into the tree
4. return rootNode

1. Efficient Encoding: binary tree allows for the creation of variable-length codes, where frequently occurring characters are assigned shorter codes, leading to efficient encoding and compression of data.
2. Prefix Codes: The binary tree facilitates the creation of prefix codes, ensuring that no code is a prefix of another. This property enables unambiguous decoding of the compressed data.
3. Optimal Prefix Codes: The binary tree is used to construct an optimal prefix code, where the most frequent characters have the shortest codes, resulting in minimal average code length and efficient compression.
4. Tree Traversal: The binary tree structure allows for efficient traversal during encoding and decoding processes, enabling quick access to the codes for each character.

1. Illustrate the significance of Threaded Binary Tree. (AKTU 2024-25,2020-21)
2. What is the significance of binary tree in Huffman algorithm? (AKTU 2023-24)
3. With Suppose character a, b, c, d,e,f has probabilities 0.07, 0.09, 0.12, 0.22, 0.23, 0.27 respectively. Find an optional Huffman code and draw the Huffman tree. What is the average code length? (AKTU 2023-24)



AKTU

B.Tech III-Sem



GATEWAY CLASSES

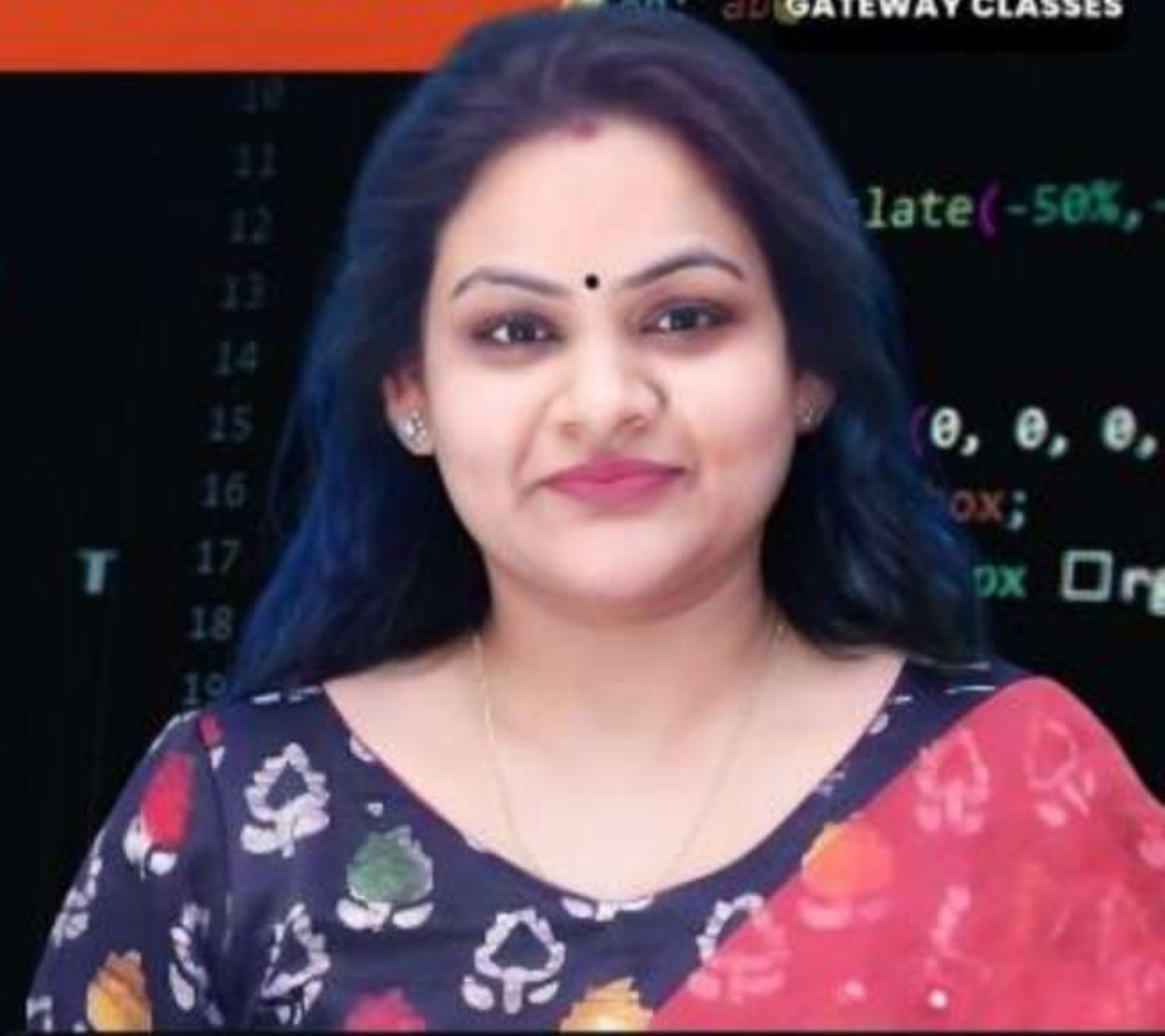
CS IT & CS Allied

DATA STRUCTURE

Unit-4 Lecture-8

Today's Target

- Creating Expression tree
- Misc types of questions
- AKTU PYQs
- DPP



By Dr. Nidhi Parashar Ma'am

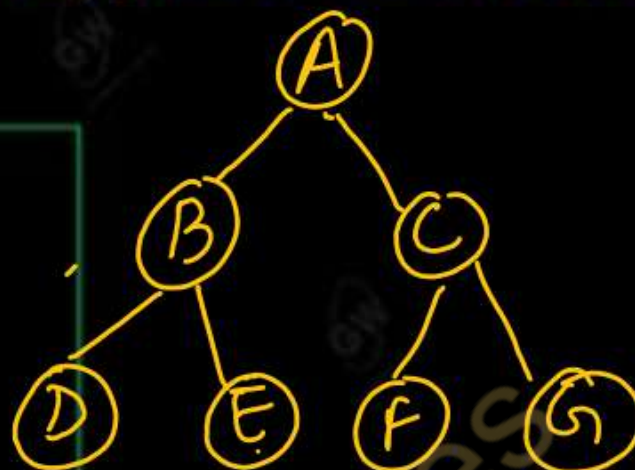
Ph.D (CSE)

- **M.Tech Gold Medalist**
- **Net Qualified**

Non-recursive(Iterative) post order traversal

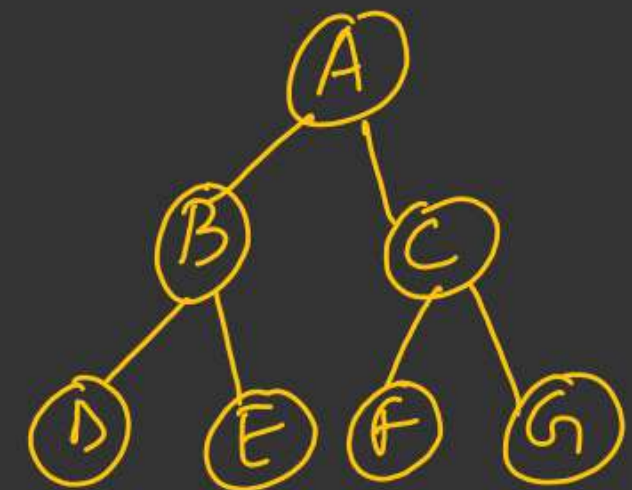
Steps -

1. Declare 2 Stacks: S1 and S2.
2. Push root to the 1st stack (S1).
3. Iteration Loop while S1 is not empty:
 - i) Pop the top node of S1 and push it to S2.
 - ii) Push node → left to S1 (if it exists).
Push node → right to S1 (if it exists).
4. Print the contents of S2.

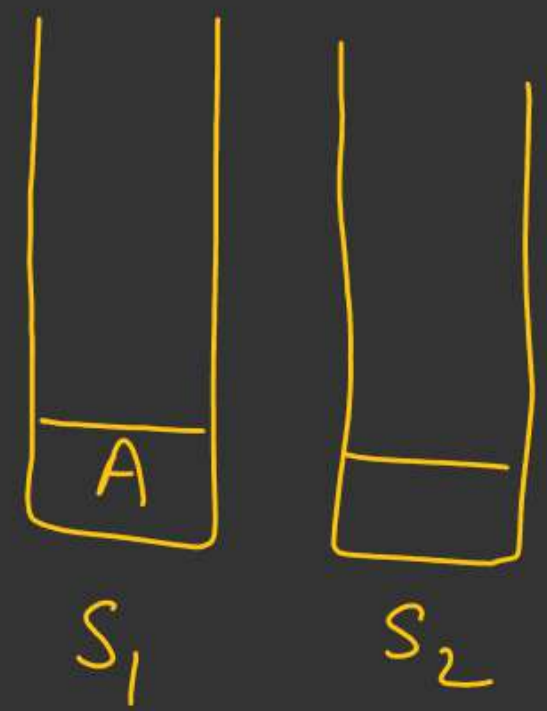


post-order ⇒ left → right → Root

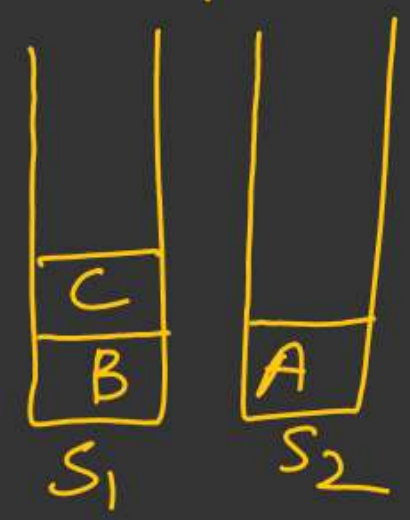
DEBFGCA



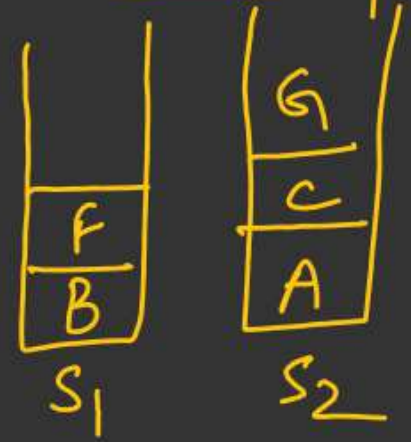
Step-1.



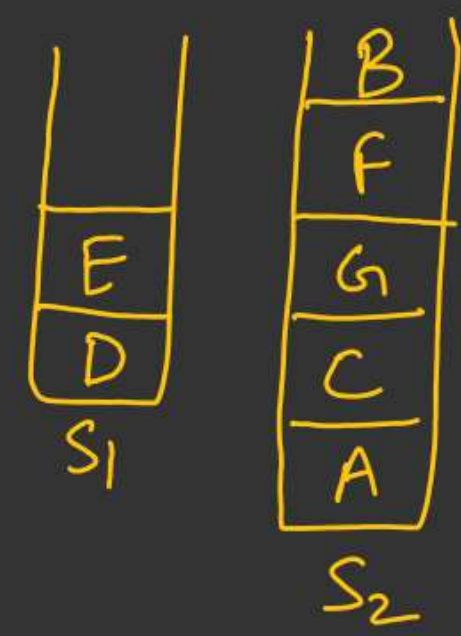
Step-2.



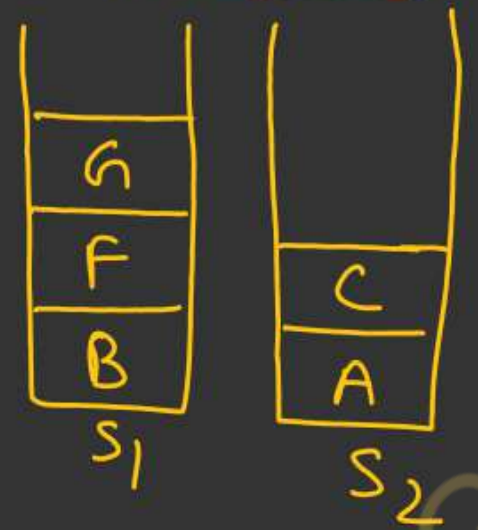
Step-4



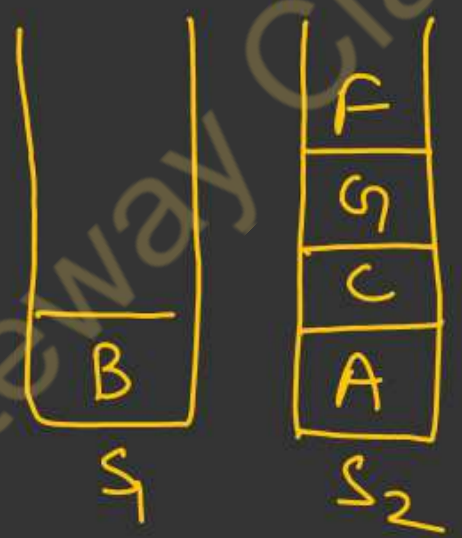
Step-6



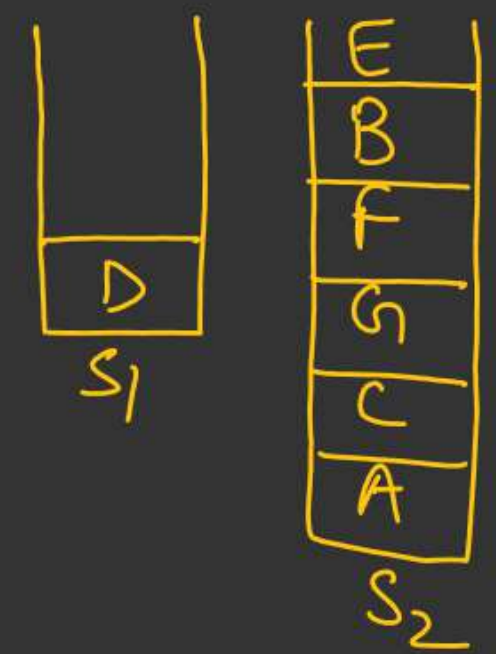
Step-3.



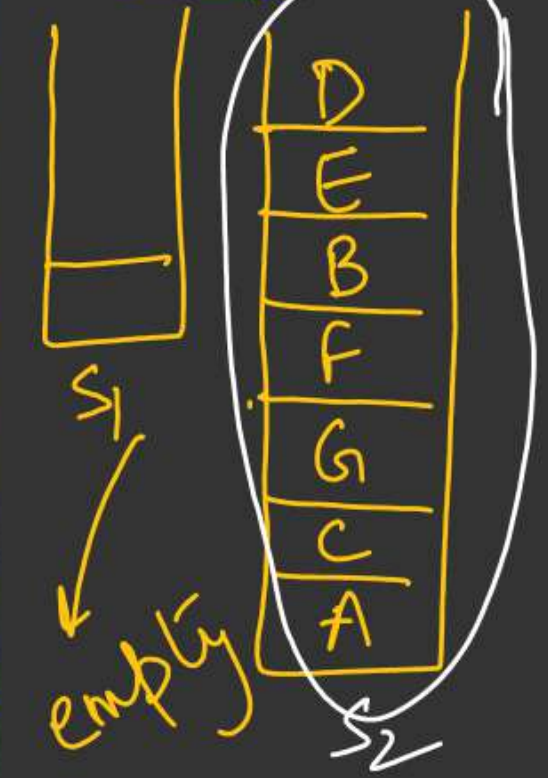
Step-5



Step-7



Step-8



Post-order

DEBFGCA

Q: Write C function for non-recursive post order traversal. (AKTU 2023-24)

```
void postOrderIterative(struct Node* root)
{
    if (root == NULL) return; // Empty tree
    struct StackNode *s1 = NULL;
    struct StackNode *s2 = NULL;
    push(&s1, root); // Push root to first stack

    while (!isEmpty(s1)) // Run while first stack is not empty
    {
        struct Node *temp = pop(&s1); // Pop an item from s1 and push it to s2
        push(&s2, temp);
        if (temp->left) // Push left and right children of popped item to s1
            push(&s1, temp->left);
        if (temp->right)
            push(&s1, temp->right);
    }
}
```

```
// Print all elements in s2 (which is in post order)
```

```
while (!isEmpty(s2))
```

```
{
```

```
    struct Node *temp = pop(&s2);
```

```
    printf("%d ", temp->data);
```

```
}
```

```
}
```

Gateway Classes

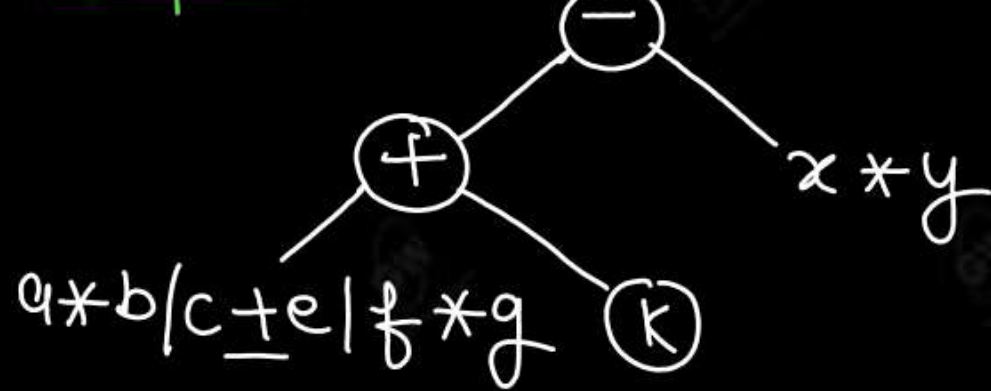
Constructing an expression tree for algebraic expressions from inorder traversal

* / $\uparrow$ precedence
+ - $\downarrow$ L $\rightarrow$ R

Step-1

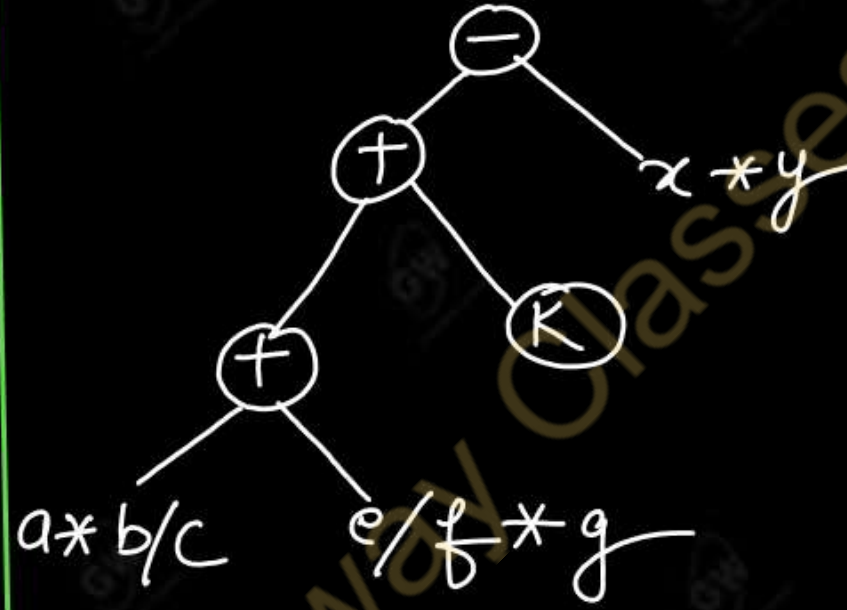


Step-2

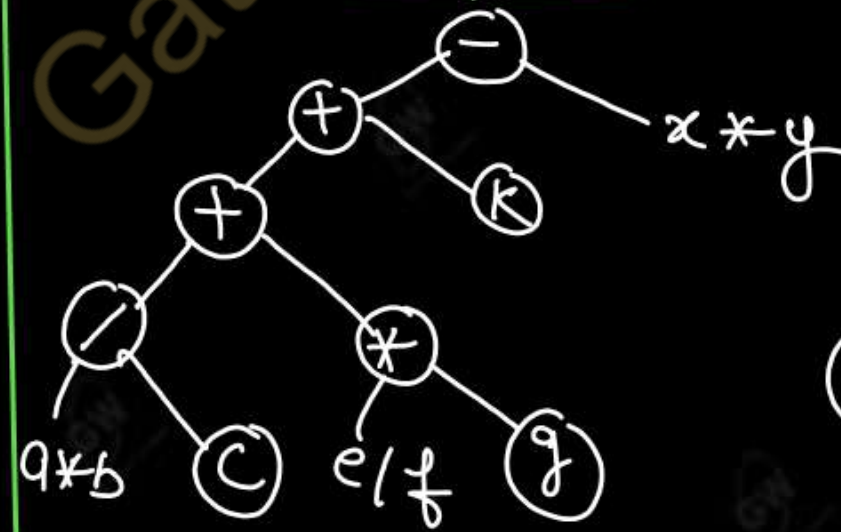


a*b/c + e/f*g + k - x*y

Step-3



Step-4

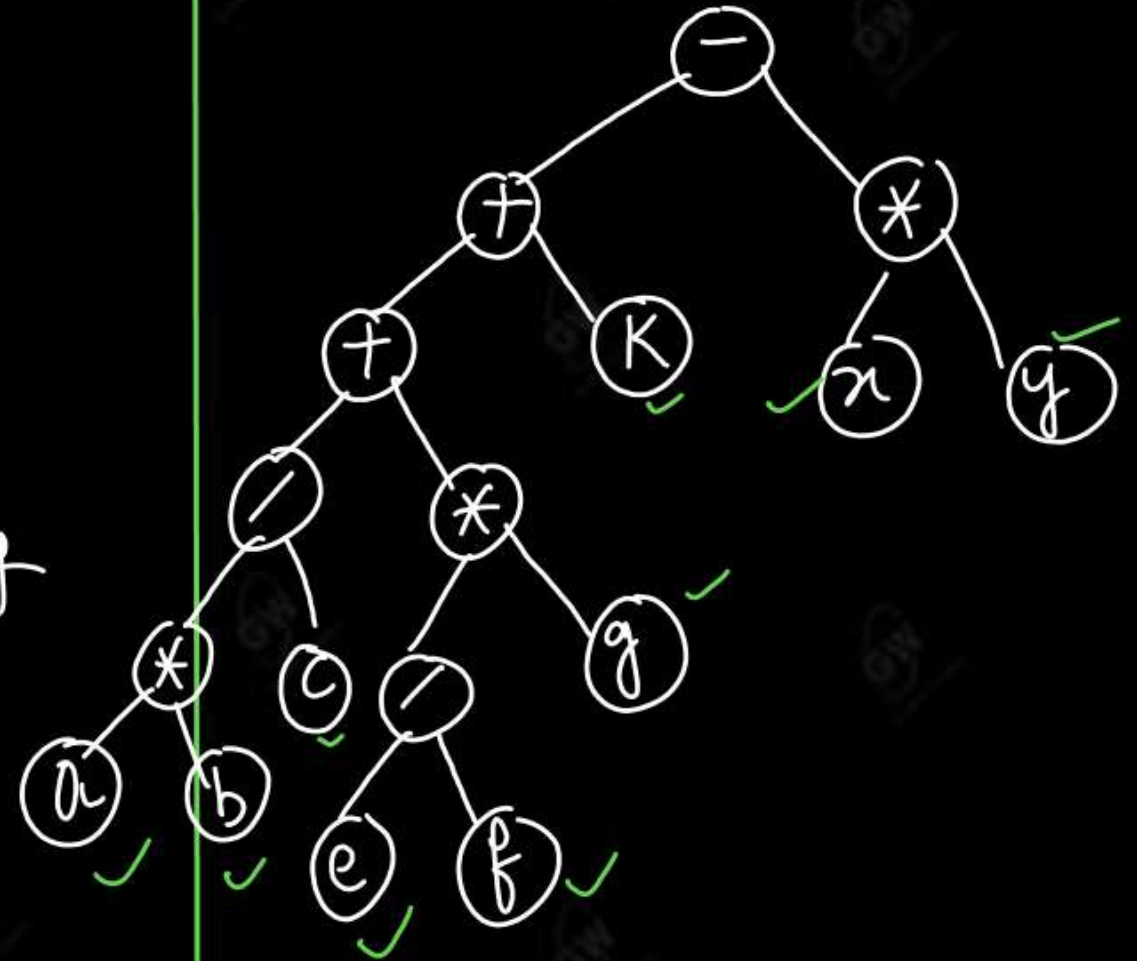


Internal nodes

— operators

leaf — operands

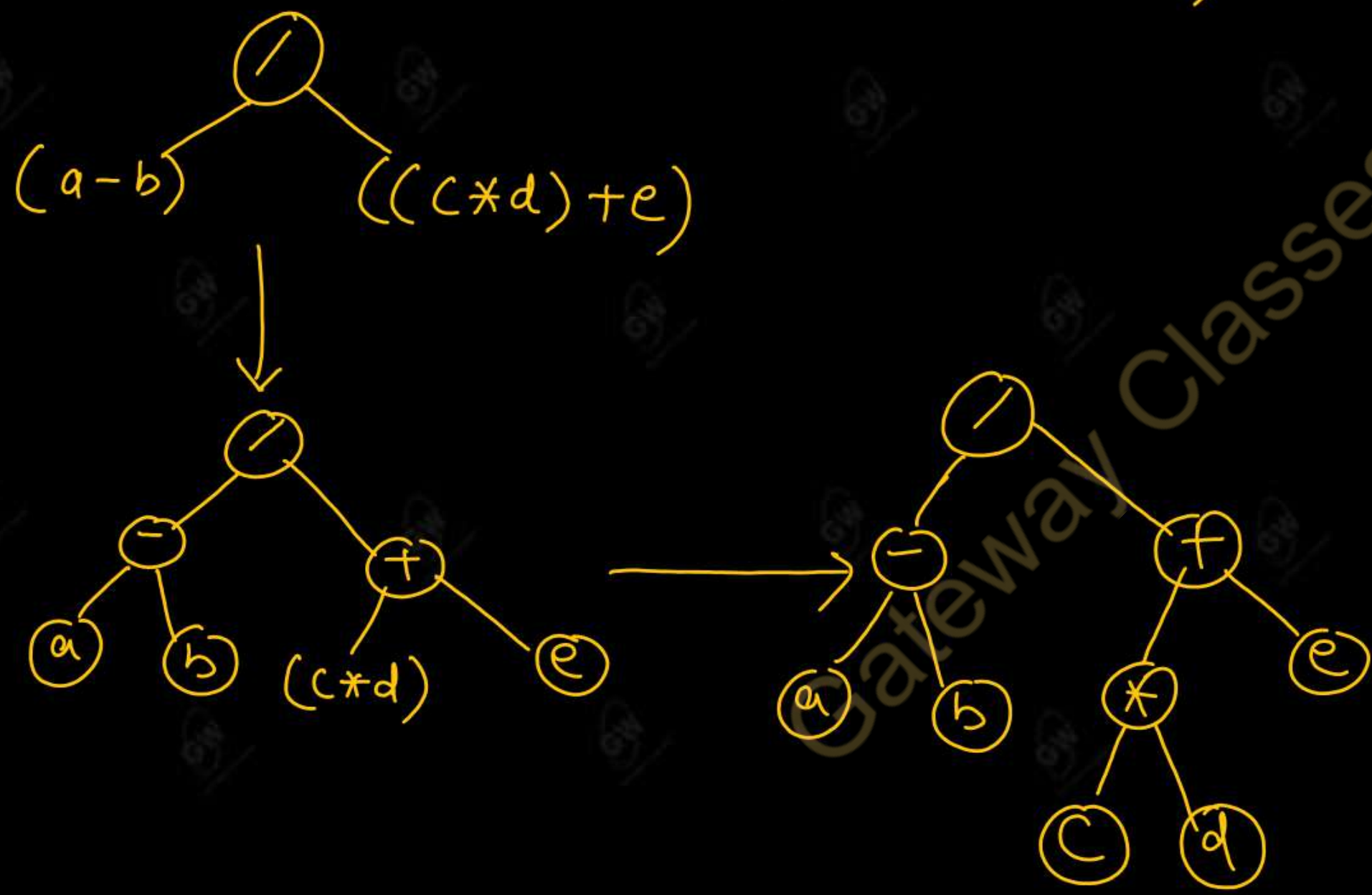
Step-6



Q: Construct an expression tree for following algebraic expression: $(a - b) / ((c * d) + e)$

$$(a-b) / ((c*d)+e)$$

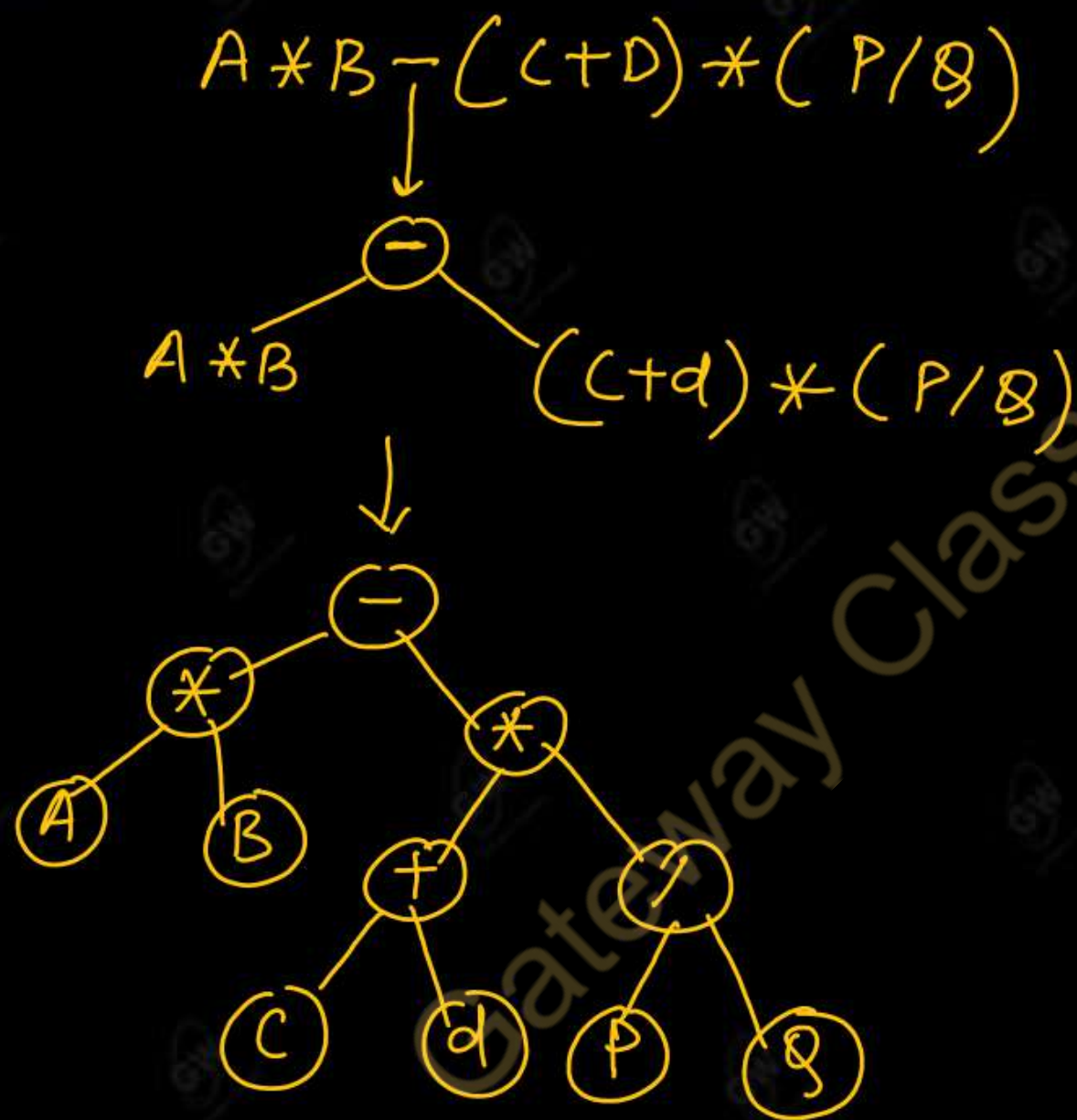
(AKTU 2022-23)

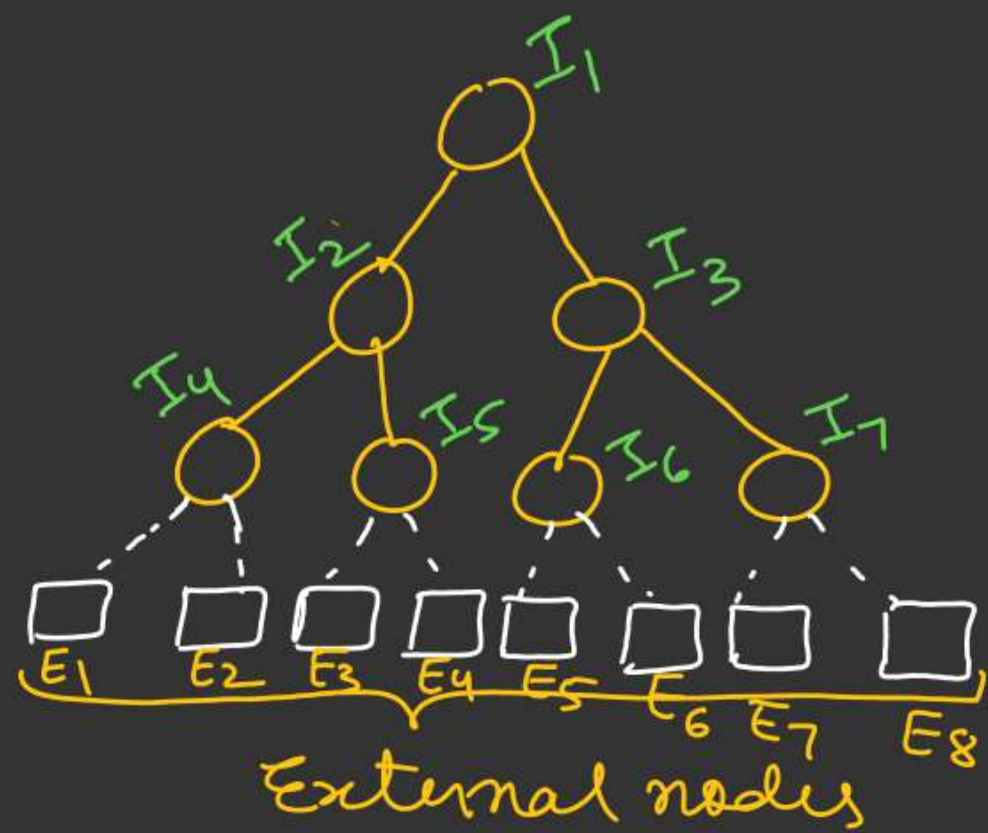


Q: Construct an expression tree for following algebraic expression: $A * B - (C + D) * (P / Q)$

(AKTU 2022-23)

2018-19





$$n = 7$$

$$E = I + 2n$$

Total no of internal nodes

$$24 = 10 + 2 \times 7$$

$$24 = 10 + 14$$

External path length ^(E) - $3 + 3 + 3 + 3 + 3 + 3 + 3 + 3 \Rightarrow 24$

Internal path length ^(I) - $0 + 1 + 1 + 2 + 2 + 2 + 2 = 10$

$$E = 1 + 1 = 2$$

$$I = 0$$

$$E = I + 2n$$

$$2 = 0 + 2 \times 1$$

$$E = 2 + 2 + 1 = 5$$

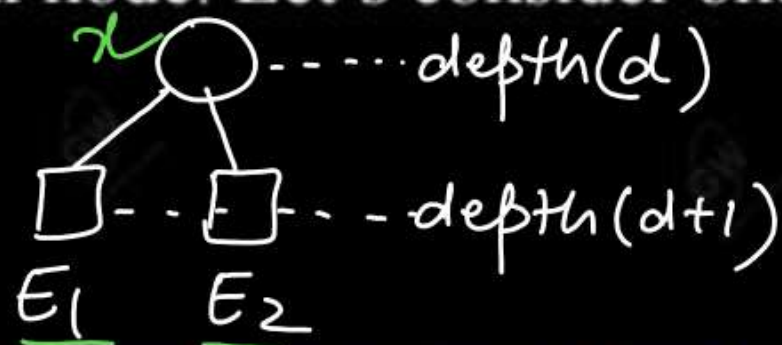
$$I = 0 + 1 = 1$$

$$E = I + 2n$$

$$5 = 1 + 2 \times 2$$

Q: If E and I denotes the external and internal path length of a binary tree having n internal nodes then show that $E=I+2n$. (AKTU 2023-24)

Means: the total depth of all leaves is equal to the total depth of all internal nodes plus 2 for each internal node. Let's consider one internal node.



– If node x is at depth (d)

Total depth of E_1 and E_2 combined: $(d+1)+(d+1)=2d+2$

How much depth external nodes add?

$(2d+2)-d=d+2$ This means –

– Each internal node contributes exactly 2 more to the external path length than it contributes to the internal path length.

– So if you have n internal nodes, then this extra amount becomes $2 \times n$. So the total external path length must be:

$$E=I+2n$$

1. If E and I denotes the external and internal path length of a binary tree having n internal nodes then show that $E=I+2n$. (AKTU 2023-24)
2. Construct an expression tree for the following algebraic expression: $(a - b) / ((c * d) + e)$ (AKTU 2022-23)
3. Write C function for non-recursive post order traversal. (AKTU 2023-24)



AKTU



Gateway Classes



CS IT & CS Allied

Gateway Udaan Batch



Sem-IV

Validity Till
Sem Exam

COMBO of 5 Subjects



Video Lectures



Pdf Notes



AKTU PYQs



Most Imp. Questions

Helpline No-7819 0058 53





Gateway Classes



Empowering Learners, Transforming Futures

B.Tech Courses



GET IT ON
Google Play



Download on the
App Store

Helpline No- 7819 0058 53